

Стъпка 1 Обобщение — Основи на обучението с подсилване

Изследване върху възможностите за прилагане на изкуствен интелект в компютърните игри

Alexander Andreev

April 2026

Съдържание

1	Стъпка 1 — Основи на обучението с подкрепление	1
1.1	Как обучението с подкрепление се утвърди като самостоятелна дисциплина	1
1.2	Основни принципи — агент, действие, среда	2
1.3	Формулировка на MDP	3
1.4	Информационни множества — когато не можете да видите всичко	4
1.5	Динамично програмиране — При наличие на модел	5
1.6	Обучение с темпорална разлика — Обучение без модел	6
1.7	DQN — Дълбоки Q-мрежи	7
1.8	PPO — Оптимизация на проксималната стратегия	8
1.9	Практическо валидиране — собствени имплементации спрямо stable-Baselines3	9

1 Стъпка 1 — Основи на обучението с подкрепление

Това е съкратен обзор на основните материали по обучение с подкрепление, разгледани в Стъпка 1. Той е предназначен за две цели: да служи като бърза справка при преминаване към следващите стъпки и като основен източник за **синтеза** на публичния доклад от Стъпка 15.

1.1 Как обучението с подкрепление се утвърди като самостоятелна дисциплина

Обучението с подкрепление не възниква изведнъж като завършена дисциплина. Неговите корени се простират в множество области, които дълги десетилетия почти не са били свързани помежду си.

Най-ранната нишка идва от **животинската психология** в началото на XX век — „Законът за ефекта“ на Торндайк (1911) предполага, че действията, последвани от удовлетворяващи резултати, стават по-вероятни. Това всъщност представлява идеята за **сигнал за награда**. Успоредна нишка преминава през **теорията на оптималното управление** през 50-те години на миналия век, където Белман разработва **динамично програмиране** за решаване на **последователни проблеми на вземане на решение**. Той въвежда **функцията на стойността** и **принципа на оптималност**, които и двете са в **ядрото** на съвременното обучение с подкрепление. Трета нишка идва от **обучението чрез проба и грешка** в ранните изследвания в областта на

изкуствения интелект — програмата за дама на Самюел (1959) се учи, като играе срещу себе си, коригирайки своята **оценъчна функция** въз основа на победи и загуби.

Тези нишки остават отделни до 80-те и 90-те години на миналия век. Докторската работа на Сътън върху обучението с темпорална разлика (1988) запълва празнината между теориите за животинското учене и динамичното програмиране на Белман. След това Уоткинс формализира q-обучението (1989), предоставяйки на областта първия ясен, безмоделен алгоритъм за управление със сходимостни гаранции. Към момента, когато Сътън и Барто публикуват своя учебник през 1998 г. (актуализиран през 2018 г.), обучението с подкрепление вече се е утвърдило като разпознаваема дисциплина със собствена нотация, таксономия на проблемите и алгоритмичен инструментариум.

Революцията в дълбокото обучение достига до обучението с подкрепление през 2013–2015 г., когато DQN на DeepMind се научава да играе Atari Games, използвайки само сурови пиксели. Тази демонстрация ясно показва, че алгоритмите за обучение с подкрепление, комбинирани с невронни апроксиматори на функции, могат да решават високоизмерни проблеми от реалния свят. PPO се появява през 2017 г. като практичен метод на градиента на стратегията и оттогава обучението с подкрепление се разширява в областите на роботиката, игровия изкуствен интелект, системите за препоръки и привеждането в съответствие на езикови модели (RLHF).

Важното е да се подчертае, че обучението с подкрепление не е просто „машинно обучение с награди“. То представлява самостоятелна рамка за последователно вземане на решения при несигурност, със собствени теоретични основи, произтичащи от теорията на управлението, психологията и статистиката.

Прочетете повече: Sutton, R.S. & Barto, A.G. (2018). *Обучение с подкрепление: Въведение*, второ издание — Глава 1.

Безплатно: <http://incompleteideas.net/book/the-book-2nd.html>

1.2 Основни принципи — агент, действие, среда

Цялата рамка на обучението с подкрепление се основава на прост цикъл. **Агентът** наблюдава текущото **състояние** на **средата**, избира **действие** и получава сигнал за **награда** заедно с ново **състояние**. След това процесът се повтаря. Целта на агента е да научи **стратегия** — съпоставяне между състояния и действия — която максимизира натрупаната награда във времето.

Няколко уточнения относно терминологията:

- **Състояние** (s): описание на средата в даден момент. В cartPole това включва позицията на количката, скоростта на количката, ъгъла на пръта и ъгловата скорост на пръта. В покер биха били раздадените карти и историята на залозите.
- **Действие** (a): възможните действия на агента. Те могат да бъдат дискретни (например бутане наляво или надясно) или непрекъснати (например прилагане на конкретен въртящ момент).
- **Награда** (r): скаларна величина за обратна връзка, която агентът получава след всяко действие. Наградите могат да бъдат оскъдни (само в края на играта) или гъсти (на всеки времеви интервал).

- **Стратегия (π):** стратегията на агента. Тя може да бъде детерминистична ($\pi(s) = a$) или стохастична, като $\pi(a|s)$ дава разпределение на вероятностите върху възможните действия.
- **Функция на стойността ($V^\pi(s)$):** очакваната натрупана награда от състояние s нататък, ако агентът следва стратегия π . Тя показва на агента колко добро е едно състояние в дългосрочен план, а не само непосредствено.
- **Коефициент на дисконтиране (γ):** число между 0 и 1, което определя доколко агентът отчита бъдещите награди спрямо непосредствените такива. Висока стойност на $\gamma = 0.99$ означава, че агентът е търпелив; ниска стойност на $\gamma = 0.5$ показва, че е късоглед.

Критичното предизвикателство при проектирането на системи за обучение с подкрепление е **компромисът изследване–експлоатация**. Агентът трябва да изследва непознати действия, за да открие потенциално по-добри стратегии, но същевременно и да експлоатира наличните си знания, за да събира награда. Прекомерното изследване води до загуба на време, а прекалената експлоатация може да доведе до заседналост в локален оптимум.

Съществуват два основни класа алгоритми. **Методите, базирани на стойност** (като DQN) научават стойността на състоянията или двойките състояние-действие и извеждат стратегия въз основа на тези стойности. **Методите, базирани на политика** (като PPO) научават политиката директно. И двата подхода имат своите предимства; значителна част от областта на обучението с подкрепление е посветена на тяхното ефективно комбиниране.

Прочетете повече: OpenAI Spinning Up — „Ключови концепции в обучението с подкрепление“
https://spinningup.openai.com/en/latest/spinningup/rl_intro.html

1.3 Формулировка на MDP

Марковски процес на вземане на решения (MDP) представлява формалната математическа рамка, върху която са изградени повечето алгоритми за обучение с подкрепление. Той включва пет основни компонента:

- S : множество от състояния
- A : множество от действия
- $P(s'|s, a)$: функция на преход — вероятност за преминаване в състояние s' при дадено състояние s и действие a
- $R(s, a, s')$: функция на наградата
- $\gamma \in [0, 1)$: коефициент на дисконтиране

Частта „Марков“ означава, че бъдещето зависи единствено от текущото **състояние**, а не от историята на пътя до него. Това е силно предположение — и такова, което не е възникнало в завършен вид в много интересни ситуации (като **покер**, където историята на залозите има значение). Разглеждаме този въпрос в Раздел 4.

Целта е да се намери стратегия π^* , която максимизира очакваната дисконтирана възвръщаемост.

$$\pi^* = \arg \max_{\pi} \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r_t \right]$$

Ключовата рекурсивна зависимост е **уравнението на Белман**:

$$V^{\pi}(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V^{\pi}(s')]$$

Това означава, че стойността на едно състояние е равна на очакваната непосредствена награда плюс дисконтираната стойност на следващото състояние. Цялото динамично програмиране, tD learning и дълбокото обучение с подсилване се основават на вариации на това уравнение.

Прочетете повече: Сътон и Барто (2018). *Обучение с подкрепление: Въведение*, глава 3 — Крайни марковски процеси на вземане на решения.

Безплатно: <http://incompleteideas.net/book/the-book-2nd.html>

1.4 Информационни множества — когато не можете да видите всичко

Стандартните MDP предполагат, че агентът може напълно да наблюдава състоянието. В действителност обаче често това не е така. В покера например не виждате картите на опонента, а в игрите със стратегия в реално време действа т.нар. **мъгла на войната**. Това са **частично наблюдаеми** среди.

Формалното разширение е **Partially Observable MDP (POMDP)**, който включва функция за наблюдение — агентът получава наблюдения, представляващи шумни или непълни изгледи на истинското състояние. Но POMDP са печално известни с това, че оптималното им решаване е изключително трудно.

В теорията на игрите същата идея се изразява чрез **информационни множества**. Информационно множество обединява всички състояния на играта, които играчът не може да различи едно от друго въз основа на направените наблюдения. В кун покер (3 карти, 2 играчи), когато държите поп и все още не е имало залагания, вие се намирате в информационно множество — знаете своята карта, но не и тази на опонента си, така че не можете да определите дали действителното състояние е „Аз имам поп, опонентът има дама“ или „Аз имам поп, опонентът има цар“.

Алгоритмите, които работят в тези среди — като CFR (минимизиране на контрафактичното съжаление, разгледано в стъпка 2) — оперират върху информационни множества, а не върху отделни състояния. Това представлява фундаментално различен изчислителен проблем от стандартното RL и е причината изкуственият интелект за игра на игри да се нуждае от собствен набор инструменти, различни от тези, които DQN или PPO предоставят по подразбиране.

Разбирането на информационните множества вече е от значение, защото следващите стъпки (5–8) се занимават изцяло с игри с **непълна информация**. Алгоритмите се променят, но **ядрото** на концепцията остава:

вие вземате решения въз основа на това, което можете да наблюдавате, като разсъждавате върху това, което не можете.

Прочетете още: Shoham, Y. & Leyton-Brown, K. (2008). *Многоагентни системи: Алгоритмични, игровитеоретични и логически основи*, Глава 5.

Безплатно: <http://www.masfoundations.org/download.html>

1.5 Динамично програмиране — При наличие на модел

Динамичното програмиране (DP) представлява най-ранният подход за решаване на MDP. То е приложимо, когато разполагате с пълна информация за средата: вероятностите за преход $P(s'|s, a)$ и функцията на наградата $R(s, a, s')$. На практика това означава, че правилата на играта са ви напълно известни.

Двата основни алгоритъма при динамичното програмиране са:

Итерация на политиката редува два етапа. Първо се извършва *оценка на стратегията*: при дадена фиксирана стратегия се изчислява стойността на всяко състояние чрез многократно прилагане на уравнението на Белман, докато не се постигне сходимост. След това следва *подобрене на стратегията*: стратегията се актуализира така, че да бъде алчна спрямо новоизчислените стойности. Процесът се повтаря, докато стратегията престане да се променя. Гарантирано е, че тя сходява към оптималната стратегия.

Итерация на стойностите е пряк път. Вместо да се оцени напълно една **стратегия**, преди тя да бъде подобрена, тя комбинира оценката и подобренieto в една единствена актуализация. На всяка стъпка, за всяко **състояние**, се избира **действието**, което максимизира очакваната едностъпкова възвръщаемост плюс дисконтираната стойност на следващото **състояние**. Тя сходява към оптималната функция на стойността, от която се извлича оптималната **стратегия**.

DP е полезен, защото осигурява точни решения и има ясни гаранции за сходимост. Недостатъкът е очевиден: необходим е пълният модел, а изчислението нараства с броя на състоянията. За кун покер с 12 състояния DP е тривиален. За Го със 10^{170} състояния това е невъзможно.

Причината DP да има значение, въпреки че рядко го използваме директно, е, че всеки следващ метод — метод Монте Карло, tD learning, DQN, PPO — може да се разглежда като приближение към решението на DP, когато моделът е неизвестен или пространството на състоянията е твърде голямо. DP представлява теоретичния таван, към който практическите алгоритми се стремят да се доближат.

Прочетете повече: Сътон и Барто (2018). *Обучение с подкрепление: Въведение*, Глава 4 — Динамично програмиране.

Безплатно: <http://incompleteideas.net/book/the-book-2nd.html>

1.6 Обучение с темпорална разлика — Обучение без модел

Обучението с темпорална разлика (TD) е пробивът, който направи обучението с подкрепление приложимо на практика. За разлика от динамичното програмиране, то не изисква модел на средата. За разлика от методите на Монте Карло, то не е необходимо да чака края на един епизод, за да се учи. То актуализира стойностните оценки след всяка стъпка, използвайки самоподкрепяне: текущата оценка на стойността на следващото състояние замества истинската бъдеща възвръщаемост.

Основната актуализация на TD(0) за функцията на стойността е:

$$V(s_t) \leftarrow V(s_t) + \alpha [r_t + \gamma V(s_{t+1}) - V(s_t)]$$

Терминът в скобите е **tD грешка** — разликата между настъпилото (награда плюс оценена бъдеща стойност) и предвиденото. Ако tD грешката е положителна, състоянието се е оказало по-добро от очакваното; ако е отрицателна, то е било по-лошо.

Контролната версия на тази идея е **q-обучение** (Watkins, 1989):

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right]$$

q-обучението научава стойността на двойки състояние-действие и използва оператора $\max$, за да бъде **извънполитиково** — то се учи относно оптималната стратегия дори докато следва изследователска. Това го прави толкова практично.

tD learning е по-изгодно от динамичното програмиране, тъй като не изисква модел. То е по-изгодно и от метода Монте Карло, защото се учи онлайн (на всяка стъпка, а не на всеки **епизод**) и има по-ниска **дисперсия**. **Самоподкрепянето** въвежда известно **пристрастие**, но в практиката тази размяна е изключително благоприятна. tD методите са основата, върху която се градят DQN и повечето съвременни **алгоритми, базирани на стойности**.

SARSA е вариантът на q-обучението, който работи по текущата политика — той актуализира стойностите, използвайки действието, което агентът действително предприе след това, а не най-доброто възможно действие. Това го прави по-консервативен и понякога по-безопасен на практика, но не му позволява да научи оптимална стратегия, докато следва под-оптимална изследователска стратегия.

Прочетете повече: Сътон и Барто (2018). *Обучение с подкрепление: Въведение*, Глава 6 — Обучение с темпорална разлика.

Безплатно: <http://incompleteideas.net/book/the-book-2nd.html>

1.7 DQN — Дълбоки Q-мрежи

DQN (Mnih и др., 2015) е алгоритъмът, който доказва възможността дълбоките невронни мрежи да се използват като апроксиматори на функция в обучението с подкрепление в голям мащаб. Преди DQN обучението с Q-стойности беше ограничено до таблична среда или ръчно извлечени признаци. DQN показва, че може да се научи да играе 49 Atari Games директно от суров пикселов вход, като надмина човешкото представяне в много от тях.

Как работи. DQN заменя Q-таблицата с невронна мрежа, която приема състояние като вход и извежда q-стойности за всички действия. Обучението на мрежата се извършва чрез минимизиране на квадратичната TD грешка — разликата между предсказаната Q-стойност и целта (награда плюс отстъпена максимална Q-стойност на следващото състояние). На всяка стъпка агентът избира действието с най-висока q-стойност (експлоатация) или произволно действие с вероятност ϵ (изследване).

Два ключови похвата осигуряват достатъчна стабилност, за да работи този подход:

1. **Повторение на натрупан опит.** Вместо да се обучава върху последователни преходи (които са силно корелирани), преходите се съхраняват в голям буфер и се извличат на случаен принцип в минипартиди. Това нарушава корелацията, намалява дисперсията и позволява всеки опит да бъде използван многократно. Принципът е аналогичен на разбъркването на тренировъчните данни при обучение с учител.
2. **Целева мрежа.** Поддържа се отделно копие на q-мрежата, което се обновява само периодично (на всеки няколко хиляди стъпки или епизода). TD target се изчислява с помощта на това замразено копие. Без него целта се измества при всяка тренировъчна стъпка — вие преследвате подвижна цел, което води до колебания и разминаване. Целевата мрежа разделя процеса на обновяване и стабилизира обучението.

Защо DQN е важна в сравнение с предишните методи: табличното Q-обучение не може да се справи с високоизмерни състояния (изображения, непрекъснати наблюдения); апроксимацията на функция само с Q-обучение беше известна като нестабилна („смъртоносна троица“ от извънполитиково + апроксимация на функция + самоподкрепяне). Двата трика на DQN — повторение на натрупан опит и целеви мрежи — адресираха нестабилността, правейки дълбокото обучение с базирано на стойност подсилващо обучение практически за първи път.

Ограничения. DQN работи единствено с дискретни пространства на действията. Тя може да надценява q-стойности (решено от Double DQN). Като извънполитиков метод, тя е ефективна по отношение на пробите, но това може да доведе до остаряла информация в буфера за възпроизвеждане. И най-важното — тя научава алчна детерминистична стратегия, без естествен начин за изразяване на стохастични стратегии, което е от значение в игрите, където смесените стратегии са оптимални.

В нашата реализация от Стъпка 1 DQN реши CartPole-v1 (постигайки средна стойност за епизод от 477,5 спрямо цел от 475) след приблизително 1000 епизода обучение. Най-влиятелните хиперпараметри бяха размерът на мрежата, епсилон минимумът и честотата на синхронизация на целевата мрежа.

Бележка относно невронните мрежи в тази стъпка. q-мрежата в DQN е стандартен многослоен пер-

цептрон (MLP), използван тук като практичен инструмент — функция, която съпоставя състояния към q -стойности. За Стъпка 1 вътрешните механизми на невронната мрежа (инициализация на теглоите, потокът на градиента, функциите на активиране) бяха разглеждани като черна кутия: използвахме настройките по подразбиране на `pyTorch` и се фокусирахме върху самия RL алгоритъм. По-задълбочено разглеждане на архитектурите на невронните мрежи и тяхното взаимодействие с методите за намиране на равновесие ще бъде представено в Стъпка 5 (невронно приближение на равновесие).

Прочетете повече: Mnih, V. и др. (2015). „Контрол на човешко ниво чрез дълбоко обучение с подкрепление.“ *Nature*, 518(7540), 529–533.
arXiv: <https://arxiv.org/abs/1509.06461>

1.8 PPO — Оптимизация на проксималната стратегия

PPO (Schulman и др., 2017) прилага фундаментално различен подход към RL в сравнение с DQN. Вместо да изчислява стойността на действията и да извежда стратегия въз основа на тези стойности, PPO **научава стратегията директно** — невронна мрежа генерира разпределение на вероятностите върху действията, а мрежата се обучава да увеличава вероятността на действията, които водят до високи възнаграждения.

Проблемът, който PPO решава. Методите на градиента на стратегията са мощни в теорията, но са печално известни с нестабилността си на практика. Основният алгоритъм **REINFORCE** страда от висока дисперсия и може да предприема разрушително големи стъпки, които провалят добра стратегия. **Trust Region Policy Optimization** (TRPO, Schulman и др., 2015) се справи с това, като ограничи колко може да се промени стратегията при всяка актуализация, но TRPO изисква скъпа оптимизация от втори ред (изчисляване на хесиана).

Как работи PPO. PPO запазва идеята за стабилност от TRPO, но заменя ограничението с много по-прост механизъм: **отсичане**. Ключовото количество е отношението на вероятностите $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{old}}(a_t|s_t)$ — колко по-вероятно (или по-малко вероятно) е новата стратегия да предприеме същото действие, което старата стратегия е предприела. PPO отсича това отношение в диапазона $[1 - \epsilon, 1 + \epsilon]$ (обикновено $\epsilon = 0.2$), което означава:

- Ако едно **действие** се окаже добро (положително **предимство**) и отношението на вероятностите надхвърли $1 + \epsilon$, **градиентът** се анулира — **стратегията** не трябва да става прекалено „алчна“ за това **действие**.
- Ако едно **действие** се окаже лошо (отрицателно **предимство**) и отношението на вероятностите падне под $1 - \epsilon$, **градиентът** също се анулира — така се предотвратява прекомерна корекция на **стратегията**.

Това е **изрязаната сурогатна цел** и именно тя осигурява работата на PPO. Тя постига стабилност, сходна с тази на TRPO, като използва единствено градиенти от първи ред (без хесиан), което значително намалява необходимата изчислителна мощност и улеснява имплементацията ѝ.

Архитектура. PPO е актьор-критик: **мрежа на стратегията** (актьор) извежда вероятности за действия, а **мрежа за стойности** (оценител) оценява стойностите на състоянията. Мрежата за стойности позволява прилагането на обобщено изчисляване на предимството (GAE), което осигурява нискодисперсни оценки

на предимството, използвани при **градиента на стратегията**. И двете мрежи се обучават едновременно.

Защо PPO е важен в сравнение с DQN: PPO естествено работи както с дискретни, така и с непрекъснати пространства на действията. Той може да усвоява **стохастични стратегии**, което е от решаващо значение в игрови среди, където прилагането на **смесени стратегии** е необходимо. Тъй като е **политика на работа** (използва пресни данни от **текущата стратегия**), той избягва проблемите със застояли данни. А изрязаната цел го прави забележително устойчив — PPO е предпочитаният избор за повечето RL приложения днес, включително RLHF за езикови модели.

Ограничения. Като **политика на работа**, PPO е по-малко **ефективен по отношение на пробите** в сравнение с **извънполитикови** методи като DQN (всеки опит се използва само за няколко **епохи на обновяване**, преди да бъде отхвърлен). Той изисква внимателна настройка на оценката на **предимството (GAE Lambda)**, **бонуса за ентропия** и архитектурата на **мрежата**. Освен това все още може да среща трудности при оскъдни награди — ако **агентът** рядко постига успех, наличният сигнал за учене е недостатъчен.

В нашата реализация от Стъпка 1 PPO успя да реши lunarLander-v3 (средна стойност за епизод от 202,2 при цел от 200) след приблизително 264 хиляди стъпки на средата. Решаващи фактори бяха капацитетът на мрежата (удвоен от [64,64] на [128,128]) и нормализацията на предимството.

Бележка относно невронните мрежи в тази стъпка. Както и при DQN, мрежите за **стратегия** и **стойност** тук са стандартни MLPs, чиито вътрешни механизми не бяха във фокуса на тази стъпка. Приехме прагматичен подход „ако работи“ към невронните компоненти и се концентрирахме върху разбирането на логиката на алгоритъма PPO (**отсичане**, GAE, структура на **разгръщане**). По-задълбоченото взаимодействие между дизайна на **невронната мрежа** и **сходимостта** в обучението с подкрепление — особено в контекста на **агенти за игри** — е отложено за Стъпка 5.

Прочетете повече: Schulman, J. и др. (2017). „Алгоритми за оптимизация на проксималната стратегия.“

arXiv: <https://arxiv.org/abs/1707.06347>

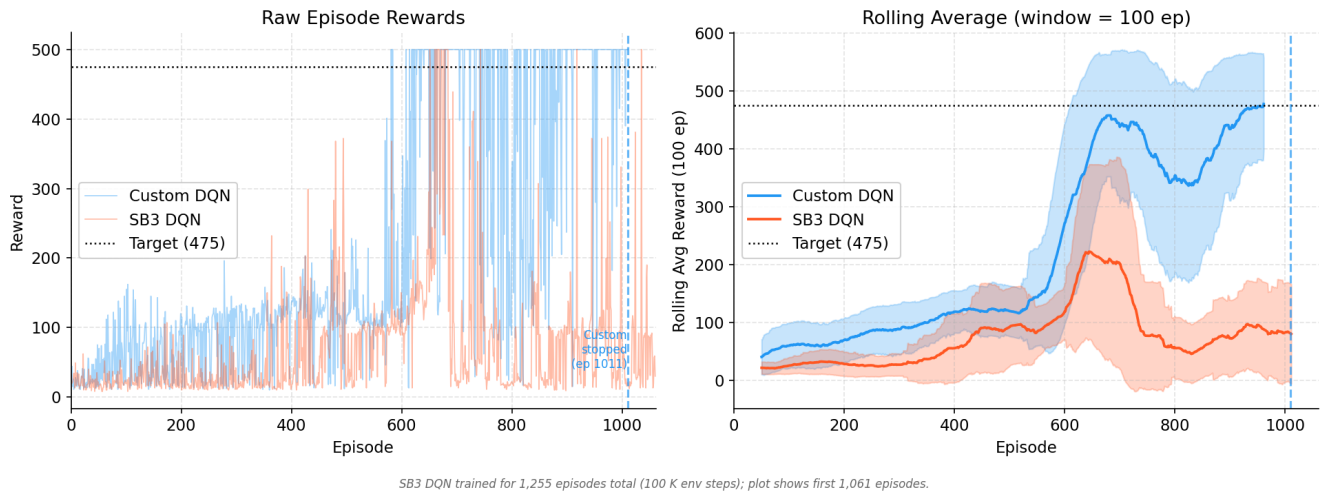
1.9 Практическо валидиране — собствени имплементации спрямо stable-Baselines3

За да проверим дали нашите имплементации от нулата функционират коректно, ги сравнихме със stable-Baselines3 (SB3) — широко използвана и добре тествана RL библиотека. На нашите имплементации и на SB3 бяха зададени еднакви хиперпараметри и еквивалентни бюджети за обучение.

1.9.1 DQN върху cartPole-v1

Нашият собствен DQN решава CartPole около епизод ~1011, като достига 100-епизодна плъзгаща средна от 477,5 (над целта от 475). DQN на SB3, с предоставени 750K стъпки на средата (17 176 епизода), не успява да се сближи — неговата плъзгаща средна остава около 30 и никога не достига целта.

Разликата се дължи на три различия на ниво изпълнение:



Фигура 1: DQN comparison — Custom vs SB3

- **График на изследване.** Нашият епсилон намалява с всеки епизод ($\times 0,995$), като така концентрира изследването там, където то е най-необходимо. SB3 намалява епсилон линейно спрямо броя стъпки, което в среди с кратки епизоди като cartPole води до загуба на много епизоди в почти случайна мода.
- **Синхронизация на целевата мрежа.** Нашият код синхронизира на всеки 5 епизода — адаптивен план, който осигурява по-бърза обратна връзка в ранните етапи на обучението. SB3 използва фиксиран интервал от 1000 стъпки.
- **Ранно спиране.** Нашият агент запазва най-добрия модел и прекратява обучението при достигане на пикова ефективност, като по този начин избягва катастрофално забравяне. SB3 изпълнява целия предвиден бюджет и стратегията му започва да се колебае.

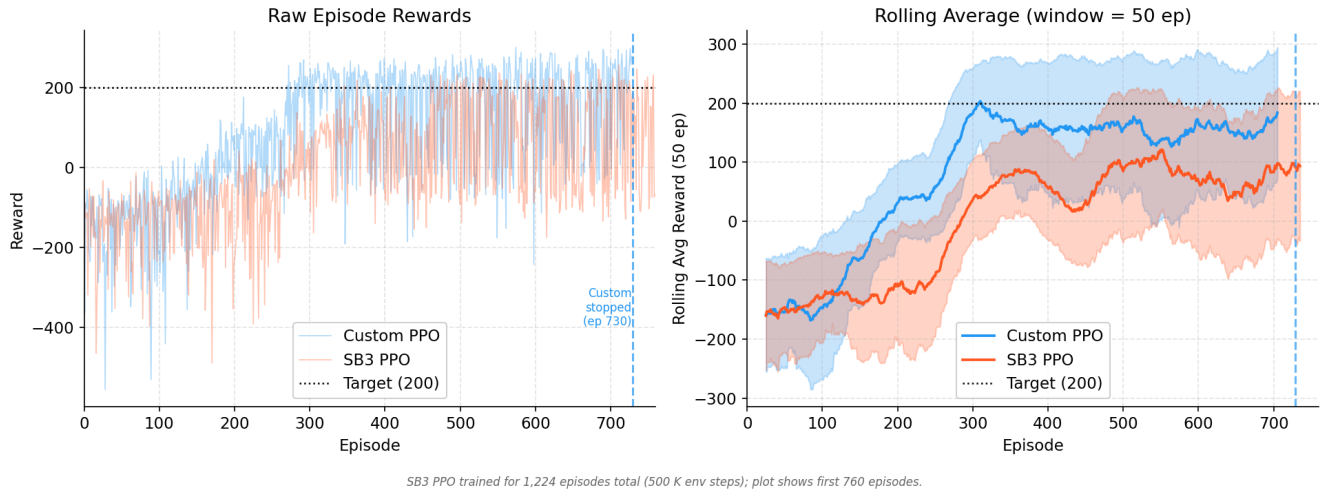
Това са истински алгоритмични решения, а не несправедливост при хиперпараметрите. Настройките по подразбиране на SB3 са проектирани за стабилна работа в разнообразни и дългосрочни области на обучение като Atari Games — а не в класически среди за управление с кратки епизоди като cartPole; нашите избори, съобразени със спецификата на задачата, просто дават по-добри резултати при cartPole.

1.9.2 PPO върху lunarLander-v3

Нашият персонализиран PPO преминава целта от 200 около епизод 543 (с бюджет от 264K стъпки). PPO на SB3 със същия бюджет от 500K стъпки достига връх с най-добра плъзгаща се средна стойност за 100 епизода от 131,2 — все още се покачва, но не е сходим.

Разликата тук е по-малка, отколкото при DQN. И двата използват идентични хиперпараметри на PPO. Оставщата разлика вероятно се дължи на ортогоналната инициализация на теглата и вградената нормализация на наблюдението в SB3, които подпомагат по-дългите серии от изпълнения, но могат да забавят ранната сходимост. При предоставяне на по-голям обучителен бюджет (1–2M стъпки), PPO на SB3 вероятно би достигнала целта.

PPO on LunarLander-v3 — Custom vs SB3



Фигура 2: PPO comparison — Custom vs SB3

1.9.3 Извод

Чиста реализация от нулата обхваща около 300–500 реда код, докато тази на SB3 достига приблизително 10 000. Компромисът е по-малко функции срещу пълна прозрачност и лесна възможност за отстраняване на грешки. За изследователски цели — особено при планираната работа върху експлоатацията на противника в стъпки 7–8 — персонализираните реализации дават възможност за прецизни модификации (например добавяне на наблюдения на състоянието на убежденията), които биха изисквали значително подклаждане в SB3. Именно тази практическа гъвкавост беше причината да изградим решение от нулата.

Стъпка 2 Обобщение — Теория на игрите и основи на CFR

Изследване върху възможностите за прилагане на изкуствен интелект в компютърните игри

Alexander Andreev

April 2026

Съдържание

2	Стъпка 2 — Основи на теорията на игрите и CFR	1
2.1	От single-agent RL към многоагентно стратегическо взаимодействие	1
2.2	Разширени игри и информационни множества	2
2.3	Теорема за минимакс	3
2.4	Равновесие на Наш	3
2.5	Напасване на съжалението — Градивният елемент	4
2.6	Минимизиране на контрафактичното съжаление (CFR)	4
2.7	Кун покер — Най-простото изпитвателно поле	6
2.8	Експлоатируемост — Мярка за отклонение от наш равновесие	6
2.9	Връзки със стъпка 1 и бъдещи насоки	7
2.10	Емпирични визуализации	8

2 Стъпка 2 — Основи на теорията на игрите и CFR

Това е съкратен преглед на материала по теория на игрите и минимизиране на контрафактичното съжаление, разгледан в Стъпка 2. Той има две предназначения: да служи като бърза справка при работа по следващите стъпки и като основен източник за синтеза на публичния доклад от Стъпка 15.

2.1 От single-agent RL към многоагентно стратегическо взаимодействие

Стъпка 1 разглеждаше средите като пасивни — количката с прът не оказва съпротива, а кацаният модул не се опитва да предизвика катастрофа. Агентът оптимизира спрямо фиксирана динамика. Стъпка 2 въвежда фундаментално различен проблем: средата включва друг *стратегически* вземащ решения участник, чиито действия зависят от вашите.

Тази промяна нарушава MDP рамката. В едноагентна среда оптималната стратегия е фиксирана — съществува един-единствен най-добър начин за действие, независимо от това каква стратегия сте обмисляли преди. В многоагентна среда обаче „най-добрата“ стратегия зависи от действията на противника, а неговата най-добра стратегия — от вашите действия. Тази взаимна зависимост е в основата на предизвикателството, което поставя теорията на игрите.

Формализацията започва с **нормални игри** (едновременни ходове, като камък-ножица-хартия) и се разширява до **разширени игри** (последователни ходове с дървовидна структура, като покер). И в двата случая концепцията на решение се измества от „оптимална стратегия“ към **равновесие на Наш** — стратегически профил, при който никой играч не може да подобри резултата си чрез едностранно променяне на собствената си стратегия.

Прочетете повече: Шохам, Й. и Лейтън-Браун, К. (2008). *Многоагентни системи*, глави 3–4.

Безплатно: <http://www.masfoundations.org/download.html>

2.2 Разширени игри и информационни множества

Игра в разгърната форма представя стратегическото взаимодействие под формата на дърво. Всеки възел е точка за вземане на решение от даден играч (или „случайност“ при случайни събития като раздаване на карти). Ръбовете обозначават действия, а крайните възли съдържат изплащанията за всеки играч.

Ключовото разграничение е между **съвършена** и **несъвършена** информация:

- **Пълна информация** (шах, Го): всеки играч има достъп до пълното състояние на играта. В дървото на играта няма групирани възли — всеки възел представлява отделно информационно множество. Тези игри се решават чрез минимакс и алфа-бета подрязване.
- **Непълна информация** (покер, много реални проблеми): играчите не могат да наблюдават всички аспекти на състоянието. В покера виждате само собствените си карти, но не и тези на опонента. Това означава, че множество състояния на играта са *неразличими* за действащия играч.

Информационно множество обединява всички състояния на играта, които играчът не може да различи едно от друго. В рамките на такова множество играчът трябва да приложи една и съща стратегия към всички състояния в него (тъй като те са неразличими за него). Именно това ограничение прави игрите с **непълна информация** фундаментално по-сложни от тези с **пълна информация** — вместо да се избира най-доброто действие за всяко отделно състояние, трябва да се подбере едно действие, което да работи добре *в средноаритметичен смисъл* за всички състояния в информационното множество.

В кун покер информационното множество "2рб" включва две състояния на играта: „Аз държа Дама, опонентът държи Вале, историята е пас–залог“ и „Аз държа Дама, опонентът държи Поп, историята е пас–залог“. Играчът с Дама не може да различи тези състояния и трябва да използва една и съща стратегия (вероятност за залог) и в двата случая.

Прочетете повече: Шохам и Лейтън-Браун (2008). *Многоагентни системи*, Глава 5 — Игри в разширена форма.

2.3 Теорема за минимакс

Теоремата за минимакс, установена от Джон фон Нойман през 1928 г., представлява фундаментална концепция на решение за двуигрови игри с нулева сума. Тя гласи, че за всяка крайна двуигрова игра с нулева сума съществува стратегия и за двамата играчи, при която максималната очаквана загуба е минимизирана. С други думи, всеки играч може да гарантира конкретна очаквана печалба, известна като „стойност на играта“, независимо от стратегията на противника. Това създава ситуация, в която оптималната стратегия на единия играч е да максимизира своята минимална награда (максимин), докато противникът се стреми да минимизира максималната награда на първия играч (минимакс). Когато тези две стойности са равни, играта достига равновесие. Този алгоритъм е пряко свързан с равновесието на Неш: в конкретния случай на двуигрови игри с нулева сума минимаксен стратегически профил е еквивалентен на равновесие на Неш. Откриването на това минимаксно решение гарантира защита срещу всяка експлоатативна стратегия, която противникът може да приложи.

2.4 Равновесие на Неш

Равновесие на Неш представлява стратегически профил, при който стратегията на всеки играч е най-добър отговор на стратегиите на всички останали играчи. Формално, за всеки играч i :

$$u_i(\sigma_i^*, \sigma_{-i}^*) \geq u_i(\sigma_i, \sigma_{-i}^*) \quad \forall \sigma_i$$

Нито един играч не може да подобри своята очаквана печалба чрез едностранно отклоняване. Това не означава, че всички са доволни от резултата (дилемата на затворника), а само че никой не може да постигне по-добър резултат, като промени единствено собствената си стратегия.

В игрите, разширени до повече от двама играчи (n -игрочни игри) или игри с обща сума, свойствата на равновесието на Неш стават значително по-сложни. В тези условия намирането на точно равновесие на Неш е известно като изчислително неразрешим проблем, често попадащ в класа на сложност $\text{P}^{\text{PAD-complete}}$. Освен това равновесията губят своята гаранция за безопасност: играенето на нашава стратегия в 3-игрочна игра не предпазва от двама опоненти, които могат да се отклоняват от равновесие, потенциално формирайки временни коалиции, които експлоатират третия играч. Тази динамика изисква изцяло различни обучителни рамки и методи за оценка (като тези, обсъдени в стъпки 9 и 11), тъй като пряката еквивалентност между равновесието на Неш и минимакс оптималността вече не важи.

Ключови свойства:

- **Съществуване:** Неш (1950) доказва, че всяка крайна игра има поне едно равновесие на Неш (възможно в смесени стратегии). Това е основополагаща теорема, но не помага при изчисленията.
- **Единственост:** Игрите могат да имат множество равновесия. Кун покер има еднопараметрично *семејство* от равновесия на Неш, индексирани от $\alpha \in [0, 1/3]$.
- **Смесени стратегии:** В много игри равновесието изисква рандомизация. В кун покер равновесието на Неш изисква Играч 2 да блъфира с поп точно $1/3$ от времето — всяка друга честота е експлоатируема.

За 2-игрaчни игри с нулева сума (където печалбата на единия играч е загуба за другия), равновесията на Наш имат специално свойство: те са **оптимални по минимакс**. Играенето на нашава стратегия гарантира поне стойността на играта, независимо от действията на опонента. Това е гаранцията за безопасност, която прави равновесието на Наш базова линия за изкуствен интелект за покер.

Прочетете повече: Наш, Дж. Ф. (1950). “Равновесни точки в игри с n играчи.” *PNAS*, 36(1), 48–49.

2.5 Напасване на съжалението — Градивният елемент

Преди CFR да може да минимизира съжалението в дърво на играта, е необходим механизъм за минимизиране на съжалението в отделна точка на вземане на решение. **Напасването на съжалението** изпълнява тази функция.

Идеята е следната: след множество изиграни рундове, за всяко **действие** a се изчислява **натрупаното съжаление** — с колко по-добър резултат бихте се справили, ако винаги бяхте избирали a вместо действително използваната **смесена стратегия**. След това следващата **стратегия** се определя пропорционално на *положителните* стойности на **съжалението**:

$$\sigma^{T+1}(a) = \begin{cases} \frac{R^{T,+}(a)}{\sum_{a'} R^{T,+}(a')} & \text{if } \sum > 0 \\ \frac{1}{|A|} & \text{otherwise} \end{cases}$$

Гаранция за сходимост: Блекуел (1956) и Харт & Мас-Колел (2000) доказват, че при използване на **напасване на съжалението** средното **съжаление** се сближава към нула със скорост $O(\sqrt{T})$. В **двуйгрови игра с нулева сума**, ако и двамата играчи прилагат **напасване на съжалението** в своите **решаващи точки**, **средният стратегически профил** се сближава до **равновесие на Наш**.

Прост пример е **камък-ножица-хартия**. Ако прилагате **напасване на съжалението** срещу постоянен противник, който винаги избира **камък**, вашето натрупано съжаление за **хартия** нараства (тъй като хартията побеждава камъка), докато съжалението за **ножица** остава нула. Постепенно стратегията ви се измества към игра с **хартия** с увеличаваща се вероятност, което представлява правилният най-добър отговор.

Прочетете повече: Нелър, Т. У. и Ланкото, М. (2013). „Въведение в минимизирането на контрафактичното съжаление“, Раздели 1–2.

2.6 Минимизиране на контрафактичното съжаление (CFR)

CFR (Zinkevich и др., 2007) разширява **напасването на съжалението** към игри в екстензивна форма. Ключовата идея е да се разложи общото **съжаление** на стратегията на играта на **локални съжаления** във всяко **информационно множество** и след това всяко локално **съжаление** да бъде минимизирано независимо чрез

напасване на съжалението. Ако **съжалението** за всяко **информационно множество** се сближи до нула, цялостната стратегия се сближава до **равновесие на Нaш**.

„**Контрафактичната**“ част е от решаващо значение. Във всяко информационно множество I съжалението за действие a не е просто „колко по-добро би било a “. То представлява *контрафактичното* съжаление — подобрението, което би настъпило, ако играчът умишлено бе избрал да достигне до I (вероятност за достигане = 1 за играча), като останалите условия (стратегия на противника и случайни събития) останат непроменени. Формално:

$$R^T(I, a) = \sum_{t=1}^T \pi_{-i}^t \cdot (v^t(I, a) - v^t(I))$$

където π_{-i}^t е вероятността за достигане на противника, а $v^t(I, a)$ представлява контрафактичната стойност на действието a в I при итерация t .

Защо вероятността за достигане на противника? Съжалението в дадено информационно множество има по-голяма тежест, когато съществува по-голяма вероятност противникът да е изиграл така, че да се достигне до него. Претеглянето с π_{-i} осигурява съжалението да бъде пропорционално на честотата, с която информационното множество е „от значение“ за крайния резултат от играта.

Алгоритъм (обикновен вариант на CFR с вероятностна извадка):

1. Инициализирайте всички кумулативни съжаления и суми на стратегиите с нула.
2. За T итерации: а. Направете вероятностна извадка на случайно раздаване на карти (вероятностна извадка). б. Рекурсивно обходите дървото на играта, започвайки от корена. в. Във всяко информационно множество изчислете стратегия чрез напасване на съжалението. г. За всяко действие рекурсивно влезте в поддървото (отрицавайки полезността за игра с нулева сума). д. Обновете кумулативните съжаления, претеглени с вероятността за достигане на противника. е. Натрупайте стратегия, претеглена със собствената вероятност за достигане на играча.
3. Изведете **средната** стратегия (а не стратегията от последната итерация).

Сходимост: Средният профил от стратегии се сближава до равновесие на Нaш със скорост $O(1/\sqrt{T})$ по отношение на експлоатируемостта, където T е броят на итерациите (Теорема 4, Zinkevich и др. 2007). Границата на сходимост е:

$$\text{exploit}(\bar{\sigma}^T) \leq O\left(\Delta\sqrt{|I|/T}\right)$$

където Δ е максималният диапазон на печалбите, а $|I|$ — броят на информационните множества.

Прочетете повече: Zinkevich, M. и др. (2007). „Минимизиране на съжалението в игри с непълна информация.“ *neurIPS*. Neller & Lanctot (2013). Раздели 3–5 (подробно разглеждане на **кун покер**).

2.7 Кун покер — Най-простото изпитвателно поле

Кун покер (Kuhn, 1950) е стандартният минимален пример за алгоритми в игри с непълна информация. С едва три карти, двама играчи и 12 информационни множества той е достатъчно малък, за да бъде решен аналитично, но същевременно достатъчно богат, за да демонстрира ключовите явления: блъфиране, информационна асиметрия и смесени равновесия.

Правила: Три карти $\{J, Q, K\}$; всеки играч внася начален залог от 1 жетон и получава по една тайна карта. Играч 0 действа пръв – пас или залог. Играч 1 отговаря – пас или залог. Ако Играч 0 е пасувал, а Играч 1 е направил залог, Играч 0 има право да отговори отново. Победител е играчът с по-висока карта при разкриването.

Равновесие на Наш (параметризирано с $\alpha \in [0, 1/3]$):

- **Играч 0 с J:** Залог (блъф) с вероятност α . Ако се изправи срещу залог след пас, винаги пас.
- **Играч 0 с Q:** Винаги пас. Ако се изправи срещу залог след пас, залог с вероятност $1/3 + \alpha$.
- **Играч 0 с K:** Залог с вероятност 3α . Ако се изправи срещу залог след пас, винаги залог.
- **Играч 1 с J:** След пас – залог (блъф) с вероятност $1/3$. След залог – винаги пас.
- **Играч 1 с Q:** След пас – винаги пас. След залог – залог с вероятност $1/3 + \alpha$.
- **Играч 1 с K:** Винаги залог, винаги залог.

Стойност на играта: При нашето равновесие очакваната печалба на Играч 0 е $-1/18 \approx -0.0556$. Играч 0 е в структурно неблагоприятна позиция, тъй като действа пръв.

Защо кун е важен: Всяка ключова концепция при решаването на игри с непълна информация се проявява тук в умален вид — блъфиране (J залага, въпреки че държи най-слабата карта), информационна асиметрия (Играч 2 вижда действието на Играч 1, преди да вземе своето решение), смесени стратегии (точно определена честота на блъф $1/3$), безразличие (Q е безразличен между залог и пас в определени информационни множества). Ако вашият алгоритъм не може да реши кун коректно, той няма да може да реши нищо друго.

Прочетете повече: Kuhn, H.W. (1950). “Simplified Two-Person Poker.” *Contributions to the Theory of Games*, Vol. 1.

2.8 Експлоатируемост — Мярка за отклонение от наш равновесие

Експлоатируемостта е основният оценителен показател за стратегии в игри с непълна информация. Тя измерва доколко една стратегия може да бъде експлоатирана от противник в най-лошия случай:

$$\text{exploit}(\sigma) = BR_0(\sigma_1) + BR_1(\sigma_0)$$

където $BR_i(\sigma_{-i})$ представлява очакваната печалба на играч i при прилагането на неговия най-добър отговор спрямо стратегията на противника.

В равновесие на Наш експлоатируемостта е точно нула — никой играч не може да подобри резултата си чрез отклонение от стратегията. За приблизителни равновесия по Наш експлоатируемостта измерва качеството на приближението.

Изчисляването на най-добър отговор в игри с непълна информация е деликатно. Най-добре реагиращият играч трябва да избере едно и също действие във всички състояния в рамките на едно информационно множество (тъй като той не може да ги различава). Наивен подход, който избира най-доброто действие за всяко състояние на играта (използвайки знанието за тайната карта на противника), изчислява „оракул“ стойност, а не истински най-добър отговор. За кун покер правилният подход е да се изброят всички $2^6 = 64$ чисти стратегии и да се оцени всяка една от тях.

Скорост на сходимост: При обикновен вариант на CFR експлоатируемостта намалява като $O(1/\sqrt{T})$. В двулогаритмична диаграма това се проявява като права линия с наклон ≈ -0.5 . Нашият измерен наклон беше -0.489 , което потвърждава теоретичните предвиждания.

Защо не просто стойност на играта? Една стратегия може да постигне правилната стойност на играта, но въпреки това да бъде експлоатируема. Представете си кун стратегия, при която Играч 1 винаги блъфира с J (вместо $1/3$ от времето). Средната стойност на играта може да е близо до $-1/18$, но стратегията е силно експлоатируема — Играч 0 винаги може да направи залог с Q и да реализира печалба. Експлоатируемостта улавя това; самата стойност на играта не го прави.

Тази метрика се появява многократно в дисертацията. Стъпки 7–8 (моделиране на противника, безопасна експлоатация) използват експлоатируемостта като основна мярка. Стъпка 14 изгражда обща рамка за оценяване около нея.

2.9 Връзки със стъпка 1 и бъдещи насоки

Локална оптимизация → глобална сходимост. В Стъпка 1 DQN минимизира tD error за всяко състояние поотделно, но цялостната q-функция се сближава до оптималната. В Стъпка 2 CFR минимизира съжалението за всяко информационно множество поотделно, но цялостната стратегия се сближава до равновесие на Неш. И двете демонстрират един и същ принцип: локалните актуализации, когато са правилно структурирани, постигат глобални цели.

Натрупване на стратегии $\approx$ повторение на натрупан опит. CFR извежда *средната* стратегия, а не стратегията от последната итерация. Това осредняване стабилизира сходимостта, по същия начин както повторението на натрупан опит в DQN стабилизира обучението, като предотвратява преобучението на агента върху последните преходи. И двете представляват механизми за памет, които изглаждат шумовите колебания.

Напред: Стъпка 2 разглежда CFR с *пълно обхождане на дърво* — всяко информационно множество се посещава при всяка итерация. При по-големи игри това е неразрешим проблем. В стъпка 3 се въвежда методът Монте Карло CFR (MCCFR), който взема проби от части на дървото, като заменя точността с мащабируемост. Стъпка 4 добавя абстракция на играта, намалявайки размера ѝ. Стъпка 5 заменя табличните стратегии с невронни мрежи.

Отворени въпроси: - Защо *средната* стратегия се сближава, а не текущата стратегия? (Интуиция: текущата стратегия превишава корекциите, докато осредняването затихва колебанията — същата причина като усредняването на Поляк в оптимизацията.) - CFR е доказан за игри с нулева сума между двама играчи. А какво да кажем за N -ма играчи или игри с обща сума? (Стъпки 9 и 11 изследват тази граница.) - Как се справяме с игри, които са твърде големи за пълно обхождане на дърво? (Стъпка 3 — метод Монте Карло CFR.)

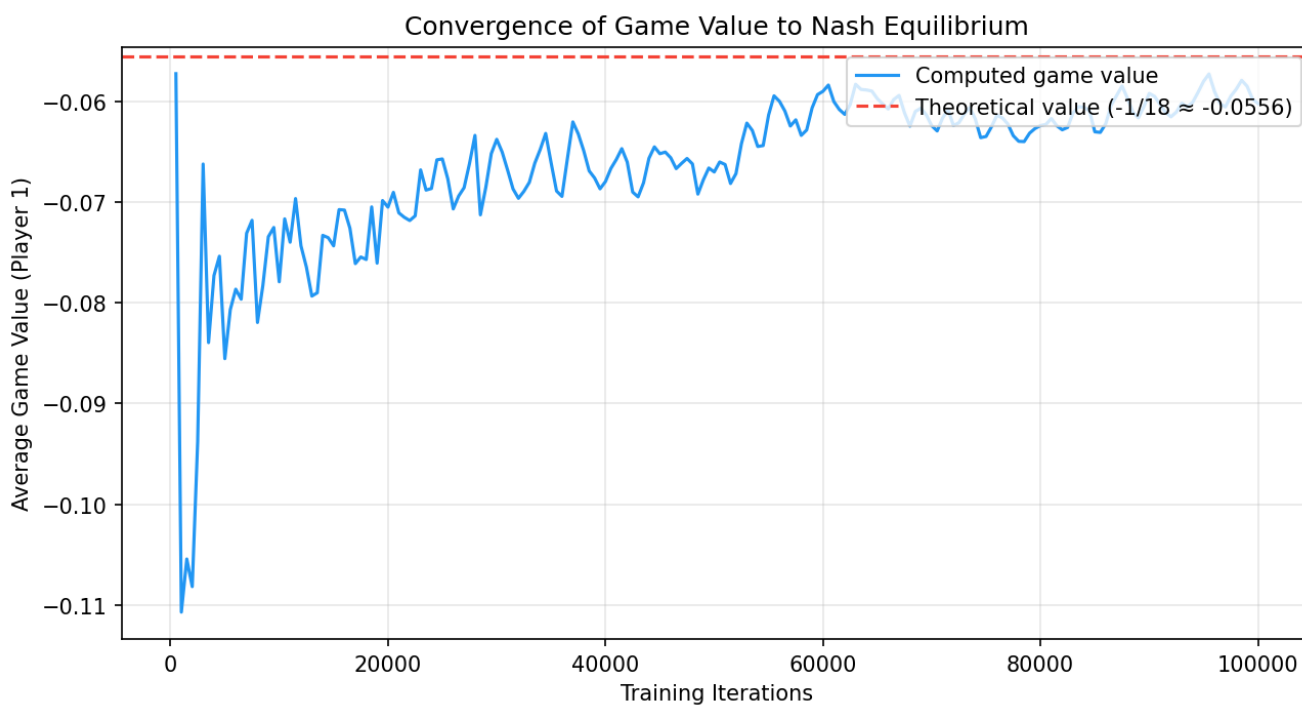
Прочетете повече: Bowling, M. и др. (2015). “Хедс-ъп лимит тексаски холдем покер е решен.” *Science*, 347(6218), 145–149.

В тази статия се използва CFR+ (вариант) за решаване на хедс-ъп лимит тексаски холдем — първата нетривиална игра с непълна информация, която по същество е решена.

2.10 Емпирични визуализации

За да се потвърдят **теоретичните гаранции** на CFR и приложението му към **кун покер**, в хода на обучението бяха наблюдавани няколко показателя.

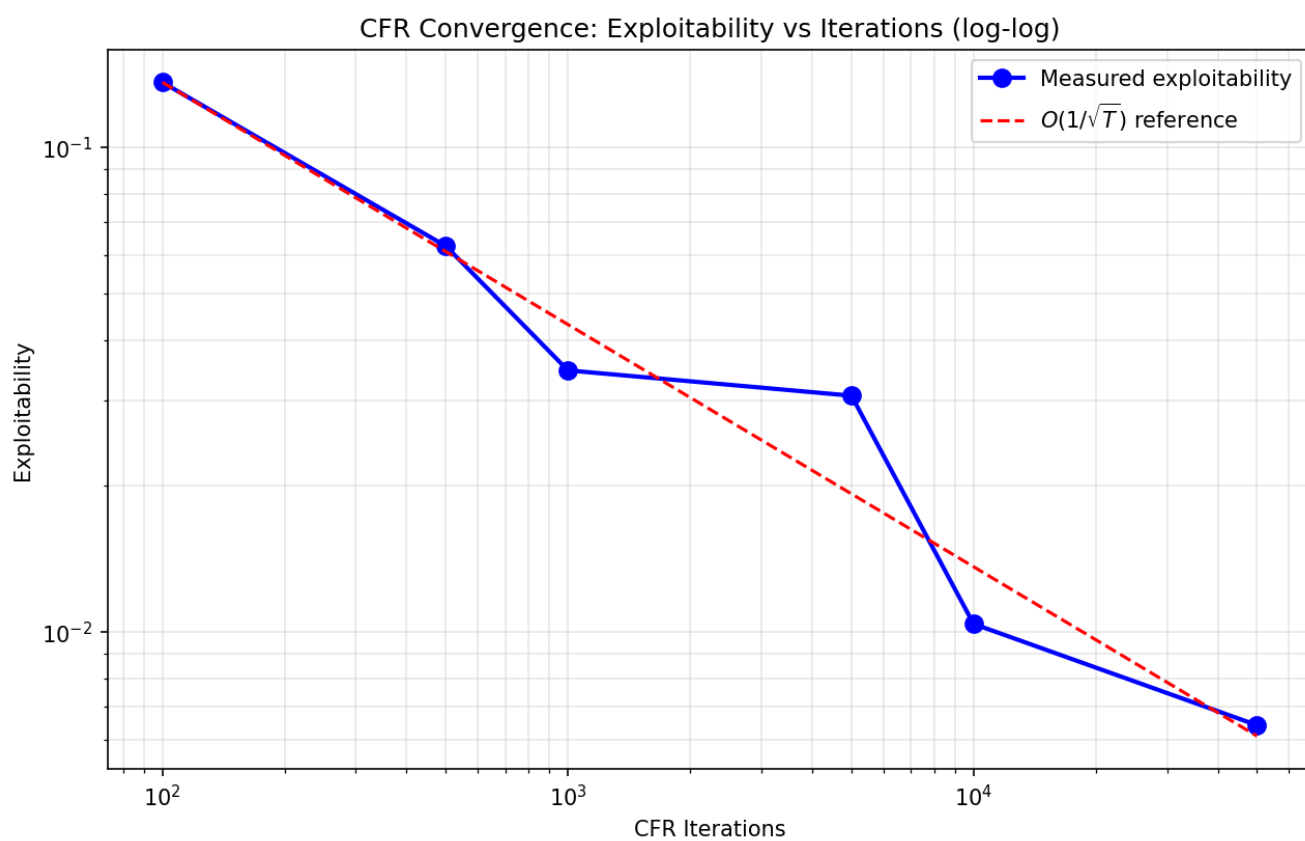
Сходимост на стойността на играта



Фигура 1: Game Value Convergence

Графиката показва как емпиричната средна стойност на играта се доближава до теоретичното очакване от $-1/18$ при нарастване броя на итерациите на CFR, което потвърждава стабилизирането на стратегията.

Сходимост на експлоатируемостта

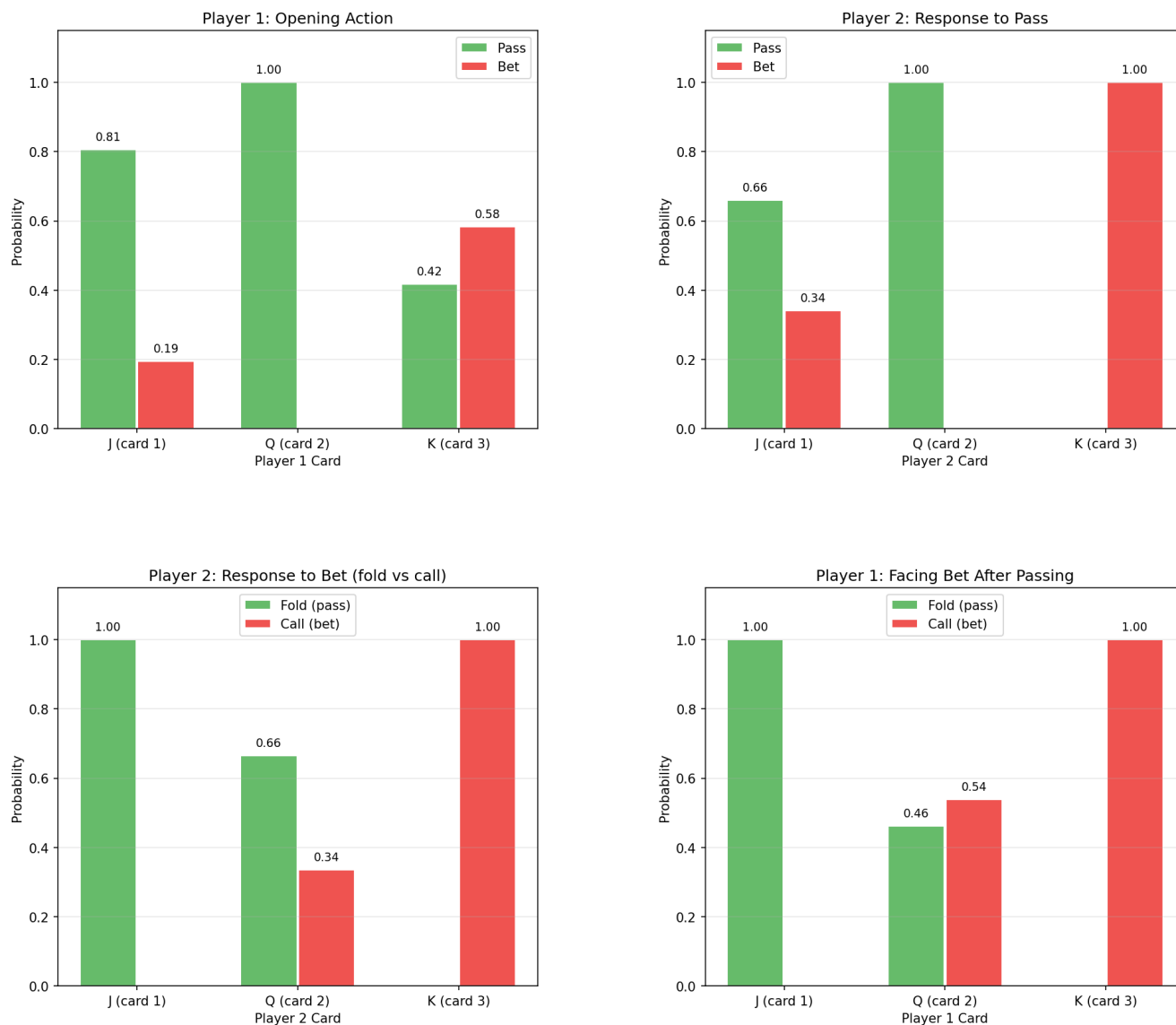


Фигура 2: Exploitability Convergence

Проследяването на експлоатируемостта във времето показва скорост на сходимост $O(1/\sqrt{T})$. Двубога- ритмичната диаграма демонстрира линеен спад, което потвърждава, че средната стратегия формира приблизително равновесие на Наш.

Анализ на стратегията

Kuhn Poker — CFR Strategy Analysis



Фигура 3: Strategy Analysis

Тази визуализация представя конкретните вероятности за **действие** в различни информационни множества, показвайки как се формират оптималните честоти на **блъфиране** и **колване**, определени от семейството на **равновесията на Наш** в кун покер.

Стъпка 3 Обобщение — Варианти на CFR и методи на Монте Карло

Изследване върху възможностите за прилагане на изкуствен интелект в компютърните игри

Alexander Andreev

April 2026

Съдържание

3	Стъпка 3 — Варианти на CFR и методи на Монте Карло	1
3.1	Защо обикновеният вариант на CFR се проваля при голям мащаб	1
3.2	Търсене с метод Монте Карло в дърво като вдъхновение	2
3.3	Маркови вериги и законът за големите числа	3
3.4	Математиката на покера	4
3.5	Ледюк покер — Разширяване на еталонния тест	5
3.6	CFR+ — Трикът с подовото ограничаване на съжалението	6
3.7	MCCFR — Извадка от дървото на играта	7
3.8	Компромисът скорост–вариация	9
3.9	Кога MCCFR надделява — точката на пресичане	11
3.10	Връзки със стъпка 2 и бъдещи насоки	13

3 Стъпка 3 — Варианти на CFR и методи на Монте Карло

Това е сбит преглед на разгледания в Стъпка 3 материал, свързан с мащабируемостта. Той има две предназначения: да служи като бързо опресняване при преминаването към следващите стъпки и като основен източник за синтеза на публичния доклад от Стъпка 15.

3.1 Защо обикновеният вариант на CFR се проваля при голям мащаб

Стъпка 2 установи работещ обикновен вариант на CFR решаващ за кун покер — 12 информационни множества, 58 крайни възли, решим за милисекунди. Този опит е подвеждащ. Обикновеният вариант на CFR изисква **пълно обхождане на дървото на играта при всяка итерация**, а истинските игри не са като кун покер. Ледюк покер има 936 информационни множества (~78 пъти повече), а тексас холдем с лимит — над 10^{14} . Нито един компютър не може да изброи това дърво дори веднъж.

Стъпка 3 преодолява проблема с мащаба чрез два взаимнодопълващи се механизма, разработени независимо в литературата по теория на игрите и по-късно комбинирани на практика:

1. **CFR+ (Tammelin, 2014)** — модификация на обикновения вариант на CFR, която постига значително по-бърза сходимост чрез подово ограничаване на съжалението, усредняване на линейни стратегии и

редуващи се актуализации. CFR+ е алгоритъмът, който „реши“ хедс-ъп лимит тексаски холдем през 2015 г. — първият нетривиален вариант на покер, който по същество беше решен.

2. **метод Монте Карло CFR (Lanctot и др., 2009)** — вместо да извършва пълно обхождане на дървото на играта при всяка итерация, MCCFR взема проба от малка част от него. Всяка итерация е с порядъци по-евтина, за сметка на по-шумни актуализации. Това е единственият осъществим подход за игри, при които пълното обхождане е физически невъзможно.

За дисертацията MCCFR е алгоритъмът, който изчислява базовата **нашева стратегия** — опорната точка „играй на сигурно“, от която адаптивният агент се отклонява въз основа на наблюденията върху опонента (Принос №1). CFR+ прави това изчисление осъществимо върху междинните **еталонни тестове**, използвани в стъпки 5–8.

Прочетете още: Bowling, M., Burch, N., Johanson, M. & Tammelin, O. (2015). “Хедс-ъп лимит холдем е решен.” *Science*, 347(6218), 145–149.

3.2 Търсене с метод Монте Карло в дърво като вдъхновение

Търсене с метод Монте Карло в дърво (MCTS) представлява каноничното семейство алгоритми, които използват **случайна извадка** за оценка на позиции в **дърво на играта**, без да прибегват до изчерпателно изброяване. Философията на проектирането му — „оценявай някои клонове до пълна дълбочина вместо всички клонове до фиксирана дълбочина“ — е пряка мотивация за MCCFR.

MCTS изгражда асиметрично **дърво на търсенето** чрез четири повтарящи се фази:

1. **Подбор** — слизване от корена чрез стратегия (обикновено UCB1, горна граница на доверителен интервал), която балансира експлоатацията на известни добри ходове и изследването на недостатъчно посетени такива.
2. **Разширяване** — във възел се добавят един или повече дъщерни възли към дървото.
3. **Симулация (разгръщане)** — от новия възел се играе случайна игра до нейния край.
4. **Обратно разпространение на грешката** — пропагиране на крайния резултат обратно по посещения път, като се актуализират броят посещения и кумулативните награди.

След множество **итерации**, детето на корена с най-голям брой посещения се избира за най-добър ход. Случайните разгръщания, агрегирани в хиляди **итерации**, **сходяват** към точни **стойностни оценки** — дори без никакви **експертни познания** извън правилата на играта.

Класически алтернативи като **минимакс с алфа-бета подрязване** оценяват всеки релевантен възел до фиксирана дълбочина и след това прилагат евристика в крайните възли. И двата подхода имат своите области на приложение:

Свойство	Минимакс + α - β	MCTS
Покритие на дървото	Всички разклонения до дълбочина d	Подбрани разклонения до терминал
Оценка	Евристика при границата на дълбочината	Точна (крайна печалба) или чрез невронна мрежа
Чувствителност към коефициента на разклоняване	Експоненциална: $O(b^d)$	Грациозна: съсредоточава се върху обещаващи разклонения
Необходими експертни познания	Силна оценъчна функция	Никакви (случайно разгръщане) или придобити чрез обучение
Най-подходящ за	Умерен коефициент на разклоняване, добри евристики	Висок коефициент на разклоняване, слаби/никакви евристики

За Го (коефициент на разклоняване ~ 250 , без добра оценъчна функция преди невронните мрежи) минимакс е непрактичен. MCTS беше пробивът, който направи изкуствения интелект за Го конкурентоспособен; същият принцип „извадка вместо изброяване“ се пренася към игра с непълна информация дървета чрез MCCFR.

Прочетете повече: Browne, C. и др. (2012). “A Survey of Monte Carlo Tree Search Methods.” *IEEE Transactions on Computational Intelligence and AI in Games*, 4(1), 1–43.

3.3 Маркови вериги и законът за големите числа

„Метод Монте Карло“ в MCCFR обозначава широко семейство изчислителни методи, които използват **повтаряща се случайна извадка** за получаване на числени резултати. Наименованието произхожда от проекта „Манхатън“, където Станислав Улам и Джон фон Нойман прилагат **случайна извадка** за симулиране на дифузията на неутрони — проблем, твърде сложен за аналитично решаване.

Математическата основа се крепи на **марковски вериги** — стохастични процеси, при които следващото състояние зависи единствено от текущото състояние и не от историята на достигането му (марковска свойственост). Обхождането на дърво на играта представлява марковска верига: от всяко **състояние на играта** преходът зависи само от текущото състояние и избраното действие.

Ключовата теоретична гаранция е **законът за големите числа**: ако се извлекат достатъчно траектории през

марковската верига, средната **стойност** на извадените стойности се сближава до истинската очаквана **стойност**. За MCCFR:

$$\frac{1}{T} \sum_{t=1}^T \hat{v}_I^{(t)} \xrightarrow{T \rightarrow \infty} v_I$$

където $\hat{v}_I^{(t)}$ представлява извадената контрафактична стойност в информационно множество I на итерация t , а v_I е истинската стойност, която обикновеният вариант на CFR изчислява точно. Тези извадени стойности са **непристрастни оценители** — тяхната очаквана стойност съвпада с истинската стойност — което гарантира сходимост към същото равновесие на Наш като при Full-Traversal CFR.

Цената е **дисперсия**. Отделните извадки могат значително да се отклоняват от истинската стойност, което налага повече итерации за постигане на същата прецизност. Този компромис между вариация и скорост е основната тема на тази стъпка.

Прочетете повече: Сътон, Р.С. и Барто, А.Г. (2018). *Обучение с подкрепление: Въведение*, Глава 5 — Методи на Монте Карло.

3.4 Математиката на покера

Покерът е каноничният тестов полигон за теорията на игрите с непълна информация, тъй като съчетава три източника на сложност, които рядко се срещат в комбинация:

1. **Скрита информация** — всеки играч вижда само своите карти. Едно и също наблюдавано състояние на играта (информационно множество) може да съответства на множество различни основни състояния на играта.
2. **Стохастични елементи** — раздаването на картите въвежда случайни възли в дървото на играта. Оптималната стратегия трябва да отчита всички възможни раздавания, а не само наблюдаваното.
3. **Стратегическо заблуждение** — за разлика от игрите с пълна информация, покерът възнаграждава смесените стратегии. Играч, който винаги залага със силни ръце и пасува със слаби, е тривиално експлоатируем. Равновесието на Наш изисква **рандомизирано блъфиране с математически точно определени честоти**.

Разширената форма на играта от Стъпка 2 се запазва: дърво на играта, информационни множества, стратегии (съответствия между информационни множества и разпределения на действия), равновесие на Наш. Ключовите величини за оценяване на стратегиите са:

- **Очаквана стойност (EV)** — средната печалба при дадена двойка стратегии.

- **Най-добър отговор** — оптималната контрастратегия срещу постоянен противник; най-силната възможна експлоатация.
- **Експлоатируемост** — средната печалба, която противник може да реализира, като играе перфектен най-добър отговор. Равновесието на Наш има нулева експлоатируемост.

Чен и Анкенман формализират тези концепции чрез решения на опростени игрови модели, при които честотите на блъфиране в наш равновесие могат да бъдат изведени аналитично. Техните полуулични и пълни модели изграждат интуиция защо изходните стратегии на CFR съдържат точните съотношения между блъф и залог — балансираният играч трябва да блъфира с честота, пропорционална на шансовете на пота, които предлага.

Прочетете повече: Chen, B. & Ankenman, J. (2006). *Математиката на покера*. ConjeLCo — аналитични извеждания на **наш равновесие** за опростени **покер** игри.

3.5 Ледюк покер — Разширяване на еталонния тест

Кун покер улавя същността на непълната информация в най-малкото възможно дърво на играта. Ледюк покер увеличава мащаба с приблизително два порядъка — все още решимо точно, но достатъчно голямо, за да станат значими разликите между алгоритмите:

Свойство	Кун покер	Ледюк покер
Карти	3 (J, Q, K)	6 ($\{J, Q, K\} \times \{\spadesuit, \heartsuit\}$)
Кръгове	1	2 (префлоп + обща карта)
Обща карта	Няма	1 разкрита между кръговете
Класиране на ръка	Само висока карта	Чифт (частна = обща) бие висока карта
Размери на залози	Фиксирани (1 чип)	Променливи (2 в първи кръг, 4 във втори кръг)
Максимални повишения/кръг	1	2
Информационни множества	12	936
Възли на игрово дърво	58	10,200
Случайни изходи	6	120

Три нови качествени характеристики се появяват:

- **Многорунтов строеж** — информацията се променя между кръговете (разкриване на обща карта), което изисква стратегии, адаптиращи се към новата информация.
- **Чифтове раздавания** — класирането на ръка вече зависи от общата карта; вале може да стане най-добрата ръка, ако вале бъде раздадено на борда.

- **По-големи залози в по-късните кръгове** — повишаването с 4 чипа във втори кръг означава, че решенията в по-късните кръгове имат по-голямо тегло.

Въпреки че е приблизително 78 пъти по-голям от **кун**, **ледюк** остава достатъчно малък за точно изчисление (пълно обхождане на дърво отнема около 50 ms). Това го прави идеален **еталонен тест**: достатъчно голям, за да бъдат значими разликите в производителността, и достатъчно малък, така че всички алгоритми да могат да достигнат почти пълна сходимост в рамките на минути.

Прочетете още: Southey, F. и др. (2005). „Блъфът на Бейс: моделиране на противника в покера.“ *UAI*.

3.6 CFR+ — Трикът с подовото ограничаване на съжалението

CFR+ въвежда три малки изменения в обикновения вариант на CFR, всяко от които е лесно за реализиране, но води до съществени резултати.

Подово ограничаване на съжалението. При обикновения вариант на CFR кумулативното **съжаление** може да става произволно отрицателно:

$$R^{T+1}(I, a) = R^T(I, a) + r^T(I, a)$$

CFR+ задава долна граница на съжаленията при нула след всяка итерация:

$$R^{T+1}(I, a) = \max(R^T(I, a) + r^T(I, a), 0)$$

Това предотвратява натрупването на големи отрицателни съжаления за дадено действие, които биха отнели множество итерации, за да се „изплатят“, преди действието отново да бъде разгледано. Аналогията с ReLU активациите в невронните мрежи е пряка: и двете ограничават отрицателните стойности до нула, като по този начин предотвратяват мъртви единици (или мъртви действия) да изискват дълъг период на възстановяване.

Усредняване на линейни стратегии. Обикновен вариант на CFR включва стратегията от всяка итерация с еднаква тежест в плъзгащата средна. При CFR+ всяка итерация t се претегля със самата нея, тоест t .

$$\bar{\sigma}^T(I, a) = \frac{\sum_{t=1}^T t \cdot \sigma^t(I, a)}{\sum_{t=1}^T t}$$

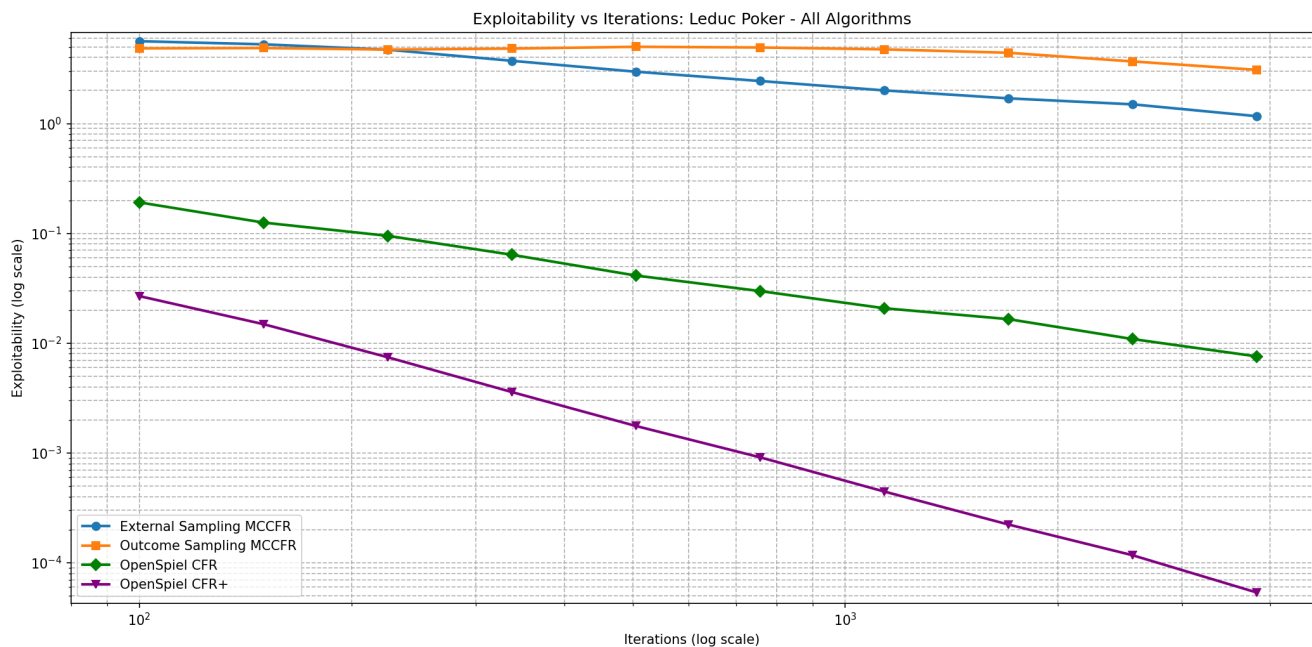
По-късните итерации — които се характеризират с по-малко съжаление и по-добри стратегии — оказват по-голямо влияние върху средната стойност. Ранните, шумни итерации получават по-ниска тежест.

Редуващи се актуализации. Вместо при всяка итерация да се актуализират съжаленията и на двамата играчи, CFR+ актуализира съжаленията на играч 0 при нечетни итерации, а на играч 1 — при четни. Това намалява

наполовина изчислителната работа за една итерация и носи малка полза по отношение на сходимостта, като предотвратява едновременните промени в стратегията.

Комбиниран ефект: емпиричната сходимост се подобрява от $O(1/\sqrt{T})$ до приблизително $O(1/T)$. С 1000 итерации CFR+ постига резултат, за който на обикновения вариант на CFR са необходими около 1 000 000 итерации. Именно това направи възможно решаването на хедс-ъп лимит тексаски холдем през 2015 г. Забележка: макар че скоростта $O(1/T)$ се наблюдава последователно, тя няма формално доказателство — Tammelin и др. представят алгоритъма, но не и теорема за сходимост, съответстваща на наблюдавана скорост.

Емпирично, при ледюк с 5000 итерации CFR+ достига експлоатируемост $\approx 5.4 \times 10^{-5}$, докато обикновеният вариант на CFR все още е около $\sim 7.6 \times 10^{-3}$ — подобрене приблизително 140 пъти, постигнато чрез три малки модификации.

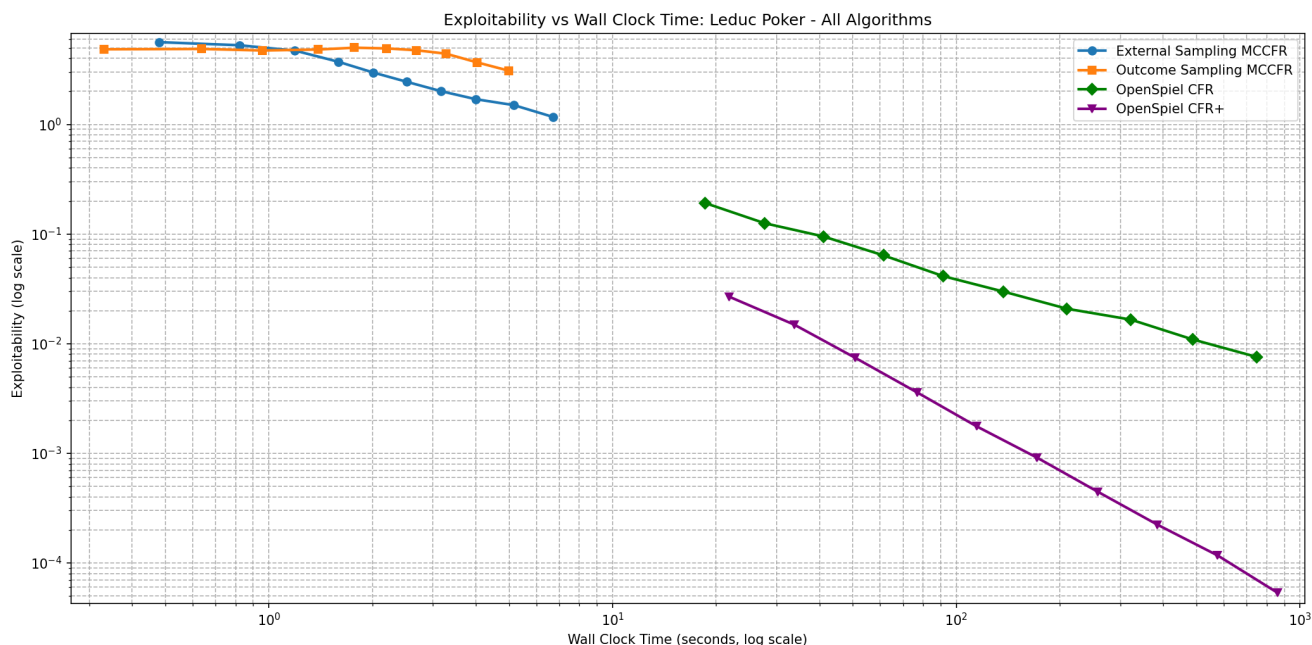


Фигура 1: Leduc Poker — Exploitability vs Iterations

Прочетете повече: Tammelin, O. (2014). “Решаване на големи игри с непълна информация чрез CFR+.” *AAAI*.

3.7 MCCFR — Извадка от дървото на играта

MCCFR заменя пълното обхождане на дърво с частично извадково обхождане. Два варианта се разполагат върху една и съща крива дисперсия–скорост в различни точки:



Фигура 2: Leduc Poker — Exploitability vs Wall-Clock Time

Вариант	Възли на шанса	Възли на traverser-a	Възли на опонента	Разход на итерация
Външно вземане на проби	Извадка от едно раздаване	Разглеждане на всички действия	Извадка от едно действие	~42 възела (~945× по-бързо от пълното)
Извадка по резултати	Извадка от едно раздаване	Извадка от едно действие (ϵ -on-policy)	Извадка от едно действие	~5.5 възела (~2,228× по-бързо)

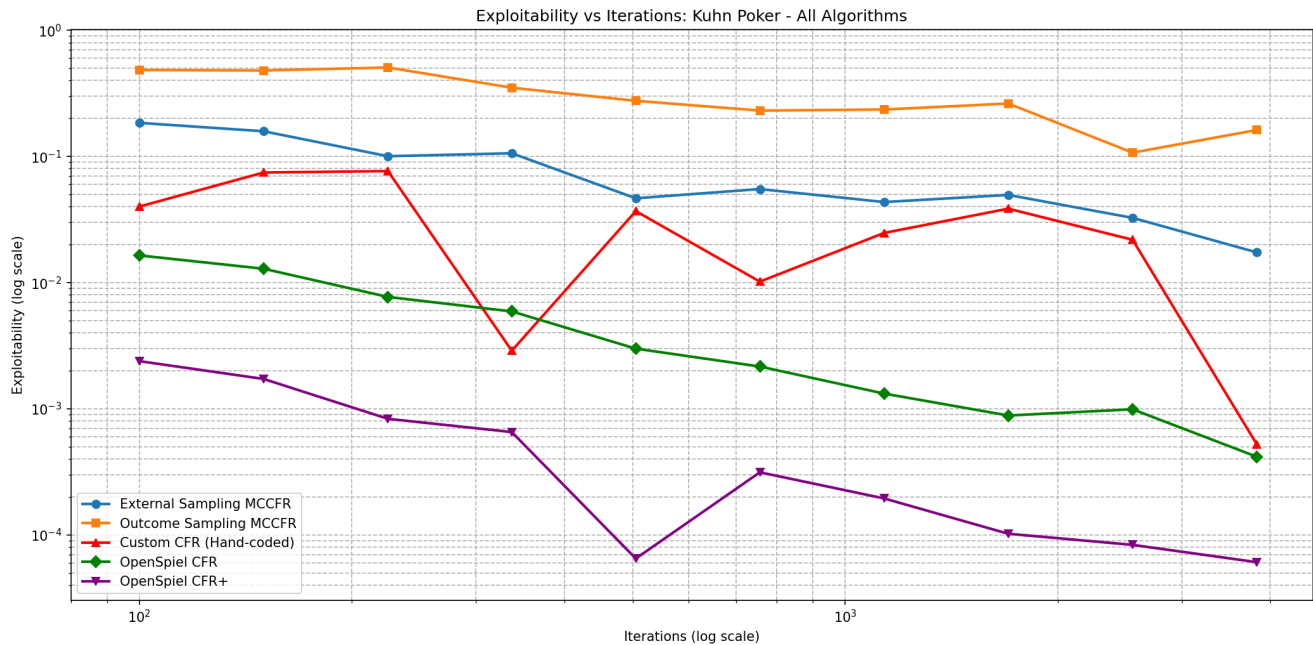
Външно вземане на проби разглежда всички възможни действия във възлите на играча, който актуализира стратегията си, но извършва извадка от възлите на шанса и възловите състояния на противника. Съжаленията за traverser-a се актуализират въз основа на извадковото поддърво. Тъй като се посещава само едно раздаване и един път на опонента, всяка итерация обхваща малка част от дървото.

Извадка по резултати избутва извадката още по-напред: тегли се една корен–краен траектория. Тъй като действията във възлите на играча, който актуализира стратегията си, също се изваждат, актуализацията на съжалението трябва да бъде коригирана чрез **извадка по важност** — отношението между истинската вероятност за достигане и вероятността за извадка. Една ϵ -on-policy смес (с вероятност ϵ се избира равномерно; в противен случай се следва текущата стратегия) осигурява разглеждане на всички действия дори когато текущата **политика на работа** им присвоява нулева вероятност.

Критичното теоретично свойство и за двата подхода е, че извадковите контрафактни стойности са **не-изместени оценки** на действителните стойности (Lanctot и др., Теорема 1). Това гарантира сходимост към същото равновесие на Наш като при обикновения вариант на CFR, въпреки шума от извадката. Цената, както

винаги при метода Монте Карло, е дисперсията. Всяка извадкова актуализация се отклонява от действителната контрафактична стойност, тъй като отразява само едно възможно раздаване, а не очакваната стойност за всички раздавания.

Върху 12-те информационни множества на **кун покер** и четирите алгоритъма (**обикновен вариант на CFR**, **CFR+**, двата варианта на **MCCFR**) достигат **почти равновесие на Наш** за секунди — **компромисът скорост–вариация** е невидим при този **мащаб** и изборът на алгоритъм е по-скоро теоретичен.



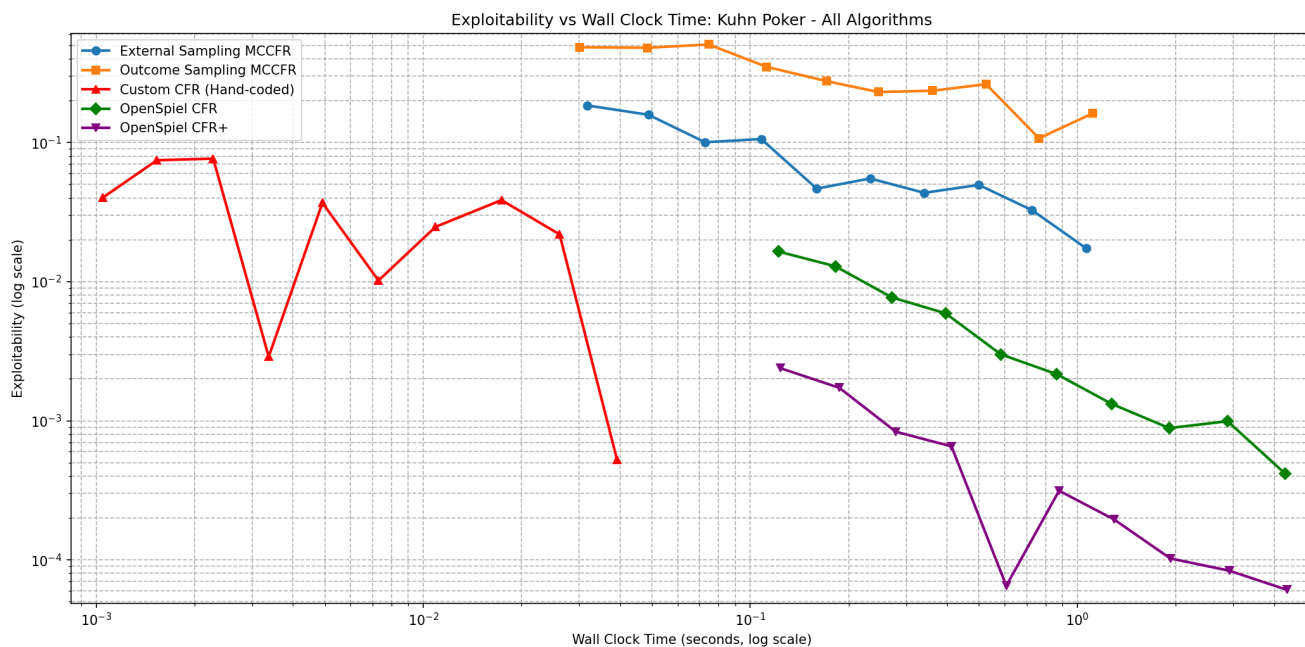
Фигура 3: Kuhn Poker — Exploitability vs Iterations

За да се прояви компромисът, дървото на играта трябва да бъде достатъчно голямо, така че разходът на итерация да има значение. Именно затова е необходим **ледюк**.

Прочетете повече: Lanctot, M., Waugh, K., Zinkevich, M. & Bowling, M. (2009). “Метод Монте Карло за минимизиране на съжалението в разширени игри.” *neurIPS*.

3.8 Компромисът скорост–вариация

Всички четири алгоритъма демонстрират $O(1/\sqrt{T})$ сходимост, но стойностите на константите им се различават съществено. При изразяване на експлоатируемостта като $\epsilon(T) = C/\sqrt{T}$ и преобразуването ѝ в реално време чрез $C_w = C/\sqrt{\text{speed}}$:



Фигура 4: Kuhn Poker — Exploitability vs Wall-Clock Time

Алгоритъм	C_{iter}	Скорост		Относително към обикновен вариант на CFR
		(итерации/сек)	C_w (реално време)	
Обикновен вариант на CFR	0.34	20.6	0.075	1.0×
CFR+	0.34	20.6	~0.002	~38 пъти по-бърз
MCCFR	107	19,470	0.767	10.2× по-бавен
External MCCFR	245	45,901	1.143	15.3× по-бавен
Outcome				

Константата в реално време определя представянето в практиката. Въпреки че външното вземане на проби е 945 пъти по-бързо на итерация, неговата константа на дисперсията (107 спрямо 0.34) е 315 пъти по-голяма. Предимството в скоростта **не компенсира наказанието от дисперсията**:

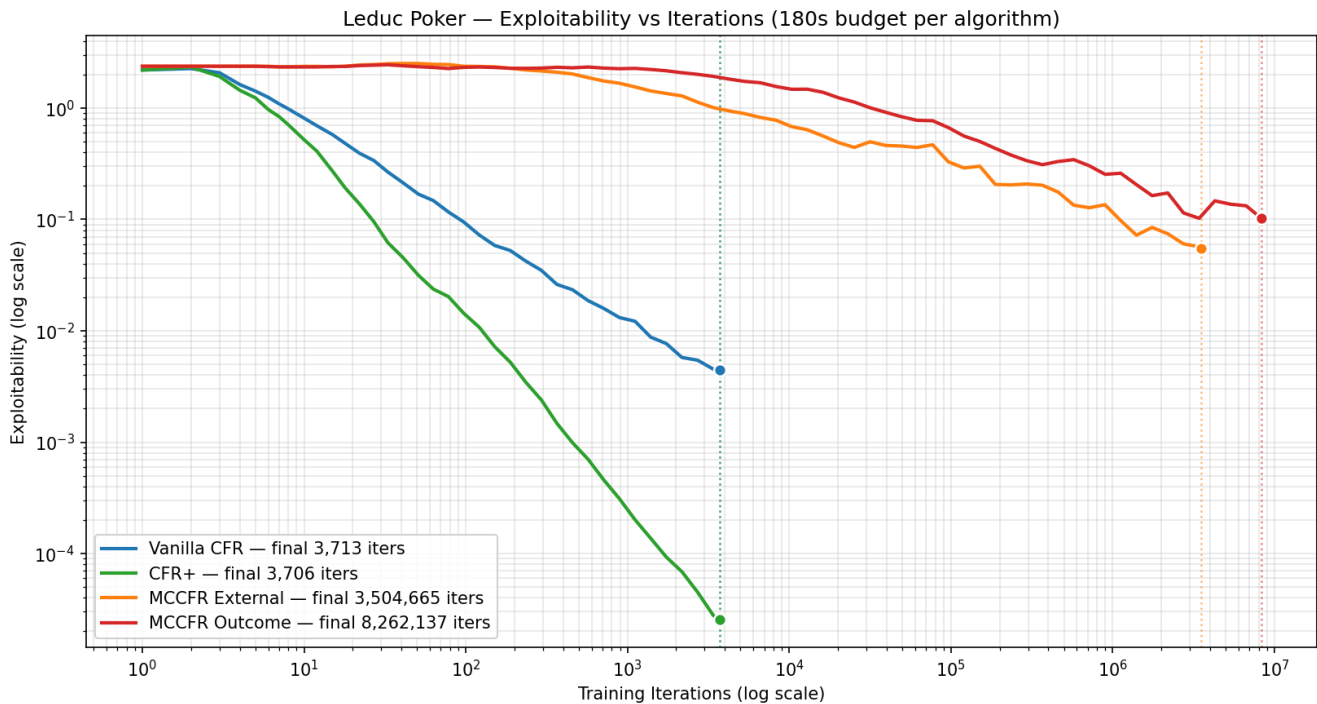
$$\frac{99,000 \times \text{more iterations needed}}{945 \times \text{faster per iteration}} \approx 105 \times \text{more wall-clock time}$$

Защо се наблюдава дисперсия? Обикновеният вариант на CFR изчислява **точни** контрафактни стойности чрез изброяване на всички 120 раздавания. Неговото обновяване на съжалението има нулева дисперсия. MCCFR взема проба от едно раздаване на итерация. Дори ако обходи цялото поддърво за това раздаване,

пак ще се наблюдава дисперсия между различните раздавания — следствие от самата скрита информация, а не недостатък на схемата на извадка:

$$\text{Var}[\hat{v}_I] = \mathbb{E}_{\text{deal}} [(\hat{v}_I - v_I)^2] \propto \frac{|N|}{|I|}$$

Формален 180-секунден еталонен тест с ограничение във времето за изпълнение на всичките четири варианта върху ледюк прави сравнението в реално време конкретно. Гледна точка, базирана на итерациите (вариантите с извадка завършват милиони обновления, докато вариантите с пълно обхождане — само няколко хиляди):



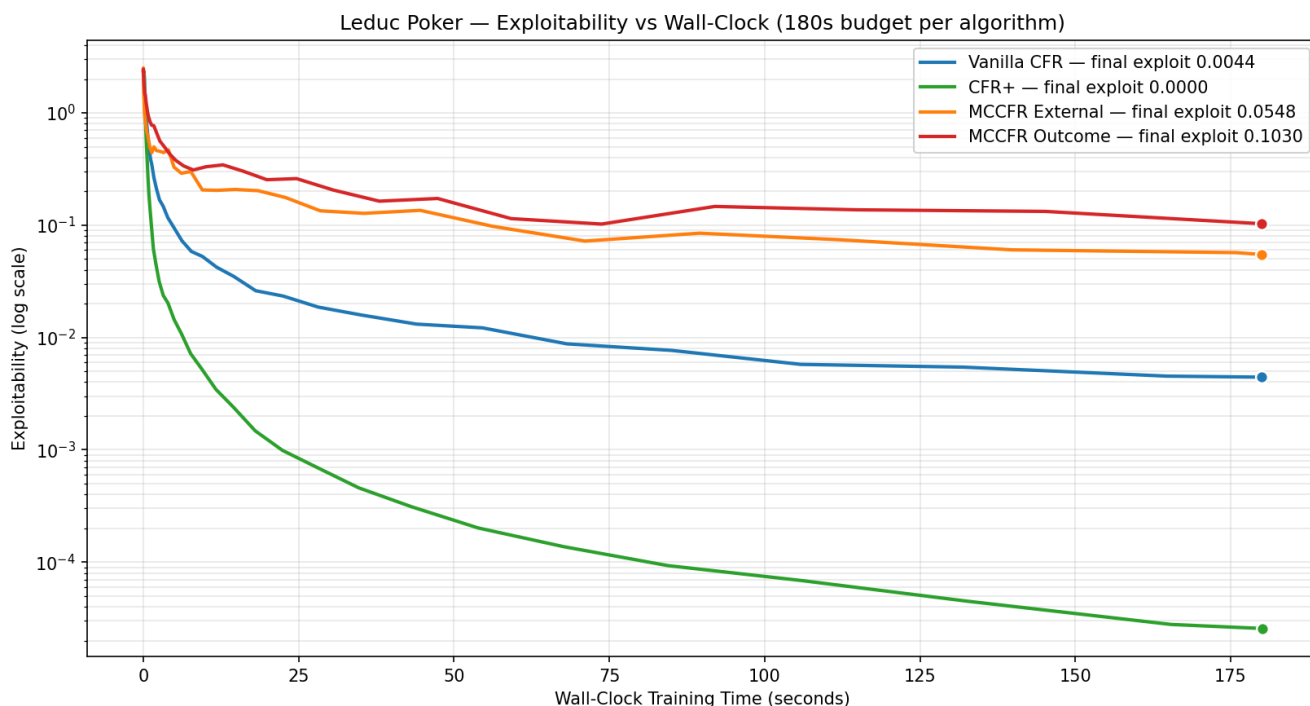
Фигура 5: Exploitability vs Iterations

Гледна точка в реално време — справедливото сравнение (при еднаква изчислителна мощност кой алгоритъм генерира най-добрата стратегия?):

CFR+ достига експлоатируемост от 2.6×10^{-5} — практически точно наш равновесие — за 3 минути. Обикновеният вариант на CFR следва с 4.4×10^{-3} . И двата варианта на MCCFR изостават с няколко порядъка при този размер на играта.

3.9 Кога MCCFR надделява — точката на пресичане

Горният модел поставя очевиден въпрос: ако MCCFR винаги е по-слаб в ледюк, защо тогава беше изобретен? Отговорът се крие във факта, че разходът на итерация при обикновения вариант на CFR нараства линейно с



Фигура 6: Exploitability vs Wall-Clock Time

размера на дървото $|N|$, докато този на MCCFR е приблизително постоянен. При достатъчно голям мащаб линейната цена надделява. Критичният размер на дървото е:

$$|N|_{\text{crossover}} = |N|_{\text{Leduc}} \cdot \left(\frac{C_{mc}}{C_v} \right)^2 \cdot \frac{\text{speed}_v}{\text{speed}_{mc}}$$

При заместване на измерените стойности:

Вариант	$ N _{\text{crossover}}$
Външно вземане на проби	~2.1 милиона възела
Извадка по резултати	~4.8 милиона възела

Сравнение с реални игри:

Игра	$ N $	Външни победи?	Победи по резултати?
Ледюк покер	10 200	Не (210 пъти твърде малко)	Не (466 пъти твърде малко)
Тексас холдем с лимит	$\sim 10^{14}$	Да (10^7 пъти над прага)	Да

Игра	$ N $	Външни победи?	Победи по резултати?
No-Limit Texas Hold'em	$\sim 10^{17}$	Да (10^{10} пъти над прага)	Да

Ледюк е 210 пъти под прага за външно вземане на проби и 466 пъти под прага за извадка по резултати. Пълното обхождане доминира, тъй като цялото дърво на играта се побира в паметта и може да бъде изброено за милисекунди. При реални мащаби на покера заключението се обръща напълно — дори една итерация на пълно обхождане на холдем би отнела повече време от възрастта на Вселената. В такъв мащаб MCCFR (с разширения за намаляване на дисперсията) е **единственият** приложим подход.

3.10 Връзки със стъпка 2 и бъдещи насоки

Същият принцип, различен режим. Стъпка 2 установи Full-Traversal CFR върху кун и доказа, че минимизирането на съжалението локално във всяко информационно множество води глобалната стратегия към равновесие на Наш. Стъпка 3 запазва този принцип, но въвежда две независими усъвършенствания: CFR+ променя начина на акумулиране на съжаленията (подохо ограничаване на съжалението + линейно претегляне), а MCCFR модифицира начина на измерването им (среднопретеглена извадка вместо точна очаквана стойност). И двете все още извеждат **средната** стратегия, а не последната итерация — същият механизъм за осредняване, който стабилизира обикновения вариант на CFR, стабилизира всеки негов вариант.

Скорости на сходимост. Стъпка 2 измери наклон в лог-лог мащаб, близък до -0.5 , за експлоатируемостта на играта кун, потвърждавайки $O(1/\sqrt{T})$. Стъпка 3 показва, че тази скорост е свойство на цялото семейство алгоритми, а не само на конкретна реализация — единствено емпиричният $O(1/T)$ при CFR+ е по-бърз, но без формално доказателство. MCCFR остава със скорост $O(1/\sqrt{T})$, но с драстично по-голяма константа.

Напред:

- **Стъпка 4 (Абстракция на играта и мащабиране)** — атакува проблема с мащабируемостта от противоположната посока: вместо да се прави извадка от дървото, самото дърво се редуцира чрез абстракция на състоянията и действията. Комбинира се с MCCFR, за да направи реалния покер управляем.
- **Стъпка 5 (Невронно равновесие / Deep CFR)** — заменя табличното съхранение на стратегията с невронни апроксиматори на функции, използвайки рамката за извадка на MCCFR като двигател за генериране на данни. Свойствата на дисперсията, установени тук, обясняват защо Deep CFR изисква специфични техники за намаляване на дисперсията (мрежи за предимство, важни целеви стойности с тегла).
- **Стъпка 6 (Игрови изкуствен интелект от край до край)** — надгражда както върху CFR+, така и върху MCCFR за създаване на пълноценни агенти. Машината Leduc, разработена за тази стъпка, се превръща в еталонна среда през Стъпка 8.

Отворени въпроси:

- Защо CFR+ емпирично постига $O(1/T)$, когато няма доказателство за такава скорост на сходимост? Най-добрата текуща хипотеза е, че подовото ограничаване на съжалението преоформя траекторията на динамиката на минимизиране на съжалението по начин, който класическите граници на сходимост не улавят.
- При какъв размер на дървото намаляването на дисперсията (среднопретеглена извадка, MCCFR с намаляване на дисперсията) става необходимо, а не просто опция? Формулата за пресичане ни показва къде печели MCCFR — но не и къде спира да бъде конкурентен без допълнителни техники.

Прочетете повече: Brown, N. & Sandholm, T. (2017). “Безопасно и вложено решаване на под-игри за игри с непълна информация.” *neurIPS* — свързва табличния CFR с декомпозицията на под-игра за реалния покер.

Стъпка 4 Обобщение — Абстракция на игри и мащабиране на игри с непълна информация

Изследване върху възможностите за прилагане на изкуствен интелект в компютърните игри

Alexander Andreev

May 2026

Съдържание

4 Стъпка 4 — Абстракция на играта и мащабиране на игри с непълна информация	1
4.1 От изброени игри към абстракция	1
4.2 Защо е необходима абстракцията	2
4.3 Две оси на абстракцията	4
4.4 Експлоатируемата разлика	5
4.5 Разстояние на Васерщайн: Кратко въведение	6
4.6 Критерият за сливане	7
4.7 Бюджетът на грешки	9
4.8 Конвейерът по време на компилация	10
4.9 Runtime Patching	12
4.10 Архитектура: План и актуализации по време на работа	13
4.11 Имплицитен маршрут: Еквиваленти на дълбокото обучение с подсилване	14
4.12 Практическо валидиране	14
4.13 Връзки и бъдещи перспективи	16

4 Стъпка 4 — Абстракция на играта и мащабиране на игри с непълна информация

Това е съкратен обзор на материала, разгледан в Стъпка 4, посветен на **абстракцията** на играта. Той изпълнява две функции: служи като бързо припомняне при преминаване към следващите стъпки и представлява основен източник за **синтеза** на публичния доклад от Стъпка 15.

4.1 От изброени игри към абстракция

Стъпки 1–3 изградиха алгоритмите (DQN/PPO; обикновен вариант на CFR; CFR+ и MCCFR външен/изходен). Всеки от тях се демонстрира върху игра с достатъчно малък размер, за да може да бъде изброена изцяло: **кун** (12 информационни множества) и **ледюк** (936). Този набор от инструменти е завършен, но той работи единствено когато цялото дърво на играта се побира в паметта.

Стъпка 4 е мостът между „игри-играчки, които могат да бъдат изброени“ и „игри, които не могат“. Механизмът е **абстракция**: умишлено опростяване на части от играта, така че същите алгоритми да могат да се приложат върху по-малък, структурно по-опростен модел, като се измерва цената на това опростяване в качеството на стратегията. До края на тази стъпка резултатът включва количествен отговор на централния въпрос на всеки практически **изкуствен интелект за покер** след 2007 г.:

*Доколко може да бъде свита играта, преди абстрактната **нашева стратегия** да престане да бъде добра **стратегия** в реалната игра?*

Този отговор се представя под формата на крива на Парето: по едната ос е размерът на **абстрахираната** игра, а по другата — **експлоатируема разлика** — с колко е по-слаба абстрактната **стратегия** спрямо точното **наш равновесие** в реалната игра. И двете метрики се въвеждат формално в раздела по-долу.

4.2 Защо е необходима абстракцията

Размерът на дървото на играта се определя като произведение от три фактора: (а) броят на различните скрити състояния, които възелът на случайността може да генерира, (б) коефициентът на разклоняване във всяка точка за вземане на решение и (с) дълбочината (решения до терминалното състояние). Всеки от тези три фактора нараства независимо:

- **Скрити състояния** — В тексас холдем се раздават 2 скрити карти от тесе от 52, след което до 5 общи карти. Броят на различимите ситуации „ръка срещу борд“ е от порядъка на 10^{17} , преди да бъде взета предвид историята на залозите.
- **Коефициент на разклоняване** — В покера без лимит всеки размер на залога може да варира от „минимално повишаване“ до „ол-ин“. Дори при дискретизация до няколко възможни размера, броят на разклоненията във всеки възел нараства от 3 (пас/зalog/повишаване при фиксиран лимит) до 5–10.
- **Дълбочина** — Множеството рундове на залагане, като всеки от тях може да включва няколко повишаващи, водят до експоненциално нарастване на дълбочината.

Комбинираният брой информационни множества в хедс-ъп безлимит покера е $\sim 10^{161}$ — повече от броя на атомите в наблюдаемата вселена с коефициент $\sim 10^{80}$. Нито един от алгоритмите от Стъпка 3 не може да бъде приложен върху това дърво.

Рецептата е една и съща за всеки подход: *създайте по-малка игра, чиито стратегии могат да бъдат преведени обратно в приложими стратегии за реалната игра, изпълнете алгоритмите от Стъпка 3 върху тази по-малка игра и след това ограничете щетите*. Тази рецепта има два маршрута.

4.2.1 Два подхода към абстракцията

Думата „абстракция“ в тази литература всъщност обозначава две операционно различни неща. И двете се използват в съвременния игрови изкуствен интелект; и двете ще се появят в тази дисертация; смесването им представлява категориална грешка. Тази секция очертава границата, а конвейерът за абстракции я следва стриктно: всяка фаза има *изрична* подсекция (табличният / алгоритмичен маршрут) и *неявна* подсекция

(съответствието с дълбоко обучение с подсилване в същото концептуално място).

Аспект	Изричен / табличен	Неявен / в стил IB (дълбоко обучение с подсилване)
Къде се осъществява компресията	Разпределение върху информационни множества / крайно избран набор от действия	Непрекъснат скрит вектор $z = f_{\theta}(s)$
Кой я извършва	Човешки проектирано правило или клъстеризация върху ръчно изведени признаци	Оптимизаторът, чрез градиентен спуск върху загуба
„Врътката“	k в k -средни кофи, заложна комбинация, правило за изоморфизъм на боя	β умножава $I(S; Z)$ в загубата
Гаранция	Ограничена експлоатируема разлика в същата игра (формална граница на експлоатируемостта)	Информационнотеоретична граница върху $I(S; Z)$; няма теорема за запазване на Неш равновесие
Кога се изчислява	Преди решаването — фиксиран вход към CFR/MCCFR	По време на решаването — мрежата <i>е</i> стратегията
Тип на изхода	Дискретен идентификатор на кофа за всяко информационно множество	Реален вектор
Къде се появява в тази дисертация	Тази стъпка (4)	Стъпка 5 (Deer CFR), стъпка 6 (от край до край), стъпки 11–12 (последователни модели)

И двата маршрута споделят една и съща **интуиция** — компресиране при запазване на стойността — а тази интуиция е най-добре изразена чрез лагранжиана на информационното тясно място, като β представлява обменният курс между паметта и стойността:

$$\mathcal{L} = \underbrace{\text{Complexity}(Z)}_{\text{memory cost}} - \beta \cdot \underbrace{\text{Value}(\pi_Z)}_{\text{strategic worth}}$$

Когато $\beta = 0$, алгоритъмът компресираща цялата игра до едно състояние и играе изключително зле. Когато $\beta \rightarrow \infty$, той отказва всякакво сливане и използва пълната игра. Всеки алгоритъм в конвейера за абстракции представлява конкретна реализация на тази „врътка“:

- **Беззагубно сливане** — граничен случай $\beta \rightarrow \infty$: извършва сливане само когато $\text{Value}(\pi_Z) = \text{Value}(\pi)$ е точно.
- **Ограничена загуба** — средата на кривата с гаранция: избира се β така, че да бъде изпълнено условието $|\text{Value}(\pi_Z) - \text{Value}(\pi)| \leq \varepsilon_{\text{abs}}$.

- **Емпирична сходност (HSD + разстояние на Земъов-Мъри)** — измерва самата крива: разстоянието на Земъов-Мъри между абстрактното и пълното разпределение служи като заместител на члена за загуба на стойност.
- **Корекция по време на изпълнение** — ортогонален подход: допуска се загуба на стойност при всяко избрано β , която след това се коригира по време на изпълнение чрез решаване на под-игра.

Реализациите в тази стъпка са изцяло изрични; имплицитните подсекции представляват само препратки към следващи раздели и не включват извършена тук работа.

През останалата част от това резюме стратегията, получена чрез решаване на абстрактната игра и използвана като отправна точка по време на изпълнение, ще бъде наричана **план**.

4.3 Две оси на абстракцията

Всяка конкретна техника за абстракция попада в една от двете ортогонални оси — *как агентът обединява състоянията* (информационна абстракция) и *кои действия разглежда* (абстракция на действията). Третата ос, *усъвършенстване във време на изпълнение*, е ортогонална и на двете и има собствен раздел (Корекция по време на изпълнение, по-долу).

4.3.1 Информационна абстракция

Обединете информационните множества, които агентът ще третира като едно състояние. Това е централната операция в конвейера — всяка фаза от критерия за сливане до конвейера по време на компилиране е свързана с *как и кога* да се извършват сливания на информация.

Бележка относно режимите. При изрична информационна абстракция в обикновен вариант на CFR съществува фино разграничение между абстрахирането (а) на **картата на възлите** (само памет) и (б) на **обхождането** (реално време за итерация). Само второто води до ускоряване в реално време.

Запомнете: Информационната абстракция определя кои скрити ситуации агентът възприема като еднакви.

4.3.2 Абстракция на действията и проблемът на транслацията

Ограничете множеството от действия, които решаващът разглежда, изпълнете CFR върху ограниченото игрално поле и след това обработете действията на реалния противник, които попадат извън това ограничение. Разграничават се два режима:

- **Дискретни множества действия.** Базовото множество вече е крайно. Абстракцията означава *подрязване* (премахване на доминирани или незаинтересовани действия преди решаването) или *макродействия* (групиране на последователности от примитиви в едно абстрактно действие — указател напред към временната абстракция в стъпки 1 и 11). Преводът обикновено е тривиален: ако абстрактното мно-

жество действия е подмножество на легалното, собствените ходове на агента винаги са в рамките на абстракцията.

- **Непрекъснати множества действия.** Базовото множество е безбройно (размери на залози в покер без лимит, съвместни въртящи моменти при непрекъснато управление). За табличните методи абстракцията на действията е *задължителна* — CFR не може да работи върху непрекъснатото дърво, без първо да го сведе до краен заместител (типична покерна мрежа: {fold, call, 0.5×pot, 1×pot, 2×pot, all-in}). Алтернативата на дълбокото обучение с подсилване е параметризиран актьор, който директно извежда непрекъснатото разпределение върху действията (PPO/SAC), елиминирайки проблема на превода за сметка на формалните гаранции.

Проблемът на транслацията е **остър** при изричната дискретизация: когато противникът направи залог от 0.7×пот, но абстракцията съдържа само 0.5×пот и 1×пот, агентът трябва да преведе този залог към възел, върху който е бил обучен. Три преводача се използват най-често:

1. **Най-близко действие** — закръгляне до най-близкия абстрактен залог по линейния мащаб. Най-лошо при загуба на стойност, когато залозите се групират около абстрактните размери.
2. **Вероятностно разпределение (линейно)** — разпределяне на тежест между двата най-близки абстрактни залога пропорционално на тяхното разстояние от реалния залог.
3. **Псевдохармонично отобразение** — интерполация в пространството *пот-фракционни коефициенти* вместо линейно пространство на сумата на залога, което съответства на това колко стратегически еквивалентни са двата залога. Най-съвременно ниво при превод на залози в покер.

Грешките при превода се натрупват в кръговете на залагане — противник, който открие закръглянето, може систематично да залага точно под или над точките на мрежата, за да предизвика грешно оразмерени отговори. Емпирично това е мястото, където практическите изкуствени интелекти за покер губят най-голяма стойност, и е оперативната причина за съществуването на корекция по време на изпълнение (по-долу): преизчислявам реалната под-игра с реалния залог, вместо да разчитам на преводача.

Запомнете: абстракцията на действията определя кои ходове може да обмисля агентът преди превод или разрешаване.

4.4 Експлоатируемата разлика

За стратегия σ , приложена в игра G , експлоатируемостта е стандартната метрика от трета степен:

$$\text{exploit}_G(\sigma) = \frac{1}{2}[v_G(\text{BR}(\sigma_1), \sigma_1) + v_G(\sigma_0, \text{BR}(\sigma_0))]$$

където σ_0 и σ_1 са стратегиите на двамата играчи, а $\text{BR}(\sigma)$ (най-добър отговор) е стратегията, която оптимално максимизира печалбата именно срещу стратегията σ .

Стъпка 4 въвежда производна метрика, **експлоатируема разлика**: с колко е по-зле една абстрактна стратегия спрямо точното наш равновесие на реалната игра, *измерено в самата реална игра*.

Нека G е реалната игра, а $\hat{G}$ – нейната абстракция. Нека $\hat{\sigma}^*$ е наш равновесието на $\hat{G}$, а $T(\hat{\sigma}^*)$ – неговият превод обратно в играема стратегия за G (идентичност, ако $\hat{G}$ представлява единствено информационна абстракция; нетривиален, ако включва и абстракция на действията — виж абстракция на действията). Тогава

$$\Delta_{\text{abs}}(\hat{G}) = \text{exploit}_G(T(\hat{\sigma}^*)) - \text{exploit}_G(\sigma_G^*)$$

с σ_G^* – точното наш равновесие на G . По дефиниция $\Delta_{\text{abs}} \geq 0$, като равенството е изпълнено тогава и само тогава, когато абстракцията е беззагубна (разгледано в критерия за сливане по-долу).

Това е централното количество в стъпка 4. Всяка парето граница на парето фронта има Δ_{abs} по една от осите. Всяко твърдение за „качество на абстракцията“ се проверява чрез неговото изчисляване.

Два допълващи се инструмента количествено оценяват Δ_{abs} преди пълното **решаване на абстрактната игра**:

- **Грешка на границата с тегло на обхвата** (обхваната в бюджета на грешки по-долу) — сумира грешките на полезност/вероятност за всяко сливане, претеглени с вероятността за достигане, за да се получи *горна граница* на Δ_{abs} единствено от структурните свойства на абстракцията.
- **Разстояние на Васерщайн между абстрактни и реални разпределения** (обхваната в бюджета на грешки по-долу) — *измерена заместителна величина* за Δ_{abs} , която не зависи от решаването на каквото и да било: просто изчислете разстоянието на Васерщайн между разпределенията на силата на ръката на слетите информационни множества и определете колко информация сливането изхвърля. Емпирично най-силният предиктор за експлоатируемостта след решаване.

Двете са допълващи се: границата е строга, но широка; проксито на разстоянието на Земъв-Музер е емпирично, но стегнато. Парето изгледът представя и двете заедно с директно измерената Δ_{abs} за всяка **конфигурация на абстракцията**.

Запомнете: експлоатируемата разлика е цената, която се плаща за решаването на по-малката игра вместо на реалната.

4.5 Разстояние на Васерщайн: Кратко въведение

Разстоянието на Земъв-Мъри се среща в почти всеки от следващите раздели и поради това заслужава кратко самостоятелно въведение.

Какво измерва. При дадени две вероятностни разпределения върху една и съща опора, разстоянието на Земъв-Мъри представлява минималното количество „работа“, необходимо за трансформиране на едното в другото, където *работа* = *преместена маса* × *преместено разстояние*. Може да си представим всяко

разпределение като купчини пясък върху числова ос — тогава разстоянието на Земъов-Мъри е най-малкото общо усилие за преоформяне на купчина А в купчина Б.

Произход и други наименования. Идеята води началото си от проблема за *оптималния транспорт*, формулиран от Монж през 1781 г. (преместване на купчини пръст, за да се запълнят дупки с минимално усилие), формализиран по-късно от Канторович през 1942 г. като линейна програма. Същото количество се среща в машинното обучение под няколко наименования — **разстояние на Васерщайн-1**, **разстояние на Канторович–Рубинщайн** или просто **разстояние на Васерщайн**. Рубнер, Томаси и Гуибас (2000) популяризират името „Разстояние на Земъов-Мъри“ в компютърното зрение за извличане на изображения. В по-ново време то е в основата на Wasserstein GANs и се използва широко при сходството на документи, адаптацията на домейни и всяка задача, при която сравняването на *формата* на разпределенията е от значение.

Защо превъзхожда сравненията, базирани на средна стойност. Разстоянието на Земъов-Мъри отчита *геометрията* на разпределението на масата, а не само средните стойности. Две разпределения, концентрирани близо едно до друго, са близки; две разпределения, концентрирани в противоположните краища, са далеч едно от друго, независимо къде съвпадат техните средни стойности. Сравненията на база очаквана стойност игнорират тази структура.

В контекста на тази стъпка. Всяко информационно множество има *разпределение на силата на ръката (HSD)* — хистограма на вероятностите за победа в края на играта, изчислени чрез разгръщане на невидими карти. Две HSD със същата средна стойност могат да имат напълно различни форми: заострен „стабилен среден ръка“ спрямо бимодален „бум или провал“ ръка. Групирането по очаквана сила на ръката ги обединява; разстоянието на Земъов-Мъри не го прави. Затова всеки качествен критерий, всеки заместител на грешка и всеки клъстеризационен конвейер по-долу използва разстоянието на Земъов-Мъри между HSD като основен сигнал за сходство.

Ефективно при едномерни хистограми. Когато поддържащото множество е едномерно и подредено (биново *вероятност за победа* $\square [0, 1]$), разстоянието на Земъов-Мъри се свежда до **разстоянието на Манхатън** между кумулативните разпределения — еднократно линейно **обхождане** на биновете. Затова HSD + разстояние на Земъов-Мъри е достатъчно бързо, за да клъстеризира милиони **информационни множества**.

Запомнете: разстоянието на Земъов-Мъри сравнява *формата* на разпределението, а не само средната стойност — затова две **раздавания** с еднакви вероятности за победа все още могат да бъдат стратегически различни.

4.6 Критерият за сливане

Допълнителна литература: <https://www.cs.cmu.edu/~gilpin/papers/extensive.JACM.pdf> · https://www.cs.cmu.edu/~sandholm/imperfect_recall_abstraction.arxiv14.pdf · <https://poker.cs.ualberta.ca/publications/AAMAS13-abstraction.pdf>

Кога могат две информационни множества да се обединят в едно? Представени са три вложени нива на

строгост, като най-слабото е първото.

4.6.1 Ниво 1 — Беззагубен

Сливане на две **информационни множества** се извършва единствено когато те са *стратегически еквивалентни*: еднаква вероятност за достигане, еднаква рекурсивна структура и еднакви последствия за **полезността** при всяко възможно продължение от страна на опонента. Последното условие е решаващо — ако някаква реакция на опонента може да ги различи, те не могат да бъдат сляти без загуба.

Когато това условие е изпълнено, сливането е безплатно: всяка оптимална стратегия в абстрактната игра се пренася в оптимална стратегия в оригиналната игра с **нула** разход за експлоатируемост.

Интуиция: Червена дама и черна дама в игра, в която боите нямат значение (невъзможни са **флъшове**), могат да бъдат сляти без загуба. В игра с **флъшове** обаче червената дама не може — боята мълчаливо влияе върху шансовете на опонента за **флъш** чрез премахване на карти от тестето.

Запомнете: беззагубната абстракция е безплатна компресия: същото стратегическо значение, по-малко съхранени информационни множества.

4.6.2 Ниво 2 — Ограничена загуба

Отпуснете „идентични полезности“ до „полезностите се различават най-много с ε “; същата идея важи и за вероятностите на случайността. Цената е количествена грешка — всяко сливане допринася към бюджет на отклонение, който се разпространява в граница върху общата експлоатируемост (използвана в следващия раздел). Последователностите от действия по слетите пътища все пак трябва да съвпадат, така че това е *контролирано забравяне*, а не произволно.

Интуиция: Поставянето на J и Q в една „ниска карта“ кофа принуждава агента да ги играе по един и същи начин, въпреки че оптимално те се различават леко. Системата поема тази грешка в замяна на значително по-малка **игра**.

Запомнете: ограничено загубно абстрахиране е контролирано забравяне: играта се свива, но всяко сливане получава цена за грешка.

4.6.3 Ниво 3 — Емпирична сходност

Когато аналитичната граница е твърде песимистична или входовете са твърде високи по размерност, за да бъдат изброени (тексас холдем има $\sim 10^9$ канонични река борда), премахнете границата при сливане и я заменете с *научена функция за разстояние*:

- Изчислете вектор на характеристики за всяко информационно множество — обикновено **разпределение на силата на ръката (HSD)**: хистограма на вероятностите за победа в края на играта след разгръщане на невидими карти.
- Използвайте **разстояние на Васерщайн** между векторите на характеристики като метрика за сходство.
- Групирайте информационните множества с близки разпределения на признаци и извършете сливане в рамките на групите.

Няма формална гаранция за експлоатируемост тук — качеството се проверява *след* решаването чрез измерване на експлоатируемостта на получената стратегия.

Запомнете: Емпиричната абстракция е предположение с ограничен бюджет, което трябва да бъде измерено след решаването.

4.6.4 Избор измежду трите

Редът на вземане на решения е следният: първо се използват всички налични беззагубни сливания; след това се приемат ограничени загубни обединявания само когато бюджетът на грешки е поносим; а когато точните проверки са твърде песимистични или твърде скъпи, се извършва групиране с HSD + разстоянието на Васерщайн и получената абстракция се проверява емпирично.

4.7 Бюджетът на грешки

Допълнително четене: https://www.cs.cmu.edu/~sandholm/imperfect_recall_abstraction.arxiv14.pdf · <https://poker.cs.ualberta.ca/publications/AAMAS13-abstraction.pdf>

Три инструмента за количествено определяне, подредени по строгост — от най-строгия към най-малко формалния.

4.7.1 Инструмент 1 — Аналитична граница

Дадена е абстракция със загуби с константи на отпуснатата връзка за всяко обединяване; тяхната сума, претеглена според честотата на достигане на всеки набор от информация по време на игра, дава горна граница на експлоатируемата разлика.

Ключовата идея е теглото на достижимостта. Рядко достиганите информационни множества могат да бъдат агресивно обединявани, без това да влоши общата експлоатируемост; небрежните обединения в гъсти и често посещавани региони са катастрофални. Това е принципът, който всяка практическа абстракция в покера след 2010 година използва.

Интуиция: Грешката при **сливането** на **кофа** се претегля според честотата, с която играчите действително достигат тази ситуация. Небрежно **сливане** в рядка **крайна игра** почти не влияе върху общия резултат, което позволява агресивна **абстракция** без загуби **в очакване**.

Запомнете: аналитичната граница е строга, но обикновено хлабава; най-важната ѝ идея е теглото на достижимостта.

4.7.2 Инструмент 2 — прокси на разстоянието на Земъв-Музер

Горна граница, която се *изчислява* и не изисква изброяване на листата: просто **изчислете разстоянието на Земъв-Мъри** между **хистограмите на силата на ръката** на две **информационни множества**. **Разстоянието на Земъв-Мъри** е *прокси*, а не граница — то корелира добре с **експлоатируемостта** след решаване, но

не носи формална гаранция. (Вижте въведението за **разстоянието на Земьов-Мъри** по-горе за основните механизми.)

Запомнете: разстоянието на Земьов-Мъри отчита формата на разпределение, докато очакваната сила на ръката отчита единствено средната стойност.

4.7.3 Инструмент 3 — директен оценител на CFR-BR

Най-силната мярка: тя връща най-близкото представимо наш равновесие, което може да бъде изразено чрез абстракцията, като по този начин *изолира* грешката на абстракцията от грешката при решаване.

В експериментите стратегиите на CFR-BR показват експлоатируемост, достигаща до 1/3 от съответните обикновени CFR стратегии върху *същата* абстракция. Изводът е, че значителна част от измерената „грешка на абстракцията“ в литературата всъщност представлява *скрита грешка при решаване* — самата абстракция е позволявала по-добри резултати, но използваните решавачи не са достигнали сходимост.

Запомнете: CFR-BR пита какво може да изрази абстракцията, а не доколко добре се е обучил даден решавач.

4.7.4 Емпирични резултати, които си струва да се запомнят

Две надеждни сравнения при еднакъв бюджет за **информационни набори**:

- **Разпределение-съзнателен спрямо очакваностойностен.** Комбинацията HSD + разстояние на Земьов-Мъри строго доминира клъстеризацията, базирана на очакван процент победи, както по отношение на експлоатируемостта, така и по отношение на процента победи в директен сблъсък срещу фиксирани опоненти.
- **Непълна спрямо пълна памет.** Абстракциите с непълна памет превъзхождат тези с пълна памет при еднакъв бюджет на множеството от информационни състояния — свободата за преразпределение на кофите е емпирично по-ценна от изгубената непрекъснатост.

И двете са причината конвейерът по време на компилация да задава по подразбиране **непълна памет, HSD и разстоянието на Земьов-Мъри**.

4.8 Конвейерът по време на компилация

Допълнителна литература: <https://www.cs.cmu.edu/~gilpin/papers/extensive.JACM.pdf> · <https://poker.cs.ualberta.ca/publications/AAMAS13-abstraction.pdf> · https://www.cs.cmu.edu/~sandholm/imperfect_recall_abstraction.arxiv14.pdf

Два допълващи се конвейера, по един на ниво на строгост на критерия.

4.8.1 Конвейер 1 — GameShrink (беззагубен)

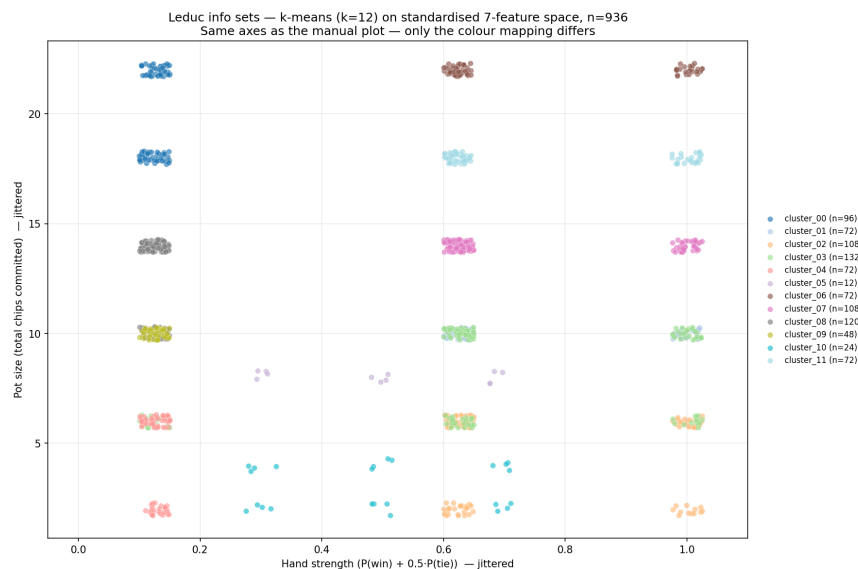
Изчерпателно сливане, което обхожда *сигналното дърво* — структура, по-малка от дървото на играта, изброяваща само последователности от публични и частни сигнали — **отдолу-нагоре**. На всяко ниво се проверяват двойки поддървета брата за стратегическа еквивалентност и се сливат всички двойки, които я удовлетворяват.

Защо сигналното дърво е по-малко: дървото на играта умножава всеки различен **сигнал от скрито състояние** по всяка различна **последователност от залози**, която може да доведе до него, докато сигналното дърво колабира всички тези пътища на залагане в един **сигнален възел**. При Rhode Island Hold'em сигналното дърво е около 6,6 млн. възела спрямо около 3,1 млрд. възела на дървото на играта — приблизително 500-кратна компресия преди прилагането на каквато и да е **загубена стъпка**. GameShrink е *нълен* по отношение на своя **критерий за сливане**: всяко **беззагубно сливане**, изразимо чрез критерия, се открива.

Този конвейер в комбинация с линейно програмиране представляваше двигателят, който реши Rhode Island Hold'em през 2007 г., четири порядъка надхвърляйки всяка покер игра, решавана преди това.

Запомнете: GameShrink търси в сигналното дърво всяко свободно сливане, преди да бъде разгледана каквато и да е компресия със загуби.

4.8.2 Конвейер 2 — HSD + разстояние на Земьов-Мъри + k-средни + непълна памет (загубен)



Фигура 1: Infoset grouping via K-means

Когато беззагубното сливане достигне сходимост и играта все още е твърде голяма, се преминава към клъстеризация:

Конвейерът изчислява HSD за всяко информационно множество, групира получените разпределения с мощта на разстоянието на Земьов-Мъри като метрика на разстояние и съпоставя всяко информационно множество към идентификатор на кофа. Предлагат се два режима на извличане:

- **Пълна памет** — идентификаторът на кофата включва предходната траектория на кофите.
- **Непълна памет** — кофата в по-късен кръг може да игнорира по-раншната траектория, като по този начин освобождава повече кофи за кръга, в който новата информация е от най-голямо значение.

Непълната памет последователно надделява при фиксиран бюджет на кофите — капацитетът се използва за кръгове, които имат по-голямо значение, вместо да се изразходва за запамятаване на историята.

Запомнете: HSD + разстоянието на Земъв–Мъри определя *какво изглежда стратегически по-добно*; непълната памет определя *къде да се използва бюджетът на кофите*.

4.8.3 Бележка относно трудността

Изборът на разделяне, което минимизира аналитичната граница, е **nP-complete**, дори за една малка еднопотребителска игра с дълбочина два нива. Затова никой не минимизира границата точно — практическите конвейери я приближават ниво по ниво (един кръг наведнъж), вместо глобално. При разумни условия едно ниво се свежда до клъстеризация около центрове в метрично пространство, за която съществуват приблизителни алгоритми с полиномиално време с гаранции с постоянен коефициент.

Това е *причината*, поради която всяка практическа абстракция в покера след 2010 г. представлява клъстеризационен конвейер, работещ ниво по ниво, а не глобален оптимизатор.

4.9 Runtime Patching

Допълнителна литература: <https://arxiv.org/pdf/1705.02955v3>

Когато играта премине в под-игра и абстрактният план е твърде общ, преизчислявам под-играта с по-висока точност *в реално време*. Два пача са от значение — заедно те бяха основните елементи на Libratus, първият изкуствен интелект, победил топ играчи в хедс-ъп безлимит покер.

4.9.1 Защо решаването на под-игра не може да се извършва изолирано (хвърляне на монета)

Прост **контрапример**, наречен *хвърляне на монета*: една монета пада с Ези или Тура с равна вероятност, като само P_1 вижда резултата. P_1 избира дали да *продаде* (с печалба, която зависи от изхода на монетата) или да *играе* (където P_2 трябва да познае страната). Оптималната P_2 **стратегия** в под-играта *Играй* **не е** функция единствено на самата под-игра *Играй* — тя зависи от **стойността** P_1 , която би получил, ако избере *продажба* вместо това. При промяна на печалбата от *продажба* оптималната **стратегия** за *игра* се променя, въпреки че самата под-игра *Играй* остава непроменена.

Това е централната патология, в която попада всяко наивно решаване на под-игра с непълна информация. Решението: решаване на *допълнена под-игра*, която включва оригиналната под-игра плюс допълнителни възли „алтернативна печалба“, кодиращи какво всеки играч би могъл да постигне, ако *не беше влязъл* в тази под-игра.

4.9.2 Пач 1 — Безопасно решаване на под-игри

Допълнената под-игра се основава на план-стойности: всяко информационно множество в началото на под-играта получава алтернативна печалба, равна на обещаната от плана стойност за играча в този момент от играта. Решаването на допълнената игра води до усъвършенствана стратегия с гаранция за безопасност — експлоатируемостта е доказуемо не по-висока от тази на плана и строго по-ниска, когато локалните условия го позволяват.

Практическо усъвършенстване (*Обхват*) пренася „подаръци“ – разлики в **стойността** от по-ранни точки по пътя, при които играчът е можел да постигне строго по-добър резултат – за допълнително подобрение.

На практика консервативните план-печалби могат да бъдат заменени с *оценки* на равновесната стойност. Това премахва строгата гаранция, но обикновено води до по-ниска експлоатируемост в реална игра, тъй като самите консервативни план-печалби са неточни.

Запомнете: безопасното решаване на под-игри работи, защото под-играта не се решава самостоятелно; тя е закотвена чрез план-стойности.

4.9.3 Кръпка 2 — Вложено решаване на под-игри (унищожителят на превода на действия)

Когато *съперникът* изиграе действие a извън абстракцията (залог $0.7 \times \text{pot}$, когато в абстракцията са предвидени само $0.5 \times \text{pot}$ и $1 \times \text{pot}$), вместо да се закръгли a до известно действие чрез преводач, *се преизчислява нова под-игра, която съдържа a .*

Евтината версия изгражда под-игра непосредствено след действието извън дървото, преизчислява я с помощта на безопасна подигра-скеле и добавя новата подстратегия към плана. Ако по-късно се появи друго действие извън дървото, процесът се повтаря — планът нараства само там, където действително се стигне до игра.

Емпирично въздействие. При хедс-ъп безлимит покер експлоатируемостта на вложено решаване на под-игри срещу действия на съперника извън дървото е **10–100 пъти по-ниска** от тази на всеки предходен метод за преобразуване на действие, в зависимост от степента на абстракция.

Рекурсията е плитка на практика. В повечето реални случаи действията извън дървото не се свързват последователно — съперникът прави един необичаен залог, агентът преизчислява и новото абстрактно дърво го включва. Статичните преобразуватели на действия остават подходящият избор само когато латентността не позволява live CFR solve (онлайн игра, вградени приложения).

Запомнете: преобразуването на действие закръгля хода на съперника; вложеното решаване запазва хода и решава около него.

4.10 Архитектура: План и актуализации по време на работа

Целият конвейер вече се свежда до един архитектурен модел, който всеки състезателен изкуствен интелект за хедс-ъп безлимит покер от 2017 г. насам (Libratus, Modicum, Pluribus) използва:

1. **Време на компилация** — прилагат се беззагубна и загубена абстракция за свиване на играта; решава се получената абстрактна игра с CFR / CFR+ / MCCFR; получената стратегия се замразява като **план**.
2. **Време на изпълнение** — когато играта премине в под-игра, която планът покрива грубо, тя се преизчислява чрез безопасно решаване на под-игри (Кръпка 1). Когато противникът извърши действие извън абстракцията, се преизчислява нова под-игра, съдържаща това действие (Кръпка 2).

Двете части се допълват взаимно: **абстракцията** прави проблема по време на компилация решима, а **актуализациите по време на работа** възстановяват прецизността, загубена вследствие на абстракцията, в онези моменти от играта, където тя е от значение. Стъпка 6 разглежда пълните интегрирани системи; тази стъпка предоставя примитивите за **абстракция** и **актуализации**, върху които те се изграждат.

Запомнете: производствената архитектура е една част предварително изчислен план и една част live re-solving — нито едната половина не може да съществува самостоятелно.

4.11 Имплицитен маршрут: Еквиваленти на дълбокото обучение с подсилване

Всяка фаза от експлицитния конвейер има съответстващ подход в дълбокото обучение с подсилване, който заема същото концептуално място, но заменя формалните гаранции с обучение от край до край. Това са насочващи указатели — реалните имплементации се реализират в стъпки 5–6.

- **Критерий за сливане** — трите нива на строгост съответстват на различни мрежови архитектури: беззагубен ↔ инвариантни спрямо разместване мрежи (Deep Sets), ограничена загуба ↔ тесни места в рекурентни/трансформър политики, емпирична сходност ↔ латентни представяния, научени от край до край (както при Deep CFR).
 - **Бюджет на грешки** — не се задават изрични граници за всяко сливане. Качеството се следи чрез криви скорост–изкривяване (Information Bottleneck) и проследяване на тестовата загуба, като вместо алгоритмични гаранции се приема асимптотична сходимост.
 - **Конвейер по време на компилация** — няма отделна фаза на компилация. Латентното пространство на мрежата научава собствено абстрактно представяне, оформено директно чрез градиентен спуск върху загубата за минимизиране на съжалението.
 - **Корекция по време на изпълнение** — точните решаващи устройства за равновесие се заменят с евристично търсене с ограничена дълбочина, подкрепено от невронни мрежови функции на стойността (DeepStack, ReBeL).
-

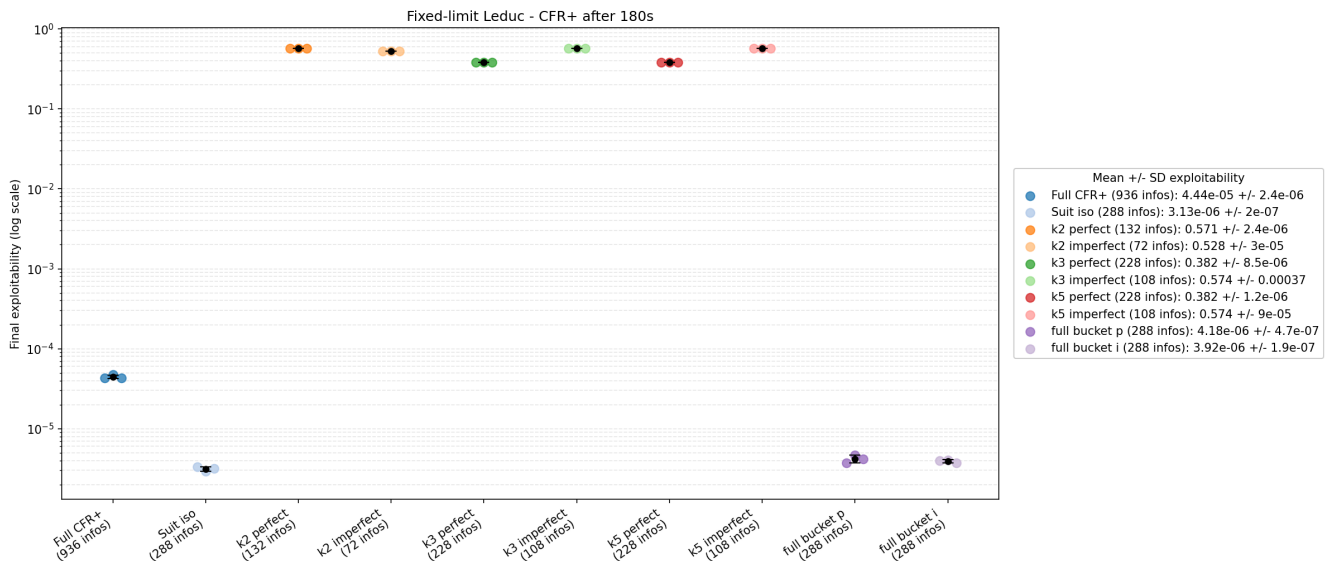
4.12 Практическо валидиране

Фазата на внедряване превърна абстракциите от резюмето в малък, но цялостен конвейер тип „Людък“:

- **Беззагубен изоморфизъм на боите** при ледюк с фиксиран лимит и Extended Leduc.

- **Загубено групиране на карти**, базирано на характеристика за силата на ръката, разстояния от ЕМД тип и конфигурируем брой кош за карти.
- **Абстракция на действията** върху Mini-NL Leduc, с код за превод по най-близък съсед, вероятностно разпределение и псевдохармоничен метод.
- **Extended Leduc** с четири ранга и две бои, предоставящ по-голяма тестова среда с непълна информация.
- **Комбинирана абстракция**, съставена от редуциране на боите, редуциране на действията и групиране на карти.
- **Оценка на качеството** чрез разлики в експлоатируемостта, ЕМД проксита, кръстосана проверка с OpenSpiel и парето граница.

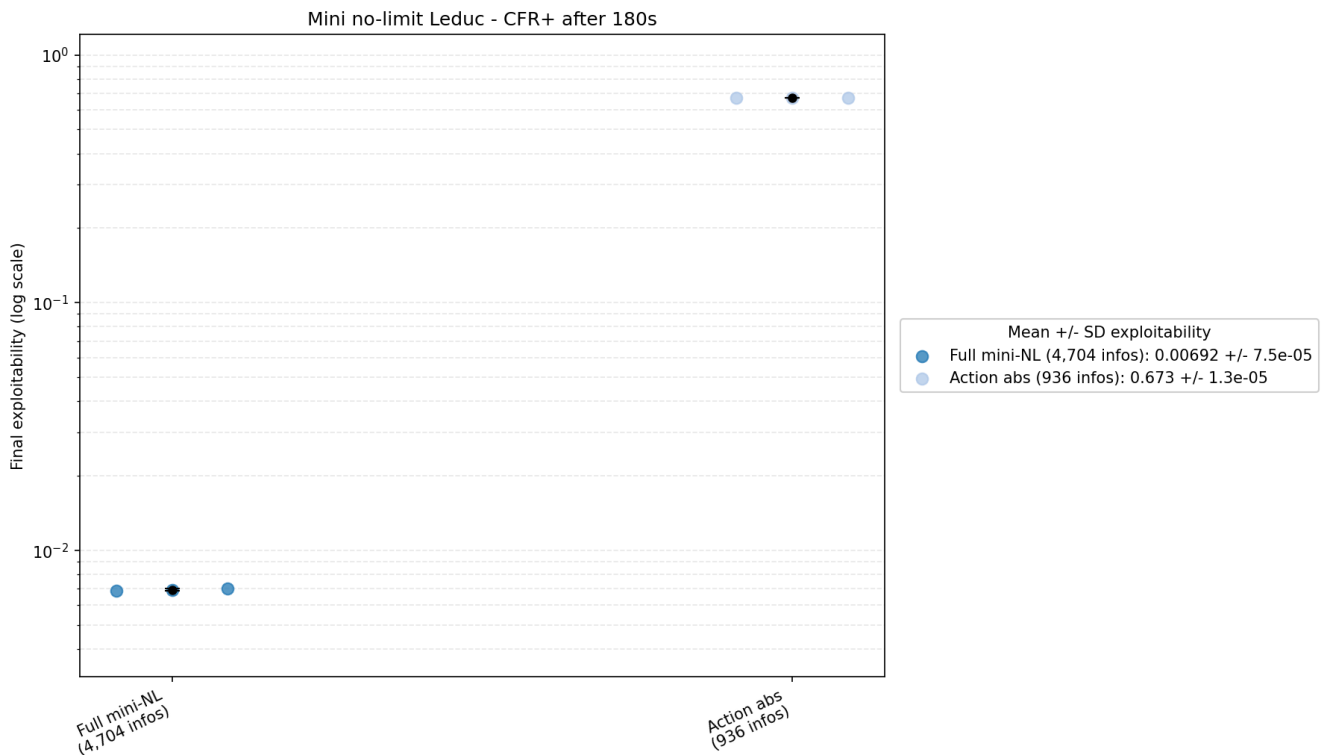
Основните емпирични изводи съвпадат с теорията: беззагубната абстракция е почти безплатна стратегически и много полезна изчислително, докато загубената информация и абстракцията на действията водят до постоянни прагове на експлоатируемост. При бюджети от 180 секунди за CFR+, изоморфизмът на боите намали ледюк с фиксиран лимит от 936 до 288 информационни множества и достигна по-ниска експлоатируемост от пълната игра, тъй като завърши значително повече итерации. Върху Extended Leduc същата идея намали играта от 10 304 до 2 968 информационни множества и подобри крайната експлоатируемост от около 2.7×10^{-2} до около 1.3×10^{-3} .



Фигура 2: Fixed-limit Leduc CFR+ abstraction results

Загубените експерименти с кофи показват и другата страна на компромиса. По-малките игри с кофи се обучават по-бързо, но грешката не изчезва с повече **итерации** на CFR+, тъй като стратегията се решава върху грешната игра. В ледюк с **фиксиран лимит** грубите абстракции с кофи останаха в диапазона 0.38–0.57 ек-

експлоатируемост, въпреки че завършиха повече **итерации** от пълния CFR+. Това е практическото значение на **експлоатируемата разлика**: грешката на **абстракцията** не е грешка на оптимизатора.



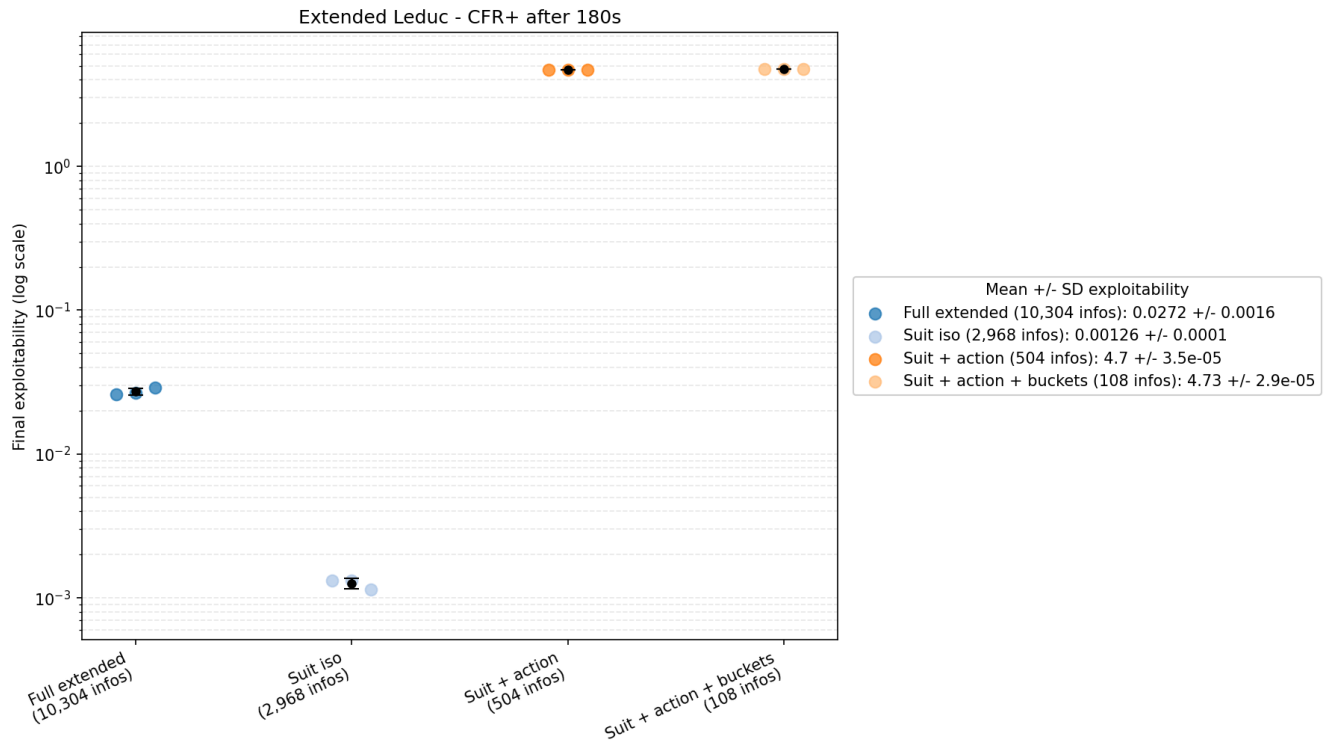
Фигура 3: Mini-NL Leduc CFR+ abstraction results

Абстракцията на действията беше най-рисковата част от стъпката. В Mini-NL ледюк ограничаването на множеството действия намали броя информационни множества от 4704 до 936 и позволи извършването на значително повече CFR+ итерации в рамките на същия времеви бюджет, но експлоатируемостта остана висока. В Extended Leduc добавянето на абстракция на действията върху изоморфизма на боите доведе до компактно дърво, но получената стратегия чрез преобразуване беше силно експлоатируема. Поради това в литературата се преминава от статично преобразуване на действие към вложено решаване на под-игри: пълното действие, което всъщност играе противникът, често има твърде голямо значение, за да бъде закръглено.

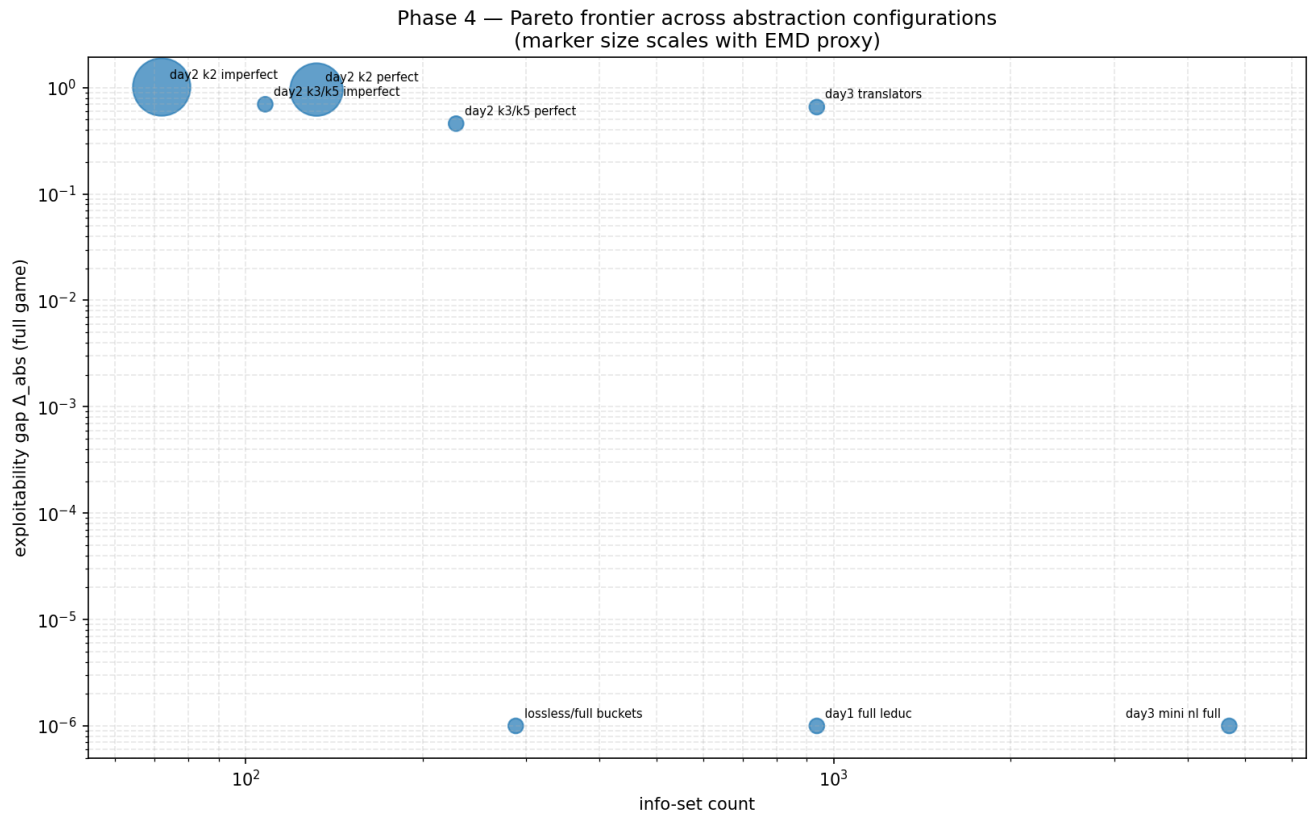
Парето изгледът представлява правилната окончателна диагноза. Всяка точка показва: с колко е намаляла играта и каква експлоатируемост е постигната или загубена чрез тази компресия? Беззагубният изоморфизъм на боите попада върху атрактивната част от границата. Грубите интервали и абстракция на действията могат да намалят играта още повече, но те преминават в различен режим, при който по-малкият размер се заплаща с качеството на стратегията.

4.13 Връзки и бъдещи перспективи

Стъпка 4 продължава развитието, започнато в стъпки 2 и 3. В стъпка 2 Белман въвежда CFR като метод за локално минимизиране на съжалението върху информационните множества. Стъпка 3 прави този решаващ по-бърз и по-ефективен чрез CFR+ и MCCFR. Стъпка 4 променя самата входна игра: преди да започне ре-



Фигура 4: Extended Leduc CFR+ abstraction results



Фигура 5: Abstraction Pareto frontier

шаването, тя определя кои състояния и действия могат да бъдат обединени, без това да доведе до загуба на стратегическия сигнал.

Мостът към стъпка 5 е разграничението между явна и неявна абстракция. Абстракциите от стъпка 4 представляват ръчно изградени или получени чрез клъстеризация разделяния на информационни множества и множества от действия. Deep CFR и невронното приближение на равновесие заменят тези таблици с научени функционални апроксиматори: представянето остава компресирано, но компресията се реализира вътре в мрежа, вместо във фиксирана карта на кофи.

Мостът към стъпка 6 е архитектурният план. Съвременните покер агенти първо решават груба абстрактна игра, след това закръпват слабостите онлайн чрез решаване на под-игра. Стъпка 4 предоставя речника: план, преобразуване на действие, експлоатируема разлика, парето граница и безопасно/вложено усъвършенстване. Стъпка 6 превръща тези елементи в пълни системи за игра.

За дисертацията абстракцията има значение, тъй като адаптацията към съперника функционира единствено с резолюцията, която се запазва в представянето на играта. Ако абстракцията обедини две стратегически различни състояния, ориентирани към противника, то нито един последващ модел на противника не би могъл да възстанови тази разлика. Обратно, твърде фино представяне може да се окаже прекалено скъпо за решаване или оценяване. Поради това парето границата от стъпка 4 се превръща в част от методологията за оценяване: качеството на стратегията трябва да бъде докладвано заедно с размера и грануларността на представянето на играта, което я е породило.

Стъпка 5 Обобщение — Невронни мрежи за игри с непълна информация

Изследване върху възможностите за прилагане на изкуствен интелект в компютърните игри

Alexander Andreev

May 2026

Съдържание

5 Стъпка 5 — Невронни мрежи за игри с непълна информация	1
Защо невронни мрежи	1
5.1 Част 1 — Основни положения	2
5.2 Част 2 — Архитектура и композиция	5
5.3 Част 3 — Невронни мрежи в CFR	11
5.4 Част 4 — Други невронни приложения	14
5.5 Част 5 — Експериментална / граница	16
5.6 Синтез	16

5 Стъпка 5 — Невронни мрежи за игри с непълна информация

Това е съкратен обзор на невронни методи за игри с непълна информация — материалът, който Стъпка 5 разширява от първоначалния си фокус върху Deep CFR до по-широко разглеждане на изграждането, обучението и внедряването на невронни мрежи в игри, при които играчите не могат да възприемат пълното състояние. Той изпълнява две функции: служи като бързо опресняване при преминаване през по-късните стъпки и като основен източник за синтеза на публичния доклад от Стъпка 15. Стъпка 5 представлява умишлено еднократно отклонение от обичайния учебен цикъл — фазата на реализация от нулата се отлага, а вместо нея се разширява теорията — поради което този документ е карта на областта в ширина, а не запис на код. Математическите постановки са представени опростено: ключовите уравнения се посочват там, където изострят интуицията, докато извежданията и доказателствата остават в цитираните източници.

Защо невронни мрежи

Табличният CFR и неговите Monte Carlo variants (Стъпки 2–4) съхраняват стойност на съжалението и стойност на стратегията във всяко информационно множество, така че разходът им за памет е пропорционален на броя на информационните множества, който нараства с размера на играта. Изследването в Стъпка 5 количествено определи този лимит върху ледюк покер — игра, достатъчно малка, за да се побере в таблица: табличният MCCFR достигна експлоатируемост от 0.097, докато Deep CFR при сравним бюджет в реално

време достигна само 1.70. В този мащаб табличната методика е за предпочитане; невронната методика не предлага никакво предимство, тъй като таблицата вече се побира изцяло в паметта.

Невронна мрежа заменя таблицата с апроксимация на функция f_θ , която съпоставя характеристиките на едно състояние към стойност или разпределение на действията и обобщава към състояния, които не са били посетени по време на обучението. Това обръща структурата на разходите: паметта става независима от броя на информационните множества, за сметка на по-скъпи итерации и по-слаби гаранции за сходимост. Двата режима се срещат при пресичане в размер на играта — под него изброяването е по-евтино и точно; над него таблицата не може да бъде конструирана и апроксимация на функция е единствената възможност. Игрите, към които тази работа е насочена, са над пресичането, поради което останалата част от главата се посвещава на невронни методи.

5.1 Част 1 — Основни положения

5.1.1 Основи на невронните мрежи

Мрежа с директно разпространение представлява композиция от слоеве, като всеки слой прилага афинно преобразуване, последвано от нелинейност по елементи:

$$h = \sigma(Wx + b),$$

където x е входът на слоя, W и b са съответно обучаемата матрица на теглоите и векторът на пристрастията, а σ представлява фиксирана нелинейност, като например изправителя $\text{ReLU}(z) = \max(0, z)$. Подреждането на L такива слоя формира функцията на мрежата f_θ , при която θ обединява всички W и b . Именно нелинейностите позволяват на тази композиция да представя функции, които не могат да бъдат изразени чрез едно-единствено афинно преобразуване.

ReLU (rectified linear unit): стандартната функция за активация — тя пропуска положителните стойности непроменени и ограничава отрицателните до нула, което е изчислително евтино и избягва проблема с изчезващите градиенти, характерен за по-старите насищащи се функции за активация.

Обучението изисква **скаларна величина** за измерване на грешката. При задача за **регресия** — например при оценка на **контрафактична стойност** — стандартно се използва **средноквадратичната грешка**:

$$L(\theta) = \frac{1}{N} \sum_{i=1}^N (f_\theta(x_i) - y_i)^2,$$

където N е броят на примерите в партида, x_i — вход, а y_i — неговата цел. Когато целта представлява разпределение на действията, вместо това се използва кръстосана ентропия загуба — $\sum_a y_a \log f_\theta(x)_a$, като y_a е целевата вероятност за действие a .

Обучението представлява **градиентен спуск** върху тази **загуба**. Всеки параметър се измества с малка стъпка

в посоката, която води до най-голямо намаление на **загубата**:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L,$$

където η е скоростта на обучение, а $\nabla_{\theta} L$ — градиентът на загубата спрямо параметрите. *Обратно разпространение на грешката* изчислява този градиент, като прилага правилото за верижно диференциране слой по слой от изхода към входа и използва повторно междинните стойности, съхранени във всеки слой; така изчисляването на един градиент струва приблизително колкото едно преминаване напред. Автоматичното диференциране извършва тази процедура механично.

На практика градиентът се оценява от малка случайна партида, а не от целия набор от данни (*стохастичен градиентен спуск*) — по-шумен, но значително по-евтин на стъпка. Импулсът намалява този шум, като извършва стъпките с плъзгаща средна на градиентите, $v \leftarrow \beta v + \nabla_{\theta} L$ и $\theta \leftarrow \theta - \eta v$, където $\beta \in [0, 1)$ определя степента, в която се запазват миналите градиенти. Adam, оптимизаторът по подразбиране в методите на тази глава, освен това разделя стъпката за всеки параметър на плъзгаща оценка на неговата големина на градиента, което осигурява стабилен напредък с минимално ръчно настройване на η .

Adam (адаптивна оценка на моментите): комбинира импулс с размери на стъпките за всеки параметър, извлечени от плъзгащи статистики на градиента, поради което се обучава надеждно с малко настройване.

Мрежа с достатъчен капацитет може да напасне точно една тренировъчна партида, включително и нейния шум, и след това да се представи слабо при невиджани състояния — *пренастройване*. Регуляризацията противодейства на това. Отслабването на теглоите добавя наказание $\lambda \|\theta\|^2$ към загубата, като λ контролира степента, в която се обезкуражават големите тегла; отпадането произволно занулява част p от активациите по време на обучението, така че мрежата да не може да разчита на нито един отделен елемент; и ранното спиране прекратява обучението, след като грешката върху задържаните данни започне да нараства.

Една идея от обучението с подкрепление се повтаря през цялата глава и си струва да бъде изразена изрично: *самоподкрепяне*. Актуализация чрез темпорални разлики оценява стойността на едно състояние въз основа на собствената оценка на мрежата за следващото състояние, а не от краен резултат:

$$V(s) \leftarrow V(s) + \alpha [r + \gamma V(s') - V(s)],$$

където s и s' са последователни състояния, r е наградата, $\gamma \in [0, 1]$ е коефициентът на дисконтиране, а α е размерът на стъпката; величината в скобите представлява грешка при темпорална разлика — разликата между текущата оценка и целта с бутстрапиране. Самоподкрепянето прави обучението ефективно по отношение на пробите, а както показва следващият раздел, също така го прави и по-малко стабилно. Основният материал е представен в глави 9 и 11 от Sutton & Barto (2-ро издание); всяка стандартна литература за дълбоко обучение разглежда подробно обратното разпространение на грешката и оптимизацията.

5.1.2 Обединяване на основите в ледюк покера

Тези елементи се комбинират в най-простия **невронен решавач** за игра. Вземете **ледюк покер**, тестовата среда от стъпки 3–4: неговото **информационно състояние** — **тайната карта, историята на залозите и потът** — се кодира като вектор от около тридесет числа. Малка **мрежа** картографира този вектор към **стойност** за всяко действие; **обратното разпространение на грешката** с **adam** го настройва спрямо **целевите стойности**, генерирани по време на игра, а леката **регуляризация** предотвратява запамятането на шума в тези цели. Използвана правилно, мрежата предоставя две възможности, които таблицата няма: тя обобщава, така че **стойността**, научена в едно **информационно множество**, информира подобни невиджани такива, а цената на паметта ѝ е фиксирана от броя на параметрите, а не от броя на информационните множества.

Същата техника се влошава, когато бъде претоварена, а **изследването** направи режимите на отказ конкретни. Капацитетът е най-ясният пример. При **ледюк** най-малката тествана **мрежа** — два скрити слоя с тридесет и две единици — достигна по-ниска **експлоатируемост** от мрежи няколко пъти по-големи, защото при ограничен бюджет на пробите по-големите мрежи задържаха твърде много параметри за твърде малко данни и пасваха шум, вместо структура. **Скоростта на обучение**, зададена твърде висока, дестабилизира спускането; прекомерната **регуляризация** изглажда острите, почти **детерминистични** стратегии, към които CFR се сближава, тъй като силното **отслабване на теглоите** или **отпадане** дърпат изхода към **равномерното разпределение**. Общият принцип е, че капацитетът и регуляризацията трябва да бъдат съобразени с количеството и качеството на данните, а не да се максимизират по подразбиране — точка, която част 2 развива в изрични насоки за оразмеряване.

5.1.3 Нестационарност и смъртоносната троица

Комбинирането на горните компоненти въвежда нестабилност. Сътон и Барто идентифицират една *смъртоносна троица*: когато **апроксимация на функция, самоподкрепяне и обучение извън политиката** се използват заедно, параметрите могат да се разминават, вместо да **сходяват**, тъй като **целевите стойности по метода „bootstrap“** за всяка актуализация зависят от същите параметри, които се променят. Всеки един компонент е **безопасен** самостоятелно; комбинацията не е. Средствата за защита са практични, а не теоретични. Една *целева мрежа* — забавено копие на параметрите, използвано само за изчисляване на **целевите стойности по метода „bootstrap“** — и един *буфер за възпроизвеждане*, който декорелира последователните проби, заедно правят дълбоките Q-мрежи обучаеми, и двата се появяват отново в **невронните решаващи средства за игри** от части 3 и 4.

Игрите въвеждат втори източник на нестабилност, който липсва при обучението на един агент. При обучението с учител разпределението на данните е фиксирано, а при обучението с подкрепление с един агент то се стабилизира, след като стратегията **сходи**. В една игра противникът също се адаптира, така че разпределението на ситуациите, които мрежата среща, се променя по време на обучението; функцията, която се апроксимира, сама по себе си е подвижна цел. Тази нестационарност не е случайна — тя е централната трудност на средата, към която това изследване е насочено. Няколко метода в следващите части са най-добре разбрани като отговори на нея: самообучението срещу замразени или осреднени във времето противници възстановява стационарна цел, популационните методи заменят подвижната цел с изричен набор от фикси-

рани противници, а техниките за моделиране на противника и бърза адаптация от части 4 и 5 се опитват да я проследят директно.

5.2 Част 2 — Архитектура и композиция

5.2.1 Индуктивно пристрастие: Съгласуване на архитектурата със структурата на играта

Всяка архитектура кодира предположения относно структурата на своя вход още преди да види каквито и да е данни; тези вградени предположения представляват нейното *индуктивно пристрастие*. Напълно свързаната мрежа не предполага нищо — всяка входна характеристика може да взаимодейства с всяка друга — което я прави обща, но изисква голям обем данни, тъй като мрежата трябва да научи структурата от нулата. Изборът на архитектура, чието пристрастие съответства на структурата на играта, намалява броя на параметрите и количеството необходими данни за постигане на определена точност, както и подобрява обобщаването към състояния, които не са били част от обучението. Решенията за архитектура в останалата част от Част 2 по същество представляват избор на индуктивно пристрастие.

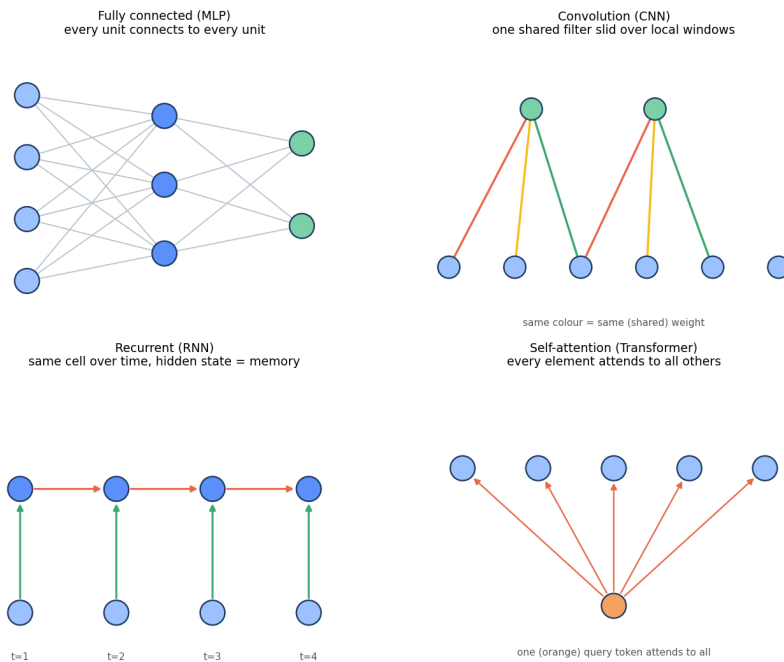
Три съответствия покриват повечето случаи в игри с **непълна информация**. Когато **състоянието** е пространствено — дъска или мрежа — **конволюционната мрежа** предполага, че един и същ локален модел е смислен, независимо къде се появи, така че характеристика, научена в една област, може да бъде приложена към други; тази *еквивариантност спрямо трансляция* обяснява защо конволюциите доминират в мрежовите области. Когато част от **състоянието** е неупотребена колекция — ръка карти или множеството от други играчи — **мрежата** трябва да бъде *инвариантна спрямо пренареждане*, като произвежда един и същ изход, независимо от реда, в който тези елементи са представени, за да не се използва капацитет за научаване на това, че редът е без значение. Когато **състоянието** представлява история, разкрита с течение на времето, както при **частична наблюдаемост**, **рекурентна мрежа** или **механизъм за внимание** позволяват на **мрежата** да обобщи последователността и да пренесе напред изводите от по-ранните наблюдения. Съгласуването на тези пристрастия със структурата на играта е разликата между **мрежа**, която се учи ефективно, и такава, която само би могла по принцип.

5.2.2 Типове слоеве на практика: многослоен перцептрон, CNN, RNN, внимание/трансформър

Семействата се различават по своята *свързаност* — кои единици са свързани с кои — и дали свързващите *тегла* се *споделят* между позиции или във времето. Всичко останало (неврона, нелинейността, обратното разпространение на грешката) е идентично. На фигурата по-долу цветът на ръбовете обозначава отделно тегло, така че повтарящият се цвят маркира споделено тегло; следващите параграфи описват всеки панел.

Напълно свързан слой — многослоен перцептрон, или MLP — свързва всяка единица с всяка единица в следващия слой, като всяка връзка носи свое собствено тегло. Той не предполага никаква структура и може да представи всяка зависимост по принцип, но броят на теглата е произведение от размерите на двата слоя и цялата структура трябва да бъде научена от данните. Това е правилният избор за плосък вектор на характеристики без пространствено или последователно значение, като например информационно състояние в покер.

Свързъчен слой (CNN) свързва всяка изходна единица само с малък локален прозорец от входа — неговото



Фигура 1: Connectivity of the four layer families: a fully connected MLP, a convolution that reuses one filter across local windows, a recurrent cell reused across time, and self-attention.

приемно поле — и прилага един и същ малък набор от тегла, *филтърът*, на всяка позиция. От тази една промяна в свързаността следват две последствия: броят на теглата зависи от размера на филтъра, а не от размера на входа, и модел, научен на едно място, се разпознава на всяко друго място (еквивариантност спрямо превод). Подреждането на свързки увеличава ефективното приемно поле, така че по-късните слоеве комбинират локални модели в по-големи. Това е стандартният избор за дъски и мрежи.

Споделяне на теглои: повторно използване на едни и същи тегла на много позиции (CNN) или времеви стъпки (RNN). Това е, което прави тези подредби параметърно ефективни и е механизмът зад тяхната еквивариантност.

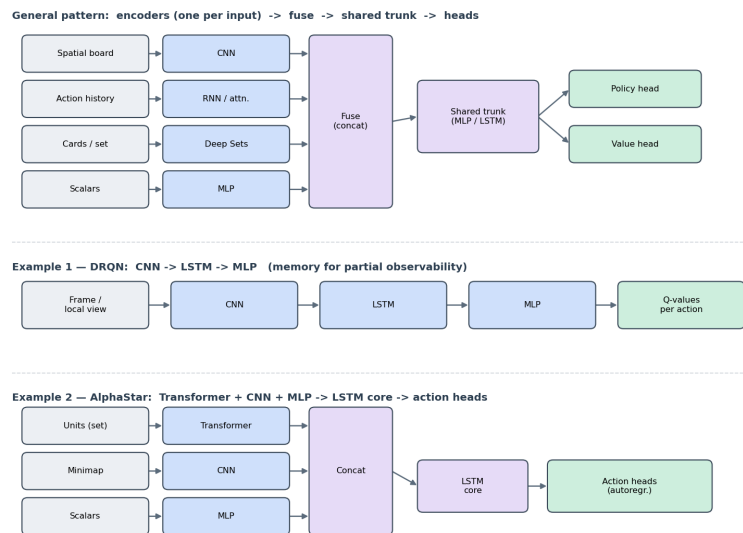
Повтарящ се слой (RNN) прилага една и съща клетка последователно по дължината на последователността: на всяка стъпка той приема текущия вход заедно със *скритото състояние* от предишната стъпка и произвежда ново скрито състояние, което се предава напред. Тъй като клетката е споделена между стъпките, броят на теглата не зависи от дължината на последователността, а скритото състояние функционира като памет за всичко видяно до момента — свойството, необходимо когато състоянието представлява история, разкривана с течение на времето, както при частична наблюдаемост. Вариантите с гейтове (LSTM, GRU) добавят научени гейтове, които контролират какво да се запази и какво да се изхвърли, което им позволява да задържат информация за по-дълги интервали от време.

Слой за **самовнимание** (трансформър) позволява на всеки елемент да взаимодейства директно с всеки друг. Всеки елемент излъчва *заявка*, *ключ* и *стойност*; изходът за даден елемент представлява претеглена средна стойност на стойностите на всички елементи, като теглата се определят от сходството между неговата заявка и всеки ключ. Тъй като операцията не отчита реда на елементите, тя е **инвариантна спрямо разместване**,

а редът трябва да бъде предоставен изрично чрез **позиционни кодировки**, когато има значение. Самовниманието улавя **дългосрочни зависимости**, до които повтарящ се слой достига само косвено, за сметка на изчисления, чиято сложност нараства квадратично спрямо броя на елементите. То е подходящо за истории с променлива дължина и входове със стойности от множества.

Вход, който е наистина неуреден — например ръка карти или множеството от други играчи — изисква **инвариантност спрямо пермутации**, която не се постига чрез нов тип слой, а чрез подредба: прилага се една и съща малка мрежа към всеки елемент, след което резултатите се комбинират с операция, независима от реда, като сума, **средна стойност** или максимум. Това е принципът зад Deep Sets и представлява естественият **кодировчик** за колекция, чийто ред няма значение.

На практика една мрежа рядко използва само едно семейство архитектури. Обичайният дизайн е конвейер: отделен **кодировчик**, съобразен с всяка част от входа, изходите на кодировчика се **обединяват** чрез конкатенация в общ *ствол*, и една или повече *глави*, които извличат прогнозите от ствола — обикновено глава на стратегията и стойностна глава. Deep Recurrent Q-Network (DRQN) комбинира конволюционен енкодер, LSTM и изходен MLP, като рекурентността осигурява памет, която едно преминаване напред няма при частична наблюдаемост — естественият шаблон за агент в мрежа с „мъгла на войната“. AlphaStar отива по-далеч, кодирайки множество единици с трансформър, пространствената миникарта с CNN и скаларни величини с MLP, след което обединявайки трите и преминавайки ги през LSTM ядро преди изходните слоеве за действие. Практическият урок за останалата част от Част 2 е, че „коя архитектура“ обикновено означава „кой кодировчик за всеки вход и как да ги комбинираме“, а не един-единствен избор.



Фигура 2: The general encoder / trunk / heads pattern (top); two instances below — DRQN and AlphaStar.

5.2.3 Кодирание на състоянието на играта и историята

Преди някоя от тези архитектури да може да работи, състоянието на играта трябва да бъде превърнато в числа. Информационното състояние — всичко, което действащият играч може законно да наблюдава — трябва

да стане тензор с фиксирана форма, а начинът, по който е подредено, определя коя архитектура може да го използва. Категориалните факти се кодират или като *one-hot* вектори — вектор с дължина k , съдържащ една единствена единица, която маркира коя от k категории е валидна — или, когато k е голямо, като *вграждания*: кратки научени вектори, които се извличат за всяка категория и се обучават заедно с мрежата. Ледюк покер е достатъчно малък, за да използва *one-hot* навсякъде: тайната карта, общата карта и историята на залозите стават *one-hot* полета, а потът — една скаларна величина, което води до приблизително трийсетмерния вектор, използван в стъпки 3–4.

One-hot срещу вграждане: *one-hot* е точен, но неговата дължина нараства с броя на категориите; едно вграждане компресиращо голям речник — всяка карта или всеки елемент от дъската — в няколко научени измерения и позволява на подобни категории да споделят структура.

По-богатите входове изискват по-богати подредби. Пространственото състояние се кодира като купчина от мрежи, по една *канална* равнина за всеки тип обект, така че сгвката да може да го прочете директно. Pommerman — игра в стил Bomberman на базата на мрежа, в която агентите се движат по дъска, поставяйки бомби, за да разрушават стени и да прихващат противници — е ясен пример: неговата дъска 11×11 е разделена на отделна двоична равнина за всеки тип обект (проходи, дървени стени, твърди стени, бомби, пламъци, бонуси, действащият агент и всеки противник), подредени в тензор (11, 11, канали); скаларни величини като оставащите боеприпаси и обхватът на взрива след това се добавят като кратък вектор или се излъчват като постоянни равнини. История, чийто ред има значение, се кодира като *последователност* от вектори за всяка стъпка за повтарящ се слой или трансформър, вместо да бъде сплескана в един вектор, който пренебрегва реда. Неподредена колекция, като текущия набор от противници, се кодира елемент по елемент и се обединява. Във всеки случай кодирането е мястото, където индуктивното пристрастие от предишните раздели всъщност се прилага: мрежата се представя като мрежа именно за да важат предположенията на сгвката.

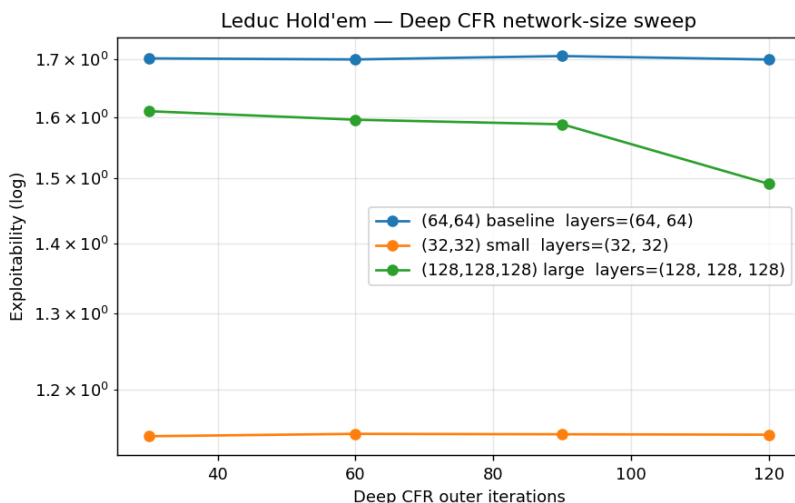
Една дисциплина е специфична за игри с **непълна информация** и лесно може да бъде нарушена: кодирането трябва да съдържа само това, което играчът може да наблюдава, и нищо скрито. Ако каквато и да е **привилегирована информация** — **тайната карта** на противника или истинското съдържание на клетка, покрита с мъгла — попадне в тензора за вход, мрежата ще се научи да я използва и ефективно ще „вижда“ през мъглата, което води до стратегия, която не може да бъде приложена срещу реален противник, и до показател за **експлоатируемост**, който е безсмислен. Ето защо рамки като OpenSpiel дефинират изрично представяне на **информационно състояние** за всеки играч; правилното получаване на това представяне е изискване за коректност, а не избор за настройка.

Две по-малки точки завършват изработката. Непрекъснати признаци като **пота** или **размера на стака** се нормализират до сравним мащаб, преди да постъпят в мрежата, така че нито един признак да не доминира градиента. А кодировчикът не е задължително да бъде проектиран ръчно от край до край: една мрежа може да се научи да компресираща дълга история в компактен обобщен вектор — идея, която Част 4 развива като научена абстракция, а Част 5 — като научени състояния на убеждение.

5.2.4 Оразмеряване и капацитет

Капацитетът на една *мрежа* — определен от нейната ширина, броя на единиците във всеки слой, и нейната дълбочина, броя на слоевете — е размерът на семейството от функции, които тя може да представи. При твърде малък капацитет мрежата *подпасва* (underfits), неспособна да изрази целевата стратегия; при твърде голям капацитет тя *свърхпасва* (overfits), пасвайки шума в ограничен набор от данни, вместо основната структура. Правилният капацитет е този, който съответства на количеството информативни данни, което процедурата за обучение действително предоставя — а при решавач на игра това често е значително по-малко от суровия брой състояния, тъй като самите стойности, които се пасват, представляват шумни оценки.

Стъпка 5 от изследването показва това директно. Прилагайки Deep CFR върху ледюк при фиксиран бюджет, но с различни размери на мрежата, най-малката мрежа — два скрити слоя от тридесет и две единици — постигна по-ниска експлоатируемост в сравнение с мрежите, които бяха два и четири пъти по-широки; по-големите мрежи съдържаха повече параметри, отколкото няколкото хиляди обхождания можеха да ограничат, и изразходваха излишъка за пасване на шум от извадката. Изводът не е, че по-малкият размер винаги е по-добър, а че капацитетът трябва да съответства на бюджета за данни: при по-голяма игра или с много повече обхождания, класирането би се променило.

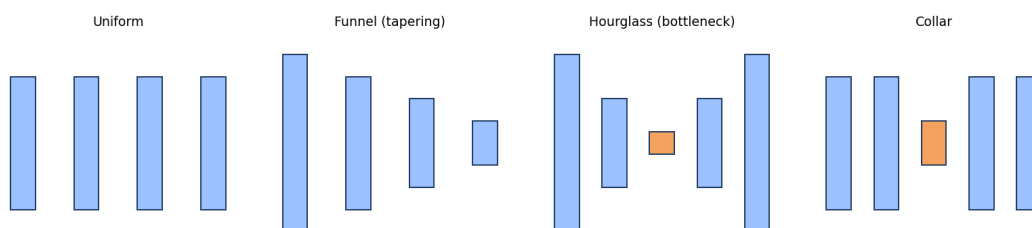


Фигура 3: Deep CFR exploitability on Leduc across network sizes (32x32, 64x64, 128x128x128).

Това аргументира за умишлено оразмеряване на мрежата, вместо да се използва по подразбиране най-голямата, която се побира в паметта. Работеща рецепта е да се започне с малък размер и ширината и дълбочината да се увеличават само докато задържан показател — загуба при валидиране или експлоатируемост за решавач — продължава да се подобрява, като се спира, когато достигне плато. Ширината позволява на мрежата да представя повече различни модели на дадено ниво на абстракция; дълбочината ѝ позволява да съставя прости модели в по-абстрактни, поради което пространствените и последователните проблеми се възнаграждат с няколко слоя, докато плосък вектор на характеристики често не. Капацитетът трябва да се мащабира спрямо размера и богатството на играта, а не да бъде максимизиран по подразбиране.

Ширината на мрежата не е задължително да остава постоянна във всичките ѝ слоеве. Те могат да се *свиват*

към изхода, като компресират входа до малко обобщение (кодировчик или *фуния*), или да се *разширяват* от малък вход, за да реконструират структуриран изход (декодер). Комбинирането на двете формира *пясъчен часовник*, чиято тясна среда — *тясно място* — принуждава цялата информация да премине през нискоразмерен код; тъй като мрежата трябва да действа или реконструира въз основа на този код, тя запазва единствено най-същественото, което регулира модела и води до компактно, многократно използвано представяне — *семе*то на научена абстракция или състояние на убеждението (Част 4 и 5). Същото стесняване, вградено в иначе равномерна купчина от слоеве, представлява *яка*. Фигурата по-долу илюстрира тези профили.



Фигура 4: Common network shapes: a uniform stack, a tapering funnel, an hourglass with a central bottleneck (orange), and a uniform stack with a single narrow collar (orange). Bar height represents layer width.

Тясното място ограничава *ширината*; *остатъчните (residual) връзки* адресират *дълбочината*, като добавят входа на даден слой към неговия изход, така че градиентите да достигат до дълбоките слоеве и много дълбоките мрежи да стават обучаеми — принципът, залегнал в основата на resNets и дълбоките мрежи за стойности от Част 4. И двете допълват отпадането, отслабването на теглоите и нормализацията, споменати по-рано: всяка от тях определя начина, по който се използва капацитетът, а не само неговия обем.

Остатъчна (residual) връзка: пряк път, който добавя входа на слой към неговия изход, така че слой трябва да научи само корекция; тя поддържа градиентите силни през дълбоки мрежи и е това, което прави много дълбоките модели обучаеми.

5.2.5 Стабилност на обучението и диагностиката

Две стабилизиращи средства от обсъждането на „смъртоносната тройка“ — целевата мрежа и буферът за възпроизвеждане — вършат по-голямата част от работата по предотвратяване на разминаването при обучение с оценка на стойности. Още две се появяват конкретно в решаващите игри по-нататък. *Извадка от резервоар* поддържа фиксиран по размер, равномерно случаен образец от всички данни, видени досега, което е важно, когато дадено количество трябва да бъде осреднено върху цялата история на обучението, а не само върху последните данни; Деер CFR го използва, така че неговата стратегическа мрежа да се обучава върху безпристрастна извадка през всички итерации (Част 3). *Намаляване на дисперсията* — чрез изваждане на базова линия от взетите извадкови стойности или използване на контролни променливи — намалява шума, който извадката внася в градиента, така че мрежата да се сближава с по-малко извадки. Нормализиране на входа и разумна инициализация на теглоите, споменати по-рано, допълват списъка: и двете поддържат активациите и градиентите в използваем диапазон още от първата стъпка.

Извадка от резервоар: метод, който поддържа извадка с фиксиран размер, при която всеки елемент, видян досега, има еднаква вероятност да бъде запазен, така че плъзгащата средна остава

безпристрастна, без да се съхраняват всички данни.

Дори и с тези мерки на място, невронният решаващ се проваля по-често тихо, отколкото шумно, така че четенето на диагностиката е от значение. Загуба, която намалява стабилно, докато оценителният показател — експлоатируемост за решаващ — застива, е знак за мрежа, която е *жива, но не сходим*: обучението работи, но бюджетът е твърде малък, за да може средната стратегия да се установи, което показва мрежите от Стъпка 5 при ледюк на няколко хиляди обхождания. Обратният и по-опасен случай е, когато обучението изобщо не протича, въпреки че изглежда така: при изследването едносимволна грешка в рутина на OpenSpiel остави мрежите за предимство необучени, но програмата завърши изпълнението си и произведе правдоподобен резултат, като експлоатируемостта просто замръзна на нивото на случайната стратегия. Урокът е да се потвърди, че загубата действително се променя и че оценителният показател реагира, преди да се вярва на който и да е резултат — тиха неоперация е неразличима от труден проблем, освен ако кривите не бъдат проверени.

5.3 Част 3 — Невронни мрежи в CFR

5.3.1 От табличен CFR към апроксимация на функцията

Табличният CFR поддържа по две стойности за всяко информационно множество: кумулативно контрафактично съжаление за всяко действие, от което се извежда текущата стратегия чрез напасване на съжалението, както и кумулативна стратегия, чиято плъзгаща средна представлява нашето приближение, което алгоритъмът в крайна сметка връща (Стъпки 2–4). И двете таблици се разрастват с увеличаването на броя на информационните множества — стената, описана в началото на тази глава.

Замяната им с невронни мрежи е концептуално проста: апроксиматор на функции приема тензор на информационно състояние като вход и предсказва това, което таблицата би съхранявала. Една мрежа може да научи съжаленията — еквивалентно, контрафактичните предимства — които определят текущата стратегия; друга може да научи средната стратегия директно. Тъй като една мрежа обобщава подобни информационни състояния, тя никога не трябва да ги изброява и нейният размер е фиксиран от параметрите ѝ, а не от играта. Методите по-долу се различават основно в *ко* от двете таблици те апроксимират и *как* генерират данните за обучението ѝ: Deep CFR научава предимства от извадкови обхождания на дървото, докато NFSP научава средната стратегия от самообучение без изобщо да обхожда дървото.

5.3.2 Deep CFR и неговите варианти с една мрежа

Deep CFR (Brown et al., 2019) е директният невронен превод на MCCFR. При всяка итерация той обхожда дървото на играта чрез външно вземане на проби и във всяко информационно множество, което посещава, изчислява извадковото контрафактично предимство на всяко действие — колко по-добро е било това действие от текущата стратегия средно. Тези двойки (информационно състояние, предимство) се записват в резервоарен буфер и *мрежа за предимства*, по една за всеки играч, се обучава да ги предсказва; текущата стратегия във всяко информационно множество след това се извежда от предсказаните предимства на мрежата чрез напасване на съжалението, точно както би била използвана таблицата. Отделна *стратегическа мрежа*, обучавана от собствен резервоарен буфер, научава средната стратегия през всички итерации —

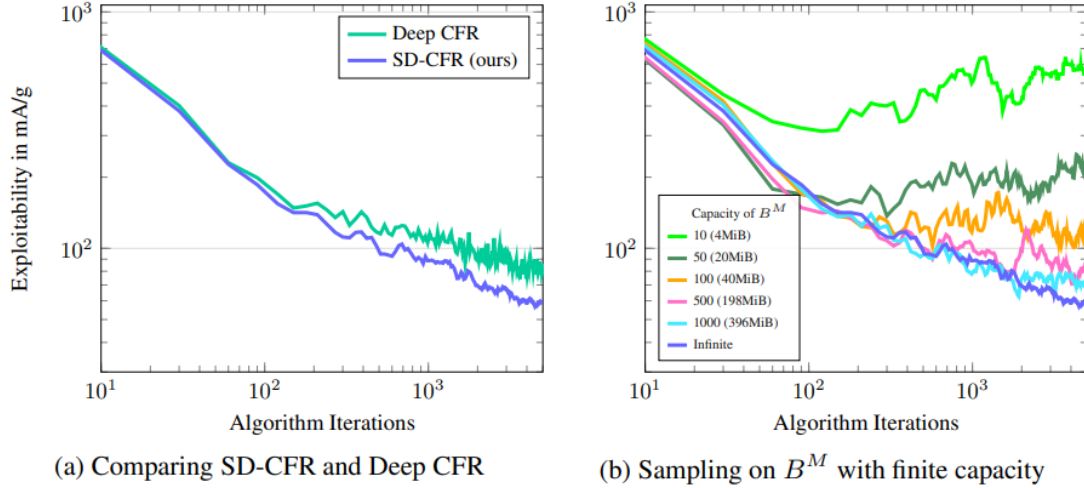
нашовото приближение, което се връща в края.

Единственото уравнение, което отличава Deep CFR от табличния MCCFR, е целевата функция на мрежата за предимства: средноквадратична грешка между нейната прогноза и извадковите предимства,

$$L(\theta) = \frac{1}{|B|} \sum_{(I, \tilde{a}) \in B} \|f_{\theta}(I) - \tilde{a}\|^2,$$

където B е партида, изтеглена от резервоара, I тензор на информационно състояние, $\tilde{a}$ векторът на извадковите предимства за неговите действия и f_{θ} мрежата за предимства (на практика пробите се претеглят според итерацията, отразявайки линейния CFR). Всичко останало — обхождането на дървото, стъпката на съвпадение по разкаяние, осредняването — се запазва непроменено спрямо табличния алгоритъм.

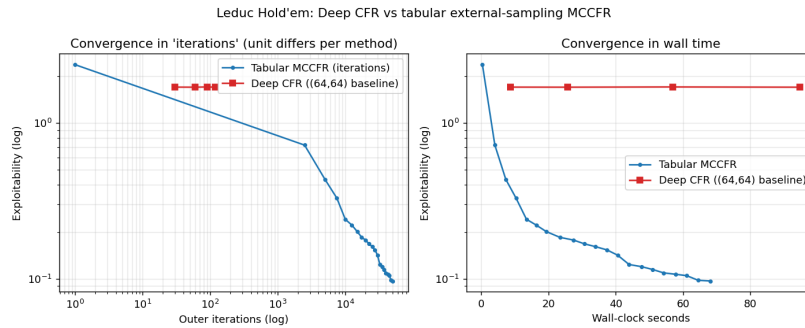
Две усъвършенствания намаляват разхода на итерация. *Single Deep CFR* премахва отделната стратегическа мрежа и възстановява средната стратегия директно от съхранените модели за предимство, като по този начин намалява наполовина броя на мрежите за обучение. *DREAM* заменя външното вземане на проби с извадка по резултати — една траектория на итерация — и добавя научена базова линия като контролна променлива, за да абсорбира допълнителната дисперсия, която извадката по резултати поражда, същата идея за намаляване на дисперсията от Част 2. При големи покер игри *Single Deep CFR* съвпада или леко превъзхожда *Deep CFR*, докато обучава една мрежа вместо две, както показва сравнението в оригиналната статия.



Фигура 5: Single Deep CFR (SD-CFR) matches or slightly beats Deep CFR while training only one network (left); the right panel shows how reservoir-buffer capacity affects convergence. From the Single Deep CFR paper — not an experiment from this work.

На ледюк изследването потвърди както обещанието, така и уловката. Deep CFR се обучи правилно — загубите му на предимство намаляха, както се очакваше — но на 120 итерации достигна експлоатируемост близо до 1,70, срещу 0,097 за табличен MCCFR при сравними реално време. Това не е дефект: ледюк е достатъчно малък, че табличният подход печели категорично, а на Deep CFR са необходими няколко пъти повече итерации, за да се доближи до наш равновесие. Неговото предимство се проявява едва когато играта е твърде

голяма, за да съществува таблица — режимът, който заключителният раздел на тази част уточнява.



Фигура 6: Deep CFR vs. tabular MCCFR exploitability on Leduc.

5.3.3 NFSP — Neural Fictitious Self-Play

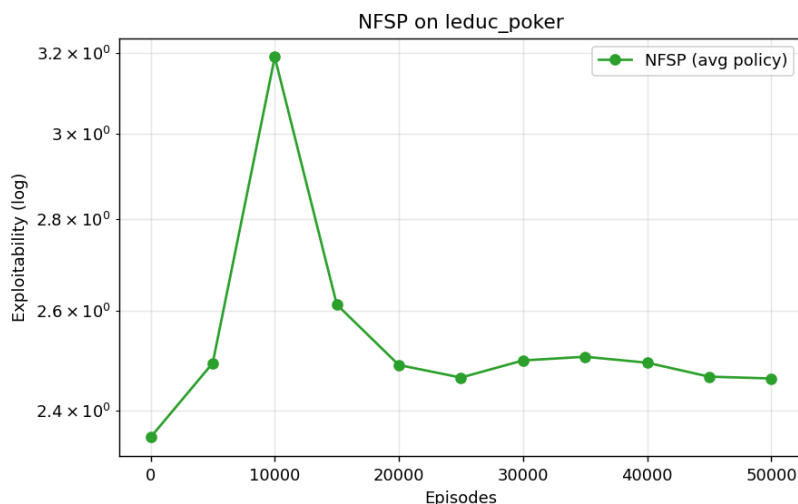
NFSP — Neural Fictitious Self-Play (Heinrich & Silver, 2016) — достига приблизително равновесие на Наш от страната на обучението с подкрепление, без никога да обхожда дървото на играта. Всеки играч поддържа две мрежи: *мрежа за най-добър отговор* (DQN), която се обучава да изчислява алчен най-добър отговор спрямо текущото поведение на опонента, и *мрежа с усреднена политика*, обучена чрез обучение с учител върху собствените минали действия на играча, която се учи на усредненото време на тези най-добри отговори. Средната стратегия е тази, която се сближава към равновесие на Наш — същото осредняване, което CFR извършва, но постигнато чрез игра, а не чрез съжаление.

Двете мрежи са свързани чрез *предварителен параметър* η : на всяка стъпка агентът действа от своята мрежа за най-добър отговор с вероятност η , а в останалите случаи — от своята мрежа с усреднена политика. Играта с най-добър отговор продължава да генерира свежи данни от разпределението, които средната стратегия усвоява, докато средната стратегия осигурява стабилен опонент, спрямо когото се изчислява най-добрият отговор. Не е необходим резервоар от обхождания на дървото — данните идват единствено от епизоди на самообучение.

Цената е ефективност на извадката. В играта ледюк изследването стартира NFSP за десетки хиляди епизоди и наблюдава колебание на експлоатируемостта около 2,5 вместо спад, тъй като достигането на използваемо равновесие дори в тази малка игра изисква порядъка на един милион епизоди или повече — собственият еталонен тест на OpenSpiel използва още по-голям брой. NFSP се мащабира до големи игри, където неговата безмоделна простота е предимство, но при еталонен тест за обучение той се сближава значително по-бавно от табличен CFR или Deep CFR.

5.3.4 Компромиси: Кога невронната CFR е от полза

Отстъпвайки назад: каква е реалната полза от невронните мрежи в CFR и каква е цената им? Ползата е обобщаване и ограничена памет — една мрежа споделя структура между сходни информационни множества и съхранява стратегия във фиксиран брой параметри, така че изчисляването на равновесие вече не изисква изброяване или дори посещение на всяко информационно множество. Цената е висок разход на итерация и по-слаби гаранции: всяка итерация обучава мрежи вместо да увеличава броячи, сходимостта е по-шумна,



Фигура 7: NFSP exploitability on Leduc over training episodes.

а чистото монотонно поведение на табличен CFR става приблизително. Резултатите от изследването в ледюк показват тази страна на компромиса при малки игри: когато таблицата се побира в паметта, тя е по-ефективна.

Компромисът се променя при увеличаване на мащаба. С разрастването на играта табличните таблици в крайна сметка изобщо не се побират в паметта, докато отпечатъкът на мрежата остава непроменен; след този преход невронната методика не е просто конкурентна, а единствената възможна. Затова Deep CFR и неговите разновидности са важни за пълномащабен покер, въпреки че губят в ледюк.

Едно структурно ограничение определя къде се прилагат невронните методи, базирани на съжаление. CFR изчислява контрафактни стойности чрез разсъждения върху цялото поддърво под едно информационно множество, което е евтино, когато дървото е *широко, но плитко* — покерът има огромен брой **раздавания**, но завършва след няколко рунда на залагане. В игри, които са *дълбоки* — със стотици последователни хода, както в една мрежова среда — това разсъждение става непосилно скъпо и безмоделните методи за самообучение от Част 4 се превръщат в по-естественото решение. Семейството, базирано на съжаление, продължава да расте в своята ниша — neuRD, например, преформулира обновяването на съжалението като стъпка на градиента на стратегията, за да комбинира стабилността на CFR с оптимизация на невронна стратегия — но неговата най-добра област на приложение остават широките, плитките игри с непълна информация.

5.4 Част 4 — Други невронни приложения

5.4.1 Четирите водещи фамилии алгоритми

Част 3 представи един от начините за прилагане на невронни мрежи в игри с непълна информация — чрез приближаване на CFR. Това е едно от четирите основни семейства, които определят съвременното равнище, като изборът между тях до голяма степен зависи от съответствието им с мащаба на играта, структурата на дървото и броя на участниците. Четирите са *базирани на съжаление* методи (Deep CFR, DREAM, neuRD) от Част 3; *популационни* методи (PSRO и неговите разновидности, включително NFSP); *търсене плюс обучение*

с подкрепление методи (Recursive Belief-based Learning, играч на игри); и *самообучение без използване на модел* (PPO, DQN, MAPPO, QMIX). Таблицата ги представя; следващите раздели разглеждат трите, които все още не са обхванати.

Семейство	Основна идея	Най-подходящо за	Цена
Базиран на съжаление (Deep CFR, DREAM, NeuRD)	Приближаване на CFR с невронни мрежи	Широки, плитки игри; неексплоатируемост	Висока
Популационен (PSRO, NFSP, XFP)	Оптимално отговаряне на нарастваща популация от противници	n-играчен; моделиране на противника	Средна
Търсене + RL (Recursive Belief-based Learning, играч на игри)	Научена стойност плюс онлайн търсене по време на извод	Игри с високи залози; изчислителна мощност по време на извод	Висока
Самообучение без използване на модел (PPO, DQN, MAPPO, QMIX)	Оптимизиране на награда чрез самообучение	Пространствени, с дълъг хоризонт (решетки)	Ниска–средна

5.4.2 Популационни методи: PSRO, NFSP, XFP

Популационните методи преформулират **намирането на равновесие** като итеративен турнир. Започвайки от малък набор от **стратегии**, се формира *мета-игра* чрез изиграване на членовете на набора един срещу друг; след това се обучава нова **стратегия** като приблизителен **най-добър отговор** на текущата **популация**, добавя се обратно към набора и процесът се повтаря. Това е идеята зад PSRO (Policy-Space Response Oracles) и неговите мащабируеми варианти като Pipeline PSRO; фиктивната **самоигра**, включително NFSP от предишната част, е специалният случай, при който **най-добрият отговор** е насочен към **усредненото време** на набора. Тъй като **популацията** представлява изричен набор от противници, а не една **подвижна цел**, тези методи естествено се разширяват отвъд двуигровите **игри с нулева сума** до *n*-игровия сценарий за **моделиране на противника**, който е предмет на тази дисертация — и именно чрез тях **alphaStar** достига нивото на гротмайстор в StarCraft II. Цената е поддържането и съхраняването на множество агенти, а гаранциите за **равновесие** са по-слаби от тези на CFR.

5.4.3 Търсене + RL: Recursive Belief-based Learning и играч на игри

Методите търсене плюс обучение с подкрепление представляват идеята на AlphaZero, адаптирана към скрита информация. В игра с пълна информация може да се извършва търсене напред от текущото състояние, тъй като то е известно; при непълна информация търсещият играч не знае действителното състояние, затова търсенето трябва да обхваща разпределение на вероятностите върху възможни състояния. ReBeL (Brown и др., 2020) прави това прецизно чрез *общодостъпното състояние на вярванията* — разпределение на вероятностите върху частната информация на играчите, предвид всичко публично — и изпълнява CFR върху малка, ограничена по дълбочина под-игра по време на игра, като научена мрежа за стойности предоставя стойностите на листата. Играч на игри (Schmid и др., 2023) обобщава този подход в един алгоритъм, който

обработка както пълна, така и непълна информация. Тези системи достигат най-силно ниво на игра, но са най-сложни за изграждане и зависят от наличието на изчислителна мощност *по време* на игра, а не само по време на обучение; те са предмет на Стъпка 6, така че тази секция само ги позиционира в общата картина.

5.4.4 Самообучение без използване на модел: PPO, DQN, MAPPO, QMIX

Методите за самообучение без използване на модел изцяло игнорират дървото на играта и оптимизират директно сигнала за награда, усъвършенствайки стратегията от епизоди на игра срещу собствени копия. Те са основните методи в многоагентното обучение с подкрепление: PPO и DQN за едноагентното ядро и многоагентните разширения MAPPO и QMIX за кооперативни и смесени сценарии. Тъй като никога не обмислят поддържането под дадено състояние, те не се затрудняват от дълбочината, която пречи на методите, основани на съжалението, което ги прави естествения избор за пространствени игри с дълъг хоризонт — мрежи от клетки, Pommerman и подобни. За да се справят с частичната наблюдаемост, те използват рекурентните и attention архитектури от Част 2, предоставяйки на агента памет за това, което мъглата е скрила. Тяхната слабост е огледален образ на тяхната сила: без концепция за контрафактивно съжаление, те не притежават вграден натиск към неексплоатируемост, така че един безмоделен агент може да бъде силен средностатистически, но все пак да остане експлоатируем от противник в най-лошия случай — което е именно причината експлоатируемостта да трябва да се измерва отделно, тема на по-късна секция.

5.4.5 Невронни мрежи за абстракция, моделиране на противника и представяне на убеждението

5.4.6 Оценяване на невронните стратегии: експлоатируемостта в мащаб

5.4.7 Обобщаване отвъд покера: среди и инструменти

5.5 Част 5 — Експериментална / граница

5.5.1 Памет и време

5.5.2 Модели на латентни убеждения

5.5.3 Бърза адаптация

5.5.4 Моделиране на чужди умове и отношения

5.5.5 Несигурност и симетрия

5.5.6 Съзнателно извън обхвата на изследването

5.6 Синтез

5.6.1 Карта на решенията: Кой метод с невронни мрежи за коя игра

5.6.2 Връзки и бъдещи насоки

Стъпка 6 Обобщение — Архитектури за изкуствен интелект в игрите от край до край

Изследване върху възможностите за прилагане на изкуствен интелект в компютърните игри

Alexander Andreev

June 2026

Съдържание

6	Стъпка 6 — Архитектури на игрови изкуствен интелект от край до край	1
6.1	Въведение	1
6.2	DeepStack (2017)	3
6.3	Либратус (2017/2018)	9
6.4	Pluribus (2019)	17
6.5	ReBeL (2020)	24
6.6	Student of Games / SoG (2023)	33
6.7	Синтез	42

6 Стъпка 6 — Архитектури на игрови изкуствен интелект от край до край

6.1 Въведение

Глава 6 е крайъгълният камък на този изследователски план. Стъпки 1–5 сглобяват отделните елементи — игровитеоретичния речник на игрите в разширена форма и равновесията на Наш, минимизирането на контрафактичното съжаление (CFR) и неговите **Monte-Carlo variants**, абстракцията на играта и невронните апроксиматори на функции, които заменят табличното съхранение, когато една игра го надрасне — и тази глава е мястото, където тези елементи се сглобяват в пълноценни системи от състезателно ниво. Подходът е умишлено **архитектурен**: вместо да се извеждат отново математическите зависимости, които са представени в техническите глави и цитираните статии, всяка система се разглежда на нивото на това как е *изградена* — какво изчислява **извън линия**, какво изчислява в **реално време**, къде е обучението и какво всъщност може да гарантира. Кратък отчет за всяка система открива всяка секция, така че същите девет измерения да се подредят с един поглед, а текстът след това проследява дизайнерските решения, инженерните компромиси и изоставените подходи зад тях.

Петте системи обхващат седем години и, прочетени в ред, проследяват еволюцията на свръхчовешкия игрови изкуствен интелект: **DeepStack** (2017), **Libratus** (2017/2018), **Pluribus** (2019), **ReBeL** (2020) и **Student of Games** (2023). Изкушаващо е да се прочете такъв списък като класация — всяка следваща по-силна от предишната — но това нито се е случило, нито така е организирана тази глава. Прогресията е последователност

от *умишлени компромиси*: всяка система придобива нова способност, като се отказва от нещо друго, а най-показателните моменти са тези, в които областта прави странична стъпка или дори отстъпление. **DeepStack** и **Libratus** са почти съвременници, които предлагат *противоположни решения* на едно и също заболяване — **DeepStack** изоставя парадигмата „абстракция и план“, която доминираше в компютърния покер близо две десетилетия, докато **Libratus** я запазва и излекува единствения ѝ фатален недостатък с решаване на под-игри в реално време, което е доказуемо безопасно. **Pluribus** след това пробива *барьерата на броя играчи*, достигайки свръхчовешка игра с шестима участници, но само като се отказва изцяло от гаранции за безопасност и остава изцяло табличен. **ReBeL** се връща към двама играчи, но прави огромен скок в обобщаването — научавайки стойности на състоянието на убеждението със самообучение в стил **AlphaZero-style Self-Play**, разтваряйки както абстракцията, така и плана, и *възстановявайки* безопасността, от която **Pluribus** се беше отказал. **Student of Games**, като завършек, обединява игра с пълна и непълна информация в един алгоритъм — и плаща за тази широта с пикова сила, губейки решително от специализиран **AlphaZero** в Го. Прочетено по този начин, главата е изследване на цената на всеки напредък, а не класация на победители.

Под отделните компромиси три оси на движение преминават през всичките пет системи и придават гръбнака на главата. Първата е **представителна**: преходът от ръчно изградена *абстракция* — групиране на подобни раздавания и допускане само на няколко размера залози — към *научено невронно приближение*, при което мрежата обобщава ситуации, които таблица никога не би могла да изброи. Втората е **времева**: преходът от *решаване на цялата игра извън линия* в съхранена стратегия към *научаване на компактен модел и търсене с него в реално време*, така че изчислението се извършва в момента на вземане на решение, а не е предварително заложено. Третата ос се появява едва в самия край, със **Student of Games**: обединяването на игра с **пълна и непълна информация** в един надежден алгоритъм, съчетаващ двете големи традиции на игровия изкуствен интелект — линията минимакс / Monte-Carlo-tree-search / AlphaZero и линията CFR / покер — които са се развивали по отделни пътища в продължение на седемдесет години. Първите две оси описват четирите покер системи; третата е това, което прави петата завършек на главата.

Една идея свързва тези нишки и заслужава да бъде назована преди самите системи, именно защото тя *не е* една от тях: **ограничено по дълбочина решаване**. Формализирано от Brown & Sandholm (2018), то е принципът, че може да се търси само малко напред в игра с непълна информация и остатъкът да се замести с *научена или предварително изчислена стойност* — при условие че заместването се извършва така, че скритата информация да не го прави невалидно. Това е теорията, която ретроспективно обединява непрекъснатото пререшаване на **deepStack** с вложеното решаване на под-игри на **Libratus**, която стои в основата на продължителните стратегии на **Pluribus** и която, във формата на състояние на убеждението, се превръща във вътрешния цикъл на **Recursive Belief-based Learning** и **Student of Games**. Ние го разглеждаме като свързваща тъкан — споменавано навсякъде, където една система го реализира — а не като шеста точка, защото ограниченото по дълбочина решаване е механизмът *чрез който* оста извън линия към търсене всъщност функционира.

Накрая, тъй като тази глава е връзката между основите на стъпки 1–5 и изследванията върху моделирането на противника и експлоатацията в стъпки 7–15, си струва още в началото да се посочи единствената нишка, към която синтезът се връща. Всяка система тук е, по умишлен дизайн, **независима от противника**: всяка изчислява стратегия, която е трудна за побеждаване *в най-лош случай* и след това я прилага без да обръ-

ща внимание кой всъщност седи срещу нея — Pluribus дори не знае самоличността на своите опоненти, а както тя, така и Libratus отказват по принцип да ги моделират или да се адаптират към тях, за да не бъдат контраексплоатирани в отговор. Този подход, поставящ устойчивостта на първо място, е голямата сила на областта и, за дисертацията относно *адаптивна* игра, нейното определящо ограничение: именно съзнанието за противник, което тези системи пропускат, стъпки 7–15 се стремят да добавят. Затова главата завършва не с победител, а със синтез — карта на това, което петте системи споделят, какво всяка от тях е жертвала и къде всъщност се намират откритите проблеми, които мотивират останалата част от тази работа.

6.2 DeepStack (2017)

DeepStack (Moravčík et al., 2017), от компютърно-покер групата на Университета на Алберта с колаборатори в Прага, беше първата програма, която побеждава професионални покер играчи в хедс-ъп безлимит покер (HUNL) със статистическа значимост и първата, която поставя евристичното търсене — двигателят зад шахмата и Го — на теоретична основа в игра с непълна информация. В рамките на четириседмично проучване тя победи група от 33 професионалисти с 492 мили-големи блайнда на игра (mbb/g, стандартната покер единица за процент победи; 50 mbb/g представлява значително професионално предимство) в 44 852 ръце и нито една известна техника не успя да открие пропуск в играта ѝ. Нейният водещ *какво-ако* въпрос е този, който рамкира цялата тази глава: **може ли програма да играе покер по начина, по който alphaGo играе Го — търсейки локално от текущата ситуация и доверявайки се на научена функция на стойността за всичко извън хоризонта — дори когато в покера самата ситуация е частично скрита?**

С един поглед	DeepStack (2017)
Играчи	2 (хедс-ъп)
Тип игра	HUNL — хедс-ъп безлимит тексаски но-лимит покер за двама (двуйграчева игра с нулева сума)
План (офлайн)?	Не — офлайн обучението е за мрежите за стойност, а не за съхранена стратегия
Невронна компонента	Deep Counterfactual Value Networks (флоп, търн, спомагателни); само стойност
Механизъм за търсене	Продължително пре-решаване (ограничено по дълбочина предвиждане с CFR при всяко решение)
Абстракция?	Няма ограничения в играта; 1000-кълъстерна кълъстеризация само на входа на мрежата, плюс оскъдни залози и групиране на действията на ривъра при предвиждането
Също и с пълна информация?	Не (само непълна информация)
Изчислителна мощ	Офлайн-интензивна (~175 процесорно-ядрени години за етикетирание на мрежата от ходове); времето за игра работи на една GPU, < 5 s/решение

С един поглед	DeepStack (2017)
Ключова иновация	Продължително пре-решаване + научени контрафактни стойности: първото <i>звуково</i> евристично търсене за игри с непълна информация

6.2.1 Затворената празнина

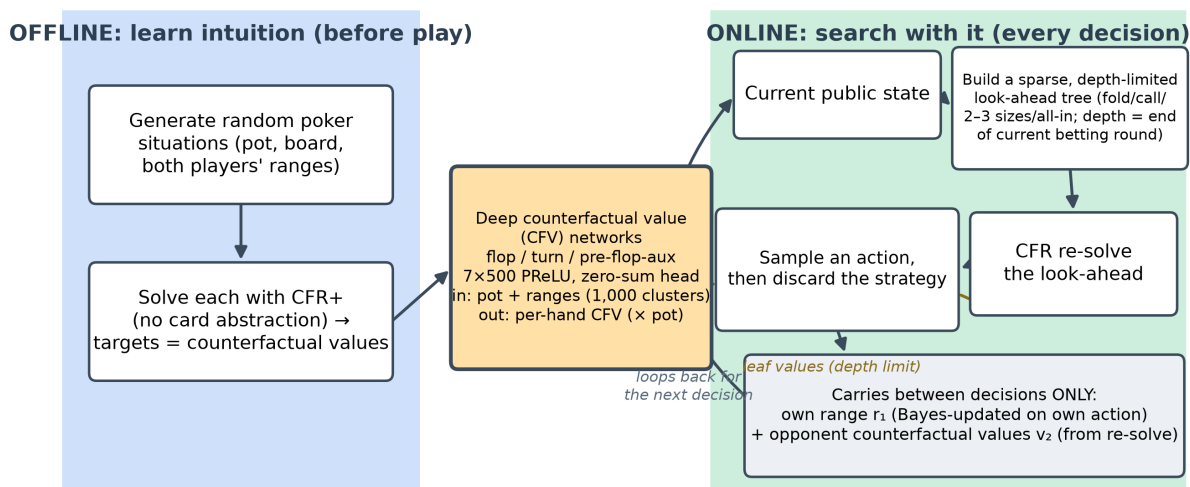
Всеки предишен етап в развитието на изкуствения интелект за игри — табла, шах, Го — се основаваше на *локално търсене*: от текущата позиция се разглеждат няколко хода напред и останалото се замества с евристична стойност. Тази рецепта предполага, че позицията е известна, а покерът нарушава това предположение — правилното действие зависи от вероятностното разпределение на скритите карти на опонента, което се разкрива единствено чрез неговите залози, а те от своя страна зависят рекурсивно от предварителните му убеждения относно вашите карти. Отвореният въпрос, който deepStack се опита да реши, е дали евристичното търсене може да бъде направено *звуково* при наличие на скрита информация в мащаба на HUNL — игра с приблизително 10^{160} решавачи точки — сравнима по обем с Го, но с непълна информация.

Почти две десетилетия доминиращият отговор беше нещо съвсем различно: **абстракция плюс офлайн равновесие плюс превод**. Свийте 10^{160} ситуацияите на HUNL до приблизително 10^{14} абстрактни такива чрез групиране на подобни ръце (*абстракция на карти*) и позволяване само на няколко размера залози (*абстракция на действията*), решете тази по-малка игра офлайн с минимизиране на контрафактното съжаление (CFR), за да произведете пълна стратегия — *план* — съхранете я и в момента на играта *преведете* всяка реална ситуация и залог на опонента към най-близката абстрактна. Компресията е загубна и загубата се проявява като експлоатируемост — колко може да спечели опонент в най-лошия случай, качественият показател на областта, нула при равновесие на Наш. През 2015 г. абстрактно-базираната програма Claudico загуби от професионалисти с 91 mbb/g, а сондаж за най-добър локален отговор (LBR, лесноизчислима долна граница на експлоатируемостта) по-късно показва, че топ състезателните ботове са експлоатируеми с повече от 3000 mbb/g — четири пъти по-зле, отколкото ако се откажеш от всяка ръка. Плановите също бяха огромни (една стратегия без абстракции на картите отне около 2 TB и 14 процесорно-ядрени години за изчисляване и все още беше *експлоатируема* чрез залози извън дървото), а стъпката на превода сама по себе си беше източник на слабост. DeepStack затваря тази празнина, като изоставя цялата конструкция: никога не изгражда абстракция на цялата игра и никога не съхранява план, разсъждавайки за всяка ситуация *така както тя действително възниква* и замествайки само далечното остатъчно продължение на играта с научена оценка.

6.2.2 Архитектура

deepStack се разделя ясно на **извън линия** фаза за изграждане на интуиция и **онлайн** фаза за търсене с нея (Фигура 6.1).

Извън линия системата генерира милиони случайни покер ситуации и ги решава с cFR решавач, за да получи целеви *контрафактни стойности* — условни „какво-ако“ печалби при държане на всяка възможна **раздаване** — и тези двойки (ситуация → стойност) обучават **дълбоките контрафактни мрежи за стой-**



Фигура 1: DeepStack architecture: offline intuition-building (left) feeding a shared deep counterfactual-value network, reused as the leaf evaluator inside the online continual-re-solving loop (right).

ности (CFV Network). Онлайн deepStack не използва план: между отделните решения той запазва само два вектора — своя *диапазон* (разпределението върху възможните раздавания, които може да държи) и *контрафактичните стойности* на опонента — а при всеки ход изпълнява cFR върху малко дърво за предвиждане, вкоренено в истинското текущо състояние, като използва CFV Network за изчисляване на стойностите на листата на границата на дълбочината. Тъй като cFR вече изчислява както диапазоните, така и контрафактичните стойности, той се вписва по естествен начин в този цикъл.

Това е евристично търсене с три компонента: надеждно изчисляване на локална стратегия (продължително пререшаване), предвиждане с ограничена дълбочина, при което научената функция на стойността замества останалата част от играта, и ограничен набор от действия за предвиждане, който поддържа всяко решаване бързо. Невронната мрежа се използва *само* на границата на дълбочината като оценител на листата; търсенето се извършва от CFR; и — най-важното — в цикъла не се прилага абстракция на цялата игра.

6.2.3 Ключова иновация: продължително пре-решаване с научени контрафактични стойности

Приносът на deepStack е в обединяването на две идеи, като всяка от тях прави другата осъществим.

Първата е **продължително пре-решаване**: възстановяване на свежа локална стратегия при всяко решение въз основа на поддържаения диапазон и вектора на контрафактуални стойности на опонента, след което тя се изхвърля, така че агентът никога не съхранява или не се ангажира с глобална стратегия. Класическото пре-решаване (Burch и др., 2014) показва, че за да се възстанови стратегия за под-игра, не е необходима цялата стратегия — достатъчни са само собственият диапазон при влизане в под-играта и векторът на контрафактуалните стойности на опонента. DeepStack избутва това до предела: то *никога* не съхранява стратегия за цялата игра. Всеки път, когато трябва да действа, то пререшава текущото публично състояние въз основа на тези два вектора, извършва едно действие и изхвърля стратегията.

Това, което прави това възможно, е воденето на отчетност. След всяко събитие deepStack актуализира своите

два вектора по прости правила: при **собствено действие** то заменя пререшените контрафактични стойности за избраното действие и извършва байесови актуализации на своя диапазон; при **случайно събитие** то заменя стойностите на тази карта и анулира вече невъзможните раздавания; а при **действието на опонента** то не прави *абсолютно нищо*. Този последен момент е тихият майсторски ход: тъй като то проследява *стойностите* на опонента, а не неговия *диапазон*, и никога не се нуждае от конкретното действие на опонента, за да поддържа тези стойности, то заобикаля стъпката на преобразуване на действие, която парализира абстракционните ботове. Опонентът може да направи всеки залог с всякакъв размер; deepStack просто пререшава от състоянието, което този залог е произвел.

Втората идея е **научена контрафактична стойност мрежа** — „интуицията“ на DeepStack. В игра с пълна информация оценител на възел съпоставя едно състояние към едно число; при непълна информация той трябва да съпостави *цяло публично състояние плюс диапазоните и на двамата играчи* към *вектор* от контрафактични стойности, по една за всяка ръка, защото стойностите в един възел се променят с диапазоните, които достигат до него. DeepStack научава това съпоставяне с директна мрежа от седем скрити слоя с по 500 единици, приемайки размера на пота и двата диапазона (компресирани в 1000 ръчни клъстера) като вход и извеждайки контрафактични стойности за всяка ръка като части от пота. Специално външно ниво налага ограничението с нулева сума: то формира двете имплицитни стойности на играта от диапазоните и суровите изходи и изважда половината от тяхната сума, така че оценките да са взаимно съгласувани и цялото нещо да остане диференцируемо. С тази мрежа, която доставя стойности в края на текущия рунд за залози, преизчисляването с ограничена дълбочина свива играта от 10^{160} решаващи точки до около 10^7 — достатъчно малко, за да бъде решена за по-малко от пет секунди на една GPU.

Сдвояването е доказуемо коректно. Ако грешката на мрежата за стойности е най-много ϵ , а преизчисляването изпълнява T итерации на CFR, експлоатируемостта на получената стратегия е ограничена от

$$\text{exploitability} < k_1 \epsilon + k_2 / \sqrt{T},$$

с игрово-специфични константи k_1, k_2 . Първият член представлява цената на несъвършената интуиция, а вторият — обичайната **сходимост на CFR**. Тази граница е теоретичното ядро на статията — гаранцията, че евристичното **търсене** може да бъде пренесено в условия на **непълна информация**, без **стратегията** да стане **експлоатируема** — и същият шаблон „**ограничено по дълбочина решаване** плюс **научена стойност**“, формализиран допълнително от Brown & Sandholm през следващата година, е в основата на всяка система в тази глава.

6.2.4 Ограничения, задънени улици и недостатъчно описаното в статията

Чистият разказ по-горе представлява основния текст, докато инженерната реалност се съдържа в приложението, където именно акцентът върху „ограниченията и еволюцията“ оправдава своето място.

Най-важната забележка е, че **внедрената система не е тази, за която важи теоремата**. За да играе с човешка скорост, deepStack ограничава своето предвиждане до ограничен набор от залози (пас, залог, два или три размера на залога, ол-ин) и в статията ясно се посочва, че това „отменя свойството на коректност на Те-

орема 1⁴. Така гаранцията на внедрената система е емпирична — подкрепена от резултатите LBR по-долу — а не доказана. Второ отклонение утежнява това: доказателството за коректност предполага стойности на *най-добър отговор*, но deepStack всъщност използва стойности от *самообучение*, които „нямат теоретично обоснование“, но са били по-малко експлоатируеми в ранните тестове. Доказаният алгоритъм и печелившият алгоритъм, строго погледнато, са различни алгоритми.

Абстракцията също така се промъква в периферията. deepStack рекламира, че не използва абстракция на карти за ограничаване на играта — но *прави* групиране на ръцете в 1000 кофи на входа на мрежата за стойности и на ривъра изцяло се отказва от мрежата, решавайки до края на играта, като използва **абстракция на действия с кофи** за постигане на управляемост. За да ускори пререшаването, тя също така прави **топли стартове** на диапазона на опонента, в консервативна комбинация през повечето време, но в агресивен вариант (когато действа първа), който „жертва гаранциите за пререшаване, когато оценката на диапазона на опонента е грешна“. И префлопът, далеч от евтин, изисква изброяване на всички 22 100 възможни флопове чрез флоп мрежата, смекчено единствено чрез кеширане на повтарящи се последователности от залагания. И накрая, изборът на приспособление за пререшаване (приспособлението CFR-D пред алтернативата с максимален марж) е направен, защото то „се представи по-добре в ранните тестове“, а не е изведено — и, отразявайки Стъпка 5, полезната дълбочина на мрежата беше ограничена от данните, а не от архитектурата, като грешката при валидиране се изравняваше след петия слой при бюджет от десет милиона проби. Нито едно от тези не подкопава резултата, но заедно те показват, че заглавието „коректно търсене без абстракции“ е стремеж, който имплементацията приближава, а не постига.

6.2.5 Изчислителна мощност и достъпност

Разходите на deepStack са почти изцяло **извън линия и почти изцяло в CFR solving**, използвано за **етикиране на мрежите за стойности** — не в обучението на невронна мрежа и не във времето за игра. Генерирането на целите означаваше приблизително решаване на милиони случайни под-игри: самата **мрежа от ходове** изразходва около **175 процесорно-ядрени години** на 6,144-ядрен клъстер, като **флоп мрежата** добави приблизително половин **GPU-година** на 20 GPU; за разлика от това, самите мрежи се обучиха за около два дни всяка на едно GPU и по време на игра deepStack работи на **едно стандартно GPU с под пет секунди на решение**. Формата на тази сметка е истинският урок: „интуицията“ се купува веднъж, предварително, чрез грубо **намиране на равновесие**, след което внедряването е евтино — огледален образ на система, чиито разходи са доминирани от **извод**. По отношение на достъпността това постави *от нулата* изграждането в обсега само на добре ресурсирана лаборатория по онова време (**предварителната цена** е с мащаб на клъстер), въпреки че обучената система след това работеше на хардуер, който всеки ентузиаст притежава; десетилетие по-късно тази **предварителна цена** е много по-постижима на стандартни **облачни изчисления** и с по-бързи модерни решаващи устройства, а публична референтна имплементация на **ледюк** допълнително намалява бариерата за влизане.

6.2.6 Силни страни и ограничения

Централната сила на deepStack е **коректност с ниска експлоатируемост**: това е първият метод за търсене при непълна информация с реална гаранция, като емпирично LBR — което излага състезателните ботове

като губещи хиляди mbb/g — не успява да намери начин да го победи, а самият той печели над 350 mbb/g. Той **не изисква пълна абстракция на играта и преобразуване на действие**, така че залозите на опонента извън дървото се обработват точно, без закръгляне; неговата **времеемкост на играта е скромна**; и той се обучава **без човешки данни и с минимални експертни познания**. Допълнително предимство е оценъчната синергия — функцията на стойността на deepStack съвпада с изискванията на оценяващия метод за намаляване на дисперсията AIVAT, като намалява стандартното отклонение в човешките проучвания с 85% и прави постигането на значимост възможно само в няколко хиляди ръце.

Ограниченията определят дневния ред за останалата част от главата. DeepStack е **само двуигрови игра с нулева сума**; неговата **коректност** се основава на тази структура. Той запазва **рядка абстракция на залозите** при предвиждането и **абстракция на действията** на ривъра, както и наследява **грешка на приближението на мрежата за стойности** (член $k_1 \epsilon$), най-голяма на флопа, където разчита най-много на мрежата. Той плаща **високата предварителна цена**, отбелязана по-горе и **преизчислява от нулата** при всяко решение, като е валидиран единствено срещу хора и LBR — никога в директен сблъсък срещу най-силните абстрактни ботове.

6.2.7 Наследство и съвременна значимост

Отстранете специфичните за покера детайли и основната идея на DeepStack е **търсене с ограничена дълбочина с научен модел на стойност в листата, адаптиран към скрита информация** — аналогът при непълна информация на търсенето с насочване по стойност зад AlphaGo и AlphaZero. Тази идея не остаря и не беше забравена; тя се превърна в шаблон. Brown & Sandholm формализираха **теорията на решаването с ограничена дълбочина** през следващата година, а парадигмата след това беше обобщена в пълни **рамки за обучение с подкрепление и търсене**: ReBeL (2020) преформулира научаването на стойността на листата около публични състояния на убеждения със **AlphaZero-style самообучение**, а Student of Games (2023) — който споделя няколко автори с DeepStack — обедини **игра с пълна и непълна информация** в един алгоритъм. Същата „рецепта за **планиране в цикъл с научен модел на стойност**“ по-късно достигна отвъд **двуигровия покер с нулева сума**, например в човешкото ниво на игра на Diplomacy на CICERO. Още по-широко, DeepStack е ранен и чист пример за принципа, който сега е централен за водещия изкуствен интелект: **изразходвайте изчислителна мощност във времето на вземане на решение чрез търсене, насочвано от научена интуиция, вместо да вграждате всичко в една гигантска предварително изчислена стратегия** — същата теза за „**тест-тайм изчислителна мощност**“ зад днешните **модели с възможност за мислене**, с уточнението на самата област (изказано от Noam Brown), че непълната информация се нуждаеше от *осъзнато убеждение* търсене именно защото обикновеното **търсене в дърво на Монте Карло**, което работи за Го, не работи за покер.

Няколко конкретни подсистеми остават използвани и днес: самият **шаблон „ограничена дълбочина плюс научена стойност на листо“**; **диференцируемата глава за изходна нулева сума** за двуигрови мрежи за стойности; **намаляване на дисперсията в стил AIVAT**, което превръща научен модел на стойност в контролна променлива за нискодисперсна оценка на всеки стохастичен агент; и малкият, но преносим трик да се изразяват стойностите като **части от пота** за обобщаване, инвариантно спрямо мащаба. Това, което е наистина остаряло, е ръчно проектираната техника *около* идеята — **клъстеризацията k-средни** със 1000 кофи,

отделните мрежи за всеки рунд, ръчно настроеното пререшаване — които по-общите научени представяния на ReBeL и Student of Games заменят. Честната оценка: **като внедрена покер система DeepStack е остаряла, но далеч не е просто стъпало** — нейната централна парадигма спечели и сега е основна; а за тази конкретна дисертация нейното състояние (диапазон, контрафактични стойности на опонент) и границата на разпространение на грешката са директни семена за базирано на убеждения моделиране на опонента (**принос 1**) и анализ на безопасното използване (**принос 2**).

6.3 Либратус (2017/2018)

DeepStack отговори на своето *какво-ако*, като изостави парадигмата „абстракция и план“, но все пак запази рядка абстракция на залозите в собственото си предвиждане и никога не беше тестван в директен сблъсък срещу най-силните предварителни ботове или срещу човешки играчи в дълъг, строг мач. Libratus (Brown & Sandholm, 2017; Carnegie Mellon) — на латински *балансиран*, както в приближаване към равновесие на Наш, и *настойчив*, за своята игра — беше създаден независимо и обявен същата година, и направи противоположния залог: запази парадигмата „абстракция и план“ и излекувай нейната единствена фатална болест. Неговият водещ въпрос е огледален образ на този на DeepStack: **какво ще стане, ако решим груб план на цялата игра извън линия, след това го поправим в реално време там, където абстракцията е твърде груба — с доказуема гаранция, че поправката никога няма да ни направи по-експлоатируеми?** През януари 2017 г. Libratus стана първата програма, която победи най-добрите човешки играчи в дълъг мач, побеждавайки четирима професионалисти с 147 мили-големи блайнда на игра (мили-големи блайнда на игра, единицата за процент победи от предишния раздел; ~50 мили-големи блайнда на игра е значително професионално предимство) в 120 000 ръце при 99,98% значимост — и, за разлика от DeepStack, първо демонтира предишния най-добър изкуствен интелект за покер в директен сблъсък.

С един поглед	Libratus (2017/2018)
Играчи	2 (хедс-ъп)
Тип игра	HUNL — хедс-ъп безлимит покер за двама (игра с нулева сума между двама играчи)
План (извън линия)?	Да — абстрахирана стратегия за цялата игра, решена извън линия с MCCFR (подробна в началото, груба в края)
Невронна компонента	Няма — чисто табличен CFR / абстракция (без невронни мрежи никъде)
Механизъм за търсене	Вложено безопасно решаване на под-игри (решаване в реално време на късната игра с CFR+ и преизпълнение за всеки залог на противника извън дървото)

С един поглед	Libratus (2017/2018)
Абстракция?	Да — абстракция на карти (търн/ривър, само в плана) + асиметрична абстракция на действията; разтворена до <i>без абстракция на карти</i> вътре в под-игрите в реално време
Също и с пълна информация?	Не (само непълна информация)
Изчислителна мощ	Тежка извън линия и онлайн: ~25 милиона часа ядро на процесор на свръхкомпютъра Bridges; ~50 възела и десетки секунди за всяко късно решение; без GPU
Ключова иновация	План + вложено в реално време <i>безопасно</i> решаване на под-игри + самоусъвършенстване: точни, доказуемо безопасни отговори на залози извън дървото вместо преобразуване на действие

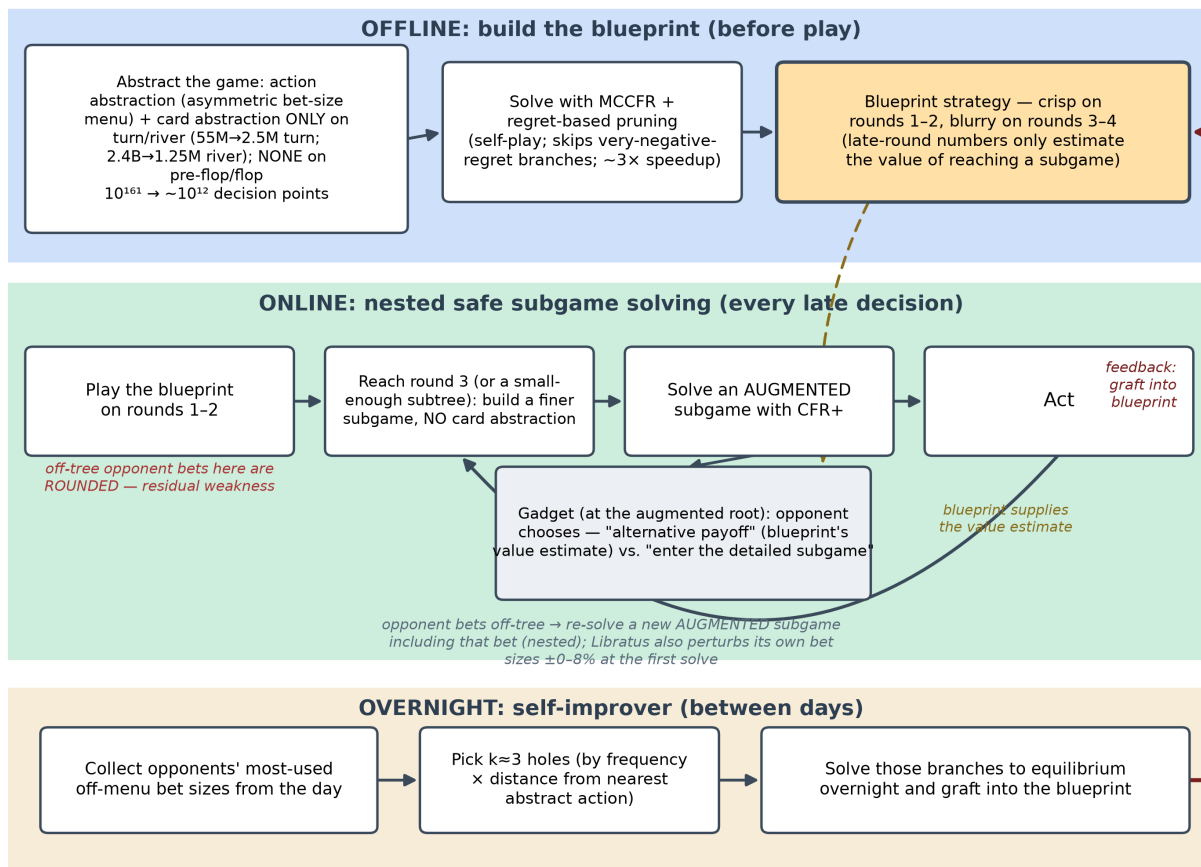
6.3.1 The gap it closed

Предишният раздел очерта парадигмата, която **DeepStack** отхвърли — абстракция плюс офлайн равновесие плюс превод — и посочи нейния фатален недостатък. **Libratus** се насочва точно към този недостатък, без да отхвърля парадигмата. Планът се решава предварително върху *фиксирано* меню от размери на залози; по време на игра всеки залог на противника, който не е в това меню, се *превежда* — закръгля се до най-близкия размер, който ботът познава — след което ботът отговаря така, сякаш противникът е направил закръгления залог. Това закръгляне е единствената най-голяма експлоатируема слабост в абстрактно-базирания покер: сондаж за най-добър локален отговор показва, че водещите състезателни ботове губят хиляди мили-големи блайнда на игра от най-лошия възможен противник, а през 2015 г. предшественикът на **Libratus** — **Claudico** — загуби първия мач *Мозъци срещу III* срещу професионалисти с 91 мили-големи блайнда на игра, отчасти защото опонентите можеха да усетят и накажат неговите граници на превод.

Концептуалният въпрос, на който **Libratus** отговаря, следователно е по-тесен и по-целенасочен в сравнение с този на **deepStack**: *може ли парадигмата на абстракцията, съществуваща от десетилетия, да бъде издигната до свръхчовешко ниво чрез поправка единствено на поведението ѝ в реално време — а именно точното отговаряне на залози извън менюто вместо чрез закръгляне — и може ли тази поправка да се извърши с гаранция, че локалните корекции никога не увеличават глобалната експлоатируемост?* Отговорът е тримодулната архитектура по-долу. Докато **deepStack** премахна напълно плана и разчиташе на научена оценъчна функция, **Libratus** запазва плана като евтин скелет и добавя решавач в реално време, който, когато противникът излезе извън дървото, изгражда и решава по-фина под-игра *съдържаща действителния залог* и вплита резултата обратно в плана. Същата година тогава предложи две противоположни решения на един и същ проблем — едно невронно и без план, друго таблично и базирано на план — и **Libratus** е доказателството, че по-старата парадигма, правилно поправена, все още беше достатъчна, за да достигне свръхчовешка игра.

6.3.2 Архитектура

Libratus представлява конвейер от три модула, които функционират на три различни времеви мащаба — **извън линия** (преди мача), **онлайн** (при всяко вземане на решение) и **през нощта** (между отделните дни на игра) — и, за разлика от всички останали системи в тази глава, той не съдържа **никаква невронна мрежа** (Фигура 6.2).



Фигура 2: Libratus's three-module pipeline: an offline blueprint (top), the online nested safe-subgame solver that plays it (middle), and the overnight self-improver that grafts solved holes back into the blueprint (bottom, feedback arrow).

Модул 1 — планът (извън линия). Libratus първо компресираща приблизително 10^{161} решаващи точки на HUNL до около 10^{12} чрез два вида абстракция: *абстракция на действията*, която запазва само дискретно меню от размери на залозите (главно кръгли дробни и кратни на пота, взети от размерите, предпочитани от водещите конкурентни ботове, с няколко ранни размера, настроени чрез алгоритъм за оптимизация на параметри), и *абстракция на карти*, която групира стратегически сходни ръце. От решаващо значение е, че **не се използва абстракция на карти в първите два рунда на залагане** — достатъчно малки, за да се позволи пълна резолюция — а се групират само търнът и ривърът, и то дори там единствено в плана. След това тази абстрактна игра се решава чрез самообучение с усъвършенстван метод **Монте Карло за минимизиране на контрафактичното съжаление (MCCFR)**, който вероятно премахва клонове с много отрицателно съжаление, което представлява приблизително трикратно ускорение и също така тихо отстранява патология

на абстракциите с непълно извличане, при която няколко различни ситуации споделят стратегията на една кофа и „се борят“ за нея. Резултатът е **планът**: пълна, но неравномерна стратегия — детайлна в началото, груба в края — чиито стойности от късните рундове се използват не за игра, а единствено за *оценяване на стойността на достигане до под-игра*.

Модул 2 — вложено безопасно решаване на под-игри (онлайн). Libratus използва плана само в ранните кръгове. При достигане на третия рунд за залози — или всяка по-ранна точка, където останалата част от ръката е достатъчно малка — той изоставя грубия план за късната игра и вместо това **изгражда нова, по-финозърнеста под-игра без абстракция на карти и я решава в реално време** с високо оптимизиран CFR+ (бърз, детерминистичен вариант на CFR). Така всяка ръка се играе индивидуално в късната игра. Определящата характеристика е това, което се случва, когато противникът направи залог, който не фигурира в менюто на плана: вместо да го закръгли, Libratus решава нова под-игра, която *съдържа точно този залог*, и повтаря това за всяко последващо извънсписъчно действие — *вложено* решаване на под-игри. Този модул е обект на разглеждане в следващия подраздел.

Модул 3 — самоусъвършенстващият се (през нощта). Тъй като решаването в реално време се пропуска в първите два рунда (решаването толкова рано само по себе си би изисквало тежка абстракция), нестандартните залози на противника *там* все още се закръглят. Самоусъвършенстващият се стеснява този остатък между дните: той събира най-често използваните от противниците нестандартни размери на залозите, избира няколко от най-вредните пропуски, изчислява правилни игровотеоретични отговори на тях през нощта и присажда тези клонове в плана. Нарочно това **не е експлоатация на противника** — той никога не се опитва да моделира и наказва грешките на хората (което би изложило Libratus на контраексплоатация). Вместо това използва залозите на противниците единствено като насока *кои от собствените пропуски на Libratus да закръпи*, а закръпванията са универсални, подобрявайки играта срещу всеки бъдещ противник.

Класическите градивни елементи от по-ранните стъпки са на видно място: **абстракция на играта** и MCCFR изграждат **плана** извън линия, CFR+ извършва **решаването на под-игра** онлайн, а „търсене“ представлява **преизчисляване в реално време** за всяко решение. Забележителното отсъствие — липсата на **мрежа за стойности**, на **мрежа на стратегията** и на каквото и да е научено чрез **градиентен спуск** — е архитектурният подпис, който отличава Libratus от всяка друга система в тази глава.

6.3.3 Ключова иновация: вложено безопасно решаване на под-игри

Централният принос на Libratus е да направи **решаването на под-игра в реално време едновременно безопасно и вложено** — достатъчно силно, за да победи хората, и доказано неспособно да направи стратегията значително по-експлоатируема от плана, който усъвършенства. И двете прилагателни са важни и всяко от тях коригира конкретно **предварително убеждение** за неуспех.

Трудността е в това, че под-игра с непълна информация *не може да бъде решена изолирано*. В шаха, когато се достигне крайна игра, тя може да бъде решена самостоятелно, тъй като дъската е напълно известна. В покера правилната стратегия за една под-игра зависи от стратегиите в *други, недостижими* под-игри, защото именно те определят разпределението на вероятностите за скритите раздавания, които противникът може да държи тук. По-ранните решаващи системи в реално време заобикаляха това, като приемаха, че противникът

е следвал плана до този момент — **несигурно** решаване на под-игра — което противникът може лесно да накаже с отклонение, понякога произвеждайки усъвършенствана стратегия значително *по-лоша* от плана.

Libratus премахва това предположение с едно малко **приспособление**, *допълнената под-игра*. В корена на под-играта на противника се предоставя избор за всяко раздаване, което той може да държи: или да вземе фиксирана **алтернативна печалба** — оценката на плана за това колко струва това раздаване тук — или да *влезе* в детайлната под-игра и да я изиграе. Решаването на тази допълнена игра принуждава усъвършенстваната стратегия на Libratus да направи противника **не по-добре от тази оценка за всяко възможно раздаване**, което е точното значение на „безопасен“: локалната корекция не може да бъде експлоатирана спрямо плана, независимо какво прави противникът. Libratus изостря това с вариант, който авторите наричат **Estimated-Maxmargin** — той максимизира *най-малкия* резерв за безопасност във всички раздавания на противника — и с две усъвършенствания, които го правят силен, а не просто безопасен: той използва оценки на плана за стойностите на противника вместо консервативни **горна граница** (по-малко предпазлива игра) и *придава по-малка тежест на раздаванията, които противникът би могъл да държи само ако е допуснал грешка по-рано* (безсмислено е да се защитава срещу раздаване, което рационален противник никога не би довел до тази ситуация), освобождавайки капацитет за защита срещу раздаванията, които той реалистично може да има.

Комбинацията идва с гаранция, която е паралелна на тази на deepStack. Ако σ^* е най-малко експлоатируемата стратегия, която се различава от плана само в рамките на решените под-игри, а оценката на плана за стойностите на подиграта на противника има отклонение не повече от Δ , тогава експлоатируемостта на усъвършенстваната стратегия се подчинява

$$\text{exploitability}(\sigma_{\text{refined}}) \leq \text{exploitability}(\sigma^*) + 2\Delta.$$

Членът 2Δ представлява цената на несъвършените оценки на стойностите и е структурно аналогичен на члена $k_1\epsilon$ в deepStack — и двата определят колко една грешка в стойностите, подадени на решавача, може да повлияе върху експлоатируемостта в най-лошия случай. По този начин двете системи стигат до един и същи вид твърдение за коректност, но от противоположни посоки: ϵ при deepStack е грешката на приближението на невронна мрежа при границата на дълбочината; Δ при Libratus е грешката на оценката на таблична схема при границата на под-играта.

“Вложено” е втората половина. Вместо да разширява менюто за залози на плана (което би довело до експлозия на извънлинейното решаване), Libratus създава **отделен отговор на всеки нестандартен залог в реално време**: когато противникът направи залог с размер, който не е бил изброяван преди, той изгражда допълнена под-игра, чиято алтернативна печалба е най-доброто *в менюто* действие, което противникът би могъл да предприеме вместо това, решава я и го прави отново за всеки следващ нестандартен залог по време на ръката. В контролираните експерименти това превъзхожда предишния стандарт — закръгляне на залога до най-близкия размер в менюто — с повече от порядък по отношение на експлоатируемост в най-лошия случай (119 срещу 1465 mbb/g в докладвания тест с малки игри). И тъй като всяка под-игра се решава в движение, Libratus може също така да *променя собствените си размери на залозите* между ръцете — той

ги възмущава със случайна стойност от 0–8% при първото решаване — така че хората никога не се сблъскват с фиксирана цел за анализ. Това е точният механизъм, чрез който най-слабото място на парадигмата на абстракцията, преобразуването на действие, се отстранява от късната игра.

6.3.4 Ограничения, задънени улици и какво е подценено в статията

Както при deepStack, ясната тримодулна структура прикрива къде всъщност се крие инженерната работа — и откровеността относно ограниченията — с изключение на това, че тук голяма част от нея дори не е в статията в *Science*, а в придружителната статия в IJCAI и в лекционните бележки на авторите.

Най-разкривателното ограничение е, че **само планът не е свръхчовешки — той дори не побеждава предишния бот**. Срещу Baby Tartanian8, победителя от състезанието през 2016 г., суровият план на Libratus губи с 8 mbb/g; едва когато вложено решаване на под-игри беше включено, същата система спечели с 63 mbb/g. С други думи, предварителната стратегия е скеле и по същество цялото предимство на Libratus идва от търсене в реално време — резултат, който тихо преформулира цялата система и предвещава завъртането на областта към търсене при извод.

Няколко други важни детайла си струва да се отбележат. **Преобразуването на действие не е премахнато, а само ограничено**: в първите два рунда на залагане Libratus все още закръгля залозите на опонента извън менюто, а целият модул за самоусъвършенстване съществува, за да работи върху този остатъчен проблем, по три „дупки“ на нощ, без никога да го отстранява напълно. **Безопасното решаване на под-игри трябва-ше да бъде преработено, за да стане изобщо приложимо** — в продължение на три години се смяташе за непрактично, тъй като в директен сблъсък учебниково-безопасното решаване губеше от теоретично необоснования *небезопасен* вариант; използването от страна на Libratus на стойностни *оценки*, а не консервативни *горна граница*, заедно с намаляването на значението на ръцете, които опонентът би държал само по грешка, направи безопасността конкурентна и дори тогава Libratus умишлено използва *небезопасен* метод за решаване веднъж, при първото си влизане в третия рунд, защото там е по-евтин и емпирично достатъчно добър. Изборът на **Estimated-Maxmargin** сам по себе си жертва малко теоретична чистота за сметка на сила: използването на оценки вместо горна граница може по принцип да повиши експлоатируемостта *над* тази на плана, ограничена единствено от 2Δ на теоремата. А абстракцията с непълна памет в основата означава, че няколко различни ситуации споделят една стратегия и „се борят“ за нея — известно изкривяване, което подрязването, базирано на съжаление, само частично смекчава. И накрая, **изчислителната мощ почти отсъства от основната статия**: водещата публикация по същество не дава никакви данни, а истинската цена (следваща) излиза наяве едва във вторичните източници, както и фактът, че **кодът никога не е бил публикуван**, оставяйки независимата проверка да се основава единствено на публикувания псевдокод.

6.3.5 Изчислителна мощност и достъпност

Където сметката на deepStack се плаща почти изцяло извън линия, а играта му е евтина, Libratus е **скъп и в двата края**. Проектът е консумирал приблизително **25 милиона часове ядро на процесор** на свръхкомпютъра Bridges в Pittsburgh Supercomputing Center за една година — от които около 6 милиона са отишли за изграждане и решаване на плана, около 3 милиона за решаване на под-игри в реално време по време на мача,

около 3 милиона за самоусъвършенстващия се модул и останалите ~13 милиона за експериментални опити и оценка. Оперативно планът е работил като е заемал приблизително 195 възела за един до осем седмици наведнъж; всяко **решаване на под-игра в реално време е използвало около 50 възела и е отнемало порядък на десетки секунди**; самоусъвършенстващият се модул през нощта е работил върху до няколкостотин възела в продължение на часове; а стратегиите и моментните снимки са консумирали около 2,6 петабайта дисково пространство. **Не е имало gPUs и обучение на невронна мрежа никъде** — всеки час ядро е CFR или абстракция.

Формата на тази сметка носи поуката. Libratus не постига евтино внедряване чрез еднократно скъпо обучение, както прави deepStack; неговата *изчислителна мощност по време на игра* сама по себе си е в мащаба на свръхкомпютър, защото силата произтича от решаването на нова под-игра при почти всяко късно решение. От гледна точка на достъпността това направи Libratus, както беше внедрен, по същество невъзможно да бъде възпроизведен извън голям център за свръхкомпютри както по време на обучението, така и по време на играта — много по-висока летва от агента на deepStack с една GPU — а липсата на публикуван код подсили това. Основните техники са независими от приложението и работят значително по-евтино на съвременния хардуер и решаващите програми днес, но артефактът от 2017 г. беше демонстрация на това, което е възможно при наличието на изобилна изчислителна мощност, а не на това, което е широко достъпно.

6.3.6 Силни страни и ограничения

Сигналната сила на Libratus е **решителна, строго демонстрирана свръхчовешка игра**. Той е първият изкуствен интелект, който побеждава водещи специалисти по HUNL в дълъг мач — 147 mbb/g за 120 000 ръце при 99,98% значимост, като побеждава всеки от четиримата професионалисти индивидуално — и, за разлика от deepStack, той също така побеждава **предходния най-добър изкуствен интелект за покер в директен сблъсък** с 63 mbb/g, отделяйки своята сила от въпроса за човешкото умение. Основната му техника — **вложено безопасно решаване на под-игри** — е едновременно **доказуемо безопасна** (2Δ границата) и **емпирично с порядък по-малко експлоатируема** от преобразуването на действие, което заменя, като не използва **човешки данни и никакви специфични за домейна знания**. Модулният дизайн сам по себе си е принос: евтин план, решаващ в реално време, който го коригира там, където е необходимо, и нощен цикъл, който закръпва собствените му слабости — всеки модул компенсира недостатъка на друг — като цялата система е с **устойчивост на първо място**, отказвайки да експлоатира опонентите, за да не бъде контраексплоатиран.

Ограниченията са също толкова ясни и подготвят останалата част от главата. Libratus е **само за игра за двама с нулева сума**; неговите гаранции за безопасност се опират на тази структура. Той остава **абстрактно базиран** и все още **преобразува залози извън менюто в ранните рундове**, което представлява експлоатируема слабост, която самоусъвършенстващият се агент единствено стеснява. Той не използва **никакво невронно обобщаване**: нищо не се прехвърля между ситуациите, планът е огромен (петабайти от съхранена стратегия), а времето за игра е в **мащаба на свръхкомпютър**, а не едно GPU, както беше необходимо на deepStack. Компромисът на estimated-Maxmargin допуска малко, ограничено повишаване на експлоатируемостта в името на по-голяма сила; едно небезопасно решаване се промъква в конвейера и — тъй като както хората, така и изкуственият интелект се адаптираха по време на мача — дори значимостта в заглавието е,

стриктно казано, „сякаш-независима“ стойност, въпреки че предимство от 147 mbb/g за 120 000 ръце не оставя никакво реално съмнение.

6.3.7 Наследство и съвременна значимост

Извадете абстракционния механизъм и трайната идея на Libratus е **търсене в реално време, насложено върху груба предварително изчислена стратегия, направено безопасно при скрита информация**: решавайте локалната ситуация точно в момента на вземане на решение, в съответствие с евтин глобален план, и реагирайте на това, което опонентът действително прави, а не на закръглено приближение на него. Тази идея не само оцеля — тя се превърна, заедно с непрекъснатото пререшаване на deepStack, в половината от основата, върху която е изградена останалата част от главата. Вложеното безопасно решаване на под-игри на Libratus е пряк предшественик на търсенето в реално време в **Pluribus** (2019), то е обединено с пререшаването на deepStack чрез теорията за **ограничено по дълбочина решаване**, публикувана от авторите през следващата година, и същият модел „усъвършенствайте глобална оценка на стойността с локално решаване“ се появява отново, сега с *научени* стойности, заменящи табличната схема, в **ReBeL** (2020).

Libratus е също така, в точен смисъл, **високата граница на парадигма, която след това полето изостави**. Той постигна свръхчовешка игра **без никакво дълбоко обучение** — чиста абстракция и CFR — и въпреки това последващите системи са всички невронни, защото ръчно създадената абстракция и петабайтовите планове нито се обобщават между ситуациите, нито се мащабират отвъд двама играчи. Честната оценка е, че Libratus е *надминат като архитектура, но оправдан като теза*: неговият залог, че **търсенето в реално време има по-голямо значение от по-голяма предварително изчислена стратегия**, беше напълно точен, дори когато залогът му върху табличната абстракция беше надминат. Тази първа теза оттогава се превърна в централна тема на границата на изкуствения интелект — наблюдението, че добавянето на търсене във време на тестване струваше далеч повече от мащабирането на извънлинейното изчисление, е ранен, конкретен пример за аргумента „изчислителна мощност по време на тестване“, който сега се прави за модели с възможност за мислене, а разделянето план-тогава-търсене на Libratus предвещава съвременната рецепта предварително обучение и търсене.

Няколко подсистеми остават директно използвани: **безопасно решаване на подучастъци / ендшпил** като начин за локално усъвършенстване на глобална стратегия *без да се нарушават нейните гаранции*; **подпомагащият подигра** — чистата конструкция „вземете известна алтернативна стойност или влезте в подучастъка“ за закрепване на локално решение към глобална оценка на стойността; **подрязване, базирано на съжаление** за мащабиране на CFR; и моделът на **самоусъвършенстващ се**, който използва наблюдавана игра, за да открие и *поправи собствените си слабости*, вместо да експлоатира съперника. Това, което наистина е остаряло, е заобикалящата го структура — ръчно настроената **абстракция на действията** и картите, много-петабайтовата **таблична схема** и внедряването с клъстер по време на игра — всички от които се разтварят чрез научените представяния на по-късните системи. За тази теза конкретно Libratus допринася с две основи: неговата граница на експлоатируемостта при безопасно решаване на подучастъци е шаблон за **безопасна експлоатация при грешка в стойността (Принос 2)**, а умишленият му отказ да експлоатира — „поправете собствените си слабости, не моделирайте съперника“ — е точният контрапункт, спрямо който може да бъде дефинирана *ограничена, целенасочена адаптация към съперника* (Принос 1).

6.4 Pluribus (2019)

Libratus реши двуигровия No-Limit Hold'em, но неговите гаранции за безопасност — и всъщност самото значение на „решаването“ на играта — се основаваха на структурата на игра с нулева сума за двама играчи: там равновесието на Наш е непобедимо и решаването в реално време може да бъде доказано *сигурно* спрямо него. Холдемът, както хората всъщност го играят обаче, събира шестима. Pluribus (Brown & Sandholm, 2019; Carnegie Mellon и Facebook AI) се изправи директно пред многопотребителския въпрос: **какво се случва с рецептата план-плюс-търсене в реално време, когато премахнете двуигровия патерица и седнете на маса за шестима — обстановка, в която равновесието на Наш не е нито уникално, нито ефективно изчислимо, нито дори гаранция, че няма да загубите?** Отговорът му беше емпиричен и категоричен. В два формата — петима професионалисти, седнали с едно копие на Pluribus, и един професионалист срещу пет копия — той победи въртящ се състав от тринадесет елитни професионалисти, няколко от които шампиони от Световните серии или Световния покер тур, печелейки около **48 мили-големи блайнда на игра** срещу петима души наведнъж (mbb/g, мерната единица за процент победи от предишните раздели — приблизително пет големи блайнда на сто ръце, решаващ шестпотребителски марж) при 95% статистическа значимост. И го направи след обучение в продължение на **осем дни на един 64-ядрен сървър за около \$150 облачни изчисления** — с порядък хиляда по-малко от това, което свръхкомпютърът зад Libratus консумираше.

С един поглед	Pluribus (2019)
Играчи	6 (шестмакс) — първият свръхчовешки изкуствен интелект в която и да е референтна игра с повече от двама играчи/отбора
Тип игра	6-max NLHE — шестпотребителски No-Limit Texas Hold'em (непълна информация; многопотребителски, <i>не</i> двуигрови с нулева сума)
План (офлайн)?	Да — цялостен план за играта, решен офлайн чрез Linear MCCFR; използва се <i>директно</i> само в първия рунд за залози, след това служи като скеле
Невронна компонента	Няма — изцяло табличен CFR / абстракция (без невронни мрежи никъде, както при Libratus)
Механизъм за търсене	Търсене в реално време с <i>ограничена дълбочина</i> с $k=4$ стратегии за продължение в листата (вложени <i>несигурни</i> решавания, пререшаване от началото на всеки рунд за залози); Linear CFR вътре в под-игрите
Абстракция?	Да — действия (1–14 размера на залозите в плана; 1–6 при търсене) + информационна абстракция (без загуби в първия рунд; с загуби в по-късните, по-фини при търсенето, отколкото в плана)
Също и пълна информация?	Не (само непълна информация)

С един поглед	Pluribus (2019)
Изчислителна мощ	Прочуто евтин: план ~12 400 ядро-часа / 8 дни / < 512 GB на един 64-ядрен сървър (~\$144); игра на 2 процесора (28 ядра), < 128 GB, без GPU, 1–33 s/решение
Ключова иновация	Свръхчовешка шестпотребителска игра чрез план + търсене с ограничена дълбочина със стратегии за продължение — спечели <i>емпирично</i> , с минимален бюджет, без гаранция за безопасност при n играчи

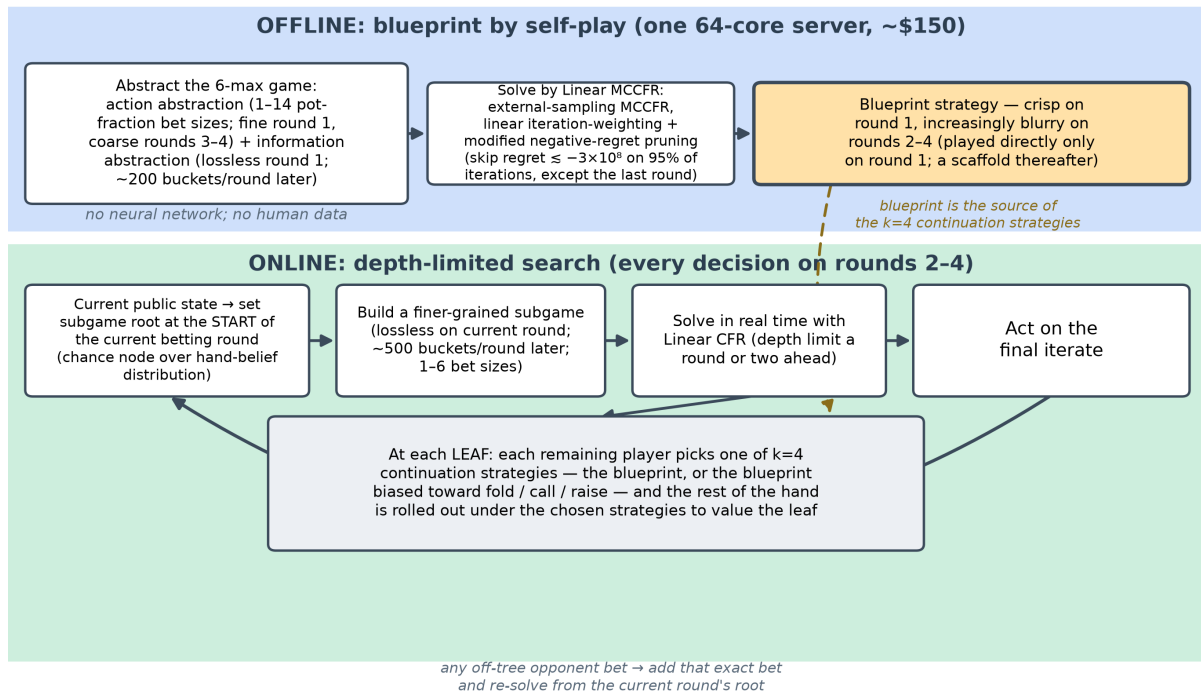
6.4.1 Пропуските, които запълни

Всеки свръхчовешки игрови изкуствен интелект преди Pluribus — дама, шах, Го и двете предходни покер програми — споделяше едно скрито предположение: двама играчи, нулева сума. В тази обстановка равновесието на Наш носи свойство, което го прави очевидната цел: всеки играч, който възприеме такова, е *гарантирано, че няма да загуби* в очакване, независимо какво прави противникът, и ако двама играчи независимо изчислят равновесия, техните стратегии все още се комбинират в равновесие. „Решаването“ на такава игра следователно *означава* приближаване на равновесие на Наш, а deepStack и Libratus бяха, по същество, два начина да направят точно това. Шестпотребителският покер разтваря всичко това. При повече от двама играчи намирането — дори приближаването — на равновесие на Наш е изчислително неразрешим проблем като цяло; обикновено има *много* равновесия; и, фатално, ако всеки играч независимо избере едно, получената съвместна стратегия може да не бъде равновесие изобщо, така че играчът може да *загуби*, докато играе безупречна равновесна стратегия. Brown и Sandholm илюстрират това с играта „Щанд за лимонада“, в която играчите се разполагат около пръстен: има безкрайно много равновесия и независимо избраните рядко се съчетават. Заключение е рязко — в многопотребителския покер равновесието на Наш *не е нито уникално, нито гаранция за безопасност*, а дефиницията на успеха в тази област от две десетилетия просто изчезва.

Pluribus затваря тази празнина, като променя целта и след това я постига. Вместо да преследва концепция на решение, на която не може да се довери, тя цели единствено *емпирично и последователно да побеждава елитни хора*, и предварително приема, че нейните алгоритми не носят гаранция за сближаване към равновесие извън играта за двама с нулева сума. Тази отстъпка е цялата същност — и това е точната празнина, която принос 2 на тази дисертация се стреми да запълни. Другата половина на празнината е механична. Libratus постигна свръхчовешка игра, като решаваше всяка под-игра в късната фаза *докрай* в реално време; с шестима играчи под-игрите експлодират експоненциално и решаването докрай става непрактично, докато алтернативата на deepStack — стойности на листата, обусловени от разпределения на убеждения — вече беше скъпа в двуйгровия покер и се влошава с увеличаване на броя играчи. Следователно Pluribus се нуждаеше от търсене в реално време, което да гледа само малко напред и да спира — в игра, в която, както показват следващите раздели, ранното спиране е точно това, което простото търсене не може безопасно да направи.

6.4.2 Архитектура

Както и при Libratus, Pluribus се разделя на офлайн фаза, която изгражда план, и онлайн фаза, която извършва търсене — но балансът на силите между тях е обърнат и, отново както при Libratus, **няма невронна мрежа никъде** (Фигура 6.3).



Фигура 3: Pluribus’s two-phase architecture: an offline Linear-MCCFR blueprint (top) reused as the source of $k=4$ continuation strategies inside the online depth-limited search (bottom).

Извън линия Pluribus изчислява план за цялата игра чрез самообучение, започвайки от случайна игра и подобрявайки се чрез **метод Монте Карло CFR (MCCFR) с външна извадка**, който при всяка итерация избира един играч като „traverser“, симулира ръка и насочва този играч към действията, които са се представили по-добре. Той абстрахира играта по два начина — ограничен брой размери на залозите (*абстракция на действията*) и групи от стратегически сходни ситуации с карти (*информационна абстракция*) — но само грубо, и използва този план *директно* само в първия от четирите кръга на залагане, където абстракцията на действията се запазва по-фина и не се използва групиране на карти. Навсякъде другаде — флоп, търн, река — Pluribus извърля грубия план локално и **търси в реално време** по-фина стратегия, точно както направи Libratus, с една решаваща разлика: той не решава до края на играта. Той разглежда само един или два кръга напред до граница на дълбочината и спира, което прави шест играчи изчислително достъпни.

Запознатите градивни елементи са всички налице — абстракция и MCCFR изграждат плана, линейният CFR извършва онлайн решаването, а „търсенето“ е преизчисляване при всяко решение — и двата архитектурни отличителни белега са *отсъствия*. Няма **невронна мрежа** (системата е изцяло таблична, като Libratus) и, за разлика от Libratus, няма **самоусъвършенстващ се**: Pluribus никога не поправя своя план между сесиите, тъй като търсенето с ограничена дълбочина в три от четирите кръга вече извършва поправките. Две състав-

ки правят ранното спиране разумно и те са предмет на задълбочения анализ по-долу: внимателен отговор на въпроса какво изобщо може да означава „стойност на възел“ при скрита информация, и търсене, което започва не от текущото решение, а от *началото* на текущия кръг на залагане.

6.4.3 Ключова иновация: търсене с ограничена дълбочина с продължителни стратегии

Евристичното търсене в шах или Го работи, като гледа на фиксирано разстояние напред и отчита стойност от хоризонта при предположението, че и двете страни ще играят добре след това. Това предположение напълно се проваля при скрита информация, а Браун и Сандхолм илюстрират провала с еднократна последователна игра „камък–ножица–хартия“: ако търсещият приеме, че противникът ще играе равновесие (всеки ход една трета от времето) отвъд хоризонта, тогава всяко действие изглежда еднакво оценено на нула и търсещият може да се установи на „винаги играй камък“ — при което противникът преминава към „винаги хартия“ и истинската стойност рязко спада. Възел в игра с непълна информация следователно няма *една единствена стойност*: неговата стойност зависи от стратегията, която търсещият ще възприеме, което е именно това, което търсенето се опитва да определи.

Централният принос на Pluribus е начинът, по който все пак се определя стойност за възлите. Когато търсенето достигне границата на дълбочината, вместо да се фиксира една продължителна стратегия, **всеки играч, който все още е в ръката, избира измежду четири различни продължителни стратегии** за остатъка от играта и може да ги смесва. Четирите са умишлено опростени: самият план и три пристрастни копия, които умножават вероятността за *пас*, за *кол* и за *рейз* (всеки след това се ренормализира). Тъй като противникът винаги може да премине към продължителна стратегия, която най-много наказва търсещия, небалансирана стратегия — покер еквивалентът на винаги играене на камък — вече не се възнаграждава и търсещият се насочва към баланс. Тази идея първоначално беше доказана в двуигровия предшественик modicum (Brown, Sandholm & Amos, 2018), който победи двама бивши шампионски бота, докато работеше на 4-ядрен лаптоп с 16 GB памет — поразителен знак, че търсенето с ограничена дълбочина с продължителни стратегии може да замести свръхкомпютър.

Pluribus обобщава идеята от двама играчи на шестима и добавя фин, но важен обрат. В modicum само *противникът* избираше измежду продължителни стратегии, докато търсещият винаги следваше плана — подход, който е обоснован в **двуигрови игра с нулева сума**, но дава инициативата на противника и оставя търсещия колеблив и с ниска стойност. Pluribus позволява на **търсещия също да избира измежду продължителните стратегии**, което балансира играчите и, по думите на авторите, е „по-ефективно, по-лесно и по-елегантно“. Освен това той не пререшава от текущата точка за вземане на решение, а от *началото на текущия рунд за залози*, като фиксира единствено вече предприетите действия. Това „несигурно“ търсене — наречено така, защото, за разлика от Libratus, не носи гаранция за **експлоатируемост** — е по-евтино (повечето раздавания с шестима играчи се пасуват веднага, така че малко от тях изобщо се нуждаят от стратегия) и, тъй като започва непосредствено след случайно събитие с множество разклонения, на практика се оказва трудно за експлоатиране.

Това, което очевидно липсва от всичко това, е една *гаранция*. deepStack ограничи своята експлоатируемост до $k_1\epsilon + k_2/\sqrt{T}$, а Libratus — до 2Δ ; Pluribus не предлага такова ограничение и пропускането е принципно,

а не поради небрежност. Двигателят на CFR все още, във всяка крайна игра, свежда *средното съжаление* на всеки играч до нула,

$$\frac{R_i^T}{T} \rightarrow 0 \quad (\text{no-regret}),$$

но изводът, който доведе двамата предшественици от безсъжаление до безопасност — *безсъжаление играта се сближава до равновесие на Наш и едно равновесие на Наш не може да бъде победено* — важи само когато има двама играчи и играта е с нулева сума. При шестима играчи първото следствие отпада (самообучението не е задължително да се доближава до равновесие), а второто е безсмислено (едно равновесие не е непобедимо). Самите итерации, които *доказаха* deepStack и Libratus безопасни, следователно носят на Pluribus само емпирична сила: свръхчовешка на практика, без нищо сертифицирано. За да направи тези итерации достатъчно евтини за изпълнение с шестима играчи, Pluribus разчита на **линеен CFR** — претегляйки приноса на итерация t чрез t , така че лошите ранни итерации да се изчистват около три пъти по-бързо — и на **модифицирано отрицателно-съжаление подрязване**, което пропуска безнадеждни действия при повечето итерации (навсякъде с изключение на последния кръг); заедно с отложено разпределение на паметта, това са нещата, които намаляват сметката до един сървър.

6.4.4 Ограничения, неизследвани пътища и пропуснати детайли в статията

Както и при двамата предшественици, публикуваната статия е изчистеният разказ, а инженерната честност се намира в допълнителните материали — още повече тук, тъй като Pluribus е шестстранична статия в *Science*, чиято архитектура почти изцяло е прехвърлена към нейните допълнителни материали и към две придружаващи статии. Най-същественото признание е, че Pluribus използва **небезопасно** решаване на подигра — той предполага, че опонентите са играли стратегията, която той изчислява за тях — което, както авторите заявяват ясно, „няма теоретични гаранции за представянето дори в двуигрови игри с нулева сума и има случаи, в които води до силно експлоатируеми стратегии“. Съществуват безопасни алтернативи, но при директен сблъсък те се представиха по-зле, така че Pluribus взема емпиричната победа и смекчава риска само като винаги пререшава от началото на рунда за залози. Втора слабост е наследена и никога не е напълно затворена: в *първия* рунд за залози, твърде отдалечени от плана на опонента действия все още се **закръглят** чрез превод на действие — същата слабост, която самоусъвършенстващият се модул на Libratus съществуваше, за да преодолее — и Pluribus, тъй като няма такъв модул, просто живее с нея.

Няколко други детайла са от значение. Основните нововъведения на Pluribus **никога не са индивидуално премахвани**: авторите признават, че дисперсията на покера без лимит и цената на човешките изпитания правят „твърде скъпо“ измерването на приноса на всяко едно от тях, така че докладваните ускорения на компонентите (приблизително $3\times$ от линейния CFR, $2\times$ от модифицираното подрязване, повече от $2\times$ в паметта от отложеното разпределение) са оценки, а общото предимство не е разложено. Допълнението обаче разсейва едно изкушаващо погрешно схващане: приемането на *единствена* план-продължителна стратегия в листата — очевидното нещо за опитване и начинът, по който основните бележки понякога описват метода — е показано в двуигровия предшественик да *губи* от двамата шампионски бота (с 10 и 1 mbb/g), докато версията с четири продължителни стратегии *печели* (с 6 и 11); наборът от продължителни стратегии вър-

ши реална работа, а не е украшение. По-малки любопитни факти допълват картината: Pluribus играе своята **последна** търсене итерация, вместо обичайното усреднено време, за да избегне остатъчни лоши действия; той се научи да **изоставя „лимпинга“** по време на самообучението, но **„донкове“ много повече от хората;** и — както Libratus — неговият **код никога не е бил публикуван**, оставяйки само псевдокод за независима проверка, тъй като покерът се играе комерсиално.

6.4.5 Изчислителна мощност и достъпност

Историята за изчислителната мощност на Pluribus е тази, която повечето хора помнят, и тя наистина обръща с главата надолу своите предшественици. Планът беше обучен за **осем дни на един 64-ядрен сървър** за около **12 400 ядро-часа** и под 512 GB памет — приблизително **\$144** при спот цени в облака — а на масата Pluribus работи на **два CPU (28 ядра) и под 128 GB, без GPU на който и да е етап**, като взема от една до тридесет и три секунди за решение и играе около два пъти по-бързо от човек. В сравнение с останалите контрастът е почти комичен: AlphaGo използва 1920 CPU и 280 GPU, Deep Blue 480 персонализирани чипа, а Libratus около петнадесет милиона ядро-часа за изграждане на своя план и приблизително сто-CPU клъстер за игра. Pluribus достигна *по-труден* етап — повече играчи, по-голяма игра — за порядък от хилядна част от изчислителната мощност за обучение на Libratus. Докато DeepStack концентрира разходите си извън линия, а Libratus плащаше скъпо и в двата края, Pluribus е **евтин и в двата края**, и това, също толкова колкото резултатът с шестима играчи, е тезата на статията.

Това преформулира изцяло достъпността. За първи път една свръхчовешка покер система беше възпроизводима, по принцип, от един добре оборудван изследовател, а не от суперкомпютърен център — авторите я представят изрично като опровержение на опасението, че граничният изкуствен интелект в игрите ще принадлежи само на екипи с милиони долари хардуер. Свивът не е магия, а алгоритмичен: съчетаването на търсене с ограничена дълбочина (което авторите оценяват, че спестява поне пет порядъка спрямо решаване до края), линеен CFR и агресивно подрязване и пестене на памет. Единственото уточнение е, че самият артефакт остава затворен — без код — така че „достъпен“ описва *метода*, демонстриран в мащаб на лаптоп от Modicum, повече отколкото изтегляема програма.

6.4.6 Силни страни и ограничения

Силата на сигнала на Pluribus е просто в това, че е **първи**: първият изкуствен интелект, който постига свръхчовешко представяне във всяка широко призната референтна игра с повече от двама играчи или два отбора и в най-популярната форма на покера. Победата беше решителна и строга — **+48 mbb/g ($p = 0,028$)** срещу петима елитни професионалисти на масата и **+32 mbb/g ($p = 0,014$)** с пет копия срещу един-единствен професионалист — измерена с намалителя на дисперсията AIVAT (който намали дисперсията около девет пъти) в десетки хиляди ръце, срещу тринадесет професионалисти, всеки от които е спечелил над един милион долара и които имаха дни да търсят слабости; процентът победи едва се колебаеше. По-късен реванш дори победи **Linus Loeliger**, широко смятан за най-добрия Six-Max Cash Player в момента. И, подобно на Libratus, Pluribus е с **устойчивост на първо място**: той играе фиксирана стратегия, никога не моделира и не се адаптира към опонентите и дори не знае техните самоличности, така че не може да бъде примамен в контраексплоатируема корекция — дисциплина, която, с **без човешки данни и без експертни познания**,

поддържа резултата чист.

Ограниченията са именно това, от което се отказва, за да стигне дотук. Pluribus **няма абсолютно никаква гаранция за безопасност** в шестпотребителската обстановка — няма сходимост към равновесие на Наш, няма граница на експлоатируемостта — така че неговото свръхчовешко състояние е *емпиричен* факт за тринадесет силни човека в десетки хиляди ръце, а не теорема; достатъчно координирана маса или просто различна игра не носят никаква увереност. Той разчита на **несигурно** търсене, все още **закръгля залозите извън дървото в първия рунд**, и остава **изцяло табличен и абстрактно-базиран**, без нищо научено, което да обобщава в различни ситуации — планът е гигантска таблица за търсене, а не модел. Авторите добавят, че целият подход може да не оцелее там, където играчите могат да **комуникират и се стоварят**, което покерът до голяма степен забранява. И неговият процент победи от 48 mbb/g в шест ръце, макар и решителен, не е същата „валута“ като 147 на Libratus в един на един — различна игра, петима опоненти, по-висока дисперсия. Повечето от тези пропуски са поправени от по-късни системи; този, който тази дисертация откроява — **че многопотребителският успех дойде без никаква гаранция за безопасност** — е изричната цел на принос 2.

6.4.7 Наследство и съвременна значимост

Две идеи надживяват спецификите на покера. Първата е техническа: **търсене с ограничена дълбочина, направено коректно при скрита информация чрез даване на всеки възел малко меню от избираеми стратегии за продължаване вместо една-единствена стойност** — чист начин да се погледне само малко напред в игра, където наивно не можеш. Втората е икономическа и областта я възприе най-дълбоко: Pluribus е каноничната демонстрация, че **по-труден проблем може да бъде решен с хиляда пъти по-малко изчислителна мощност чрез по-добри алгоритми и търсене във време на вземане на решение**, вместо повече хардуер. Ноам Браун оттогава посочи точно това — че добавянето на търсене в реално време е струвало толкова, колкото огромното мащабиране на предварително изчислената стратегия — като ранен, конкретен пример за аргумента „**тест-тайм изчислителна мощност**“, който сега е централен за модели с възможност за мислене като o1. В архитектурната дъга на тази глава Pluribus е точката, в която *барьерата на броя играчи* пада; *барьерата на обобщаването*, която той оставя изправена — всичко все още е табличен и ръчно абстрахиран — е това, което **Recursive Belief-based Learning** и **Student of Games** разрушават след това, като заменят плана с научени стойности на състоянието на убеждението.

Няколко от частите на Pluribus остават директно използвани: конструкцията **продължителна стратегия (многозначна листовка)** за стабилно търсене с ограничена дълбочина във всяка среда с непълна информация или многоагентна среда; **Линеен (дисконтиран) CFR**, вече стандартен ускорител на сходимостта и отправна точка за по-късни дисконтирани варианти на CFR; **подрязване, базирано на съжаление** за мащабиране на CFR; **AIVAT** като нискодисперсна оценка; и обработката *добавяне на действието и пререшаване* на залозите извън дървото, наследена от Libratus. Това, което е заменено, е заобикалящата таблична конструкция — ръчно настроената абстракция и планът под формата на таблица за търсене — които научените представяния на ReBeL и SoG разтварят, заедно с *несигурното* търсене, което тези методи със състояние на убеждението правят ненужно. Честната оценка отразява тази на Libratus: **заменена като внедрена архитектура, но оправдана и изострена като теза**. За тази дисертация по-специално Pluribus е ключовият камък на

принос 2 — емпиричното доказателство, че методите търсене по Наш и търсене *работят* в N-потребителски игри с непълна информация, като същевременно предлагат **абсолютно никаква гаранция за безопасност**, което е точно пропускът, който една теория на многоагентната безопасна експлоатация трябва да затвори.

6.5 ReBeL (2020)

Pluribus преодоля бариерата на броя играчи, но го направи, като остана точно това, което беше Libratus — гигантска, ръчно абстрахирана таблица за търсене без нищо научено, което да се прехвърля от една ситуация в друга — и постигна победата си с шестима играчи, като изцяло се отказа от безопасността, разчитайки на *небезопасно* търсене без никаква гаранция за експлоатируемост. ReBeL (Brown, Bakhtin, Lerer & Gong, 2020; Facebook AI Research) прави крачка назад от шестима към двама играчи и задава обратния въпрос: **какво ще стане, ако вместо ръчно да създаваме абстракция и предварително да изчисляваме план, пуснем AlphaZero — обучение чрез самоигра с подкрепление плюс търсене, както по време на обучението, така и при тестване — в игра със скрита информация, възстановявайки гаранциите, които Pluribus отхвърли, като същевременно изцяло премахнем абстракцията?** Неговият отговор, *рекурсивно обучение на основата на убеждения*, е първият алгоритъм, който прави обучението с подкрепление и търсенето доказуемо коректни в игри с непълна информация. Трикът е да се преформулира играта като „игра с пълна информация“ върху **публични състояния на убеждения** — вероятностни разпределения за това какво може да държи всеки играч, предвид общото знание — върху които функциите за стойност и политика са добре дефинирани; цикъл в стил AlphaZero след това обучава невронна мрежа за стойност (и политика) върху тези състояния, като минимизирането на контрафактичното съжаление (CFR) решава под-игри с ограничена дълбочина на листата. ReBeL доказуемо се сближава към равновесие на Наш във всяка игра за двама с нулева сума, побеждавайки топ човешкия специалист в хедс-ъп Донг Ким със 165 mbb/g (мили-големи блайнда на игра, мерната единица за резултат от предишните раздели) в рамките на 7 500 ръце, като използва *значително по-малко* експертни познания от всеки предишен покер изкуствен интелект — и, за разлика от затворените Libratus и Pluribus, неговата реализация (за Liar’s Dice) беше **отворена**.

С един поглед	ReBeL (2020)
Играчи	2 (хедс-ъп) — <i>гаранциите</i> са за игра за двама с нулева сума; алгоритъмът се обобщава за повече играчи, но без гаранциите
Тип на играта	Общи двуйгрови игри с непълна информация; оценяван върху hUNL poker + лъжливи зарове (и ендгейм холд’ем на ход). Редуцира се до алгоритъм в стил AlphaZero при игри с пълна информация
План (офлайн)?	Няма съхранен план — офлайн продуктът е <i>научена мрежа за PBS стойности (+ политика)</i> от самообучение, а не таблица на стратегиите

С един поглед	ReBeL (2020)
Невронна компонента	Мрежа за PBS стойности + (по избор) мрежа на PBS стратегията; MLP (GeLU/LayerNorm), 6×1536 скрити слоя за покер, вход = убеждение върху всеки от 1 326 ръце на играча + борда + пота + флаг за залог
Механизъм за търсене	CFR (CFR-D / CFR-AVG; също FP/FLOP), решаващ ограничена по дълбочина под-игра, кореняща се в PBS, като научената мрежа за стойности доставя (зависими от итерацията) стойности на листата — както по време на обучението , така и при тестване
Абстракция?	Няма — няма абстракция на карти/информация (със загуби или без); мрежата за стойности я замества. Запазва се само малко (≤ 9) ръчно избрано меню от размери на залозите, като залозите извън дървото се добавят в реално време
Също и пълна информация?	Не (представен/оценяван като игра с непълна информация), с уточнението, че <i>дегенерира</i> до търсене в стил AlphaZero, ако частната информация бъде премахната — пълното обединяване е претенция на Student of Games
Изчислителна мощ	Обучение на GPU: пълният hUNL използва ~ 90 DGX-1 възела $\times 8$ V100 GPU за генериране на данни от самообучение (в контраст с CPU-only $\sim \$150$ на Pluribus); CFR на един CPU поток; игра < 2 s/ръка, ≤ 5 s/решение
Ключова иновация	Публични състояния на убежденията + цикъл на самообучение в стил AlphaZero, обучаващ PBS мрежа за стойност/политика с CFR, работещ в пространството на убежденията: първото <i>коректно</i> RL+търсене за игри с непълна информация, възстановяващо доказуема 2p0s гаранция на Наш без абстракция или план

6.5.1 Затворената празнина

Предишните три системи се сходилих към една и съща рецепта — предварително изчислен план плюс търсене в реално време — а Pluribus я разшири до шестима играчи, но всяка от тях, в основата си, беше *табличен* обект: стратегия или план, съхранени върху ръчно проектирана абстракция, без нищо научено, което да се обобщава към ситуация, която никога не е била изрично решена. Само deepStack имаше невронна компонен-

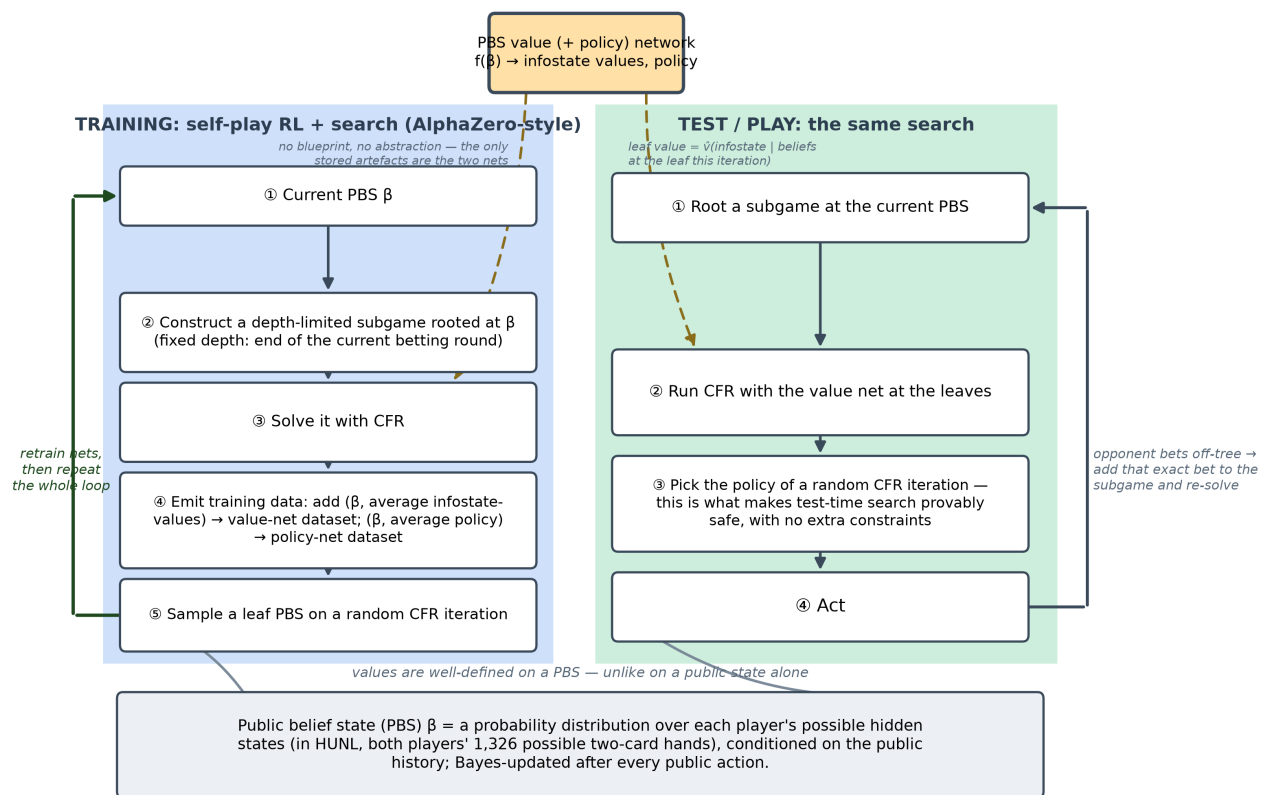
та, но дори и той разчиташе на ръчно извлечени признаци, клъстеризация в 1000 кофи на входа на мрежата и абстракция на ривъра. Междувременно най-успешният парадигма във всички игрови изкуствени интелекти — съюзът на AlphaZero между обучение чрез самоигра с подкрепление и търсене, който научава собствената си оценка от нулата и я използва както за обучение, така и за игра — беше *недостъпен* за игри с непълна информация. Отвореният въпрос, на който ReBeL отговаря, е този, който Ноам Браун нарича „свещения граал“ на областта: **може ли рецептата на AlphaZero да бъде накарана да работи коректно в игри със скрита информация?**

Причината, поради която това не беше възможно преди, е фина и представлява концептуалната същност на цялата глава. AlphaZero предполага, че всяко състояние има една единствена добре дефинирана стойност: шахматна позиция струва толкова, колкото струва, независимо колко често достигате до нея. Скрытата информация унищожава това предположение. Статията го прави нагледно с модифициран камък-ножицартия, в който ножицата печели (или губи) двойно: равновесието е да се хвърля камък и хартия 40% от времето и ножица 20%, и при това равновесие всяко действие има очаквана стойност нула. Ако Играч 1 извърши еднопластово търсене с придвиждане, което работи в шаха — замествайки равновесната стойност на листата — всяко движение изглежда еднакво добро, така че търсещият може да се спре на „винаги камък“, след което противникът преминава към „винаги хартия“ и истинската стойност на камъка се срива от 0 до -1 . **В игра с непълна информация стойността на едно действие зависи от вероятността, с която то се изиграва**, така че състояние, дефинирано само от последователността на действията, няма уникална стойност — и търсенето в стил AlphaZero е просто некоректно. Pluribus беше закрил симптома със стратегии за продължение по избор на листата; ReBeL лекува болестта, като променя това какво е едно „състояние“. Той предефинира състоянието така, че да включва разпределението на вероятностите върху скритата информация — *общодостъпно състояние на убеждението* — върху което, както показват следващите раздели, стойностите отново стават добре дефинирани и цялата машина на AlphaZero може да бъде прехвърлена. При това той също така премахва двете патерици, на които все още се опираха deepStack и системите за абстракция: ръчно създадената абстракция и предварително изчисления план изчезват, заменени от функция на стойността, научена единствено от самообучение.

6.5.2 Архитектура

ReBeL е най-добре да се възприема като **AlphaZero за непълна информация**: цикъл на самообучение, който обучава невронни мрежи за стойности и стратегии, при който „търсенето“, използвано както по време на обучение, така и при игра, представлява CFR solving на ограничена по дълбочина под-игра — но всичко се извършва върху *общодостъпни състояния на убеждението*, а не върху сурови игрови състояния, като оценителят на възел е научена мрежа за стойности, вместо разгръщане (Фигура 6.4).

Цикълът има три основни компонента. **Обучението чрез самоигра** е движещата сила: започвайки от кореново PBS, ReBeL конструира ограничена по дълбочина под-игра, решава я, записва решените стойности и стратегия като цели на обучението, избира възел, който да стане следващото кореново PBS, и продължава до края на играта — след това преобучава мрежите и повтаря процеса, по същия начин както AlphaZero редува самообучение с обучение на мрежата. **Търсенето** представлява вътрешното решаване: ReBeL изпълнява CFR (конкретно CFR-D, „CFR с декомпозиция“, или неговия вариант CFR-AVG) върху ограничената



Фигура 4: ReBeL's AlphaZero-style loop: self-play training (left) and test-time play (right) share one PBS value/policy network, both solving depth-limited subgames rooted at the public belief state (shared definition box, bottom).

по дълбочина под-игра и на всяка итерация на CFR задава стойността на всеки възел чрез заявка към **научената pBS value мрежа** — така че стойностите на възлите се променят от итерация до итерация, което е именно това, което гарантира коректността на търсенето при наличие на скрита информация. **Невронните мрежи** възпламяват научената интуиция: мрежа за стойности, която съпоставя PBS със стойностите на възможните ръце на играча, и незадължителна мрежа на стратегията, която осигурява топли стартове на CFR, намалявайки броя необходими итерации.

Архитектурните особености се изразяват в *липсата* на елементи, характерни за по-ранните системи. Няма **предварително изготвен план** — единствените резултати от обучението са двете мрежи. Няма **абстракция** от какъвто и да е вид: докато DeepStack, Libratus и Pluribus групираха стратегически сходни ръце, ReBeL изчислява отделна стратегия за всяко информационно състояние, като подава на мрежата единствено суровото разпределение на убежденията върху възможните 1326 ръце и на двамата играчи, плюс дъската, пота и един флаг „дали някой е заложил този рунд“. А **cFR търсене** работи на **един CPU поток** без никаква абстракция; основната изчислителна мощност се използва за **gPU самообучение**, което обучава мрежата за стойности (балансът е разгледан под *Изчислителна мощност*). Класическите градивни елементи от по-ранните стъпки все още са видими — **cFR търсене** (Стъпка 3) изпълнява търсенето, а **невронното приближение на стойността** (Стъпка 5) е оценителят на възел — но сега те са обединени в един цикъл самообучение плюс търсене, като **общодостъпното състояние на убеждението** (разделът по-долу) служи за представянето, което прави това обединение възможно.

6.5.3 Ключова иновация: общодостъпни състояния на убеждението и научени стойности в пространството на убежденията

Приносът на Recursive Belief-based Learning е един концептуален ход с три технически последствия. Ходът е да се спре търсенето върху **състояния** и да се започне търсене върху **убеждения за състоянията**.

Общодостъпното състояние на убеждението (PBS) е сърцевината на всичко това. Блогът на Meta AI дава най-ясната интуиция. Вземете игра с карти и я променете така, че играчите да не могат да виждат собствените си карти — само безпристрастен съдия може; на всеки ход играч обявява, за всяка карта, която *може* да държи, вероятността, с която би предприел всяко действие, и съдията избира хода за истинската карта на играча. Тъй като стратегиите на всички играчи се приемат за общоизвестни, всеки може да проследява, чрез правилото на Бейс, вероятността всеки играч да държи всяка възможна ръка. Тази модифицирана игра е **стратегически еквивалентна** на оригиналната, но тя не съдържа *частна информация*: нейното състояние — векторът от тези вероятности — е напълно наблюдаемо за всички. Recursive Belief-based Learning нарича това състояние общодостъпно състояние на убеждението: формално, **съвместно разпределение на вероятностите** върху възможните информационни състояния на играчите, дадено общите публични наблюдения. От решаващо значение е, че разглеждането на игрите с непълна информация като игри с непрекъснато състояние и пълна информация по този начин е стара идея (тя датира от работата върху кооперативни POMDPs); постижението на Recursive Belief-based Learning е, че за първи път я комбинира с обучение чрез самообучение в *състезателна* среда.

Първото следствие е, че **стойностите отново стават добре дефинирани**. Под-игра с непълна информация,

вкоренена в обикновено *публично състояние*, няма уникална стойност, защото нейната стойност зависи от скритите раздавания, които я достигат — което е точно причината евристичното търсене да се провали. Но под-игра, вкоренена в *общодостъпно състояние на убеждението*, има такава: в двуйгрови игри с нулева сума всяко общодостъпно състояние на убеждението β носи уникална стойност $V_i(\beta)$ с $V_1(\beta) = -V_2(\beta)$, дефинирана от това и двамата играчи да играят равновесие на Наш в под-играта от β . Това е именно свойството, на което AlphaZero разчита в шаха, сега възстановено за покера — и то *формализира и обобщава контрафактичните стойности при дадени убеждения на deepStack*, които бяха първият случай на функция на PBS стойност, но бяха обучени по много по-ад хок начин.

Второто следствие е, че **alphaZero loop** може да работи с **CFR като алгоритъм за търсене**. По принцип би могло да се стартира търсене в дърво на Монте Карло на AlphaZero директно върху играта на убежденията, но представянето на убеждението е много високодимензионно непрекъснато пространство (в малкия пример с карти от блога само пространството на действията има 156 измерения) и там търсенето с дърво е безполезно. Спасителният фактор е, че в двуйгрови игри с нулева сума тези представяния на убежденията са *вдлъбнати* задачи за оптимизация, а CFR е по същество решаващ, базиран на градиент, за тях. Така ReBeL търси с CFR вместо с търсене в дърво на Монте Карло — единствената съществена разлика от AlphaZero. Техническа тънкость, която статията доказва (Теорема 1), е, че количествата, които CFR действително изисква — стойностите за всяко информационно състояние — са *сверхградиент* на вдлъбнатата функция на PBS стойност, така че ReBeL обучава мрежа за стойности на информационни състояния, вместо един скалар за всеки PBS.

Третото следствие — и водещото заглавие — е, че цялото нещо е **доказуемо коректно**, възстановявайки гаранцията, която Pluribus се отказа да даде. Най-дълбокият резултат засяга *времето на изпитване*. По време на самообучение Recursive Belief-based Learning може да приеме, че стратегиите и на двамата играчи са общо знание, така че винаги знае истинското PBS; но срещу реален противник то не знае неговата стратегия и следователно не знае в кое PBS се намира, което наивно прекъсва търсенето. Елегантното решение на Recursive Belief-based Learning е да стартира *същия алгоритъм* по време на изпитване и **да действа според стратегията на произволно избрана итерация на CFR**. Статията доказва (Теорема 3), че това води до безопасно търсене — равновесие на Наш *в очакване* — без допълнителни ограничения, където всеки предишен метод за „безопасно търсене“ добавяше ограничения толкова скъпи, че никога не бяха използвани изцяло в състезателен бот. Конкретно, с мрежа за PBS стойности, чиято грешка е най-много δ и T итерации на CFR на под-игра, Recursive Belief-based Learning играе ε -равновесие на Наш — такова, което никой противник не може да експлоатира за повече от ε — с

$$\varepsilon = \delta C_1 + \frac{\delta C_2}{\sqrt{T}},$$

за специфични за играта константи C_1, C_2 . Структурата умишлено отразява тази на $k_1\varepsilon + k_2/\sqrt{T}$ от DeepStack — термин за грешка в стойността плюс обикновен термин за сходимост на CFR — но докато границата на DeepStack управлява експлоатируемостта на едно преизчисляване, тази на ReBeL управлява *сходимостта към равновесие на Наш на цялата игра*, и двата термина вече се мащабират с грешката на научената стойност δ : с подобряването на мрежата за стойности, $\delta \rightarrow 0$ и играта на ReBeL се доближават

до точно равновесие на Наш. Това е точният смисъл, в който ReBeL *се връща към двуигрови игри с нулева сума и възстановява теоретичните гаранции, които Pluribus се отказва да даде* — същият $1/\sqrt{T}$ гръбнак на CFR, който доказва безопасността на DeepStack и Libratus, сега обвит около стойност на състоянието на убежденията, научена чрез самообучение, вместо ръчно изградена абстракция.

6.5.4 Ограничения, задънени улици и какво статията описва недостатъчно

Recursive Belief-based Learning обръща обичайния модел на тази глава: *теорията* е в основния текст, а *инженерната честност* — по-специално точният смисъл, в който се използва „значително по-малко експертни познания“ — е преместена в приложенията.

Най-важният детайл е това, което ReBeL умишлено **отхвърля**, каталогизирано в Приложение D. Всеки предишен водещ изкуствен интелект за покер, включително deepStack, използваше **информационна абстракция**, за да групира стратегически подобни ръце; ReBeL не използва *никаква*, нито загубна, нито беззагубна, като изчислява уникална стратегия за всяко информационно състояние от суровото разпределение на убежденията. deepStack обучаваше своята мрежа за стойности върху *случайно генерирани* PBS, извлечени от ръчно настроен генератор; ReBeL генерира своите тренировъчни PBS изцяло от самообучение, твърдейки, че случайната извадка „би била като да се научи функция на стойността за Го чрез случайно поставяне на камъни върху дъската“ — и наистина показва (Фигура 2), че мрежа за стойности, обучена върху случайни убеждения, „се проваля в научаването на нещо ценно“. Предишните агенти предварително изчисляваха точни **таблици със стойността при отиграване докрай** и решаваха *до края на играта* на третия рунд за залози (използвайки **речна абстракция**, за да направят това постижимо); ReBeL не прави нито едно от двете, като винаги решава само до края на *настоящия* рунд и сам научава дори стойностите при отиграване докрай. Резултатът е честен, но с двойно острие: ReBeL трябва да научи *шест* „слоя“ от стойности, където deepStack се нуждаеше от три, което *увеличава* повърхността за **разпространение на грешка** — реална цена, приета в замяна на премахването на абстракцията. И не е без експертни познания: запазва ръчно избран списък с най-много осем или девет размера на залози (възпрепятствани по време на обучението), единствената отстъпка пред постижимостта, като залозите извън дървото се обработват чрез добавянето им към под-играта и **пререшаването** ѝ, à la Libratus и Pluribus.

Няколко други важни момента трябва да се отбележат. Чистите теореми стъпват върху **идеализации**: сходимостта в Теорема 2 предполага съвършен апроксиматор на функции, а гаранциите изискват стратегиите на играчите да са общо знание (резултатът от §6 премахва *време-на-тестване* версията на това предположение, но анализът на обучението все още идеализира мрежата). Вариантът, който агентът реално използва за ефективност — модифициран **CFR-AVG** — според самите автори *няма доказана теоретична обоснованост* („дали тази модифицирана форма на CFR-AVG е теоретично обоснована остава отворен въпрос“), въпреки че се представя добре в покера — позната празнина между доказания алгоритъм и внедрения. Гаранцията за безопасно търсене също зависи от избора на *случайна* итерация на CFR, която може да се окаже много лоша в ранните етапи; това се смекчава единствено чрез използването на Линеен CFR, който намалява тежестта на ранните итерации. И както при предшествениците си, по същество цялата архитектура, хиперпараметрите и най-важните данни за изчислителната мощност са представени в приложенията, а не в шестте страници основен текст — статията е, по дизайн, теоретична.

6.5.5 Изчислителна мощност и достъпност

Профилът на разходите на Recursive Belief-based Learning е огледален образ на този на Pluribus и контрастът е най-рязък в тази глава. Там, където интуицията на Pluribus беше план, решен чрез **CFR само с процесори на един 64-ядрен сървър за около \$150**, интуицията на Recursive Belief-based Learning е невронна мрежа и нейното обучение е в **кълъстерен мащаб на графични процесори**. Тясното място е генерирането на данни от самообучение — секвенциален CFR решава, чиято всяка итерация оценява всички възли листа чрез мрежата — така че статията паралелизира генерирането на данни върху „до 128 машини с по 8 gPUs всяка“, а пълният агент за HUNL в частност използва **около 90 възела DGX-1, всеки с осем 32 GB Nvidia V100 gPUs** (от порядъка на 700 gPUs), захранвайки една единствена тренировъчна машина с над 1750 епохи от по милиони примери всяка. Това е категорично сметка за обучение чрез дълбоко обучение, а не CFR сметка: самото CFR търсене се изпълнява на едно процесорно ядро. На масата обаче Recursive Belief-based Learning е евтин и подобен на deepStack — под две секунди на ръка средно и никога повече от пет секунди за решение, като предварителните под-игри са кеширани, за да бъде още по-бърз.

Историята с достъпността се разделя на две и това е най-ясният контраст със системите за покер от по-ранен период. От една страна *методът* на Recursive Belief-based Learning е най-общият и най-малко ръчно настроеният от четирите, и — решаващо — неговата **реализация беше с отворен код** (за „лъжливи зарове“), докато кодът както за Libratus, така и за Pluribus остана затворен. Това отворено издание е истинска стъпка към възпроизводимост, която системите на CMU никога не предлагаха. От друга страна, основният *покер* резултат *не* беше публикуван — авторите го задържаха с изричното основание, че Recursive Belief-based Learning „може да изчисли стратегия за произволни размери на стакове и произволни размери на залози за секунди“, което го прави готов инструмент за измама — а възпроизвеждането на този резултат в пълния му мащаб изискваше голям флот от V100, какъвто повечето групи не притежаваха. Така че Recursive Belief-based Learning е „отворен“ по начин, по който неговите предшественици не бяха, но отвореният артефакт е изследователската игра, а не свръхчовешкият покер бот.

6.5.6 Силни страни и ограничения

Силата на сигнала на Recursive Belief-based Learning е **общност с гаранция**. Това е първият алгоритъм, който прави парадигмата RL+Search на AlphaZero *доказуемо коректна* в игри с непълна информация, той доказуемо се сближава към равновесие на Наш във всяка **двуйгрови игра с нулева сума** и подкрепя теорията с резултати: побеждава предишните шампиони Slumbot (+45 mbb/g) и babyTartanian8 (+9 mbb/g), довежда **сондажа за най-добър локален отговор** до голяма загуба и побеждава топ човешкия специалист Донг Ким с 165 mbb/g в 7500 ръце — всичко това, използвайки *значително по-малко експертни познания от всеки предишен покер изкуствен интелект*, без **абстракция на карти** и най-общото представяне от четирите. Същият алгоритъм, непроменен, се сближава към **приблизително равновесие на Наш** в „лъжливи зарове“, демонстрирайки, че това е *рамка*, а не покер програма; в игри с пълна информация той елегантно се свежда до метод, подобен на AlphaZero. И той възстановява, за невронната ера, безопасността, която Pluribus трябваше да изостави — търсене, което е **доказуемо безопасен** по време на изпитване без допълнителни ограничения.

Ограниченията са точни и подготвят финалната система. Гаранциите на Recursive Belief-based Learning важат **само за двуигрови игри с нулева сума**: уникалната pBS стойност, изпъкналостта, която позволява cFR-as-Search, и доказателствата за коректност се основават на тази структура. Макар алгоритъмът да е изпълняван и с повече играчи, теорията не го обхваща в тези случаи — „2 играча“ в оценката се отнася до *гаранцията*, а не до кода. Най-конкретната му стена на мащабиране е, че **пространството на убежденията нараства с количеството скрита информация**: входът на мрежата се мащабира с броя на инфосъстоянията в публично състояние, така че игри с голяма стратегическа дълбочина, но малко общо знание — статията посочва „слепец шах с разужнаване“ — раздуват представянето, а добавянето на играчи само увеличава пространството на убежденията. Алгоритъмът предполага, че **точните правила на играта са известни** (разширение в стил MuZero към неизвестна динамика е посочено като бъдеща работа). Той наследява остатъчна **грешка на приближението на стойността δ** , която ограничава колко близо до равновесие на Наш всъщност играе, а най-силният му емпиричен резултат се основава на **един човешки противник** в 7500 ръце, вместо на многопроцесорно поле.

6.5.7 Наследство и съвременна значимост

Отстранете покера и трайната идея на ReBeL е представителна: **преобразувайте скритата информация в състояние на убеждението и пълната сила на обучение чрез самоигра с подкрепление плюс търсене се прехвърля**. Публичните състояния на убеждението вече са стандартният субстрат за коректно търсене в игри с непълна информация, а ReBeL е най-чистото изявление на „AlphaZero за непълна информация“ — търсене, вплетено както в *обучението*, така и в *играта*. Неговият пряк наследник е **Student of Games (2023)**, който пренася идеята за състояние на убеждението плюс коректно търсене по-нататък в единен алгоритъм, обхващащ игри с пълна и непълна информация; същата линия обучение чрез самоигра плюс търсене с научени модели също води до **CICERO (2022)**, агента на Meta за игра на Diplomacy на човешко ниво, който планира с научени модели в седем-игрови, със смесени мотиви, игра на естествен език далеч извън двуигровите гаранции на ReBeL. По-широко погледнато, ReBeL принадлежи към линията работа — deepStack, Libratus, Pluribus и сега ReBeL — която Ноам Браун посочва като ранен, конкретен пример за **изчислителна мощност във време на тестване**: изразходване на изчисления за търсене в момента на вземане на решение, насочвано от научена интуиция, вместо всичко да се пече в една предварително изчислена стратегия, същата теза, която сега е централна за моделите с възможност за мислене.

Няколко подсистеми остават директно използваеми. Самата **pBS representation** е най-важната — разпределение на убежденията върху скритите състояния на противниците при наличие на общо знание — и тя представлява естествената основа за базирано на убеждение моделиране на противника: Принос 1 на тази дисертация разширява pBS, така че да включва убеждения относно *типа стратегия* на противника, а не само неговите карти. **Обучената чрез самоигра функция на стойност/политика върху състояния на убеждението**, моделът на cFR-as-Search с научен оценител на възел, трикът за „безопасно търсене безплатно“ чрез произволни итерации и бързият търсач на равновесие FLOP, представен в статията, са всички преносими. Това, което е преодоляно, е основно ръчно изграденото скеле на DeepStack, разтворено от самообучението без абстракция на ReBeL, а след това и самият ReBeL като *обща* рамка, която Student of Games обхваща. Честната оценка е, че **ReBeL е жив стъпаловиден камък, а не крайна точка**: като внедрен покер

артефакт той никога не беше пуснат и сега е засенчен, но като идея — публични състояния на убеждението плюс AlphaZero-style RL и търсене — той е основополагащ и много жив. За тази дисертация конкретно той действа в две посоки: от една страна е отправната точка за по-богатото състояние на убеждението в Принос 1, а от друга — доказателството, че коректността, изоставена от Pluribus, *може* да бъде възстановена — макар все още не и при повече от двама играчи, което е точно границата, която Принос 2 се стреми да премине.

6.6 Student of Games / SoG (2023)

ReBeL възстанови коректността за игра за двама с нулева сума покер и изостави както абстракцията, така и плана — но той остана метод с непълна информация, който просто *се свеждаше* до AlphaZero в игри с пълна информация, вместо да ги обедини, решаваше *фиксирана* ограничена по дълбочина под-игра и обвързваше своето търсене по време на тестване със своята процедура за обучение. Student of Games (Schmid et al., 2023; Google DeepMind, с партньори от Университета на Алберта и EquiLibre Technologies в Прага) затваря последната празнина в главата, като задава най-амбициозния въпрос от всички: **какво ще стане, ако един единствен алгоритъм, който се учи чрез самообучение без човешки данни, може да играе шах, Го, покер един на един и Scotland Yard — разширявайки собственото си дърво за търсене според нуждите и оставайки доказуемо коректен както за игри с пълна, така и с непълна информация?** Неговият отговор обединява двете големи линии на главата — по думите на първия автор Мартин Шмид, това е „AlphaZero и DeepStack в един голям унифициран алгоритъм“. Двигателят е **Growing-Tree контрафактично минимизиране на съжалението (GT-CFR)**: търсене по всяко време, което изгражда дървото на играта постепенно, водено от мрежа за политики, с научена мрежа за стойности, оценяваща листата, които все още не са разширени. Върху поддървета с пълна информация GT-CFR се държи като MCTS на AlphaZero; върху тези с непълна информация — като CFR; и една единствена теорема за коректност покрива и двете. Системата достига **силно аматьорско до професионално ниво в шах и Го, побеждава Slumbot — най-силният публично достъпен бот за хедс-ъп безлимит покер — и надвишава най-съвременното ниво на агент за Scotland Yard**, като същевременно е доказано, че се сближава към минимакс-оптимална игра с нарастването на изчислителната мощ и точността на мрежата. Той споделя няколко автори от DeepStack (Bowling, Moravčík, Burch, Schmid) — линията от Алберта, която започна тази глава, тук в контраст с произхода на ReBeL от Facebook/FAIR — и първоначално се появи през 2021 г. под името *играр на игри*, преди да бъде преименуван за своята публикация в *Science Advances* през 2023 г.

С един поглед	Student of Games (2023)
Играчи	2 (игра за двама с нулева сума) — гаранциите <i>и</i> оценката са за 2p0s (детективът в Scotland Yard се брои като един отбор); основната формализация е по-обща, но равновесието на Наш е „по-малко смислено“ извън 2p0s

С един поглед	Student of Games (2023)
Тип игра	Както с пълна, така и с непълна информация — единствената унифицирана система: шах + Го (пълна) и HUNL покер + Scotland Yard (непълна)
План (офлайн)?	Не — офлайн продуктът е научена мрежа за стойности и политики, а не съхранена таблица на стратегиите (както в ReBeL)
Невронна компонента	Една единствена контрафактична мрежа за стойност и политика (CVPN) : една мрежа извежда контрафактични <i>стойности</i> по инфо-състояния и предварителна <i>стратегия</i> , за всички етапи на играта
Механизъм за търсене	GT-CFR — редува фаза на актуализация на съжалението в CFR (стойности на CVPN в листата) с pUCT expansion phase в стил AlphaZero, който <i>разширява</i> публичното дърво; работи вътре в продължително пре-решаване, както по време на обучение , така и по време на тестване
Абстракция?	Няма от вида на картите/информацията (CVPN я заменя); запазва само малко <i>рандомизирано залагане</i> (действие) меню в покера ($\approx 20\,000 \rightarrow 4-5$ действия), както ReBeL
Също и за игри с пълна информация?	Да — отличителният ред; единствената система в главата, демонстрирана върху игри с пълна информация, със <i>същия</i> алгоритъм и коректност, покриваща и двата класа
Изчислителна мощ	Обучена на TPU, умишлено съпоставена с бюджета на AlphaZero (Google TPUv4; Го е най-скъпата); докладвано <i>относително</i> спрямо AlphaZero, без една единствена сума в долари; търсенето е $O(kT^2)$ ($O(T)$ за игри с пълна информация); пълният агент/код не беше публикуван
Ключова иновация	Growing-Tree CFR + правилно самообучение: един алгоритъм за самообучение с търсене, коректен и за двата класа игри , който разширява дървото си постепенно с CVPN в листата и доказуемо се сближава към равновесието на Наш

6.6.1 The gap it closed

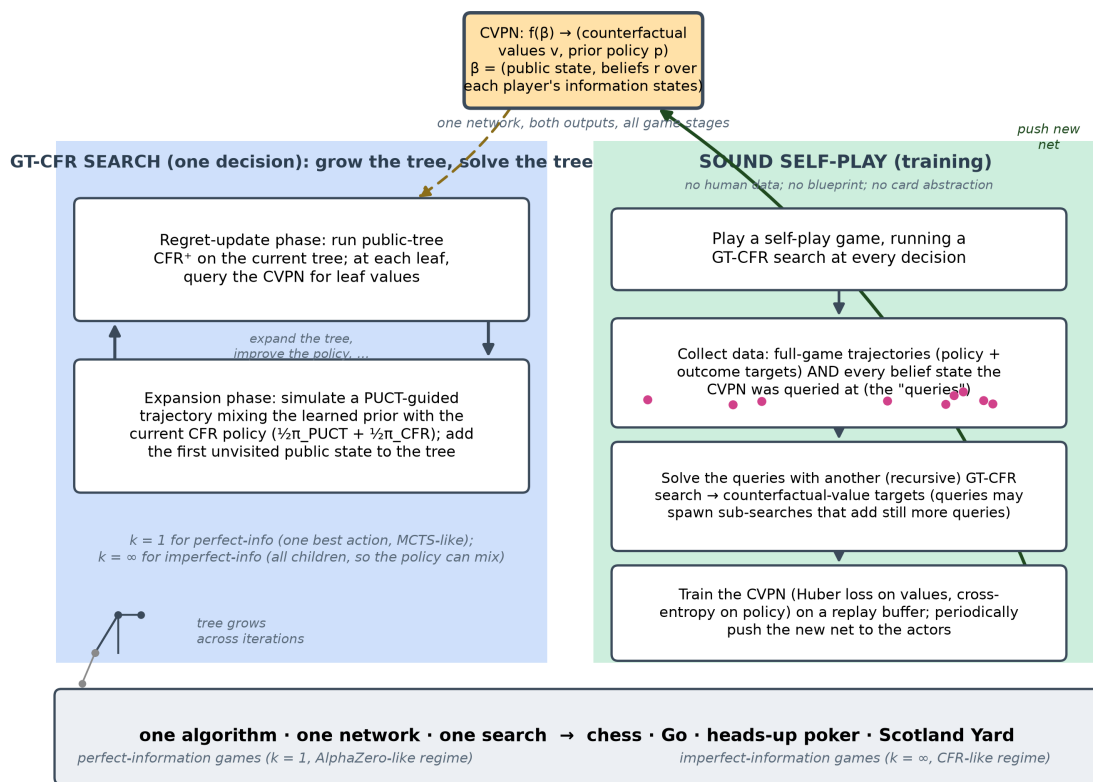
В продължение на седемдесет години двете големи традиции в игровия изкуствен интелект вървяха по отделни пътища. Едната — минимакс, алфа–бета, търсене с дърво на Монте Карло и в крайна сметка AlphaZero — завладя игрите с *пълна информация*, като комбинира търсенето с научена функция на стойността, а най-общото ѝ изражение, AlphaZero, овладя шах, шог и Го с един алгоритъм. Другата — минимизиране на контрафактичното съжаление и четирите покер системи в тази глава — завладя покера с *непълна информация* чрез игровитеоретично разсъждение. Но пътищата никога не се срещнаха. AlphaZero не можеше да играе покер: неговото търсене с дърво на Монте Карло е *некоректно* при скрита информация, по причината, която разделът за Recursive Belief-based Learning изясни — стойността на едно състояние зависи от вероятността, с която се достига до него, така че стойностите не могат просто да бъдат върнати назад от локални поддървета. А покер системите не можеха да играят Го: те бяха изградени около структурата на залаганията на една игра. Дори Recursive Belief-based Learning, най-общата от тях, беше представена и гарантирана само за непълна информация, *свеждайки се* до AlphaZero в случая с пълна информация, вместо наистина да обхваща и двата класа.

Отвореният въпрос, на който Student of Games отговаря, е дали *един* алгоритъм — една процедура за търсене, една архитектура на мрежата, един цикъл на самообучение — може да бъде **коректен и силен и в двата класа игри едновременно**. Трудността не е просто инженерна. Търсенето при пълна информация цели да разшири най-добрата линия дълбоко и да прочете една единствена стойност от всеки **възел**; търсенето при непълна информация трябва да поддържа *разпределение* върху действията (за да не изтича частна информация) и да разсъждава за *убеждение* относно скритите състояния във всеки **възел**. Съгласуването на тези два подхода в един алгоритъм означава намиране на търсене, което разширява дърво по начина, по който го прави AlphaZero — насочено, асиметрично, по всяко време — като същевременно изчислява *игровитеоретично коректните* величини, които CFR изчислява. Student of Games затваря тази празнина, като изгражда точно това търсене, GT-CFR, върху представянето на публичното убеждение, което Recursive Belief-based Learning беше установило, и като обучава своите мрежи чрез *правилно самообучение*, чиито цели произтичат от самото търсене. Резултатът е кулминацията на главата: точката, в която дъгите абстракция-към-невронна и извън линия-към-търсене са съединени от трета — пълна и непълна информация, обединени.

6.6.2 Архитектура

Student of Games е, по своята структура, **цикълът на самообучение на AlphaZero, при който търсенето с дърво на Монте Карло е заменено от GT-CFR и при който мрежата стойност/политика е дефинирана върху публични убеждения** — офлайн фаза, която обучава мрежа чрез самообучение, и онлайн фаза, която търси с нея, но при която същото търсене вече се изпълнява и в двете фази (Фигура 6.5).

Представянето е наследено от линията ReBeL: **общодостъпно състояние на вярванията** \$ pairs the public state (in poker, the betting history and board) with a **range** \$ — двойка разпределения върху информационните състояния, които всеки играч може тайно да заема. Игрите с пълна информация са просто дегенеративният случай, при който всяко публично състояние има точно *едно* информационно състояние и убеждението е точкова маса, което е именно причината една и съща репрезентация да може да обслужва и двата класа.



Фигура 5: Student of Games: the GT-CFR search loop (left) and the sound self-play training loop that feeds it (right) share one CVPN; the footer strip names the unification across perfect- and imperfect-information games.

Върху тази репрезентация се намират двата основни компонента. **Контрафактичната мрежа за стойност и политика (CVPN)** е една единствена мрежа, която при подадено състояние на убеждението излъчва както вектор от контрафактични стойности (по една за всяко информационно състояние, за всеки играч), така и предходна стратегия — една мрежа, която изпълнява и двете задачи за всеки етап от играта, докато DeepStack използва отделни мрежи само за стойност на рунд. **GT-CFR търсенето** е механизмът, който превръща тази мрежа в стратегия, а цикълът на **правилно самообучение** е този, който обучава мрежата въз основа на собствения изход от търсенето. Офлайн актьорите играят игри чрез самообучение, изпълняват GT-CFR търсене при всяко решение и генерират два вида тренировъчни данни — пълни игрови траектории и състоянията на убеждението, при които мрежата е била запитана по време на търсене — докато треньорите настройват нова CVPN и периодически я връщат обратно. Онлайн агентът изпълнява същото GT-CFR търсене (чрез продължително пререшаване), за да избере всеки ход. Класическите градивни елементи са всички видими — CFR⁺ (Стъпка 3) е вътрешният цикъл на търсенето, приближение на стойност и политика (Стъпка 5) е CVPN, а публични убеждения и разлагане (линията DeepStack/ReBeL) са репрезентацията — но свързващата новост е GT-CFR, търсенето, което *разраства* своето дърво, и правилното самообучение, което поддържа всяко търсене съгласувано с всяко друго.

6.6.3 Ключова иновация: Growing-Tree CFR и правилно самообучение

Приносът на Student of Games е търсещ алгоритъм, който за игри с непълна информация прави това, което търсенето с дърво Монте-Карло направи за игри с пълна информация — интелигентно разраства дърво към линиите, които имат значение — *без* да жертва коректността, която изисква скритата информация. Две идеи го правят работещ, а една единствена теорема го прави общ.

Първата идея е **разрастване на дървото чрез редуване на две фази**. MCTS на AlphaZero разширява един възел при всяка симулация и никога не се връща назад, защото в игра с пълна информация стойността на даден възел, след като бъде изчислена, вече не се променя — бъдещето не влияе върху миналото. При непълна информация това не е вярно: както отбелязва Шмид, наблюдението на *бъдещо* действие на противника променя вашето *убеждение* за частното състояние, което той е държал в миналото, така че всяка промяна на стратегията в дървото се разпространява навсякъде. Затова едно търсене не може просто да разширява и оценява; след всяко разширяване то трябва да преизчислява *цялото* дърво, за да го поддържа самосъгласуван. GT-CFR прави именно това. Всяка итерация изпълнява последователно две фази. **Фазата на актуализация на съжалението** извършва няколко итерации на Public-Tree CFR+ върху *текущото* дърво и където то спре — на своите гранични листа — запитва CVPN за контрафактичните стойности на подиграта отдолу, точно както решаването с ограничена дълбочина използва научена стойност на хоризонта. **Фазата на разширяване** след това разраства дървото: тя симулира една единствена траектория от корена, избирайки действия по rUCT правило, което *смесва* предварителната стратегия на мрежата с текущата cFR policy, и добавя първото публично състояние, до което достига и което все още не е в дървото. Итерирането — „разшири дървото, подобри стратегията, разшири, подобри“ — дава търсене *по всяко време*, което, подобно на MCTS, насочва изчисленията към релевантните линии, но за разлика от него решава за игрално-теоретично съгласувана стратегия на всяка стъпка.

Втората идея е **единственият параметър, който адаптира това едно търсене към двата класа игри**. Кога-

то разширява, AlphaZero добавя само едното най-обещаващо действие, което е идеално, когато оптималната игра може да бъде детерминистична. Но оптималната **игра с непълна информация** обикновено е *стохастична* — трябва да се смесват действия, за да се избегне изтичане на информация — така че ангажирането само с едно действие е грешно. GT-CFR следователно разширява първите k действия по предварително убеждение и задава $k = 1$ за игри с пълна информация (една добра линия е достатъчна; търсенето тогава на практика е това на AlphaZero) и $k = \infty$ — всички деца — за **игри с непълна информация** (така че стратегията може да смесва действия между тях). С $k = \infty$ търсенето дори получава *крайно-времева* гаранция за качеството на стратегията, а не само такава в крайна сметка. Този един превключвател е техническото сърце на уеднаквяването: същият код се свежда до търсене, подобно на MCTS, върху шахматна позиция и до итерация, подобна на CFR, върху покер решение.

Третият елемент е **правилното самообучение**, което обучава CVPN. Подобно на AlphaZero, агентът се обучава да предсказва резултата от собственото си търсене: всяко запитване към мрежата, направено по време на търсене, дефинира под-игра, тази под-игра се пререшава чрез *друго* GT-CFR търсене и мрежата се обучава към изхода от търсенето (с по-високо качество) — стойности чрез Huber loss, стратегия чрез кръстосана ентропия, като резултатите от цялата игра осигуряват останалите целеви стойности. Думата *правилно* е носещата: всяко търсене, използвано за генериране на данни, трябва да бъде съгласувано с мрежата и с другите търсения по траекторията (то не трябва да сглобява фрагменти от две различни равновесия), което се осигурява чрез изпълнение на търсенията върху спомагателна игра за безопасно пререшаване. Това е аналогът при непълна информация на самообучението с бутстрапинг на AlphaZero и именно това кара научените стойности действително да сходяват към игровитеоретичните.

Двойката идва с гаранция, която завършва семейството от **граници на коректност** в тази глава. Теорема 1 гласи, че съжалението на средната стратегия на GT-CFR след T итерации се разделя на член, който акумулира грешката ϵ на мрежата за стойности върху *границата* на дървото (листата, които CVPN оценява) и обикновен член на съжалението на CFR върху *вътрешността* на дървото (възлите, където CFR работи); разделяйки на T , експлоатируемостта на средната стратегия е ограничена от

$$\text{exploitability}(\bar{\pi}^T) \lesssim \underbrace{|\mathcal{F}| \epsilon}_{\text{value-net error (frontier)}} + \underbrace{\frac{|\mathcal{N}| UA}{\sqrt{T}}}_{\text{CFR convergence (interior)}},$$

с $|\mathcal{F}|$ и $|\mathcal{N}|$ съответно размерите на границата и вътрешността, U най-голямата разлика в стойностите и A максималният брой действия. Формата вече е позната — член за грешка в стойността плюс $1/\sqrt{T}$ термин за сходимост на CFR, пряк структурен наследник на този при deepStack's $k_1\epsilon + k_2/\sqrt{T}$ и ReBeL's $\delta C_1 + \delta C_2/\sqrt{T}$ — но две неща са отличителни. Първо, коефициентите са *размерите на растящата граница и вътрешност на дървото*, така че границата е буквално твърдение за инкрементален растеж на дървото: добавянето на възли с течение на времето не струва нищо по отношение на реда на сходимост. Второ, и решаващо, **това същото твърдение важи както за дърво на игра с пълна информация, така и за публично дърво с непълна информация** — това е единствената гаранция за коректност, която обхваща и двата класа. Съпътстващ резултат, Теорема 2, след това показва, че извикването на GT-CFR рекурсивно чрез продължително пререшаване в рамките на цял епизод поддържа агента коректен, като експлоатируемостта нараства

само *линейно* с дължината на играта (фактор приблизително $5D + 2$ за D стъпки на пререшаване) — свойството, което позволява на Student of Games да оцелее в хоризонта от двадесет и четири рунда на Scotland Yard. Заедно теоремите казват това, което AlphaZero можеше само да предполага: с подобряването на мрежата (*andsearchdeepens()*) и задълбочаването на търсенето ($\square \text{MATH110} \square$), играта сходи към равновесие на Наш — сега както за покер, така и за шах.

6.6.4 Ограничения, задънени улици и какво остава неразгледано в статията

В съответствие с модела на главата, основният текст в *Science Advances* представя чистата, високо-ниво история, а инженерната откровеност е изложена в приложението, като теоремите са представени там „само неформално“, докато архитектурите, хиперпараметрите, пълният псевдокод, изчислителната мощност и доказателствата са прехвърлени към Допълнителния текст.

Основното ограничение е открито признато: **Student of Games е по-слаб от AlphaZero в шах и Го при еднакви ресурси**, а разликата не е малка в Го — най-силната му конфигурация спечели само *2 от 400* игри срещу AlphaZero в средна фаза на обучение, т.е. загуби 99,5%, дори докато смазва класическата програма Pachi с над 1100 ело точки. Авторите наричат това „цената на общността на SoG“ и формулират причината точно: CFR просто е по-малко ефективен от търсенето в дърво на Монте Карло при игри с пълна информация, така че унифицираното търсене плаща за своята широчина с пикова сила. Шмид откровено заявява, че това е по дизайн — целта е един алгоритъм „по-добър от хората“ във всичко, а не най-добрият шахматен двигател — но това означава, че унификацията е доказателство за *коректност и компетентност* в различните класове, а не за доминация в нито един от тях.

Две стени за мащабиране са признати като реалните ограничения. Първата е **експлозията в пространството на убежденията**, която Шмид определя като основното ограничение: CVPN трябва да *изброи информационните състояния за всяко публично състояние*, така че неговата цена се мащабира с броя на частните състояния, които играч може да заеме — управляемо за 1326 раздавания в покера, но „забранително скъпо в някои игри“, където представянето „се разпада“. Това е точно ограничението за PBS-вход на ReBeL, наследено непокътнато; статията предлага да се направи приближаване с генеративен модел, който *избира проби* от състояния на света вместо да ги изброява, но не изгражда такъв. Втората е **изискването за известен модел**: подобно на AlphaZero и ReBeL, Student of Games се нуждае от перфектен симулатор на правилата, което според думите на Шмид го поставя „зад линията на AlphaZero“ спрямо muZero, който *научава* своя модел.

Няколко по-малки детайла допълват картината. Системата все още използва **рандомизирана абстракция на залозите** в покера (около двадесет хиляди действия, сведени до четири или пет) — единственото експертно познание, което не е премахнато и е посочено като бъдеща работа. Нейното търсене е **квадратично спрямо броя на итерациите** ($O(kT^2)$) извиквания на мрежата, тъй като дървото нараства с всяка стъпка) — неефективност, която авторите отбелязват, че може да бъде премахната чрез паралелизация, но не е била. Хиперпараметърът c — колко актуализации на съжалението да се изпълняват при всяко разширяване — е признато **недостатъчно проучен** („ $c = 1$ не винаги е най-добрият в практиката ... надяваме се да го проучим по-задълбочено“). А чистият аргумент за сходимост на обучението е **идеализиран**, валиден само „асимптотично, когато $T \rightarrow \infty$ и при много голяма (експоненциална) памет“, така че практиката разчи-

та на крайния капацитет на мрежата. Нито един от тези недостатъци не подкопава резултата, но заедно те определят Student of Games като първо, умишлено общо доказателство за концепция, а не като настроен и мащабируем продукт.

6.6.5 Изчислителна мощност и достъпност

Разходите на Student of Games са концентрирани **извън линия, в обучение чрез самообучение с мащаб TPU**, и статията ги отчита *спрямо AlphaZero, а не в абсолютни стойности* — умишлен избор, който позволява честно сравнение без претенции за хардуерен рекорд. Базовата линия на AlphaZero използва 3500 едновременно изпълнители, всеки на един Google TPUv4, в продължение на 800 000 стъпки на обучение, а Student of Games „е обучен с подобно количество изчислителна мощност TPU“, като обучението по домейни варира от приблизително един милион стъпки (Го) до седемнадесет милиона (Scotland Yard). Самото пояснение на Шмид е, че шахът и покерът могат да бъдат обучени „на няколко GPU“, докато Го „безспорно най-трудното“ е изисквало голям пул от TPU — главно защото хардуерът е бил наличен, а не защото методът го е изискувал. Важно е да се отбележи, че **няма една единствена сума в долари** от рода на тази, с която Pluribus стана известен: тъй като сравнението е обвързано с *бюджет за мрежови извиквания*, а не с реално време, честният извод е „сравним с AlphaZero и значителен“ — всъщност авторите включват намаляването на тази изчислителна мощност сред своите нерешени проблеми. По време на игра търсенето е по всяко време и може да се настройва, но неговият $O(kT^2)$ растеж (който се свежда до $O(T)$ за игри с пълна информация, където възелът трябва да бъде оценен само веднъж) означава, че силните конфигурации също не са евтини.

По отношение на достъпността присъдата е смесена и заслужава да бъде изразена точно. **Пълният агент Student of Games и неговите обучени мрежи не бяха публикувани** — няма декларация за достъпност на кода освен данните в статията и допълнението — така че по оста „главен артефакт“ той е заедно със затворения Libratus и Pluribus, а не с отворения ReBeL. Това, което е отворено, е заобикалящата екосистема: същата група DeepMind/Alberta поддържа **OpenSpiel**, който предоставя cFR family и самите еталонни тестове (ледюк, лъжливи зарове, Scotland Yard), така че компонентите и средите са възпроизводими, въпреки че сглобената система не е. На практика това прави Student of Games *отворен метод върху отворена основа със затворен флагман* — разумна средна позиция, но такава, която оставя пълното възпроизвеждане на резултатите за шах/Го/покер/Scotland Yard извън обсега на типична група.

6.6.6 Силни страни и ограничения

Силата на Student of Games е в **обобщеност с коректност** — комбинация, която нито една предишна система в тази глава не предлагаше. Това е първият алгоритъм, който е *доказуемо коректен както за игри с пълна, така и за игри с непълна информация* (Теорема 1–2), и той подкрепя теорията с широчина, която никой конкурент не можа да достигне: в един-единствен дизайн достига експертно до професионално ниво в **шах и Го, побеждава Slumbot**, най-силния публично достъпен бот за хедс-ъп безлимит покер (с около +7 mbb/g — единицата за коефициент на печалба от предишните раздели, която тази статия записва като mbb/hand — и решаващо е, че *не* бива експлоатиран от сондаж за най-добър локален отговор, който хваща по-слабите ботове, които той побеждава с +434 mbb/g), и **побеждава PimBot**, агента за Scotland Yard на най-съвременното ниво, изграден върху търсене в дърво на Монте Карло с ръчно настроени евристики, дори когато на PimBot

са дадени десет милиона симулации за търсене спрямо няколкостотин на Student of Games. Той използва **без човешки данни, без предварително изчислен план и без абстракция на картите**, като научава всичко от самообучение; той **се мащабира** с изчислителна мощност (повече търсене и повече обучение понижават експлоатируемостта, емпиричното лице на Теорема 1); и демонстрира, че *усъвършенства своята политика по време на тестване чрез търсене*, „за разлика от всеки чист алгоритъм за обучение с подкрепление“ — най-ясното изявление в главата за търсене като извод.

Ограниченията са точната форма на неговата амбиция. Той е **по-слаб от специалистите** в техните собствени области — особено забележимо в Го — признатата цена за един алгоритъм за всичко. Неговите гаранции, подобно на тези на Recursive Belief-based Learning, важат само за **игра за двама с нулева сума** (детективите в Scotland Yard са обединени в един отбор, за да се вместят), така че *многопотребителската* празнина в сигурността, разкрита от Pluribus, остава недокосната дори тук. Неговото **представяне на убеждения не се мащабира** до игри с огромни частни състояния и то **изисква известен модел** на средата — двете стени, които самите автори отбелязват първи. То запазва малка **абстракция на залозите**, неговото търсене е **квадратично** по отношение на итерациите и — уникално неудобно за твърдението за обобщеност — **флагманският код никога не беше публикуван**. Честният прочит е, че Student of Games доказва, че уеднаквяването е *възможно и коректно*, и плаща цената на това доказателство в пикова сила, мащаб и възпроизводимост.

6.6.7 Наследство и съвременна значимост

Student of Games е завършекът на аргумента в тази глава и обединява трите ѝ сюжетни линии едновременно. В рамките на петте системи полето премина от **ръчно изградена абстракция към научено невронно приближение** (DeepStack и ReBeL премахнаха абстракцията, от която зависеха Libratus и Pluribus), и от **решаване на всичко офлайн към обучение и търсене** (планът се сви от петабайтите на Libratus до мрежа за стойности и решаване в реално време). Student of Games добавя третото и последно уеднаквяване — **пълна и непълна информация в един алгоритъм** — и с това преформулира цялата сюжетна линия. Концептуалният извод е, че рецептата на AlphaZero (самообучение, научена мрежа за стойности и политики и дърво, пораснало чрез насочено търсене) никога не е била специфична за пълна информация; тя просто се нуждаеше от *коректно* търсене върху *убеждения*, а не върху състояния. Заменете **търсене в дърво на Монте Карло** с GT-CFR и го изпълнете върху публични убеждения, и същата рецепта обхваща шах и покер с една единствена гаранция за сходимост. Това е централната теза на главата, изразена буквално: един алгоритъм, една мрежа, едно търсене, четири много различни игри.

Отношенията му към околния фронт са изясняващи. Student of Games е *общата* рамка, към която ReBeL насочи — той развива своето дърво вместо да фиксира ограничена по дълбочина под-игра, отделя търсенето по време на изпитване от обучението (така че, подобно на AlphaZero, може просто да търси по-усилено по време на изпитване за по-силна игра) и е валидиран в различни класове игри, а не само върху игри с непълна информация. Най-видимият му *непокрит* съсед е **MuZero**: докато MuZero премахва нуждата на AlphaZero от известен модел чрез научаване на динамиката, Student of Games все още предполага перфектен симулатор, така че естествената следваща стъпка — обединяване на GT-CFR с научен модел и генеративна извадка на убеждения — би разтворило и двете му посочени ограничения наведнъж. И той се намира на една ос от **CICERO**, играча на Diplomacy на Meta, който достига човешко ниво, като пренася търсене плюс

обучение в игра със седем играчи, със смесени мотиви и на естествен език: ярко напомняне, че Student of Games уеднаквява *информационната структура*, но не и *броя играчи*. Няколко от неговите части вече са използвани сами по себе си — **GT-CFR** като търсене с нарастващо дърво, полицейно насочено и валидно за всеки клас игри; **CVPN** като единна глава за стойности и политики върху състояния на убеждения; **правилно самообучение** като начин за буутстрапинг на целеви стойности от взаимно съгласувани под-търсения; и **независимо от домейна продължително пререшаване**, което вече не се опира на специалната структура на покера.

За тази дисертация по-специално Student of Games е едновременно таван и пътепоказател. Като най-общата **рамка за звуково търсене**, базирана на PBS, тя е естественият контекст за **принос 1**: нейното убеждение — разпределение върху възможните информационни състояния на противника — е точно субстратът, който трябва да бъде *обогатен* с убеждения относно *типа* стратегия на противника, превръщайки убеждение, търсещо Неш равновесие, в адаптивно такова. И неговото внимателно ограничаване на всички гаранции до **двуйгрови игра с нулева сума** — статията изрично посочва, че Неш е „по-малко значим“ извън нея — преформулира, от най-напредналата гледна точка в литературата, точния пропуск, към който е насочен **принос 2**: уеднаквяването на *информационната структура* на игрите не направи нищо, за да затвори *многоагентната безопасност*, която Pluribus първо разкри. Това е най-чистият възможен преход от тази глава към останалата част от дисертацията, а **синтезът**, който следва, обединява петте системи — **таблица на еволюционните промени** за това какво всяка добави и загуби, диаграмата на еволюцията и картата за повторна употреба на компоненти — за да направи нерешените проблеми и приносите, които ги адресират, явни.

6.7 Синтез

С петте системи на разположение синтезът ги обединява — не за да определи победител, а за да открие три основни момента: формата на седемгодишната еволюция, общите механизми, които системите използват, и нерешените проблеми, с които тази глава предава щафетата на останалата част от дисертацията. Оценъчните карти за всяка система вече съдържат подробностите по отделните измерения, така че тук се прави обобщение, а не повторно извеждане на тези данни.

6.7.1 Дъгата в едно четене

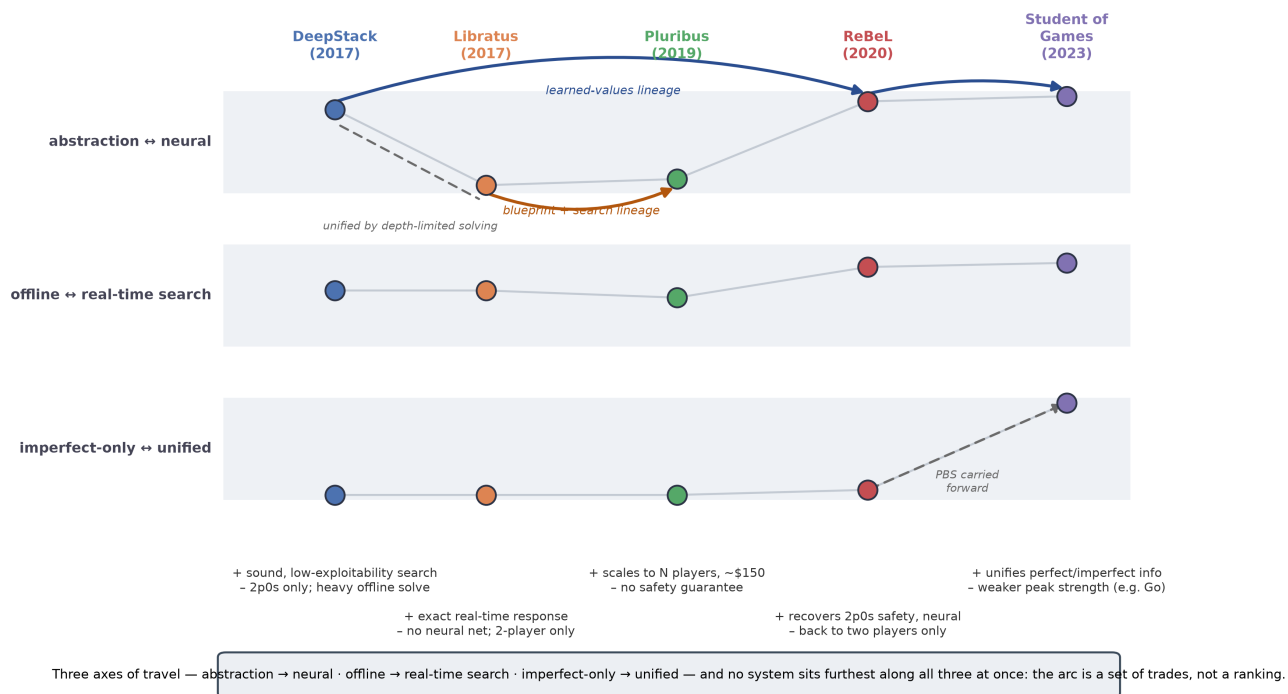
Прогресията е най-добре да се разглежда не като изкачване, а като разходка по трите оси от въведението, при която всяка стъпка напред по една ос беше платена с нещо друго. DeepStack и Libratus откриха десетилетието, като атакуваха една и съща слабост — парадигмата „абстракция и план“ със загуби — от противоположни посоки: DeepStack изостави парадигмата и я замени с научени стойности и продължително пререшаване, докато Libratus я запази и добави доказуемо безопасна корекция в реално време. И двете системи останаха двуйгрови и нито една от тях не съобщи за резултата на другата, така че още в началото „по-добър“ вече беше многоизмерен. Pluribus след това придвижи оста *брой играчи* до шест — първия свръхчовешки резултат във всеки еталонен тест отвъд двама играчи или отбори — но го направи, като се отказа напълно от безопасността и остана табличен — странично движение по представителната ос и назад по гаранциите. ReBeL обърна и двете тези отстъпки, връщайки се към двама играчи, за да *възстанови* гаранция за коректност, като същевременно придвижи представителната ос най-далеч досега — публични състояния на убеждения със

стойности и политики, обучени чрез самоигра, без абстракция и без план — a Student of Games завърши картината, като добави третата ос, обединявайки игра с пълна и непълна информация в един алгоритъм, за сметка на пиковата сила във всяка от тях. Честният едноредов обобщен извод е, че областта замени *ръчно изградена широчина с научена общност и предварително изчисление с търсене в реално време*, и че нито една система не доминира по всички оси наведнъж.

Таблицата по-долу представя тази разходка като отчет за това какво е добавила всяка система и каква е била цената, което представлява полезното сравнение при еволюционен прочит; фиксираните стойности по измерения (играчи, тип игра, план, невронна компонента, търсене, абстракция, пълна информация, изчислителна мощност, ключова иновация) са във всяка оценъчна карта на раздел и тук не се повтарят.

Система (година)	Какво добавя	Какво жертва / каква е цената
DeepStack (2017)	Първо <i>коректно</i> търсене с ограничена дълбочина при скрита информация; научени контрафактични стойности вместо съхранена стратегия; изоставя парадигмата „абстракция и план“ на цялата игра	Все още запазва рядка абстракция на залозите в рамките на своето предвиждане; никога не е тестван директно срещу предишни ботове; приблизително 175 CPU-години офлайн етикетиране на стойности
Libratus (2017/18)	Първа директна <i>и</i> човешка победа чрез вложено безопасно решаване на под-игри в реално време с доказуема граница; самоусъвършенстващ се за една нощ, който поправя собствените си пропуски	Запазва и зависи от абстракция + петабайтов план; без неврално обобщаване; изчислителна мощност в мащаба на суперкомпютър (~25 милиона процесорни часа)
Pluribus (2019)	Първа свръхчовешка многоиграща (шест играчи) игра; прочуто евтина (~150 долара на един сървър)	Отказва се от <i>всички</i> гаранции за безопасност (без N-игрови граници); разчита на <i>небезопасно</i> търсене; остава изцяло табличен и абстрахиран

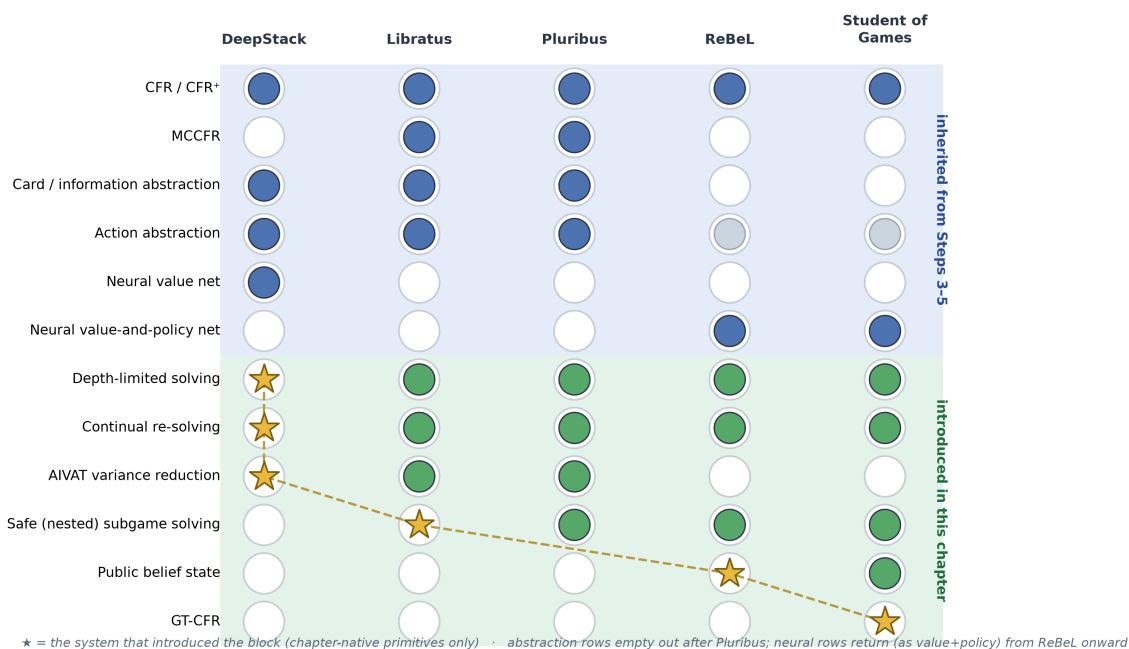
Система (година)	Какво добавя	Какво жертва / каква е цената
ReBeL (2020)	„AlphaZero за непълна информация“: публични състояния на убежденията + научена стойност/политика + CFR в пространството на убежденията; <i>възстановява</i> гаранцията за 2p0s; елиминира абстракцията <i>и</i> плана; с отворен код за Liar’s Dice	Гаранциите се връщат към двама играчи; експлозия в пространството на убежденията там, където общото знание е оскъдно; обучение върху GPU клъстер; изисква известен модел
Student of Games (2023)	Обединява пълна и непълна информация в един коректен алгоритъм (GT-CFR + една мрежа за стойност и политика + правилно самообучение)	По-слаб от специалистите (осезаемо при Го); гаранциите все още са 2p0s; експлозия в пространството на убежденията; изисква известен модел; основният код не е публикуван



Фигура 6: The seven-year arc as trade-offs: the five systems placed along three axes (abstraction → neural; offline → real-time search; imperfect-only → unified), with lineage arrows and a per-system capability-gained/given-up tag.

6.7.2 Какво се пренася напред

Почти нищо в тези системи не е изцяло ново; всяка от тях представлява рекомбинация на малък набор от градивни елементи, повечето от които са въведени в стъпки 3–5. **CFR и CFR⁺** (стъпка 3) са вътрешният решаващ на всяка система; **Монте Карло CFR** (стъпка 3) обучава планове на Libratus и Pluribus; **абстракция на действията** (стъпка 4) е в основата на Libratus и Pluribus и оцелява само като остатъчно меню за залагания в ReBeL и Student of Games; а **невронно приближение на стойността** — а след това на **стойност и стратегия** (стъпка 5) е именно това, което позволява на deepStack, ReBeL и Student of Games да разтворят абстракцията, от която зависят останалите. Върху тези глават налагат свои собствени примитиви, всеки въведен от една система и използван повторно от нейните наследници: **ограничено по дълбочина решаване, продължително пререшаване, безопасно вложено решаване на под-игри, публично състояние на убежденията и Growing-Tree CFR**. Прочетете надолу списъка с *първи появи* и еволюцията на главата се преразказва сама чрез компоненти: блоковете се натрупват, тези от ерата на абстракцията избледняват, а научените поемат контрол.



Фигура 7: Component-reuse map: Step 3–5 building blocks and chapter-native primitives (rows) against the five systems in chronological order (columns); stars mark the system that introduced each native primitive, connected by a staircase of first appearances.

6.7.3 Защо това е важно за нашето изследване

Тази глава представлява съвременното равнище, върху което дисертацията се изгражда и от което се отклонява, а нейният най-важен принос към тази работа е рамкиращ. Всяка система, разгледана тук, е **независима от противника по дизайн**: всяка изчислява най-лошосценарийна устойчива стратегия и я играе безусловно, а двете многопотребителски и тествани с хора системи правят тази позиция явна — те отказват да моделират или да се адаптират към противниците си, за да не бъдат контраексплоатирани, а Pluribus дори не знае

срещу кого играе. Прочетена като обзор, следователно, главата е каталог на пет системи от световна класа, които споделят точно една пропуск: нито една не се адаптира към конкретния, погрешим противник пред себе си. Тази обща пропуск представлява отвора, който заема цялата тази дисертация, а останалата част от този подраздел разглежда активите на главата спрямо трите планирани приноса.

За принос 1 (Рамка за поведенческа адаптация). Най-прекият принос на главата е **публичното състояние на убеждението (PBS)**, въведено от ReBeL и наследено от Student of Games — разпределение върху скритите състояния, които противникът може да заема при наличие на общо знание, което принос 1 разширява, за да носи убеждения относно *типа стратегия* на противника, а не само неговите карти. Два по-малко очевидни момента изострят това. Първо, PBS е изобретен, за да направи *търсенето доказуемо коректно*, а коректността е ортогонална на моделирането на противника — така че изграждането на адаптация върху PBS наследява целия преизчислителен механизъм и неговите гаранции за сходимост *непроменени*, позволявайки на адаптивното убеждение да бъде актуализирано вътре в доказуемо коректно търсене, вместо да бъде добавено отвън; адаптацията получава възможността да използва повторно не само структура от данни, но и доказателствата, прикрепени към нея. Второ, всяка система в главата споделя **двустепенна архитектура** — евтина глобална стратегия (план или научена мрежа за стойности), усъвършенствана чрез локално решаване в реално време — и собственият дизайн на дисертацията, „започнете от безопасна стратегия и се подобрявайте срещу слаби противници“, е същата двустепенна форма, при която целта на второто ниво е променена от *точност* към *ограничена експлоатация*. Следователно главата предоставя както субстрата за адаптация, така и системния скелет, в който тя се вписва.

За принос 2 (Многоагентна безопасна експлоатация). Ключовият елемент е Pluribus: свръхчовешки покер за шестима играчи, постигнат *без* каквато и да е гаранция за безопасност, което е точно пропускът, който една теория за многоагентна безопасна експлоатация трябва да запълни — точка, която главата заявява директно. По-фин прочит превръща собствената уговорка на системите в оправдание за приноса. Pluribus и Student of Games и двете ограничават своите гаранции до игра за двама с нулева сума и признават, че извън нея равновесието на Наш нито е уникално, нито може да бъде изчислено ефективно, нито дори представлява гаранция срещу загуба. Ако равновесието не осигурява безопасност в многопотребителската среда от самото начало, тогава отказът да се експлоатира видимо слаб противник означава отказ от гаранция, която никога не е била притежавана — което преформулира умишлената експлоатация от безразсъдно отклонение от безопасността в *рационална цел*, след като безопасността е недостижима само чрез равновесие. Това е най-чистата мотивация, която литературата предлага за многоагентната експлоатация, която принос 2 преследва.

За принос 3 (Методология за оценяване). Глава стандартизира **експлоатируемостта** — най-лошата загуба спрямо **най-добре реагиращ противник** — като универсален измерител, прилаган еднакво към всичките пет системи, и тази однородност сама по себе си представлява шаблон за **рамка за оценяване между системи**. Тя също така предоставя на дисертацията два инструмента, предназначени за нова употреба. **Локалната най-добра реакция (LBR) сонда**, използвана в цялата глава за *сертифициране* на **неексплоатируемост**, се превръща в *тест за безопасност*, когато бъде насочена към **адаптивен агент** — прави ли преследването на слаб противник вас **контраексплоатируем**? — което е точно антагонистичната, стил „учебна атака“ оценка, от която се нуждае стъпка 8. А AIVAT, **намалителят на дисперсията на научената стойност** зад докладваните резултати на DeepStack и Pluribus, е инструментът, който прави малкото, но *растящо* предимство на

адаптацията статистически видимо срещу бруталната дисперсия на картите игри, където **суровите печалби** биха подвели.

По-нататъшни опорни точки (кандидат — за запазване или премахване според крайната дължина). Следните са компактни екстраполации, а не твърдения, вече направени от системите; всяко представлява едноредово **начално число** за по-късни стъпки.

- **Многократно използвана форма на доказателство (Принос 2).** Всяка гаранция в главата има една и съща форма — термин за приближение на стойността плюс $1/\sqrt{T}$ термин за сходимост (DeepStack's $k_1\epsilon + k_2/\sqrt{T}$, Libratus's 2Δ , ReBeL's $\delta C_1 + \delta C_2/\sqrt{T}$, SoG's $|\mathcal{F}|\epsilon + |\mathcal{N}|UA/\sqrt{T}$) — което дава на Принос 2 готов шаблон за ограничаване на експлоатируемостта, *породена* от умишлено отклонение.
- **Експлоатация с ограничена дълбочина (Принос 1/Принос 2).** Многозначните листа на Pluribus (k избираеми стратегии за продължение) структурно представляват меню от поведения на противника на хоризонта; преинтерпретирането на това меню като *типове* на противника води до управляема експлоатация отвъд дълбочината на търсенето — точният проблем, който стъпка 8 „адаптиране отвъд границата на дълбочината“ разглежда.
- **Карта за изграждане спрямо цитиране (методология).** Подразделите „Изчислителна мощ и достъпност“ сортират областта на това, което е възпроизводимо в мащаба на докторска дисертация (Single-Server CFR на Pluribus, CPU решавач с един поток на ReBeL, отворената OpenSpiel среда, малки игри) спрямо това, което е само за цитиране (свръхкомпютърът на Libratus, TPU басейнът на SoG) — неясната обосновка за изграждане върху евтината, отворена линия (задачата reBeL-Lite-on-Leduc), вместо възпроизвеждане на водещ продукт.
- **Пренасочен механизъм (рамкиране).** Самоусвършенстването през нощта на Libratus е корекция на стратегията в мача, насочена към *собствените* му слабости; дисертацията изпълнява същия цикъл наблюдение—последвано от корекция, насочен към *противника*, така че приносът е пренасочване на доказан свръхчовешки механизъм, а не неговото изобретяване.
- **Кука за разказ за границата (рамкиране).** И петте системи вече са цитирани като ранни инстанции на „изчислителна мощ по време на тест“ — изразходване на търсене в момента на вземане на решение, вместо всичко да бъде вградено в предварително изчислена стратегия — така че рамкирането на адаптацията в реално време като изчислителна мощ за *адаптация* по време на тест свързва тази работа със съвременния разказ за моделите на разсъждение.
- **Съгласуваност като безопасност (Принос 2).** „Правилното самообучение“ на Student of Games изисква всяко локално търсене да остане в *съответствие* с безопасна референция, концептуално същото ограничение като регулираната с КЛ-дивергенция (PiKL) експлоатация, която Принос 2 ще използва — аналогия, която си струва да бъде заета.
- **Абстракция, прехвърлена към типово пространство (Принос 3).** Урокът от стъпка 4, че абстракцията със загуби създава експлоатируеми шевове, се пренася от пространството на състоянията към пространството на *типовете* на противника: твърде грубият типови модел е експлоатируем по същия начин, както твърде грубата абстракция на карти, предсказвайки режим на отказ за Принос 3 да улови.

6.7.4 Нерешени проблеми и предаването към стъпки 7–15

Четири нерешени проблема произтичат пряко от синтеза и всеки е начално число за по-късна стъпка. Първият е самата **невидимост за противника**: цялата граница изчислява фиксирана, **стратегия, оптимална в най-лошия случай** и я играе безусловно, оставяйки празнината между статично равновесие и динамичен, **съобразен с противника начин на игра** широко отворена — предмет на стъпка 7 (извеждане на поведението на противника от **следи от действия**) и извеждащата част на **принос 1**. Вторият е **безопасност в мултиплейър среда**: Pluribus доказва, че методите „**търсене по Наш и търсене**“ *печелят* в N-играч игри с **непълна информация**, без да предлагат никаква **гаранция за безопасност**, а Student of Games потвърди, че дори най-общата **рамка за звуково търсене** все още ограничава своите гаранции до двама играчи — така че управляема теория на *безопасна експлоатация отвъд игра за двама с нулева сума* (стъпка 8, **принос 2**) остава наистина нерешена. Третият е **адаптация отвъд границата на дълбочината**: решаващите в реално време системи в главата гледат само малко напред и носенето на **модел на противника** по звуков начин отвъд този хоризонт — вместо да се изхвърля на листа — е нерешен проблем, който стъпка 8 разглежда пряко. Четвъртият е **бюджети за изчисления в реално време**: разходите на системите варират в шест порядъка и **адаптивен агент**, който трябва да пререшава и да преоценява противник онлайн, наследява най-строгата версия на това ограничение.

Тези проблеми определят фаза D (стъпки 7–8), критичното ядро на дисертацията, и се разклоняват през следващите фази — динамика на многоагентни системи и формиране на коалиции (фаза E), поведенчески конвейери, задвижвани от данни, които основават моделите върху реални следи (фаза F), и рамката за междудоменна оценяване, която затваря плана (фаза G). Петте системи в тази глава колективно представляват платформата, върху която стъпва настоящата дисертация, както и контрапункта, спрямо който тя се определя: тя наследява техните представяния на състояния на убежденията, тяхното търсене в реално време и тяхната дисциплина по отношение на експлоатируемостта и се стреми да добави една способност, която всички те, по дизайн, пропускат — способността да забележи, че противникът не играе оптимално и да предприеме действия по въпроса, без самата тя да стане експлоатируема в замяна.

Стъпка 7 Обобщение — Моделиране на противника в игри с непълна информация

Изследване върху възможностите за прилагане на изкуствен интелект в компютърните игри

Alexander Andreev

July 2026

Съдържание

7	Стъпка 7 — Моделиране на противника в игри с непълна информация	1
7.1	1. Защо е необходимо моделиране на противника — печалбата, която наш равновесие пропуска	2
7.2	2. Байсовото ядро — убеждение, доказателство и реакция	3
7.3	3. Виждане през ключалка — проблемът с частичната наблюдаемост	6
7.4	4. Три модела с единен интерфейс	7
7.5	5. Проблемът на съгласуваността и решението във последователна форма	9
7.6	6. Когато моделът е уверен, но греша	11
7.7	7. От модела до печалбата — най-добрият отговор, таванът и самонанесеното изтичане . . .	13
7.8	8. Адаптиране към промяна — нестационарни опоненти	15
7.9	9. Връзки и бъдещи насоки	16

7 Стъпка 7 — Моделиране на противника в игри с непълна информация

Това е цялостна глава за моделиране на противника: формулировката на проблема, математическият му апарат, семейството от методи и набор от контролирани експерименти, проведени върху две малки покер игри. Тя изпълнява двойна функция — служи като самостоятелно въведение, което по-късните стъпки надграждат, и представлява основен източник за публичния синтез в Стъпка 15. Главата е написана така, че да бъде разбираема и без предварителна познатост с програмния код на проекта. Всички представени експериментални резултати са получени от възпроизводими изпълнения на двете тестови среди (кун покер и ледюк холдем) и, когато е възможно, са ограничени чрез *точни* аналитични препратки вместо симулирани такива.

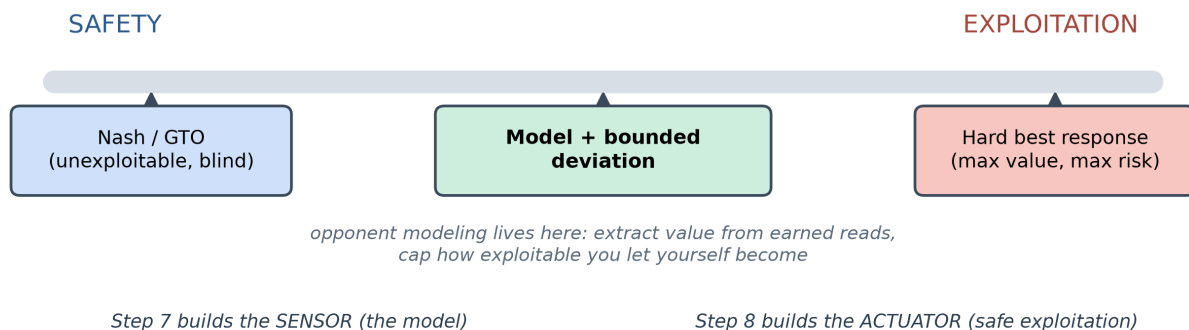
Къде се вписва това в дисертацията. Игровитеоретичен агент, изчислен в по-ранните стъпки, определя *равновесие на Наш* — стратегия, която не може да бъде победена в дългосрочен план. Настоящата стъпка изгражда противоположната способност: **сензор**, който наблюдава как *конкретен* противник действително играе и превръща тези наблюдения в оценка на неговата стратегия, така че агентът да може да се отклони от равновесието с цел да накаже грешките на този противник. Този сензор представлява първата половина от рамката за поведенческа адаптация на дисертацията (Принос №1). Втората половина — как да се експлоатира *безопасно*, без да се създаде уязвимост в собствената игра — е формализирана в Стъпка 8.

7.1 1. Защо е необходимо моделиране на противника — печалбата, която наш равновесие пропуска

Равновесна стратегия на Наш е изградена така, че в дългосрочен план никога да не води до загуба, *независимо срещу кого се играе*. В игра за двама с нулева сума тази гаранция е реална и ценна: една равновесна стратегия има фиксирана *стойност* и нито един противник — колкото и да е умен — не може да я свали под нея. Но същото свойство представлява и **ограничение**. Една равновесна стратегия играе **идентично** както срещу световен шампион, така и срещу някой, който се отказва всеки път при направен залог. Тя никога не се адаптира, тъй като именно липсата на адаптация е основната ѝ характеристика.

Този отказ да се адаптира има цена и тази цена е измерима. **Моделирането на противника** е актът на наблюдение на действията на конкретен противник, формиране на убеждение относно неговата стратегия и **отклонение от равновесие на Наш с цел експлоатиране на модела** — блъфиране повече срещу някой, който се отказва твърде често, залагане за стойност по-тънко срещу някой, който плаща твърде често.

Най-ясният начин да се види стойността е чрез картина „камък-ножица-хартия“. Вашият противник тайно хвърля камък 70% от времето. Безопасната стратегия е да **рандомизирам** равномерно и да **изравнявам** резултата завинаги. Но ако *забележите* пристрастието, хвърляте повече хартия и започвате да печелите. Две неща правят тази стъпка цяла в миниатюра: трябва да **изведа** пристрастието от шумен поток от хвърляния (няколко не са достатъчни) и ако се **ангажирам прекалено** с хартия, ставате предвидими и те ви смазват с ножица. Извеждането на пристрастието е моделиране на противника; знанието докъде да се наклоните е компромисът между експлоатация и безопасност, който преминава през всичко по-долу.



Фигура 1: The safety-exploitation dial: pure equilibrium play is unexploitable but blind; a hard best response extracts the most value but is maximally risky. Opponent modeling operates in between. Step 7 builds the sensor (the model); Step 8 builds the actuator (safe exploitation).

7.1.1 Колко е заложено — измерено в кун покер

Кун покер е най-малката нетривиална покер игра: тесте от три карти (Вале, Дама, Поп), по една карта на всеки, един рунд за залози. Тя е достатъчно малка, за да бъде решена *точно*, което ни позволява да изчислим стойността на моделирането, вместо да я предполагаме. За всеки от няколко постоянни противникови стила

ние изчислихме три точни величини:

- **Нашева очаквана стойност** — това, което равновесната стратегия носи срещу този противник (на ръка);
- **Стойност на най-добрия отговор** — това, което стратегия, която *познава противника перфектно*, носи: точният таван за експлоатация;
- **разликата между тях** — сумата, която равновесната игра оставя неизползвана.

Противник (Кун)	Нашева очаквана стойност	Стойност на най-добрия отговор	Пропуснато от експлоатация
Пасивен играч (никога не се отказва)	+0.119	+0.333	0.215
Камък (ангажира се само с Поп)	-0.060	+0.167	0.227
Маниак (винаги залага/плаща)	+0.121	+0.333	0.213
Равновесие на Наш (равновесна игра)	-0.055	-0.052	0.004

Срещу трите експлоатируеми стила разликата е огромна — **от 0.11 до 0.28 на ръка**, в игра, чиято цялостна равновесна стойност е около една двадесета от фиш. Срещу наш противник разликата е по същество **нула**, точно както изисква теорията: не можете да експлоатирате равновесие, защото най-добрият отговор не може да надмине стойността на играта. (Малките отрицателни Нашеви стойности са известният недостатък на първия играч в кун покер от $-1/18 \approx -0.056$ — това е характеристика на самата игра, а не грешка.)

Два урока вече се открояват. Първо, **моделирането си струва**: стойността е значителна и реална. Второ, **експлоатацията е еднопосочна** — точният най-добър отговор на „камък“ *блъфтира повече* (той залага най-слабата ръка в отворен пот, защото „камъкът“ се отказва от всичко освен Поп). Останалата част от тази глава е посветена на това как да реализираме тази стойност, без да я губим.

Прочетете повече: Southey, F. и др. (2005). „Блъфът на Бейс: Моделиране на противника в покера.“ *UAI*.

7.2 2. Байсовото ядро — убеждение, доказателство и реакция

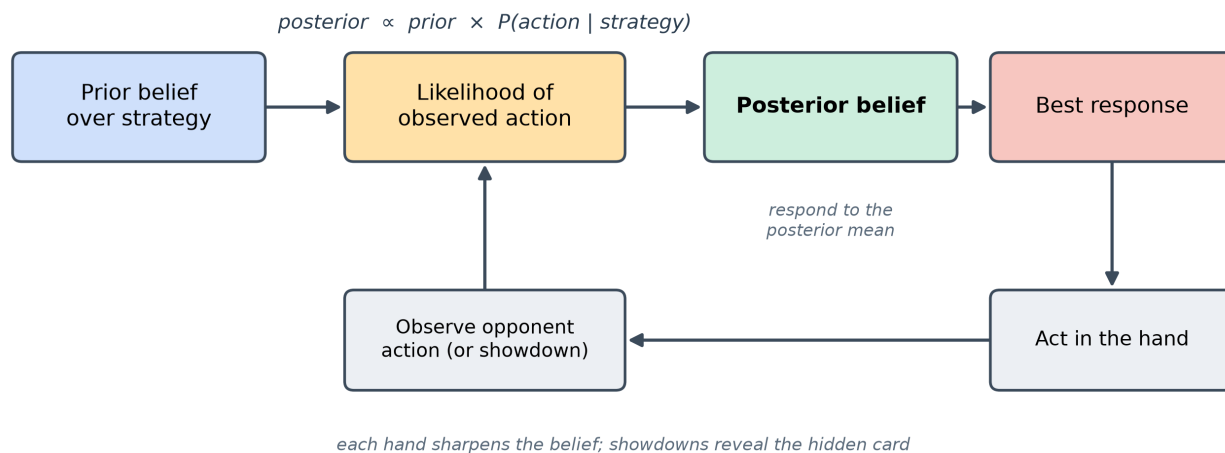
Под всеки метод в тази глава стои един цикъл, който е добре да бъде обяснен с прости думи преди въвеждането на каквито и да било означения. Представете си, че сте детектив. Започвате с предположение за начина, по който играе противникът (**предварително убеждение**). Всяко негово **действие** е следа: то прави някои обяснения по-вероятни, а други — по-малко вероятни. Вие включвате тази следа в подозренията си (**апостериорно разпределение**) и повтаряте процеса. **Моделиране на противника** *представява* именно този цикъл:

belief about their strategy $\rightarrow$ see an action $\rightarrow$ update belief $\rightarrow$ best-respond $\rightarrow$ repeat.

Формално, обновяването се извършва по правилото на Бейс. Ако σ обозначава кандидат-стратегия на противника и a е действието, което току-що беше наблюдавано,

$$P(\sigma | a) \propto P(\sigma) \cdot P(a | \sigma),$$

с думи: **апостериорното $\propto$ предварителното убеждение $\times$ колко добре дадена стратегия обяснява току-що извършеното действие.** Действие, което кандидатът смята за невъзможно, намалява вероятността му към нула; действие, което предвижда добре, я увеличава. След множество раздавания убеждението се съсредоточава върху най-подходящото обяснение.



Фигура 2: The opponent-modeling loop: a prior belief is multiplied by the likelihood of each observed action to give a posterior, which drives a best response; the outcome feeds the next observation.

7.2.1 Зоологическа градина от типове

Най-простият начин да направим това конкретно е да разсъждаваме върху малък набор от предварително дефинирани противникови **типове**, вместо върху цялото безкрайно множество стратегии. В рамките на главата използваме повтарящ се набор от типове, всеки от които изпълнява двойна роля — като противник, срещу когото да играем, и като хипотеза, която детекторът обмисля:

Тип	Поведение
Пасивен играч	Никога не залага, никога не се отказва — проверява, когато има възможност, и плаща всеки залог.

Тип	Поведение
Камък (стегнат-пасивен)	Вкарва жетони само с най-добрата ръка; се отказва от всичко останало. Най-експлоатируемият тип.
Маниак (разпуснат-агресивен)	Винаги залага или плаща, независимо от ръката си.
Наш равновесие	Балансирана равновесна игра; използва смесени стратегии; неексплоатируем.

7.2.2 Актуализацията, изчислена на ръка

Механиката е най-лесна за разбиране, когато сами завъртите механизма веднъж. Да предположим, че трима кандидати залагат определена ръка с вероятност 0.8, 0.5 и 0.1 съответно, като започват от равномерно априорно разпределение $(\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$. Наблюдавате **залог**:

$$\text{posterior} \propto (\frac{1}{3} \cdot 0.8, \frac{1}{3} \cdot 0.5, \frac{1}{3} \cdot 0.1) = (0.267, 0.167, 0.033) \xrightarrow{\text{normalize}} (0.571, 0.357, 0.071).$$

Наблюдаваме **втори** залог и отново умножаваме по (0.8, 0.5, 0.1):

$$(0.711, 0.278, 0.011).$$

Убеждението се натрупва върху типа играчи, които правят чести **залози**, и би го направило *още по-бързо*, колкото повече кандидатите се разминават в мненията си. Тази интуиция „колко много се разминават“ се появява отново в раздел 6 като обяснение защо някои опоненти се идентифицират след четири **раздавания**, а други – след двеста.

7.2.3 Защо дирихле и защо ви е нужна само средната стойност

Два факта от литературата оформят всеки практически модел. Първо, когато оценявате разпределение на действия чрез броене, естественото предварително убеждение е **дирихле** разпределението, защото то е *спрегнатото* предварително убеждение на **мултиномиалната**: актуализирането на дирихле с наблюдавани броеве води до друго дирихле разпределение, при което броевете просто се **добавят** към **псевдоброевете** на предварителното убеждение. Няма нужда от интеграл или **симулация** — средната стойност апостериори на действие i е

$$\mathbb{E}[p_i] = \frac{\alpha_i + n_i}{\sum_j (\alpha_j + n_j)},$$

където α_i представляват предварителни псевдоброеве (които едновременно изпълняват ролята на *изглаждане*, като предотвратяват стойността нула за което и да е действие), а n_i са наблюдаваните броеве. Тази **затворена форма** съответства на „**непрекъснатия**“ модел, разгледан в Раздел 4.

Второ — и това е единственият най-често използван резултат в областта — за да изберете **оптимален отговор**, никога не е необходимо цялото **апостериорно разпределение** върху стратегиите; достатъчна е само неговата **средна стойност**. Вашата **очаквана печалба** срещу *разпределение* на стратегиите на опонента е равна на вашата **печалба** срещу единствената осреднена стратегия. Това свежда **неразрешим** интеграл върху **пространството на стратегиите** до „**оптимално отговарям** на един осреднен опонент“ и именно това прави **байесовата експлоатация** практична. (Раздел 5 се връща към това: отговарянето на средната стойност е **оптимален по отношение на изплащането**, но, както се оказва, може да не *сходява* към **истинската стратегия** на опонента — тънкостта, която работата от 2025 г. разрешава.)

Прочетете повече: Ganzfried, S. & Sun, Q. (2016/2018). “Bayesian Opponent Exploitation in Imperfect-Information Games.” *IEEE CIG*. (Теорема 2.1: отговаряйте на средната стойност апостериори.)

7.3 3. Виждане през ключалка — проблемът с частичната наблюдаемост

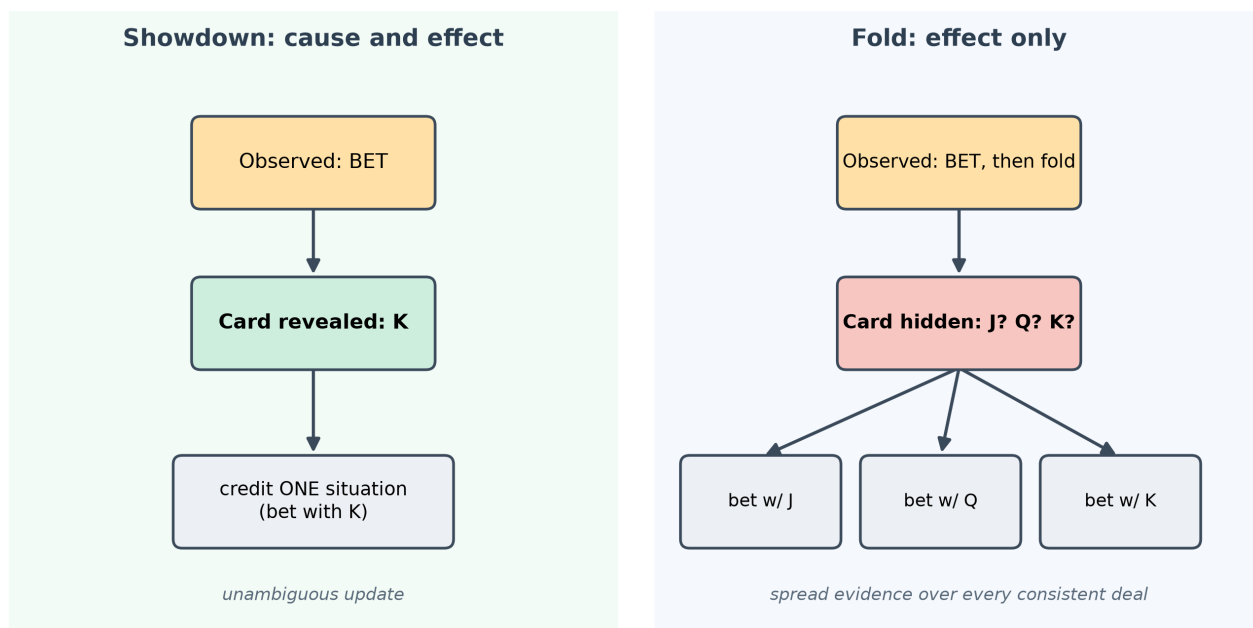
Има причина моделирането на противника в покера да е по-трудно от броенето на честоти в „камък-ножица-хартия“ и това заслужава отделен раздел, тъй като определя характеристиките на всеки следващ модел.

В покера наблюдават **действия** — залог, кол, пас — но **не** и частната ръка, която ги е породила. Един и същ „залог“ може да произтича както от силна ръка, така и от блъф. Единственото изключение е **разкриването**, когато картите се показват в края на ръката, ако тя не е приключила с пас. По този начин всяка ръка попада в една от двете доказателствени категории:

- **Разкриване — причина и следствие.** Виждате както действията на противника, *така и* частната ръка зад тях. Можете да свържете поведението с точната ситуация, която го е породила. Това представлява златния стандарт за наблюдение.
- **Пас — само следствие.** Ръката завършва без разкриване на картите. Видяхте *какво* са направили, но не и *какво са държали*, така че не можете да сте сигурни коя ситуация е причината за това.

Следствието е истински теоретичен лимит, а не неудобство при имплементацията: **без никога да наблюдават частната информация на противника, не можете да научите повече от своето предварително убеждение**. Ако единственото, което виждате, са пасове, повече **раздавания** не помага — доказателството е фундаментално двусмислено. Разкриването е именно информацията, която премахва тази двусмисленост.

Как се справя моделът с пас? Като разсъждава върху **всички възможни раздавания, които противникът би могъл да държи**. Конкретно, той изброява раздаванията, съгласувани с това, което действително е било видно, преиграва решенията на противника при всяка хипотетична тайна карта и разпределя доказателствата между тях — като претегля всяка хипотеза според нейната правдоподобност. Разкриването свежда този набор до една-единствена възможност; пасът го оставя разпределен. Този ход „маргинализиране върху скритата карта“ е техническото ядро на всеки модел в следващите два раздела и — неслучайно — формалният обект, около който е изградена теорията на съгласуваността от раздел 5.

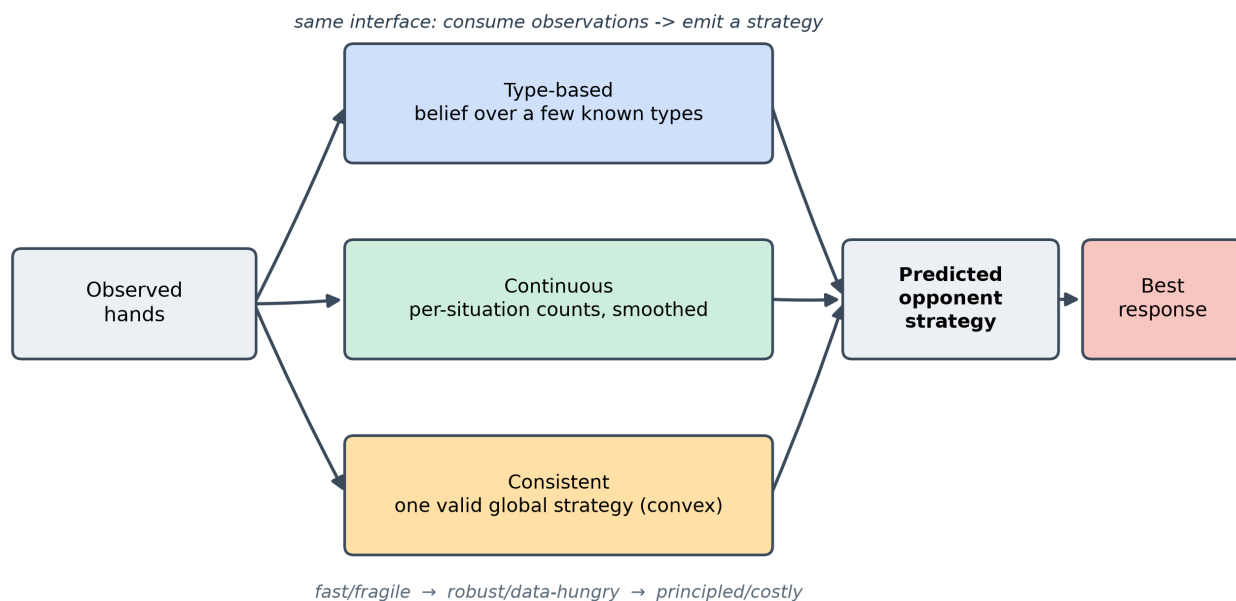


Фигура 3: Partial observability: a showdown reveals the opponent’s private card, pinning the action to one situation; a fold hides it, forcing the model to spread the evidence across every hand the opponent might have held.

7.4 4. Три модела с единен интерфейс

Няма един-единствен **модел на противника**. Има семейство, което прави компромис между скорост на сходимост, устойчивост към изненади, интерпретируемост и изчислителна мощност. Този проект имплементира и сравнява три точки от този спектър, умишлено изградени зад **общ интерфейс** — всеки приема поток от наблюдавани раздавания и генерира прогнозирана стратегия на противника — така че *същият* последващ най-добър отговор може да бъде приложен към всичките три и сравнението да е „ябълки с ябълки“.

- **Базиран на типове.** Поддържа убеждение (разпределение на вероятностите) върху фиксирано меню от типове противници, актуализира го чрез правилото на Бейс (Раздел 2) и докладва апостериорно претеглената средна стратегия. Той се сближава *изключително* бързо **когато противникът е в менюто** и изходът му е интерпретируем („80% камък, 20% маниак“). Неговата слабост също е ясна: ако реалният противник не прилича на нито един от типовете, моделът няма честен начин да го каже (Раздел 6).
- **Непрекъснат.** Премахва менюто. Оценява вероятностите за действията на противника *директно във всяка ситуация* чрез броене на това, което те са направили там, изгладено с дирихле предварително разпределение (Раздел 2). Той може да представи **всяка** стратегия, включително смеси и нови стилове, така че е устойчивият избор срещу непознати противници. Цената е данни: той научава всяка ситуация независимо, без споделяне на структура, така че се нуждае от много повече наблюдения — и, както показва Раздел 7, този глад има зъби в по-голямата игра.
- **Съгласуван.** Оценява една единствена *глобално съгласувана* стратегия — такава, която е гарантирано да бъде валидна стратегия върху цялото дърво на играта — вместо всяка ситуация поотделно. Това е най-принципиалният модел и най-новият; той е достатъчно важен и централен за тезата, за да получи



Фигура 4: Three models on one interface: a discrete belief over types, a free-form per-situation count, and a globally-consistent sequence-form estimate — all emit a predicted opponent strategy consumed by the same best-response step.

собствен раздел (Раздел 5).

Следващата таблица служи като **пътна карта** за по-нататъшната работа:

Модел	Представява	Сближаване	Устойчив на непознати противници?	Интерпретируемост	Изчислителна мощност
Базиран на типове	няколко известни типа	най-бързо	не	висока (именувани типове)	евтина
Непрекъснат	всяка стратегия за конкретна ситуация	по-бавно (жаден за данни)	да	средна (за конкретна ситуация)	евтина
Съгласуван	една валидна глобална стратегия	доказуемо към истината	да	средна	скъпа (оптимизация)

Очевидният въпрос — *кой от тях трябва да използва дисертацията?* — няма единствен отговор и това само по себе си е откритие. Естествената **цел** е **хибриден подход**: да се разчита на структурни априорни знания (типове), когато данните са оскъдни, и да се сходява към съгласуваната оценка при натрупването на доказателства. Тази глава изгражда и измерва крайните точки, така че хибридният модел да има стабилна

основа, върху която да стъпи.

Прочетете повече: Bard, N. (2013). „Онлайн имплицитно моделиране на агенти.“ *AAMAS* — основата „явни срещу неявни“, която рамкира тази таксономия.

7.5 5. Проблемът на съгласуваността и решението във последователна форма

Този раздел представлява теоретичната граница на разглежданата стъпка и е частта, която дисертацията най-пряко доразвива. Той поставя въпрос, който по-ранните модели подценяват: *ако даден модел добре описва наблюдаваното поведение, означава ли това непременно, че сме възстановили истинската стратегия на опонента?* Изненадващият отговор е **не** — дори при наличие на безкрайни данни — и съществува принципно решение.

7.5.1 Недостатъкът: напасването към средна стойност не е равносилно на откриване на истината

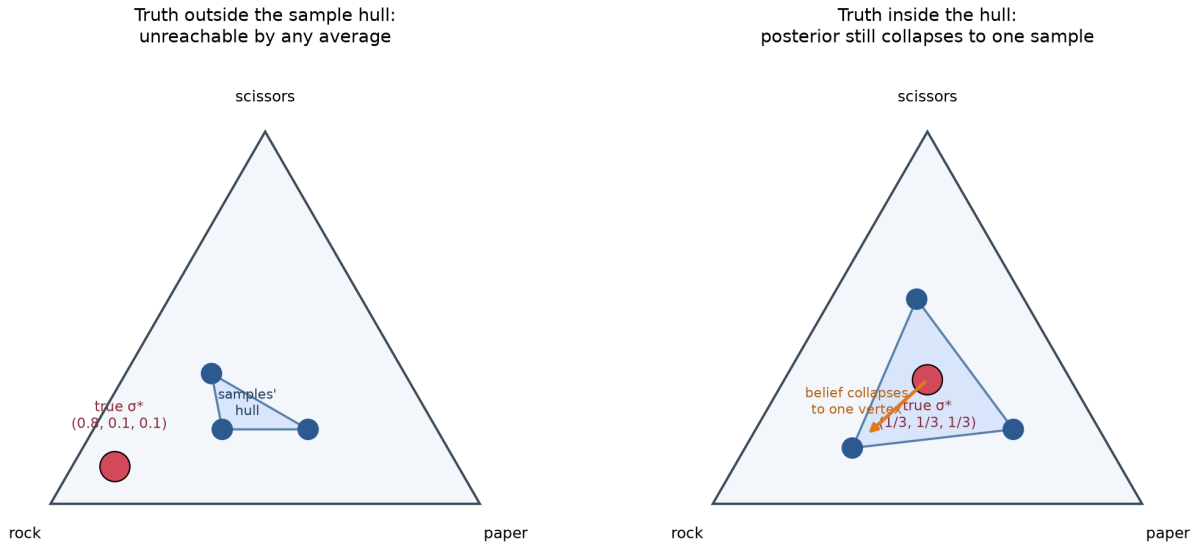
Класическата байесова рецепта отговаря на **средната стойност** на апостериорното разпределение на стратегията (Раздел 2). Един метод за моделиране на противника ще наречем **съгласуван**, ако неговата оценка се сближава със същинската стратегия на опонента σ^* при натрупване на наблюдения срещу постоянен противник. Бихме очаквали класическата рецепта да бъде съгласувана, но тя не е.

Най-ясният контрапример е камък-ножица-хартия. Да предположим, че моделиращият разсъждава върху множество от *извадени* кандидатски стратегии и винаги реагира на някаква тяхна средна стойност. Ако истинската стратегия $\sigma^* = (0.8, 0.1, 0.1)$ лежи **извън** областта, до която тези извадки могат да се комбинират, моделът — тъй като винаги е претеглена средна стойност на извадките — просто **не може да я достигне**, независимо колко много данни постъпват. Това е интуитивно. По-дълбокият резултат е, че методът може да се провали **дори когато истината лежи вътре** в достижимата област: ако истинската стратегия се намира в центъра на три извадки, чиято средна стойност съвпада с нея, задното тегло върху единствената най-добре пасваща извадка нараства неограничено спрямо останалите, така че асимптотично убеждението **колабира върху една извадка**, вместо да се установи на истинската смес. Колкото по-плътен моделът напаса данните, толкова повече се сближава с *грешната* стратегия.

Това е важно, защото обезсилва интуитивната представа за „сигурност чрез натрупване на повече данни“. Съгласуваността не е автоматична; тя трябва да бъде изградена.

7.5.2 Решението: изпъкнала програмираща задача в последователна форма

Лекарството преформулира оценката така, че „валидна стратегия“ и „съответства на наблюденията“ да се превърнат в ограничения на една единствена, добре дефинирана оптимизация. Ключовата промяна на променливите е **последователната форма**: вместо вероятности за действие във всяка ситуация, стратегията на опонента се представя чрез **реализационни тегла** y_r върху последователности от действия. Законните стратегии тогава са именно тези, които удовлетворяват набор от **линейни** ограничения $Fy = f, y \geq 0$ — полиномиални по размер на дървото на играта, вместо експоненциални.



Фигура 5: Why fitting the mean can miss the truth: the modeler can only ever produce points in the shaded hull of its samples. A true strategy outside the hull is unreachable; even one inside is abandoned as the posterior collapses onto a single vertex.

Частичната наблюдаемост (Раздел 3) се въвежда чрез **функция на наблюдаемост**: за всяка наблюдавана ръка множеството от траектории, съгласувани с това, което действително е било видно (пас скрива картата на опонента и допуска множество траектории; разкриването я показва и допуска само една). Вероятността на едно наблюдение представлява сума от реализации на тегла върху това съгласувано множество. Поставянето на **дирихле предварително разпределение** върху теглата и максимизирането на **логаритъма на задната вероятност** води до програмата

$$\max_y \sum_r (\alpha_r - 1) \log y_r + \sum_t \log \left(\sum_{r \in o(\ell_t)} q_r y_r \right) \quad \text{s.t.} \quad Fy = f, y \geq 0,$$

където $o(\ell_t)$ е множеството от съгласувани траектории за наблюдение t и q_r са (нормализирани) вероятности на случайността. Решаващото свойство е, че когато $\alpha_r \geq 1$ тази целева функция е **вдлъбната** — първият член е вдлъбнат, вторият е логаритъм на афинна функция, а ограниченията са афинни — така че това е **изпъкнал** проблем без **локални оптимуми**. Всеки решавач, който намери локален оптимум, е намерил глобалния. Достатъчен е стандартен **проектиран градиентен метод**: извършва се стъпка на градиента, след което резултатът се проектира обратно върху множеството от ограничения. В псевдокод цикълът е елементарен, което е и целта —

```

initialize y feasible (F y = f, y >= 0)
repeat until converged:
  g <- gradient of the (negative) log-posterior at y
  z <- y - eta * g                                # gradient step
  y <- project z onto { y : F y = f, y >= 0 }    # convex projection

```

Този метод връща **модата** на апостериорното разпределение (неговата единствена най-вероятна точка), а не средната стойност. Това е ключовият компромис: модата е *съгласувана* — при леки условия (истината има положителна предварителна плътност, различни стратегии водят до различни наблюдения и всяка ситуация на противника се посещава безкрайно често) оценката доказано се сближава към σ^* — докато оптималната по отношение на изплащането средна стойност обикновено е неразрешима за точно изчисляване. Следователно основната ос на дизайна, върху която стъпва целият процес, е **средна стойност срещу мода**: оптимална по отношение на изплащането, но неразрешима, спрямо разрешима и съгласувана.

7.5.3 Какво установихме, откровено казано

Ние внедрихме този съгласуван оценител и проверихме неговата работа при възстановяване на стратегия в играта кун, където той се представя според очакванията: възстановената стратегия е много близка до истинската (разстоянието по вариация е приблизително **0.004 до 0.021**), като съвпада или превъзхожда резултатите от възстановяването на непрекъснатия модел в същата игра. Това потвърждава както правилната работа на механизма, така и валидността на теорията върху малката тестова среда.

Ние **не** го стартирахме в рамките на онлайн цикъла на експлоатация, нито върху по-голямата игра. Причината е същата, която и самата литература посочва: оценката представлява решението на оптимизация, която се **пререшава с натрупването на наблюдения**, като цената ѝ нараства с историята — от части от секундата в началото до много секунди за преобучаване след изиграването на десетки хиляди раздавания. За мач в реално време, раздаване по раздаване, това е непрактично в наивната му форма. Вместо да го разглеждаме като пропуск, ние приемаме това за *емпиричен отговор* на въпрос, който стъпката поставя изрично — *достатъчно ли бързо е конвексното решаване при всяка актуализация за игра в реално време?* — а именно **не, без инкрементални методи** (топло стартиране на всяко решаване от последното и кеширане на термините за всяко раздаване), което е точно видът приближение, с който се занимава Стъпка 8. Разбирането и резултатът от възстановяването са това, от което тезата се нуждае от този модел сега; онлайн инженерството остава за бъдеща работа.

Прочетете повече: Ганцфрид, С. (2025). „Съгласувано моделиране на противника в игри с непълна информация.“ *arXiv:2508.17671*.

7.6 6. Когато моделът е уверен, но греши

Раздел 5 беше за фин провал при безкрайни данни. Този раздел разглежда груб провал при крайни данни и именно той най-пряко мотивира *безопасната* експлоатация. Това е различен провал от несъответствието: тук проблемът е **неправилно зададено меню** — противник, който типово базираният модел просто не може да представи — и как се държи моделът, когато бъде притиснат в ъгъла.

Изградохме скрит противник, който представлява 50/50 смес от действията на „камък“ и „маниак“ — съзнателно **нищо един** от четирите кандидата. Очаквано е апостериорното разпределение да разпредели убеждението си между двата най-близки типа. Това обаче не се случва. Апостериорното разпределение, изчислено

като произведение на вероятностите, се концентрира върху *единственото най-добро обяснение*, поради което убеждението прескача от един тип към друг и в крайна сметка твърдо се ангажира с **наш равновесие** — единственият кандидат, който приписва реална вероятност както на залагане, така и на проверка със средна ръка и по този начин никога не е категорично опроверган. В резултат моделът става **уверен** (апостериорното разпределение е близо до 1.0 върху един тип) и същевременно **грешен** (този тип не отговаря на действителния противник).

Това разкрива един капан, който си струва да бъде изказан ясно: апостериорното разпределение е **относителна** величина. „Деветдесет и пет процента хипотеза на Наш“ означава „най-добро приближение *сред тези четири кандидата*“, а не „добро приближение в абсолютен смисъл“. Измерването на *абсолютното* приближение на победителя издава играта — същата хипотеза на Наш, която се представя добре срещу истинско равновесие на Наш противник, се представя значително по-зле срещу тази смес, като същевременно отчита еднаква увереност на хартия. Малкото стереотипно меню няма честен начин да каже „нито едно от горните“.

7.6.1 Колко дълго може да остане грешен? Обхождане за устойчивост

Тъй като уверено-грешните убеждения са опасни, подложихме феномена на стрес-тест върху **300 случайни семена от по 500 раздавания**, като разграничихме две неща, които една **наивна метрика** обединява: *бавна сходимост* и *отпадане след сходимостта*.

Измерване (300 семена x 500 раздавания)	Резултат
Правилен дългосрочен победител до раздаване 500	300 / 300 (100%)
Някога отстъпва, след като трайно поеме лидерството	0 / 300 (никога)
Грешен тип все още води след раздаване 100	40 / 300 (~13%)
Грешен тип все още води след раздаване 200	14 / 300 (~5%)
Раздаване, на което истината окончателно се установява	медиана 23 - 90-ти персентил 125 - най-лош случай 461

Добрата новина е, че убеждението в крайна сметка винаги се установява върху най-близката представима стратегия и след като се заключи, никога не отстъпва. Тревожната новина са средните редове: в значителен малък брой случаи моделът поддържаше **уверено грешно убеждение за повече от сто раздавания** — достатъчно дълго, за да може един най-добре реагиращ агент, който експлоатира твърдо това убеждение, да се е настроил да победи илюзия. Това е емпиричното проявление на напрежението между експлоатацията и безопасността: на модела може да се вярва в *крайна сметка*, но „в крайна сметка“ понякога е след 200 раздавания, а действието въз основа на междинното убеждение, сякаш то е сигурно, е начинът, по който самият агент се наранява.

Честният поправка не е по-голямо меню — тя е да спрем да питаме „кой единствен тип?“ и да се опрема на модели, които могат да представят смеси (непрекъснатите и съгласуваните модели от Раздели 4–5), и да **мащабираме степента на експлоатация според това колко добре е заслужено прочитането**. Този принцип е нишката, която преминава през следващите два раздела и към Стъпка 8.

7.7 7. От модела до печалбата — най-добрият отговор, таванът и самонанесеното изтичане

Моделът е полезен единствено ако действията, основани на него, носят печалба. За да измерим това по ясен начин, подаваме оценката на всеки модел в **същия** точен най-добър отговор и играем пълни мачове и в двете игри, като поставяме всеки резултат между два аналитични критерия, изчислени чрез ръчно проверена математика, а не чрез симулация:

- **нашева очаквана стойност** — това, което безопасната равновесна игра носи срещу този опонент (не можете да се представите по-зле от тази стойност, ако прочитът е безполезен);
- **таван** — точната стойност на най-добрия отговор, максималното, което може да бъде извлечено при пълно познаване на опонента.

Добрият експлоататор се намира между двете и се стреми към тавана. Първо, проверка за коректност, че апаратът е надежден: решаващът възпроизвежда точната равновесна стойност на Кун $-1/18$ (остатъчна експлоатируемост ≈ 0.002 , по същество неексплоатируем), а точен най-добър отговор превъзхожда равномерната игра с предвидените разлики (+0.500 при Кун, +2.087 при Ледюк).

Кун — и двата модела достигат тавана.

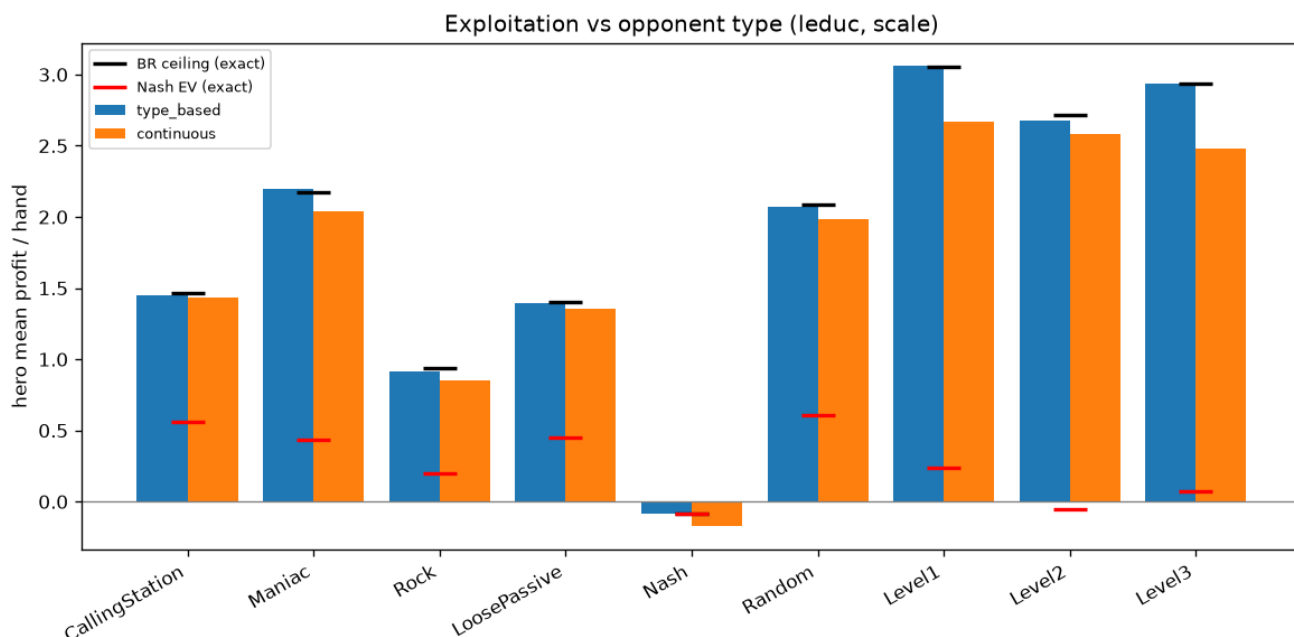
Опонент	таван	базиран на типове	непрекъснат
Камък	0.167	0.168	0.168
Маниак	0.333	0.337	0.330
наш равновесие	-0.055	-0.055	-0.053

(Чист опонент „винаги пас на залог“ е още по-експлоатируем — таван от 0.975, достигнат до 0.966 — но той не е сред повтарящите се типове, дефинирани по-горе, и поради това е пропуснат тук с цел съгласуваност.)

Ледюк — започва да се проявява значението на класа на модела. Ледюк Холдем добавя втори рунд за залози и една споделена обща карта, което води до приблизително два порядъка повече ситуации в сравнение с Кун — все още точно решима, но вече достатъчно голяма, за да различи моделите.

Опонент	таван	базиран на типове	непрекъснат
Ниво-1	3.056	3.061	2.672
Маниак	2.177	2.199	2.038
Пасивен играч	1.464	1.451	1.434
Камък	0.937	0.912	0.848
наш равновесие	-0.083	-0.085	-0.175

Два резултата подчертават извода, като вторият е от съществено значение:



Фигура 6: Exploitation against each Leduc opponent: the type-based model (which fits these opponents) hugs the exact best-response ceiling, while the continuous model tracks close but sits below for the hardest-to-fit types — and dips below the safe baseline against Nash.

1. **Не можете да експлоатирате равновесие.** Срещу нашов противник всеки модел реализира приблизително (отрицателната) стойност на играта и не повече. Таванът *по същество представлява* стойността на играта. Това потвърждава, че наблюдаваната експлоатация другаде е реална, а не артефакт на средата за тестване.
2. **Уверен, но недообучен модел ви прави уязвими.** В ледюк непрекъснатият модел *губи* от наш равновесие — -0.175 при таван от -0.083 . С несвършени данни за множеството ситуации в ледюк той реагира най-добре на *грешна* оценка на противник, който изобщо не подлежи на експлоатация, и по този начин създава **изтичане** в собствената си игра. Подбирането на ограничена извадка, което само му коства малко срещу експлоатируеми опоненти, се превръща в активно вредно срещу неексплоатируем такъв.

Това е напрежението между експлоатацията и безопасността, изразено в едно число. Когато класът на модел съвпада, най-добрият отговор извлича пълната теоретична стойност; когато не съвпада, агентът както оставя пари на масата *и* — по-опасно — връща част от тях обратно. Тази втора цена е именно това, което предпазният механизъм от Стъпка 8 (ограничаване на отклонението от наш равновесие чрез собствената увереност на модела) съществува, за да предотврати.

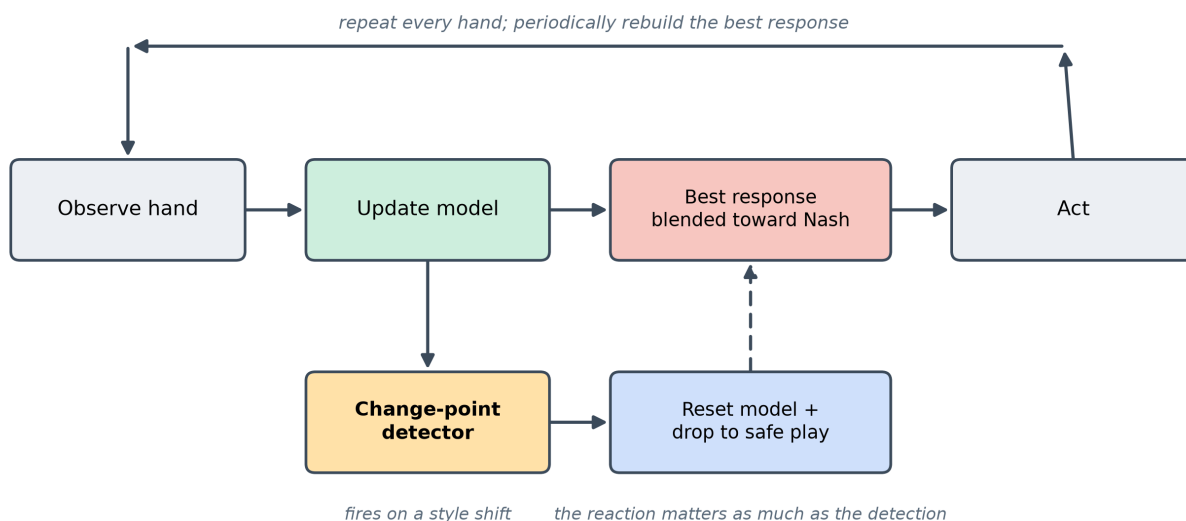
Разликите тук не се дължат на начална случайност. При пет различни начални числа стандартната грешка на печалбата на ръка е незначителна в сравнение с наблюдаваните ефекти: типово базираният модел е статистически неразличим от тавана за всеки тип и в двете игри, а недостигът на непрекъснатия модел в ледюк и неговото изтичане при наш равновесие са многократно по-големи от стандартната грешка.

Прочетете повече: Ganzfried, S. & Sandholm, T. (2015). “Safe Opponent Exploitation.” *ACM EC*
— безопасната половина от скалата и основата за Стъпка 8.

7.8 8. Адаптиране към промяна — нестационарни опоненти

Досега всичко се основаваше на постоянен противник. Реалните опоненти обаче се променят и адаптират, а модел, който е учил търпеливо десет хиляди раздавания, става безполезен в момента, в който неговият обект промени стила си — той вече *уверено* описва играч, който вече не съществува. Това е откритата граница, към която дисертацията е насочена да атакува, и стъпката включва първата контролирана сонда.

Пълният адаптивен агент изпълнява цикъла от Раздел 2 от край до край: **наблюдава** раздавания, **актуализира** модела, периодически преизгражда стратегията на героя като най-добър отговор на текущата оценка (по избор смесена към Наш равновесие за безопасност) и **действа**. За да се справи с промяната, той добавя детектор: лек статистически монитор върху агресията на опонента, който следи за промяна в режима и при задействане **нулира** модела и преминава към безопасна игра, докато отново се учи.



Фигура 7: The adaptive exploitation loop: observe, update the model, best-respond (blended toward Nash for safety), and act; a change-point detector can reset the model when the opponent’s style shifts.

Сменихме стила на опонента в средата на мач от 20 000 раздавания и сравнихме **статичен** модел (който никога не забравя) с такъв, който прилага **забравяне на точка на промяна**. Резултатът е **сценарийно-зависим**, и именно това представлява откритието:

Игра	Смяна на стила (в средата)	статичен (след смяната)	точка на промяна (след смяната)
Кун	камък → манияк	-0.116	+0.226

Игра	Смяна на стила (в средата)	статичен (след смяната)	точка на промяна (след смяната)
Ледюк	камък → маниак	+1.940	+0.525

- **В Кун забравянето печели.** Стратегията, научена срещу „камък“, е блъф-ориентирана; пусната срещу маниак, който вдига всичко, тя *активно губи* (статичният модел отива на отрицателна печалба). Откриването на смяната и преобучаването възстановява до здравословна печалба.
- **В Ледюк забравянето губи.** Маниакът там изпуска по над два жетона на ръка, така че непрекъснато адаптиращ се модел го експлоатира щедро без никакво нулиране — докато детекторът, твърде нетърпелив, задейства десетки **фалшиви аларми** по време на стабилните периоди (около шестдесет нулирания срещу една истинска промяна), всяко от които изхвърля трудно спечелени данни и преминава към безопасна игра.

Урокът не е „откриването на промени е добро“ или „лошо“ — той е, че **реакцията при открита промяна има толкова голямо значение, колкото и самото откриване**. Детектор, който задейства твърде често, съчетан с пълно връщане в безопасен режим, може да струва повече от остарялост, когато новият опонент е достатъчно експлоатируем, така че остарялостта е евтина. По-евтини и по-меки реакции — частично забравяне вместо твърдо нулиране, по-малко чувствителен детектор — са ясна следваща стъпка и те се свързват директно с експлоатацията с мащабирана увереност от Стъпка 8. (Този експеримент беше проведен при едно начално число, така че възприемайте *посоката* като стабилна, а точните величини — като илюстративни.)

7.9 9. Връзки и бъдещи насоки

Какво установява тази стъпка. Добрият модел на противника е *необходим, но не и достатъчен* за печеливша и безопасна адаптация. Когато класът на модела съответства на противника, най-добрият отговор достига точния извличащ таван — реализира се пълната теоретична стойност на моделирането. Но при частична наблюдаемост и ограничени данни, недообученият модел както оставя пари на масата, така и срещу противник, който не може да бъде експлоатиран, реагира най-добре на химера и *губи*. Нестационарността добавя втори аспект: забравянето на остарял модел е полезно само когато остарялостта е активно вредна, а фалшивите аларми на небрежен детектор могат да струват повече, отколкото спестяват. Един принцип се повтаря през всичко това: **експлоатацията трябва да бъде мащабирана според това колко добре е заслужено прочетеното**.

Обратни връзки. Тази стъпка е огледален образ на работата по равновесие, която я предшества. Докато една равновесна стратегия *игнорира* идентичността на противника по замисъл, моделът на противника е механизмът, който я *използва* — двата са крайните точки на спектъра безопасност–изследване. Механизмът за извадка, използван в предишните стъпки за обхождане на дърво на играта според *текущата* стратегия, се появява отново тук, за да разсъждава върху действията на противника според *изведения* модел. А идеята за групиране на много конкретни ситуации в няколко представителни категории — абстракция — е именно това, което типично базираният модел прави в пространство на стратегиите.

Напред към Стъпка 8 и дисертацията. Тази стъпка изгради **сензора**, а Стъпка 8 изгражда **изпълнителния механизъм** — механизма, който превръща модела в *безопасна* експлоатация, като ограничава доколко се отклонявам от равновесие според това колко е заслужено прочетеното. Самопричиненото изтичане на **непрекъснатия** модел срещу **Наш равновесие** (Раздел 7) е емпиричният случай за този механизъм; уверено, но грешно обхождане (Раздел 6) определя неговия бюджет; теорията на съгласуваността (Раздел 5) е принципният гръбнак, върху който рамката се разширява.

Отворени въпроси за бъдеща работа.

- *Съгласуваност в реално време.* Съгласуваният модел е принципен и точен, но неговата изпъкнала оптимизация при всяка актуализация все още не се вписва в онлайн мач. Постепенното решаване (топли стартове, кеширани членове) или под-игра приближенията от Стъпка 8 са пътищата към използвана версия в реално време.
- *Нестационарност, правилно разгледана.* И трите модела предполагат стационарен противник за своите гаранции. Мащабираното от доверие забравяне и по-добрите сигнали за промяна са непосредствената следваща работа; пълно теоретично третиране е цел в мащаба на дисертация.
- *Противници извън менюто.* Добре специфицираното откриване е лесният случай. Интересният режим са противниците *извън* всяко меню — смеси, дрейф, противници — където непараметричните и последователните модели трябва да се отделят от типово базирания модел, и където изричен сигнал „нито един от моите типове не пасва“ се превръща в собствен изследователски въпрос.
- *От един противник към много.* Срещу няколко опонента едновременно, съвместно-оптималният отговор не е комбинация от индивидуални най-добри отговори и изпъкналата гаранция от Раздел 5 вече не важи. Моделирането на множество опоненти е мостът към по-късните стъпки с много агенти.

Прочетете повече: Shoham, Y. & Leyton-Brown, K. (2008). *Многоагентни системи*, Гл. 7 „Учене и преподаване“ — рамката за учене в повтарящи се игри под всички горепосочени, включително напрежението, че вашите действия както *експлоатират*, така и *обучават* опонента.

Стъпка 8 Обобщение — Безопасно експлоатиране в игри с непълна информация

Изследване върху възможностите за прилагане на изкуствен интелект в компютърните игри

Alexander Andreev

July 2026

Съдържание

8	Стъпка 8 — Безопасна експлоатация в игри с непълна информация	1
8.1	1. Защо безопасната експлоатация — другата половина на скалата	2
8.2	2. Експлоатацията като ограничена оптимизация	3
8.3	3. Какво означава „безопасен“ — три определения с нарастваща степен на реализъм	5
8.4	4. Двигателят — линейно програмиране в последователна форма и генериране на ограничения	6
8.5	5. Ограничен отговор по Наш и граница тип „вкл.–изкл.“	7
8.6	6. Ганцфрид върху кун — безопасен <i>и</i> печеливш (основният резултат)	10
8.7	7. Безопасност в реално време — под-игра джаджи (SES)	12
8.8	8. В мащаб — кун работи; ледюк се разпада глобално, но се задържа локално	14
8.9	9. Връзки и бъдещи насоки	17

8 Стъпка 8 — Безопасна експлоатация в игри с непълна информация

Това е глава, изградена из основи, за *безопасна* експлоатация на противника: формулировката на проблема, математическият апарат, семейството от методи и набор от контролирани експерименти, проведени върху две малки покер игри. Тя изпълнява две функции — служи като самостоятелно въведение и опресняване, докато по-късните стъпки се надграждат върху него, както и като основен източник за синтеза на дисертацията в Част 1. Написана е така, че да може да бъде прочетена самостоятелно; не се предполага предварителна познатост с кода на проекта. Всички експериментални числа, докладвани тук, са **измерени** при възпроизводими изпълнения на двете тестови среди (кун покер и ледюк холдем) и са ограничени, където е възможно, от *точни* аналитични препратки, а не симулирани такива. Когато дадено изпълнение противоречи на това, което теорията ме е накарала да очаквам, аз го заявявам и го съгласувам — тези пропуски са най-показателните части от стъпката.

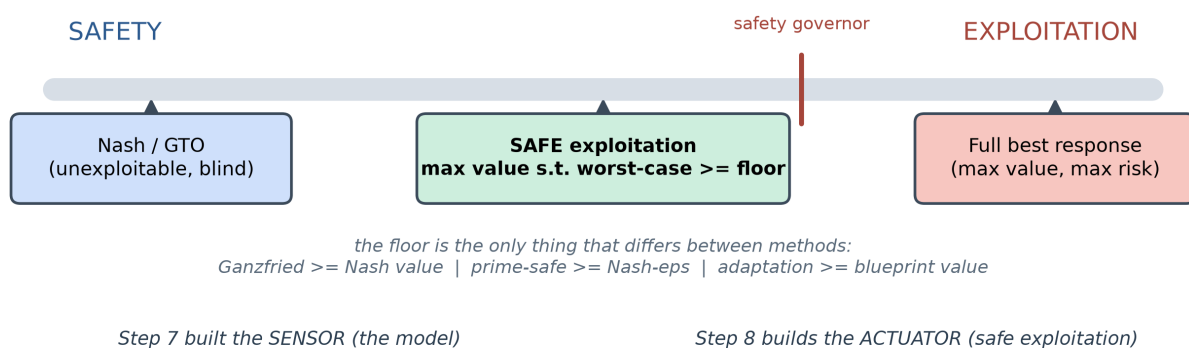
Къде се вписва това в дисертацията. Стъпка 7 изгради **сензор**: модел, който наблюдава как даден противник играе и оценява неговата стратегия. Тя също така разкри опасността от наивното действие въз основа на този сензор — уверен, но недообучен модел, към който се реагира оптимално, може да отвори дупка в собствената ви игра (в Ледик той всъщност *загуби* срещу неексплоатируем Нашов противник). Стъпка 8 изгражда **изпълнителен механизъм**: механизма, който превръща модел в печалба, **без да става експлоатируем**.

Този изпълнителен механизъм е втората половина на рамката за поведенческа адаптация на дисертацията (принос №1), а определянето *точно къде неговите гаранции зависят от предположението за игра с нулева сума за двама играчи* е отправната точка за многоагентното разширение (принос №2).

8.1 1. Защо безопасната експлоатация — другата половина на скалата

Стратегия в **равновесие на Наш** е изградена така, че никога да не губи в дългосрочен план. В **двуйгрова игра с нулева сума** тази гаранция е точна: стратегията има фиксирана *стойност* v^* , и никой противник, колкото и умен да е, не може да ви свали под нея. **Моделиране на противника** (Стъпка 7) ви дава възможността да се *отклоните* от тази безопасна стратегия, за да накажете грешките на конкретен противник. Очевидният начин да реализирате печалба от модел е да изчислите **най-добър отговор** към него и да играете така. Проблемът е, че най-добрият отговор обикновено сам по себе си е **силно експлоатируем**: за да накаже „винаги пасваш на залог“, той напълно спира да блъфира определени **раздавания**, а по-умен противник — или същият противник, след като спре да се преструва — минава право през отворения пробив.

Представете си **скала**. Ако я завъртите докрай към *безопасност*, ще играете в **равновесие на Наш**: непобедими, но без да наказвате слаб противник. Ако пък я завъртите докрай към *експлоатация*, ще **оптимално отговарям** твърдо на текущото си впечатление: максимална **печалба**, **ако** впечатлението е вярно и противникът остане на място, но сериозен риск, ако грешите, извадката ви е малка или сте били *сандбегвани* (умишлено подхранвани със слаб стил, за да се предизвика голямо **отклонение**). **Безопасна експлоатация** поставя *регулатор* върху тази скала: можете да се наклоните към експлоатацията колкото желаете, но никога не бива да минавате отвъд точката, в която противник в **най-лошия случай** може да ви свали под вашата **безопасна базова линия**.



Фигура 1: The safety-exploitation dial with a governor. Pure Nash is unexploitable but blind; a full best response extracts the most value but is maximally risky. Safe exploitation operates in between, capped by a floor on the worst-case value. Step 7 built the sensor (the model); Step 8 builds the actuator (safe exploitation). The only thing that differs between methods is where the floor sits.

Причината това да не е параноя е едно число от Стъпка 7 и отново от тази стъпка: пълният **оптимален**

отговор на стегнатия стил „камък“ в кун носи +0,167 на раздаване, но неговият най-лош стойностен случай е −0,5 — противник, който оптимално отговаря обратно, може да вземе половин чип на раздаване от него. Най-лошият случай при равновесие на Наш, напротив, е самата стойност на играта. Безопасна експлоатация е дисциплината да се запази по-голямата част от този +0,167 потенциал за печалба, като същевременно се избягва −0,5 потенциала за загуба.

Какво означава „безопасен“ в неформален план. Вашата очаквана стойност за целия мач никога не бива да пада под избран праг, **независимо какво прави противникът**. Това е твърдение за най-лошия случай, а не за конкретно раздаване — все пак ще губите раздавания; гаранцията се отнася до дългосрочната средна стойност при противников подход. Целият въпрос на дизайна в тази стъпка е *какъв праг* да бъде избран и как да се изчисли стратегия, която го спазва, като същевременно извлича възможно най-много над него.

Прочетете повече: Ганцфрид, С. & Сандхолм, Т. (2015). „Безопасна експлоатация на противника“. *ACM Trans. Economics and Computation* — статията, която за първи път формулира като теорема принципа „експлоатирай, но никога не губи спрямо базовата линия“.

8.2 2. Експлоатацията като ограничена оптимизация

Единствената най-полезна идея в тази стъпка е, че **всеки метод за експлоатация представлява един и същ проблем на оптимизация**, при който се заменя само една негова част. С други думи:

максимизиране на очакваната стойност на героя спрямо модела на противника, **при условие** за праг на безопасност за най-лошия стойностен случай на героя.

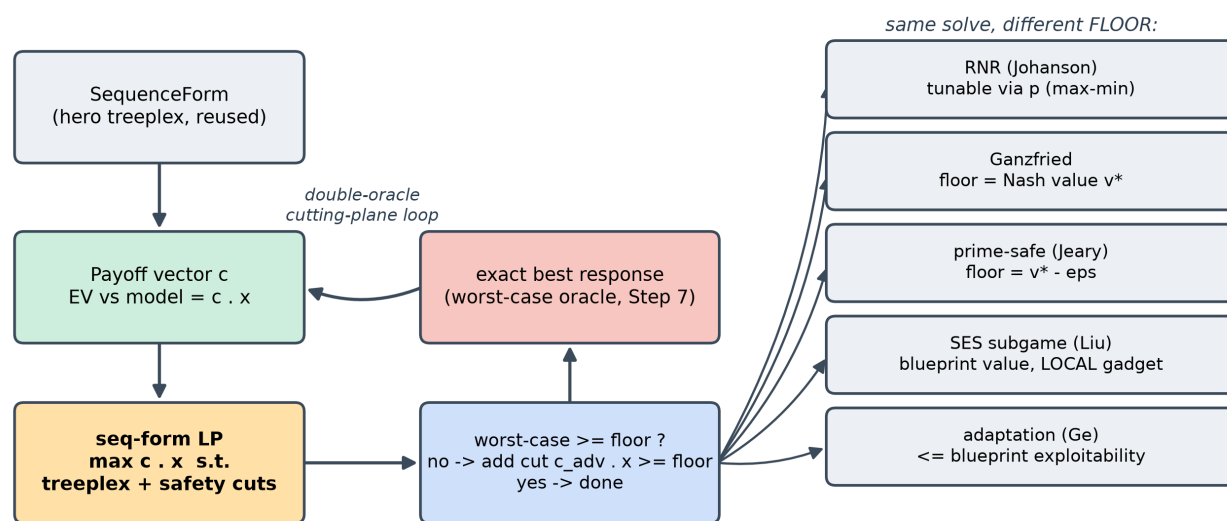
За да се превърне това в *изчислима* програма, представете стратегията на героя в **последователна форма**: вместо вероятности за действие според ситуацията, използвайте **план за реализация** x , който присвоява тегло на всяка от последователностите от действия на героя (вериги от корен до тук от собствените им избори), подчинени на линейните ограничения на **treeplex** (празната последователност има тегло 1; при всяко информационно множество теглата на децата се сумират до теглото на родителя; всички тегла са неотрицателни). Причината да се плати тази „данък“ за смяна на променливите е решаваща: срещу **фиксирана** стратегия на противника, очакваната стойност на героя е **линейна** в x ,

$$EV(x) = \sum_{\text{terminals } z} [\text{chance}(z) \cdot \text{opp-reach}(z) \cdot u_{\text{hero}}(z)] x_{\text{seq}(z)} = c \cdot x,$$

защото собственото произведение на вероятностите за действие на героя по всяка линия е x при последователността на тази линия. Така „максимизиране на очакваната стойност спрямо модела“ представлява **линейна цел** $c \cdot x$, **treeplex** е множество от **линейни ограничения**, а целият метод е **линейна програма**. Пълният най-добър отговор е просто $\max_x c \cdot x$ върху **treeplex** — което ни позволява да проверим линейната програма спрямо кода за точен най-добър отговор от Стъпка 7 (те съвпадат до 10^{-6}).

Прагът на безопасност представлява ограничение върху **най-лошия случай** стойност — „за *всеки* противник σ' , $EV(x, \sigma') \geq \text{floor}$.“ Това вътрешно „за всеки противник“ само по себе си е най-добър отговор (ми-

нимакс), поради което не го записваме като една гигантска билинейна програма; решаваме го чрез **генериране на ограничения** (двойно предсказание / цикъл с равнини на сечение): решаваме линейната програма, прочитаме стратегията на героя, извикваме **точен най-добър отговор като най-лошият случай оракул**, и ако най-лошият случай е под прага, добавяме единственото линейно сечение, което той предполага, и преизчисляваме. Това е крайно (съществува краен брой чисти най-добри отговори), прозрачно и подлежи на отстраняване на грешки.



one validated primitive (Step 7's exact best response) powers BOTH the objective and every safety check

Фигура 2: One LP engine, five safety floors. The sequence-form treeplex gives a linear objective (EV vs the model); a constraint-generation loop calls an exact best response as the worst-case oracle and adds a safety cut until the floor is met. RNR, Ganzfried, prime-safe, SES and adaptation are the SAME solve with a different floor — and one validated primitive (Step 7's best response) powers both the objective and every safety check.

Концептуалната полза от този подход е икономия: **пет метода, един двигател**. Те се различават единствено по прага, който ще бъде разгледан в следващия раздел. И тъй като както целта (векторът на печалбата), така и оракулът за безопасност използват повторно *валидирания* най-добър отговор от Стъпка 7, цялата Стъпка 8 се основава на една вече доверена примитивна операция, а не на нова поредица от математически изчисления.

Прочетете повече: Shoham, Y. & Leyton-Brown, K. (2008). *Многоагентни системи*, §3.4 (изчисляване на равновесия) и §4.6 (изчисляване на най-добри отговори) — последователната форма на механизма под всяка линейна програма в тази глава.

8.3 3. Какво означава „безопасен“ — три определения с нарастваща степен на реализъм

„Безопасността“ звучи като абсолютна характеристика, но всъщност не е еднозначна. От значение са три различни определения, а изследователската линия по същество представлява *отслабване* на изискването, за да стане то приложимо в практиката.

(а) Сигурност при ганцфрид (2015): никога да не се печели по-малко от стойността на играта. При *перфектно* нашево равновесие σ^* като базова линия, изискването е

$$EV(x, \sigma') \geq v^* \quad \text{for all opponents } \sigma'.$$

Това е най-силната и най-чистата концепция. Нейното доказателство се основава на **теоремата за минимакс**: в двуигрови игра с нулева сума нашевата стратегия *гарантира* стойността v^* срещу всеки противник, така че човек може да се отклони към експлоатация на модела в рамките на „хлябината“, създадена от грешките на противника (техните *подаръци*) и никога да не падне под v^* . Уловката е предпоставката: *перфектно* нашево равновесие, което е неизчислимо във всяка голяма игра.

(б) Безопасен спрямо прайм / ϵ -safety (Jeary & Turrini 2023): корекция за несъвършена базова линия. Всяка реална базова линия е **ϵ -равновесие** (поради абстракция и ограничена изчислителна мощност — предмет на Стъпка 4), така че самата тя е експлоатируема в известна степен ϵ . Поради това закрепването към точната v^* е неоправдано; безопасен спрямо прайм понижава пода с точно експлоатируемостта на самата базова линия,

$$\text{floor} = v^* - \epsilon, \quad \epsilon = \text{exploitability}(\text{baseline}) \geq 0,$$

т.е. „никога не печели по-малко от най-лошия случай на стратегията, която така или иначе щеше да се играе“. ϵ трябва да бъде **измерена**, а не предполагаема — точка, която експериментите приемат буквално.

(в) Безопасност на адаптацията (Ge и др. 2024): да бъдеш не по-експлоатируем от своя план. Най-практичната концепция. Една експлоатираща стратегия е *безопасна за адаптация*, ако и само ако

$$\text{exploitability}(x) \leq \text{exploitability}(\text{blueprint}) \iff \text{worst-case}(x) \geq \text{worst-case}(\text{blueprint}).$$

Тъй като планът вече е ϵ -експлоатируем, това е по-слабо от абсолютната долна граница на Ганцфрид (по-слабо точно с ϵ) и следователно *постижимо* там, където строгата безопасност не е. Опасността му е огледан образ: ако планът е *ужасен*, „не по-лошо от плана“ се удовлетворява тривиално от почти всичко — така че безопасността на адаптацията има смисъл само при разумна базова линия (отворено изискване, което отбелязвам, вместо да реша).

Понятие за безопасност	Неформална долна		
	граница	Нужди	Слабост
Ганцфрид (2015)	$\geq v^*$	едно <i>перфектно</i> нашево референтно ниво	перфектното нашево равновесие е неизчислимо в мащаб
Безопасен спрямо прайм (2023)	$\geq v^* - \varepsilon$	измерената експлоатируемост на базовата линия	трябва честно да измерите ε
Адаптация (2024)	$\leq$ експлоатируемост на плана	всеки план	безсмислено, ако планът е лош

8.3.1 Къде се крие предположението за игра с нулева сума при двама играчи — атакуващата точка на дисертацията

Всеки от горните прагове се основава на един факт: **в игра за двама с нулева сума, една нашева стратегия гарантира стойността v^* срещу всеки противник** ($\min_{\sigma'} \text{EV}(\sigma^*, \sigma') = v^*$). Именно това прави „отклонявай се към модела, но никога под прага“ последователно и приложимо ограничение, което произтича директно от теоремата за минимакс. В $N > 2$ -игрова игра това се срива: нашевата стратегия *не* гарантира фиксирана стойност срещу произволни противници (другите могат да координират действията си и техните печалби вече не се сумират до отрицанието на твоята), така че няма единна v^* , към която да се закотви. **Това е отвореният проблем за Принос №2** — дали съществува аналог на гаранцията за стойност в N -игрова игра или структурно предположение (например коалиционна структура) трябва да възстанови опорната точка. Тази стъпка не го решава; тя прави провала *прецизен*, което е точно това, от което се нуждае една глава на дисертация.

Прочетете повече: Johanson, M., Zinkevich, M. & Bowling, M. (2007). „Изчисляване на устойчиви контрастратегии.“ *NeurIPS* — Ограничен отговор по Наш, настройваемият предшественик на всички изброени по-горе. · Jeary, J. & Turrini, P. (2023). „Безопасна експлоатация на противника за ε -равновесие стратегии,“ *arXiv:2307.12338*. · Ge, Z. и др. (2024). „Безопасна и устойчива експлоатация на под-игра...,“ *ICML*.

8.4 4. Двигателят — линейно програмиране в последователна форма и генериране на ограничения

Този раздел описва „как е изградено“. Единствената примитивна операция, която цялата стъпка добавя, е HeroTreeplex, който (i) изброява последователностите на героя и дървовидните ограничения (използвайки повторно кода в последователна форма от Стъпка 7), (ii) изгражда вектора на печалбата c за постоянен противник чрез едно обхождане на дървото, и (iii) решава линейното програмиране с ограничение за безопасност със решавача HiGHS на SciPy. Всичко останало е цикълът на генериране на ограничения, който е достатъчно кратък, за да бъде представен изцяло:

```

c_model <- payoff vector of EV vs the opponent model      # linear objective
cuts    <- {}                                              # discovered safety cuts
repeat:
  x <- argmax c_model . x s.t. treplex(x) and all cuts      # one LP solve
  pol <- behavioral strategy read out of x
  wc <- worst_case_value(pol)                               # = -(opponent's exact best response) : the oracle
  if wc >= floor - tol: return pol                          # safe: done
  adv <- opponent's exact best response to pol
  add the cut c(adv) . x >= floor - slack to cuts          # a valid lower bound on the worst case

```

Всяко генерирано ограничение представлява вектора на печалбата спрямо *специфичен* най-добър отговор на противника; то е валидна линейна долна граница на истинския най-лош случай, поради което добавянето на ограничения монотонно затяга релаксираното ограничение за безопасност към действителното. Извеждането на поведенческа стратегия от план за реализация x се извършва чрез съотношението $\beta(I, a) = x_{Ia} / x_{\text{seq}(I)}$ (равномерно в случаите, когато теглото на родителя е приблизително 0).

Единственото, което се различава между петте метода, е стойността на `floor` и, за метода на под-игра (§7), набор от *цифтове*, които фиксират играта на героя извън избрана под-игра да бъде равна на плана. Ганцфрид използва `floor = v*`; безопасен спрямо прайм използва `floor = v* -`; адаптация използва `floor = worst_case(blueprint)`; RNR преформулира целевата функция като максимин в параметър p (§5).

Една практическа особеност, която си струва да се отбележи (тя доведе до реално отстраняване на грешки при изпълненията): *приблизителната* стойност от самообучението в равновесието на Наш може да бъде съвсем малко **над** истинската постижима максимин стойност на играта, така че изискването за точно `wc floor` може да направи линейното програмиране в крайна сметка недопустимо и, при неблагоприятен път на ограниченията, да върне *небезопасна* стратегия. Решението е малък резерв за допустимост върху ограниченията (`floor - slack`, като `slack = 5 · 10-6`, поддържан независим от допустимата грешка при сходимост), така че истинската максимин стратегия винаги да остава допустима. Това е видът числов детайл, който никога не се появява в научните статии, но решава дали методът всъщност ще върне безопасна стратегия.

Прочетете повече: идеята за генериране на ограничения / двойно предсказание води началото си от McMahān, Gordon & Blum (2003), “Планиране в присъствието на функции на разходите, контролирани от противник,” *ICML* — общата рецепта за “оптимизиране срещу най-лош случай, който откривате докато вървите.”

8.5 5. Ограничен отговор по Наш и граница тип „вкл.–изкл.”

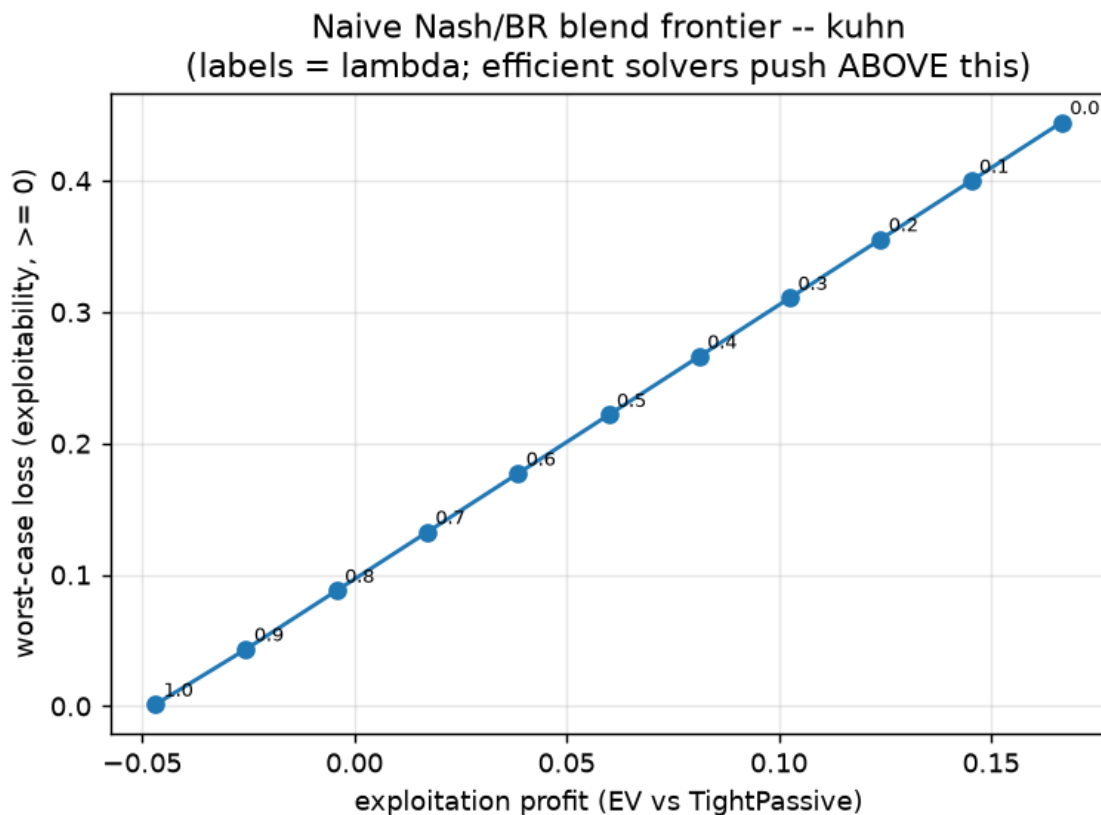
Най-старият принципен метод, **ограничен отговор по Наш** (Johanson 2007), е концептуалният предшественик на всички останали и въвежда *настройваемия параметър*. Неговата идея: да се изчисли равновесието на героя срещу p -ограничен противник — такъв, който е принуден да играе фиксирания модел с вероятност p и е свободен да играе противнически с вероятност $1 - p$. Променяйки p от 0 до 1 се проследява път от

равновесието на Наш ($p = 0$) до пълния най-добър отговор ($p = 1$). Тъй като свободният компонент дава най-добър отговор на героя, RNR е **максимин**:

$$\text{RNR}(p) = \arg \max_x \left[p \cdot \text{EV}(x, \text{model}) + (1 - p) \cdot \min_{\sigma'} \text{EV}(x, \sigma') \right],$$

която решаваме със същия механизъм с равнини на отсичане плюс допълнителна променлива за най-лошия случай.

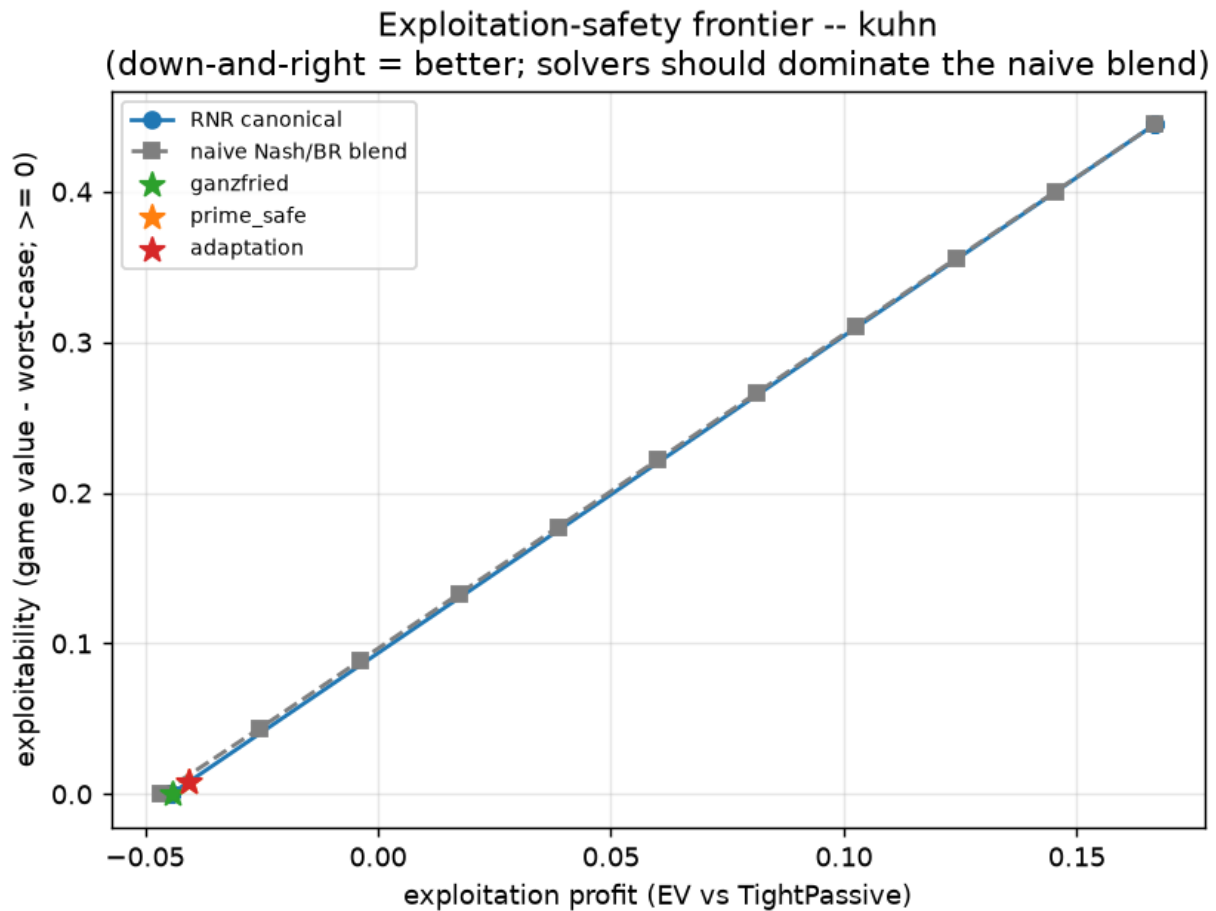
Знаме, което си струва да се запази. В първоначалните бележки на проекта RNR беше описан като *наивна смеска*, $(1 - p) \cdot \text{Nash} + p \cdot \text{BR}$. Това е полезен интуитивен инструмент, но **не представлява** алгоритъма на Johanson; ние имплементирахме *и двата* и ги обозначихме, тъй като в малка игра поведението им е коренно различно. Наивната смеска проследява плавна крива (фигура по-долу, вляво): печалбата и експлоатируемостта се увеличават почти пропорционално заедно — от **наш равновесие** до пълен най-добър отговор.



Фигура 3: The naive Nash/best-response blend on Kuhn: a smooth exploitation-safety frontier where each increment of profit costs a near-proportional increment of your own worst-case loss.

Измерената изненада — Canonical RNR е изненадващ обрат. Предвидих, че каноничното обхождане ще бъде гладка, монотонна граница, която доминира смеската навсякъде. Не е така. В играта кун срещу камък, Canonical RNR връща **една и съща безопасна стратегия** за всички $p \in [0, 0.6]$ (EV -0.044 , експлоатируемост ≈ 0) и след това **преминава направо към пълен най-добър отговор** при $p \in (0.6, 1]$ (EV 0.0 , експлоатируемост ≈ 0.5). Няма междинни точки.

	Canonical RNR —	Canonical —	наивна смеска —	наивна —
p	EV	експлоатируемост	EV	експлоатируемост
0.0	−0.044	~0.000	−0.047	0.000
0.3	−0.044	~0.000	+0.017	0.133
0.6	−0.044	~0.000	+0.081	0.266
0.7	+0.167	0.444	+0.103	0.311
1.0	+0.167	0.444	+0.167	0.444



Фигура 4: The exploitation-safety frontier on Kuhn with the LP operating points. The canonical RNR “curve” is only an interpolation between two achieved clusters (the safe corner and the full-BR corner); the smooth line is the dominated naive blend; the stars (Ganzfried, prime-safe, adaptation) sit at the efficient safe corner.

Защо (проверих, преди да се доверя на историята). Целта на RNR е **линейна** в x върху един **политоп**, така че нейният оптимум е **връх** и сменя върховете само когато p пресече критично съотношение. Политопът на стратегиите в кун е малък — малко върхове — така че **преходът се изразява в един-единствен скок**. Гладката крива на Йохансон е феномен от *големи игри* (много върхове) или идва от *данни-ориентирания* вариант, който прави p за всеки информационен набор. Наивната смеска изглежда гладка само защото линейно интерполира две фиксирани стратегии — и тя е **доминирана в безопасния връх** (при експлоатируемост ≈ 0

методите на ЛП достигат EV -0.044 спрямо -0.047 на смеската). Основният извод остава — „избери къде да се отклониш, а не *колко* равномерно“ — но механизмът („гладък RNR циферблат“) е артефакт от големи игри и в малка игра честната картина е дискретен преход от безопасен връх към BR-връх, чийто праг *се измества според това колко експлоатируем е противникът*. Тази последна точка се повтаря в §6: срещу много експлоатируем противник дори $p = 0.5$ вече е отминал скока.

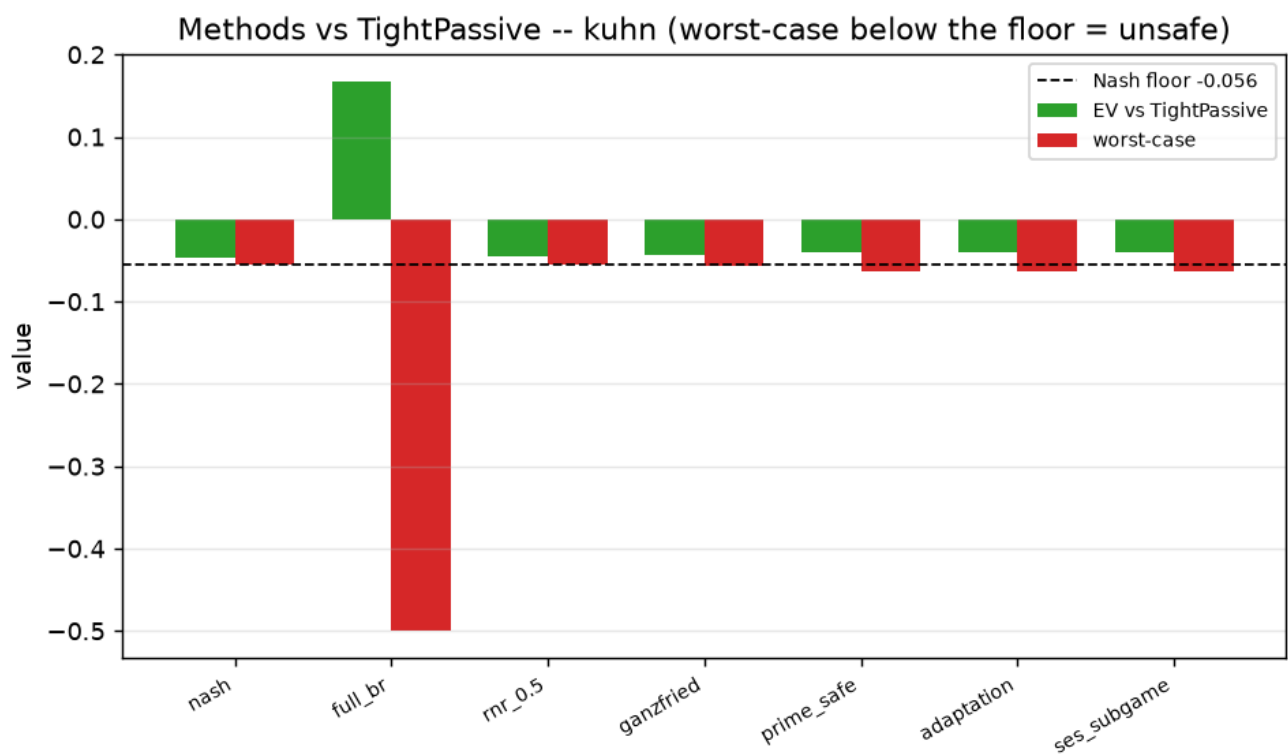
Прочетете повече: Johanson, M. & Bowling, M. (2009). “Data Biased Robust Counter Strategies.” *AISTATS* — прави лоста p да зависи от това колко данни подкрепят всяка част от стратегията, което е това, което изглажда границата на практика.

8.6 6. Ганцфрид върху кун — безопасен и печеливш (основният резултат)

Централният експеримент решава всеки метод срещу *перфектен* модел на всеки **тип противник** и го оценява по **печалба** (EV спрямо този **тип противник**) и **безопасност** (**най-лошият случай**), като подът на **наш равновесие** е $v^* = -0.056$ (известната първоначална **стойност в кун**, $\approx -1/18$). Всички числа са точни (пълно дърво), от мащабния прогон.

		наш					
Противник	метрика	равновесие	full_br	rnr_0.5	ганцфрид	prime_safe	адаптация
Камък (стегнат-пасивен)	EV	-0.047	+0.167	-0.044	-0.044	-0.040	-0.040
	най-лошият случай	-0.056	−0.500	-0.056	-0.056	-0.063	-0.063
Маниак (LooseAggr.)	EV	$+0.118$	$+0.333$	$+0.278$	+0.131	$+0.151$	$+0.151$
	най-лошият случай	-0.056	-0.167	-0.111	-0.056	-0.063	-0.063
AlwaysBet	EV	$+0.113$	$+0.333$	$+0.333$	+0.115	$+0.131$	$+0.131$
	най-лошият случай	-0.056	-0.167	-0.167	-0.056	-0.063	-0.063
AlwaysPass (най-експлоатируем)	EV	$+0.146$	+0.975	$+0.975$	+0.222	$+0.266$	$+0.266$
	най-лошият случай	-0.056	-0.500	-0.333	-0.056	-0.063	-0.063

		наш					
Противник метрика		равновесие	full_br	rn timer_0.5	ганцфрид	prime_safe	адаптация
Наш равно- весие (конт- рол)	EV	-0.056	-0.055	-0.056	-0.056	-0.055	-0.055
	най- лошият случай	-0.056	-0.333	-0.056	-0.056	-0.063	-0.063



Фигура 5: Methods versus the Rock on Kuhn: green is EV against the opponent, red is worst-case value, the dashed line is the Nash floor. Only the full best response's worst-case (red) plunges to -0.5 , far below the floor; every principled method hugs the floor while still exploiting.

Три основни резултата обобщават тази глава.

1. **Пълният най-добър отговор е поучителната история.** Той винаги печели най-много срещу фиксиран модел — до $+0.975$ срещу `alwaysPass` — но неговият най-лошият случай се срива до -0.5 . Това е единственият експлоатируем метод в таблицата и той е експлоатируем срещу *всеки* противник, защото противникът винаги може оптимално да отговаря обратно на дупката, която той отваря.
2. **Ганцфрид е безопасната и печелившата златна среда.** Срещу *всеки* противник неговият най-лошият

случай остава на долната граница на Наш (безопасен в рамките на 10^{-3}) и той превъзхожда равновесната стойност на Наш при всеки експлоатируем тип — най-ярко **+0.222 срещу +0.146 на Наш** срещу alwaysPass и +0.131 срещу +0.118 срещу маниак. Това е резултатът, който стъпката съществува, за да произведе: *можете да експлоатирате смислено, като доказуемо никога не падате под равновесната стойност.*

3. **Един глобален p не е настройка за безопасност.** `rn_r_0.5` е безопасен срещу камък, но вече е скочил до пълния най-добър отговор срещу силно експлоатируемите alwaysPass / AlwaysBet (идентична стойност с `full_br`, най-лошият случай $-0.33 / -0.17$, експлоатируем). Това е §5 прагов контрол „включено/изключено“, който се движи с експлоатируемостта на противника — и конкретната причина на Ганцфрид, който ограничава *стойността*, а не *копче*, да бъде по-добрият примитив.

Безопасен спрямо прайм и адаптацията изразходват предвидения ϵ -бюджет. Те понижават долната граница от v^* до $v^* - \epsilon$, а изпълнението *измерва* експлоатируемостта на базовата линия с ранно прекратен CFR при $\epsilon = 0.0074$; техният най-лош случай излиза $-0.063 = v^* - 0.008$ срещу всеки противник, съответствайки на долната граница, коригирана с ϵ . Изразходвайки този бюджет, те печелят малко повече от ганцфрид (+0.266 спрямо +0.222 на AlwaysPass). Те изглеждат „експлоатируеми“ в таблицата само защото флагът се сравнява с v^* ; спрямо тяхната *собствена* долна граница те са безопасни по конструкция. (Безопасен спрямо прайм и адаптацията съвпадат тук, защото за тази базова линия $\square \text{MATHI5} \square$ — двете долни граници са буквално равни, което изпълнението потвърждава, а не е грешка.)

Бележка за резолюцията на измерването, защото това ме ухапа. В *smoke* изпълнението (30 000 CFR итерации) дори стратегията на Наш е маркирана като „експлоатируема“ — нейният най-лошият случай е -0.0568 , т.е. 0.0013 под точната стойност v^* , малко над допустимата стойност 10^{-3} . Това е чиста грешка на приближението, а не провал в безопасността: при по-голям мащаб (200 000 итерации) недостигът спада до 3×10^{-4} и Наш правилно е маркиран като безопасен. Урокът за тестовата среда е, че допустимата стойност на флага за безопасност трябва да надвишава собствената грешка на приближението на базовата линия, в противен случай базовата линия ще „спъне“ собствения си тест.

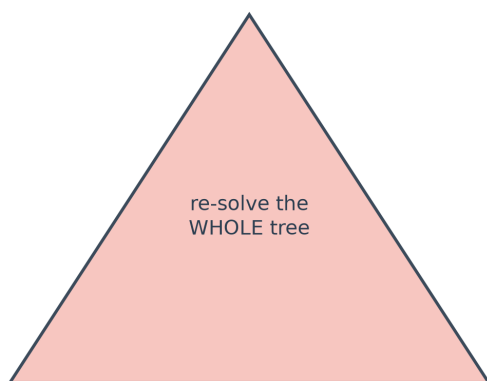
Read more: Ganzfried, S. & Sandholm, T. (2015), *op. cit.* — Theorem 1 is exactly the guarantee this table exhibits on Kuhn (най-лошият случай $\geq v^*$), and its minimax step is the 2-player-zero-sum dependency of §3.

8.7 7. Безопасност в реално време — под-игра джаджи (SES)

Гаранцията на ганцфрид е *глобална*: ограничението за безопасност се прилага върху **цялото** дърво на играта. Това е приемливо при кун, но в реална игра не можете да преизчислявате цялото дърво всеки път, когато моделът се актуализира. Идеята за **търсене на безопасна експлоатация** (Liu и др. 2022) прави безопасността **локално** свойство на една единствена **под-игра**: играйте нашия план навсякъде, но в избрана под-игра оптимизирайте отново играта на героя, за да експлоатира модела — с **приспособление**, което гарантира, че локалното отклонение никога не може да направи героя по-зле *глобално* от плана.

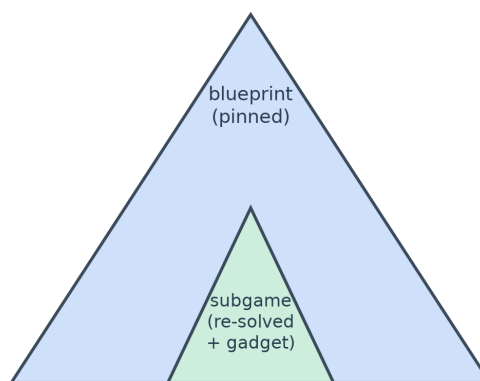
В последователна форма приспособлението се реализира директно върху treeplex: **фиксирайте** всяка последователност от герои *извън* под-играта към теглото на реализацията на плана (така че героят играе плана навсякъде, освен в под-играта) и задайте прага на безопасност на най-лошата стойност на самия план. Тъй като „играйте плана също вътре в под-играта“ винаги е възможно и достига точно този праг, линейното програмиране може само да се справи поне толкова добре — локалната експлоатация е безопасна по конструкция. Единственото изискване е под-играта да бъде **надолу-затворена** (веднъж щом сте в нея, оставате в нея), което важи за естествени избори като „ледюк кръг 2“ или „след като се падне поп“.

GLOBAL safety (Ganzfried / prime-safe / adaptation)



Leduc: capped at 40 iters, NOT converged
worst-case -0.64 .. -1.33 (grossly unsafe)

LOCAL safety (SES subgame gadget)



Leduc: converged in 194-350 iters
worst-case ~ -0.13 (near-safe), $+0.25..+0.68$ vs weak types

Фигура 6: Global versus local safety. Global methods re-solve the whole tree; on Leduc their constraint-generation loop did not converge within the iteration budget and left grossly unsafe strategies. The subgame method pins play outside a chosen subgame to the blueprint and re-solves only that subgame with a gadget — a far smaller problem that did converge and stayed near-safe.

Значението на „локално“ не е само въпрос на естетика; §8 показва, че именно то определя дали решаването ще **успее** или няма да успее.

Прочетете повече: Liu, W. и др. (2022). “Safe Opponent-Exploitation Subgame Refinement.” *NeurIPS* (приспособлението). · Ge, Z. и др. (2024), *op. cit.* — OX-Search ограничава загубата от експлоатация *по информационно множество*, като по този начин укрепва същата идея срещу атаки тип „обучение“. · Milec, D., Kovařík, V. & Lisý, V. (2025). “Adapting Beyond the Depth Limit,” *arXiv:2501.10464* — използване на информация от модел на противника *отвъд* хоризонта на търсене.

8.8 8. В мащаб — кун работи; ледюк се разпада глобално, но се задържа локално

Ледюк Холдем добавя втори рунд за залози и една обща карта — приблизително два порядъка повече ситуации от кун, все още точно решими, но достатъчно големи, за да натоварят методите. Изпълнението за ледюк използваше конфигурация с ограничен брой итерации (брой итерации 40 в цикъла на генериране на ограничения, допустима стойност 10^{-2} и под-играта зададена на крал-флоп), като записваше за всяка клетка дали решаването е **сходимо** или е достигнало лимита. Стойност на играта $v^* = -0.086$.

Опонент	метрика	наш равновесие	full_br	ses_под-игра	ганцфрид	prime_safe / адаптация
Камък	EV	+0.201	+0.937	+0.247	+0.624	+0.635
	най-	-0.089	-1.633	-0.130	-0.838	-0.744
	лошият случай					
	сходим?	□	□	□ (194 итерации)	□ достигнат лимит	□ достигнат лимит
Маниак	EV	+0.438	+2.177	+0.682	+1.806	+1.842
	най-	-0.089	-1.100	-0.130	-0.638	-0.657
	лошият случай					
	сходим?	□	□	□ (211 итерации)	□ достигнат лимит	□ достигнат лимит
CallingStationEV	EV	+0.559	+1.464	+0.663	+1.342	+1.363
	най-	-0.089	-1.000	-0.130	-0.915	-0.686
	лошият случай					
	сходим?	□	□	□ (350 итерации)	□ достигнат лимит	□ достигнат лимит
LoosePassiveEV	EV	+0.449	+1.405	+0.559	+1.306	+1.309
	най-	-0.089	-4.200	-0.133	-1.239	-1.334
	лошият случай					
	сходим?	□	□	□ (400 итерации)	□ достигнат лимит	□ достигнат лимит

Основната констатация е отрицателна и емпирична: глобалната безопасна експлоатация не се мащабира, дори до ледюк. Предвидих, че ганцфрид ще бъде безопасен на ледюк, както е на кун. Вместо това **глобалните решавачи (ganzfried, prime_safe, adaptation и rnr_0.5) всички достигнаха лимита от 40 итерации без да сходят, оставяйки стойности за най-лошия случай от -0.64 до -1.33 — грубо небезопасни.** Причината, която потвърдих, вместо да предполагам: цикълът с равнини на сечение добавя по

един съперников срез на итерация и при по-голямото дърво на ледюк множеството от релевантни чисти най-добри отговори е голямо, така че 40 среза изобщо не фиксират истинския най-лош случай; главната линейна програма продължава да връща оптимистични, но небезопасни стратегии. Това беше маркирано като номер едно „вероятно ще се счупи“ елемент *преди* изпълнението и точно там се счупи.

Положителната половина е методът на под-играта. SES **сходи** при трима от четиримата експлоатируеми опоненти (194–350 итерации), тъй като пререшава само малката под-игра „крал-флоп“, като останалата част от дървото е фиксирана — значително по-малка линейна програма с далеч по-малък съперников набор. Той извлича реална **стойност** (+0,25 до +0,68 спрямо слабите типове, надминавайки **наш равновесие**) при **най-лошия случай** от $\approx -0,13$, което е с порядък по-близо до **безопасност** в сравнение с глобалните методи. Това е разграничението *глобална срещу локална безопасност* от §7, което се проявява като измерима констатация върху игра толкова малка като **ледюк** — конкретният аргумент за методите за под-игри в реално време и против наивно глобално решаване.

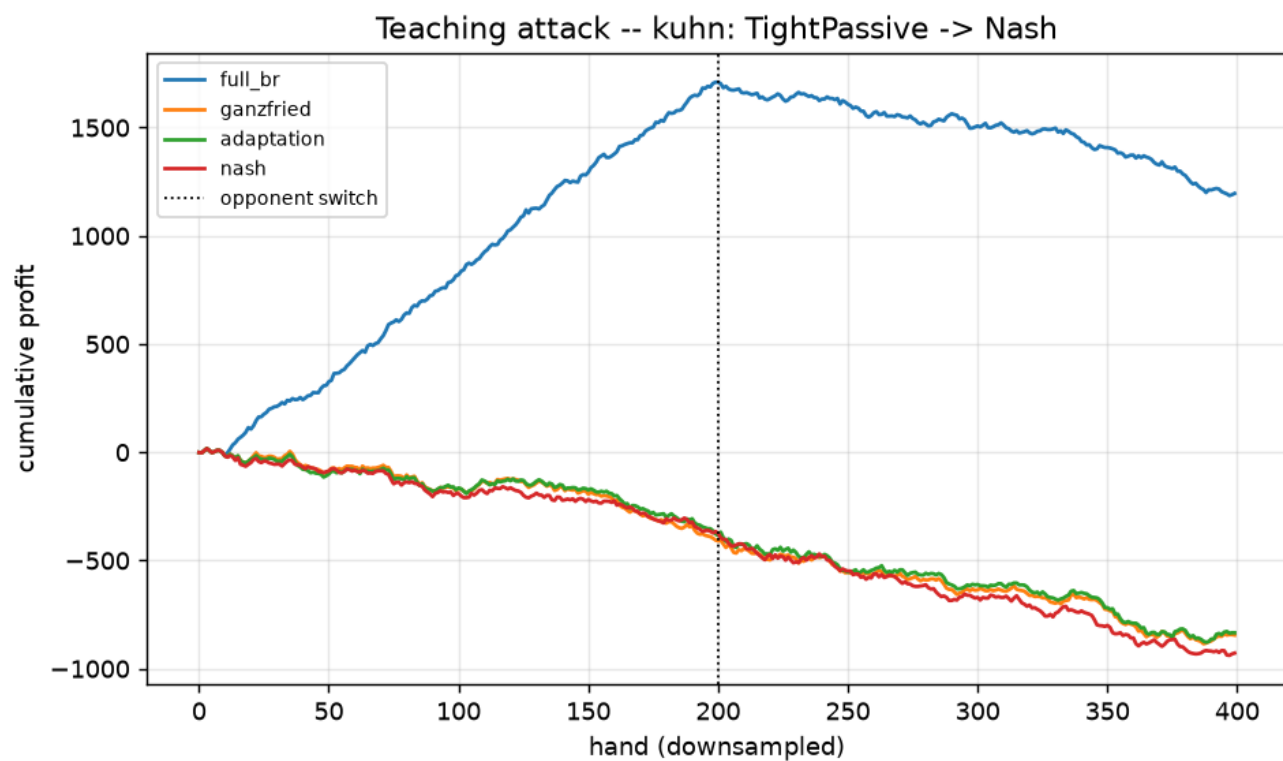
Оставам честен към себе си относно SES: неговата остатъчна експлоатируемост ($\approx 0,043$) все още надвишава допустимата стойност от 0,01, така че и той е маркиран като небезопасен, а при LoosePassive изпълни пълните 400 итерации без **сходимост**. Дали този остатък от 0,04 представлява приспособлението, което легитимно се ограничава до вече под- v^* план (най-лошият случай на самото **наш равновесие** е $-0,089$, което е с 0,003 под v^*), артефакт от толерантността при **сходимост** или малко изтичане във външното поддеревно фиксиране е основният въпрос за разрешаване, преди SES да бъде обявен за *доказуемо безопасен*.

8.8.1 Обучаващата атака — и защо осъществената печалба е погрешен критерий

Последният експеримент е стрес тест за измама, който цялата система за безопасност трябва да издържи. Измамлив опонент играе слабата примамка „камък“ в продължение на 10 000 раздавания, след което преминава към силно „наш равновесие“ „разкриване“ за още 10 000; моделът от Стъпка 7 подава всеки решаващ на всеки 500 раздавания (5 семена).

метод	средна стойност/ръка (всички)	средна стойност/ръка (след преминаване)	нарушения на безопасността / начално число
full_br	+0.051	-0.061	40, 40, 40, 40, 40
ганцфрид	-0.048	-0.055	0, 0, 0, 0, 0
адаптация	-0.046	-0.055	40, 40, 40, 40, 40
наш равновесие	-0.051	-0.061	0, 0, 0, 0, 0

Наивният разказ — „обучаващата атака наказва алчния експлойтър“ — **не се потвърди по отношение на осъществената печалба**, а разбирането защо е поучително. Пълният най-добър отговор завършва *значително положително* (+0,051/ръка общо): той реализира неочаквана печалба по време на фазата с примамката и, тъй като „разкриването“ е само *Нашов* опонент, този опонент успява да си върне едва малко повече от стойността на играта на ръка, така че неочакваната печалба никога не се възстановява в рамките на 10 000



Фигура 7: Teaching attack on Kuhn, cumulative profit. The full best response (blue) climbs on the bait to about +1700, then only drifts down after the switch — it ends far ahead, because a Nash “revealer” claws back only about the game value per hand. The safe methods refuse the bait and pay the first-player tax throughout.

ръце. Сигналът, който *наистина* отличава безопасните от небезопасните методи, е **точният най-лош случай / брой нарушения на безопасността**: пълният най-добър отговор наруши долната граница на Наш в **40 от 40** повторни настройки, а Ганцфрид — в **0 от 40**. Най-лошият случай за една стратегия се реализира от опонент, който *дава най-добър отговор спрямо нея* — и стационарното разкриване на равновесието на Наш просто не е такъв противник. Коригираният извод, в който сега вярвам, че е правилният урок за проектиране: **измервайте безопасността чрез най-лошия случай, а не чрез осъществената печалба срещу добронамерен опонент**; за да накажете алчния експлойтър *по отношение на печалбата*, разкриването трябва да бъде адаптивен контраексплойтър, а не фиксирано равновесие на Наш. (Това, че показва 40 нарушения, докато показва 0, също не е грешка: адаптацията умишлено се насочва към долна граница под v^* , така че по дизайн задейства брояча, посочен в v^* .)

8.9 9. Връзки и бъдещи насоки

Какво установява тази стъпка. Безопасното използване представлява една идея — *максимизиране на стойността спрямо модела при спазване на праг на безопасност* — и в напълно решима игра то функционира точно както предвижда теорията: „Ганцфрид“ е безопасен срещу всеки опонент, като същевременно надминава равновесната стойност при всеки експлоатируем такъв, докато наивният най-добър отговор носи повече печалба, но е катастрофално експлоатируем. „Прайм-безопасен“ и „адаптация“ разширяват гаранцията към неперфектните базови нива, присъщи на всяка реална система, като изразходват *измерен ϵ -бюджет* под стойността на играта. Но най-ценният резултат от тази стъпка е отрицателен и емпиричен: дори в игра с малък размер като „Ледюк“ глобалното решаване на задачата за безопасно използване не сходява в рамките на практически бюджет, докато локалният метод на под-играта успява — разликата между глобалната и локалната безопасност от теория към практика, измерена, а не просто заявена.

Обратни връзки. Тази стъпка представлява точния комплемент на предходната работа върху равновесието и моделирането на противника. Сензорът от Стъпка 7 създава модела на противника, който тук се превръща в *цел*; точният най-добър отговор от Стъпка 7 функционира като *най-лошият случай оракул*; а представянето в последователна форма от работата върху CFR и съгласуваността е *променливата*, която линейното програмиране оптимизира. Емпиричната мотивация за необходимостта от прага на безопасност в тази стъпка произтича от самопричиненото изтичане на непрекъснатия модел спрямо **наш равновесие** в играта **ледюк** (Стъпка 7, §7) — и действително **ганцфрид** тук никога не изтича към **наш равновесие**.

Напред към дисертацията. Резултатите на Кун потвърждават коректността на изпълнителния механизъм; несходимостта в Ледик представлява конкретният аргумент за двата мащабируеми пътя, между които дисертацията трябва да избере — **точно еднократно двойно линейно програмиране** за ограничението при най-лошия случай или ангажимент към **локална / под-игра безопасност** (SES, OX-търсене) като механизъм в реално време. И в двата случая прагът на безопасност е това, което превръща крехкия сензор от Стъпка 7 в приложим адаптивен агент.

Отворени въпроси за бъдещи изследвания.

- *Мащабируема безопасност.* Заменете цикъла с равнини на сечение с точно двойно линейно програ-

миране или възприемете изцяло безопасността на под-играта; установете дали глобалните решаващи за Ледияк са просто *бавни* или *структурно* заседнали чрез повторно изпълнение с голям бюджет.

- *Остатъчна експлоатируемост на SES джаджата.* Определете дали разликата от 0,04 над толерантността е ефект на ограничаване от плана, артефакт на толерантност или изтичане при фиксиране — това решава дали методът на под-играта е доказуемо или само приблизително безопасен.
- *Наказваща обучаваща атака.* Заменете разкриването на равновесието на Наш с адаптивен контраексплойтър, така че осъществената печалба да потвърждава най-лошия случай и може да се покаже, че безопасният метод *надминава* алчния при измама.
- *От двама играчи до N .* Всяка гаранция тук се основава на факта за двуигрови игри с нулева сума, че стратегията на Наш осигурява стойността v^* срещу всеки противник — котва, която изчезва за $N > 2$. Намирането на аналог за N играчи (или структурен заместител) е отвореният проблем, който тази стъпка прави прецизен, и сърцевината на Принос №2.

Прочетете повече: Shoham, Y. & Leyton-Brown, K. (2008). *Multiagent Systems*, гл. 7 „Обучение и преподаване“ — повтарящата се игра, в която вашите действия едновременно *експлоатират* и *преподават* на противника, което е точното напрежение, което обучаващата атака (§8) изследва и многоагентното разширение трябва да посрещне.

Стъпка 9 Обобщение — Многоагентно обучение с подсилване

Изследване върху възможностите за прилагане на изкуствен интелект в компютърните игри

Alexander Andreev

July 2026

Съдържание

9	Стъпка 9 — Многоагентно обучение с подкрепление: координация, конкуренция и комуникация	1
9.1	1. Защо многоагентното обучение с подкрепление е различен проблем	2
9.2	2. Марковски игри — мостът между стъпки 2–8 и MARL	3
9.3	3. Семейството от методи	4
9.4	4. Самостоятелно обучение и неговите режими на провал	7
9.5	5. Централизирано обучение, децентрализирано изпълнение (CTDE)	8
9.6	6. PSRO — теория на игрите върху съвкупност от политики	13
9.7	7. Научена комуникация (commNet)	17
9.8	8. LOLA — моделиране на опонента като учащ се	17
9.9	9. Честни бележки, ограничения и къде това раздаване продължава нататък	18
9.10	Основни изводи за синтеза на дисертацията	20

9 Стъпка 9 — Многоагентно обучение с подкрепление: координация, конкуренция и комуникация

Тази глава е изградена от основи върху многоагентното обучение с подкрепление (MARL): проблемът, който го отличава от всичко преди него, математиката, която го рамкира, семейството от методи, които се използват за решаването му, и набор от контролирани експерименти, проведени в малки, точно решими игри. Тя има две цели — да послужи като самостоятелно опресняване за по-късните стъпки, които се надграждат върху него, и като основен източник за синтеза на дисертацията в Част 1. Написана е така, че да може да бъде прочетена независимо; не се предполага предварително запознаване с кода на проекта. **Всички експериментални числа, представени тук, са измерени** при възпроизводими изпълнения на тестовите среди и, където това е възможно, са ограничени от *точни* аналитични препратки (наш равновесия, точни стойности на най-добър отговор), а не от други симулации. Когато дадено изпълнение противоречи на очакванията, произтичащи от теорията, запазвам първоначалното предварително убеждение и го съгласувам с получения резултат — тези разминавания са най-поучителните части от стъпката.

Къде се вписва това в дисертацията. Стъпки 2–8 съществуват изцяло в рамките на **двуйгрови игри с нулева сума**: има стойност v^* , нашава стратегия я осигурява срещу *всеки* противник и CFR доказуемо се сближава към нея. Стъпка 9 е повратната точка към **многоагентния** свят, където тези три удобства отслабват

или изчезват. Тя съдържа три теза. **LOLA** — диференциране през *стъпката на обучение* на противника — е *динамично* моделиране на противника, допълнение към статичното четене от Стъпка 7 (Принос №1). **PSRO** — игра, която се играе върху *популация от стратегии* — едновременно представлява рамката за безопасна експлоатация, когато няма теорема за минимакс (Принос №2), и обща многоагентна *методология за оценяване* (Принос №3). Мястото, където всяка двуйгрова гаранция се нарушава — липсващата минимакс котва за $N > 2$ — е назовано тук и оставено отворено за атака от дисертацията.

9.1 1. Защо многоагентното обучение с подкрепление е различен проблем

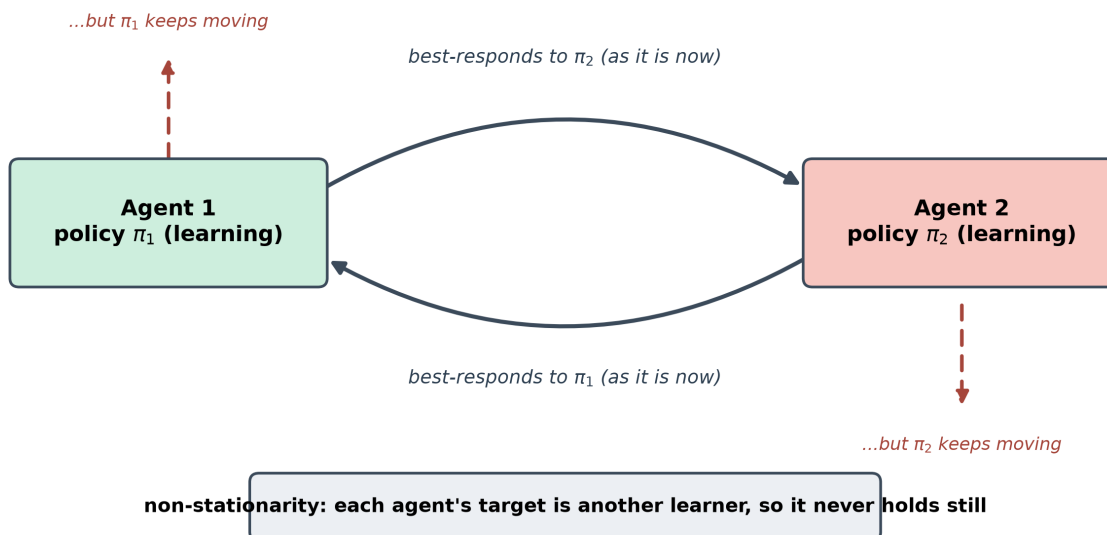
В single-agent RL (Стъпки 1 и 6) един агент се учи чрез проба и грешка в **непроменяща се** среда. Формално той се изправя пред стационарен марковски процес на вземане на решения: законът за преход $P(s' | s, a)$ и наградата $R(s, a)$ не се изменят по време на обучението, което позволява да съществува фиксирана оптимална стойностна функция Q^* , към която агентът сходява. В Стъпки 2–8 ние преминахме към едно по-високо ниво и изчислихме **равновесия** — стратегии, които са оптимални срещу *съвършено рационален* противник, който вече е завършил своите разсъждения.

Многоагентното обучение с подкрепление се намира между тези два подхода и е по-трудно и от двете. Няколко агента **се учат едновременно в споделена среда**, така че от гледната точка на всеки един агент „средата“ — която вече *включва и другите агенти* — **непрекъснато се изменя**, докато всички се обновяват. Това е **нестационарност** и тя нарушава предположението, което стои в основата на single-agent RL. От гледна точка на агент i ефективният преход и награда зависят от политиките π_{-i} на другите агенти, а те се променят при всяко обновяване, така че целта Q^* , която агент i преследва, самата тя е в движение. Това също така поражда два проблема, които просто не съществуват за самостоятелен агент: **координация** (как си сътрудническите агенти се учат да действат заедно, без да им бъде казано как?) и **разпределяне на заслугата** (когато отборът успее, чии действия са имали значение?).

Картина, която да запомните: **учене на танц с партньор, който също се учи да танцува**. Ако стъпките на вашия партньор бяха фиксирани, бихте могли да запаметите рутина, която им подхожда — това е single-agent RL. Ако вашият партньор беше безупречен професионалист, бихте могли да изучите известния му стил и да се подготвите с перфектен отговор — това е изчисляване на равновесие. Но когато *и двамата* се подобряват в реално време, всяка промяна, която правите, променя това, което другият трябва да направи, и обратното; вие си стъпвате един на друг по краката, прекоригирате и осцилирате — докато или не се заключите в споделен ритъм (координирано равновесие), или циклирате завинаги (както в игра „хвърляне на монета“, §4). Цялата област е изкуството да *подредите картите така, че синхронизацията да се случва надеждно*.

Прочетете още: Zhang, K., Yang, Z. & Başar, T. (2021). “Multi-Agent Reinforcement Learning: A Selective Overview of Theories and Algorithms.” *Handbook of RL and Control* — §1–2 за нестационарността и таксономията, използвана тук.

Single-agent RL assumes a fixed world; here the 'world' is a partner who is also learning.



Фигура 1: Non-stationarity as two partners learning to dance at once. Each agent optimizes against the other's current policy (the solid arrows), but that policy is itself moving because the other agent is optimizing back (the dashed arrows). The target each one chases is never still, so naive learners cycle or over-correct rather than converge. Every method in this chapter is a different way to make the moving target hold still long enough to learn against.

9.2 2. Марковски игри — мостът между стъпки 2–8 и MARL

Стъпки 2–8 разсъждаваха върху **игри в екстензивна форма (EFGs)**: дърво на играта, **множества информация**, групиращи истории, които играчът не може да различи, и контрафактични стойности, подавани към решавач за съжаление (CFR). Стъпка 9 разсъждава върху **Марковски (стохастични) игри**, стандартната формализация на MARL. Двете се свързват чисто и изясняването на тази връзка е това, което не позволява преминаването да изглежда като започване отначало.

Една **Марковска игра** е наредената n -торка

$$(\mathcal{S}, \{\mathcal{A}_i\}_{i=1}^N, P, \{R_i\}_{i=1}^N, \{\Omega_i\}, O, \gamma),$$

с множество състояния $s \in \mathcal{S}$, по едно множество действия $\mathcal{A}_i$ за всеки агент, закон за преход $P(s' | s, a_1, \dots, a_N)$, управляван от **съвместното** действие, награди на агент $R_i(s, a_1, \dots, a_N)$ и (при частично наблюдавания случай) наблюдения $o_i = O_i(s)$. Всеки агент има стратегия $\pi_i(a_i | o_i)$; заедно те формират **съвместна стратегия** $\pi = (\pi_1, \dots, \pi_N)$. Специалните случаи са именно предишните стъпки:

- $N = 1$ възстановява обикновено **MDP** (Стъпка 1).
- $N = 2, R_1 = -R_2$, едно състояние води до **матрична игра** (тестовата среда на тази стъпка, §4).
- Последователна игра с **непълна информация** и игра с **нулева сума** дава **EFG** (Стъпки 2–8): Markov игра, при която „състоянието“ е *история*, а **частичната наблюдаемост** се изразява чрез **информа-**

ЦИОННО МНОЖЕСТВО.

Какво преминава през моста е *речникът*. Политиката все още съпоставя това, което знаеш, с разпределение върху действията, така че поведенческата стратегия на EFG в информационно множество е точно политиката на марковска игра $\pi_i(a_i \mid o_i)$; информационното множество се превръща в наблюдението o_i ; а контрафактичната стойност на история става оценка на централизиран оценител в състояние. Това, което **не** преминава през моста, са *гаранциите*. Контрафактичното разлагане на CFR изисква дърво и пълна памет, които общите марковски игри (цикли, едновременни ходове, $N > 2$) не предоставят, така че „средна стратегия $\rightarrow$ Наш“ вече не важи; MARL се връща към обучение чрез градиент/стойност, чиято сходимост не е гарантирана (оттук и цикличността в §4). Най-съществено, **котва на минимакс стойността** изчезва: в двуговни игра с нулева сума нашата стратегия гарантира v^* срещу всеки противник — фактът, който направи безопасната експлоатация в Стъпка 8 последователна — и за $N > 2$ няма такава единична стойност. Тази липсваща котва е точната празнина, която Принос #2 наследява.

Прочетете повече: Littman, M. L. (1994). “Markov Games as a Framework for Multi-Agent Reinforcement Learning.” *ICML* — статията, която въведе тази рамка и алгоритъма minimax-Q.

9.3 3. Семейството от методи

Всеки метод на този етап представлява различно структурно решение на въпроса „другите агенти се учат, докато аз се уча“. Тук са от значение пет основни **семейства**.

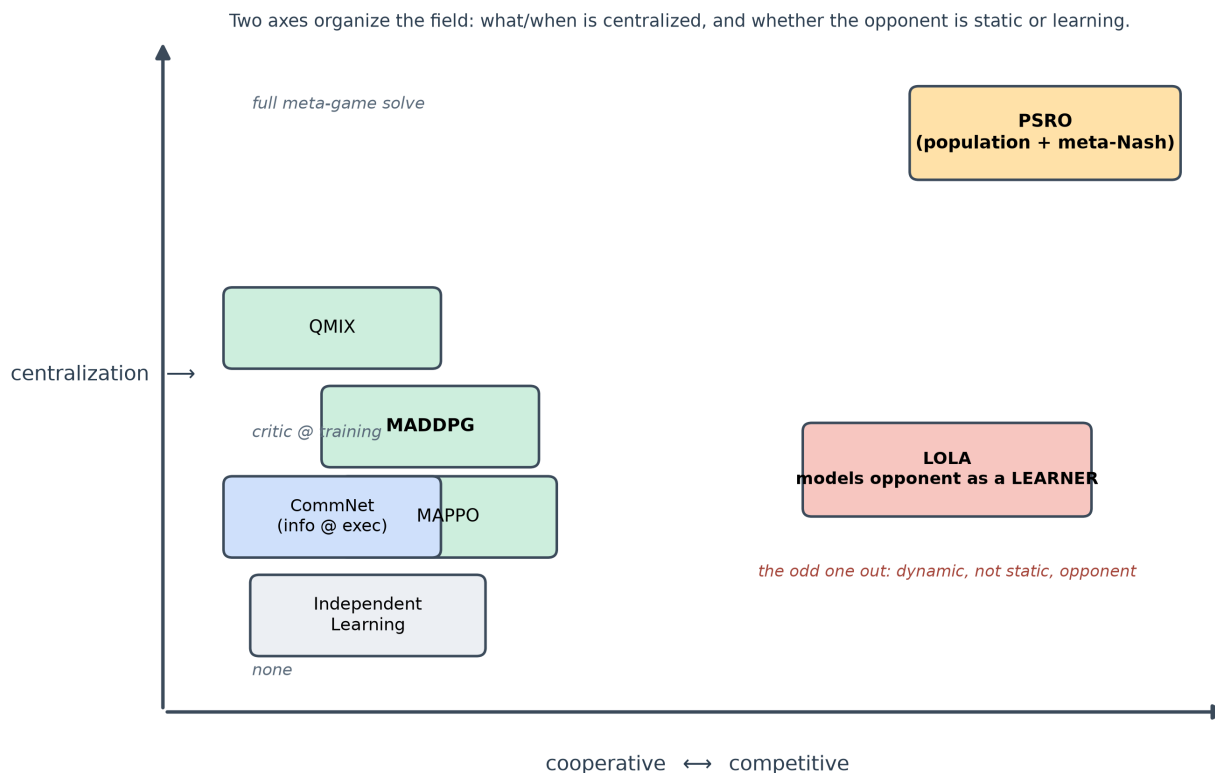
Подход	Какво прави	Кога да го използвате	Основна слабост
Самостоятелно обучение (IL)	всеки агент изпълнява свое собствено single-agent RL, третирайки останалите като среда	бърза базова линия; почти стационарни настройки	нестационарност $\rightarrow$, цикличност, координационен провал
MADDPG (CTDE)	критикът на всеки агент вижда obs+действията на всички агенти по време на обучението; всеки актьор вижда само своите obs при изпълнение	смесени кооперативно-конкурентни задачи; каноничният CTDE шаблон	входът на критика нараства с N ; актьорското обновяване е сложно за настройка
MAPPO (CTDE)	обикновен PPO с централизирана стойност $V(\text{global state})$ и споделени параметри	кооперативен MARL, когато искате проста и надеждна базова линия	цена на on-policy извадката; „прост“, но чувствителен към настройка

Подход	Какво прави	Кога да го използвате	Основна слабост
PSRO	поддържайте съвкупност от политики; решавайте мета-Наш върху нея; обучете най-добър отговор към тази смес; добавете го; повторете	конкурентни/обща игри; когато искате игровитеоретична цел на сходимост	пълен най-добър отговор на рунд; приблизителен оракул отслабва гаранцията
LOLA	оптимизирайте, приемайки, че противникът прави една стъпка на обучение ; диференцирайте <i>през</i> неговото обновяване	2-играч диференцируеми игри, където наивното обучение се проваля (IPD)	предполага, че знаете и можете да диференцирате обновяването на противника
CommNet	агентите излъчват диференцируемо съобщение ; всеки получава средното от останалите и го подава в своята политика	кооперативни задачи с частична наблюдаемост	средното обединяване изхвърля <i>кой</i> какво е казал

Две оси пресичат таблицата (фигурата по-долу). Първата е **какво е централизирано и кога?** IL не централизира нищо; CTDE централизира *оценителя/стойността само по време на обучението* (изпълнението остава децентрализирано); PSRO централизира цялостно *решаване на мета-играта* между рундовете на обучение; commNet централизира *информация по време на изпълнение* чрез научен канал. Втората е **противникът моделиран ли е като статичен или като учащ се?** Всичко освен LOLA третира стратегията на противника като фиксирана, докато ти отговаряш; **LOLA** предвижда *следващото обновяване* на противника — динамично, а не статично.

Исторически развитието протича така: научи се да говориш (commNet, 2016) → централизирай оценителя (MADDPG, 2017) → внеси теория на игрите в популацията (PSRO, 2017) → факторизирай стойността (QMIX, 2018) → предвиди обучението на противника (LOLA, 2018) → и след това установи, че простият подход често е най-успешен (MAPPO, 2022).

Прочетете повече: Albrecht, S. V., Christianos, F. & Schäfer, L. (2024). *Многоагентно обучение с подкрепление: Основи и съвременни подходи* (MIT Press), гл. 8–9 — актуално учебникарско изложение точно на тази таксономия.



Фигура 2: The method family on two axes. Horizontal: how competitive vs cooperative the target setting is (IL and PSRO span competition; MADDPG/MAPPO/QMIX/CommNet target cooperation; LOLA is the mixed-motive bridge). Vertical: how much is centralized and when (nothing for IL; the critic at training for CTDE; a full meta-game solve for PSRO; information at execution for CommNet). LOLA is the outlier that models the opponent as a learner rather than a fixed strategy.

9.4 4. Самостоятелно обучение и неговите режими на провал

Най-ясният начин да се разбере *защо* останалата част от областта изобщо съществува, е чрез изпълнението на контролен експеримент, който води до провал. При самостоятелното обучение всеки агент се поставя в собствен градиентен цикъл и се приема, че останалите агенти са част от средата. Върху четири канонични 2×2 матрични игри — всяка от които представлява различен качествен случай — това води до четири различни изхода.

Игрите и техните аналитични равновесия на Наш (обективна истина): **дилемата на затворника** (доминираща стратегия: дефектът строго превъзхожда сътрудничеството, така че уникалното равновесие е взаимен дефект); **игра „хвърляне на монета“** (игра с нулева сума; уникалното равновесие е напълно смесената $(\frac{1}{2}, \frac{1}{2})$ със стойност 0); **Stag Hunt** (две чисти равновесия на Наш — доминиращото по печалба (Stag, Stag) и рисково доминиращото (Hare, Hare) — плюс едно смесено); **битка на половете** (две чисти равновесия на Наш, които не са съгласни кое да изберат, плюс едно смесено).

Изпълнението на двама независими градиентни учещи се (с точни градиенти, така че динамиката е чиста и безшумна) дава, измерено:

Игра	Измерено равновесие	NashConv	Съвпада ли с аналитичното равновесие на Наш?
дилемата на затворника	$x \rightarrow [0.001, 0.999]$ (дефект)	0.0013	да — равновесието при доминиращи стратегии
лов на елен	всички семена $\rightarrow$ (Hare, Hare)	0.0013	да — <i>рисково доминиращото</i> чисто равновесие на Наш
битка на половете	всички семена $\rightarrow$ едно чисто равновесие на Наш	0.0013	да — чисто равновесие на Наш
игра „хвърляне на монета“	не сходява (последната итерация се отклонява към границата)	1.4–1.8	не — никога не се установява

Три от четирите игри сходяват към истинско равновесие на Наш, а интересните детайли са в „как“. В **лов на елен** обучаемите надеждно избират **рисково доминиращото** равновесие (Hare), вместо доминиращото по печалба (Stag), въпреки че Stag носи повече печалба — едностранен ход към Stag се наказва, така че градиентната динамика се плъзга към безопасния ъгъл. В **битка на половете** всяко начално число води до *едно и също* чисто равновесие; обучаемите решават проблема на координатията, но не по начин, разнообразен според началните числа. Това са проблемите с избора на равновесие и координатията, представени в конкретен вид.

Играта „хвърляне на монета“ е показателна: тя **никога не сходи към равновесие**, което е именно същината

на проблема.

Съгласуване (запази прогнозата → за това какво всъщност се случи). Прогнозирах, че Matching Pennies ще проследи чиста орбита с приблизително постоянен радиус около $(\frac{1}{2}, \frac{1}{2})$. Двама обучаеми, пуснати по два начина, и двамата не успяха да **сходяват** — но нито един от тях не обиколи чисто. Обучаемият с проектиран градиент (IGA), използван в скриптовите за изследване, *спираловидно се отдалечава*: измереното разстояние до нашово равновесие нарасна през времевите прозорци от 0.30 до 0.48. Обучаемият със softmax-logit, използван в имплементацията, *се отклонява към ъглите* (последни профили като $x = [0.96, 0.04]$, $y = [0.03, 0.97]$, NashConv ≈ 1.8). **Изводът остава непроменен и дори е по-ясен:** при наивно едновременно учене *последната итерация* не сходи към равновесие в игра само със смесено равновесие — това, което сходи, е *усредненото време* (§6). Конкретният мисловен модел за „енергозапазваща орбита“ беше грешната част; механизмът е бавно **разминаване** към границата, което, ако нещо, е още по-силен аргумент за среднопретеглящия механизъм, който следва.

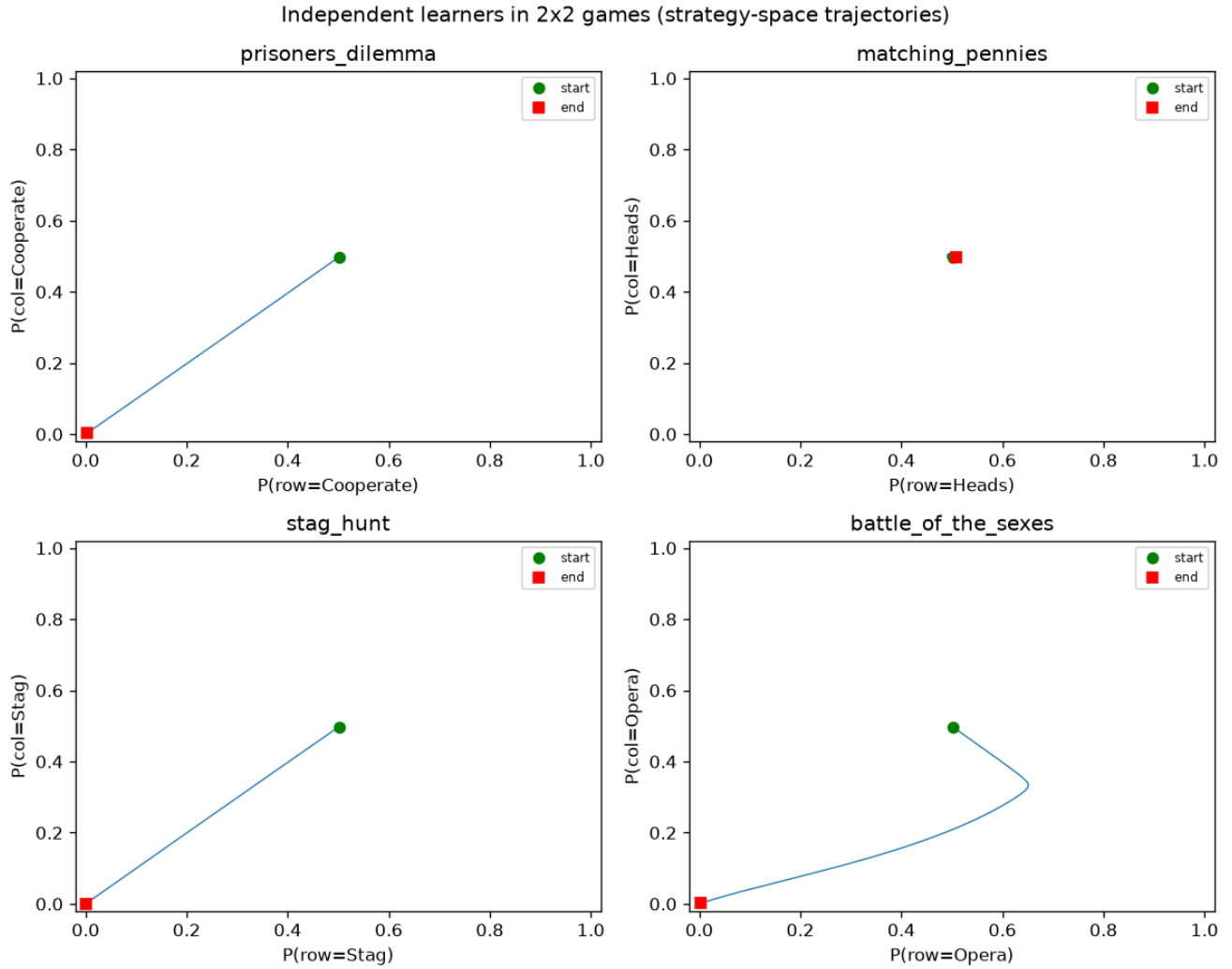
Прочетете още: Singh, S., Kearns, M. & Mansour, Y. (2000). “Nash Convergence of Gradient Dynamics in General-Sum Games.” *UAI* — анализът защо изкачването по градиент циклира, вместо да се сходи в игри като игра „хвърляне на монета“.

9.5 5. Централизирано обучение, децентрализирано изпълнение (CTDE)

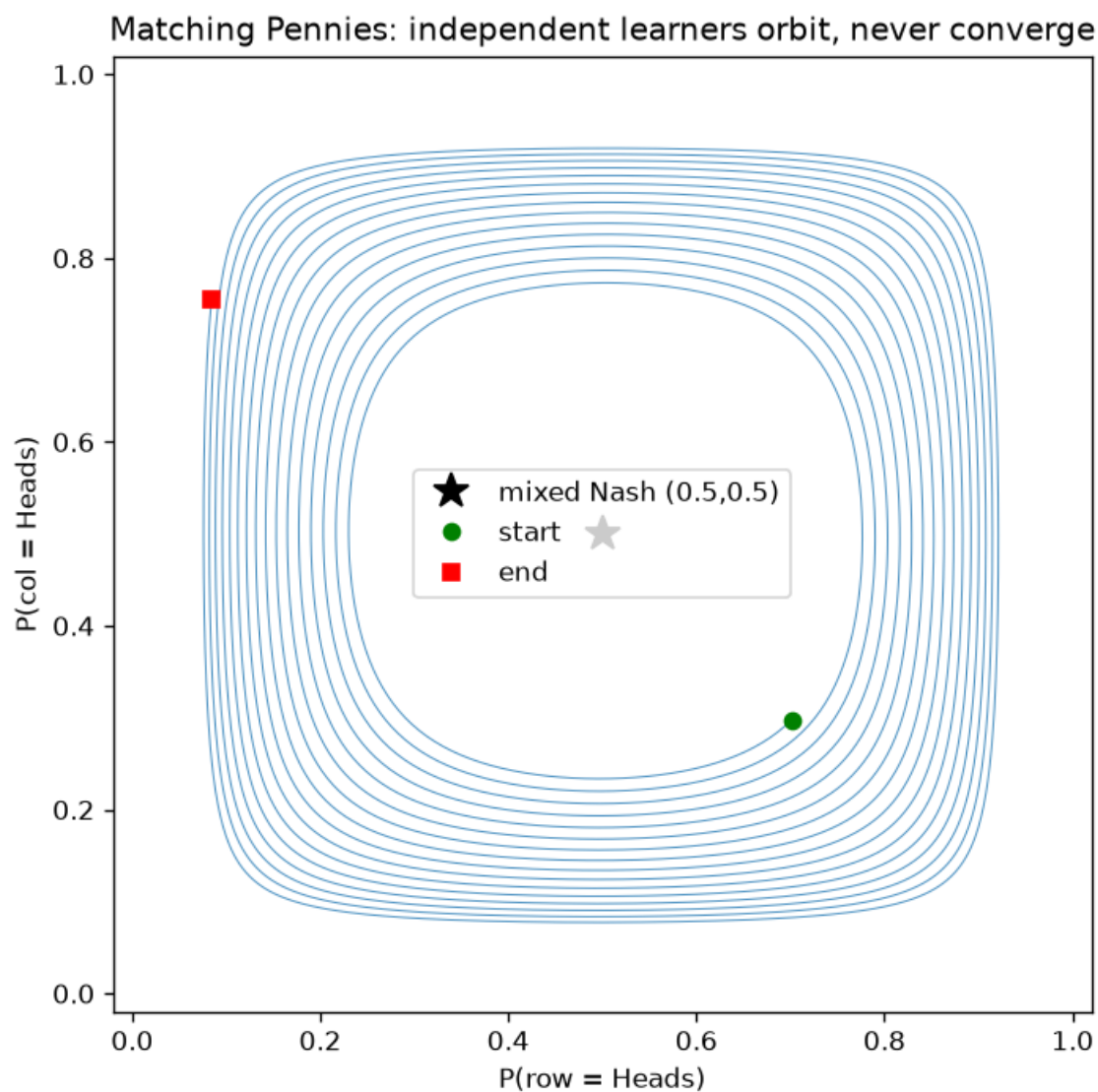
Първата структурна корекция запазва изпълнението реалистично — всеки агент все още действа въз основа на собственото си наблюдение — като същевременно предоставя на *обучаващия* привилегирован изглед по време на обучение. Архетипът е **MADDPG** (Lowe и др., 2017): актьорът $\pi_i(a_i \mid o_i)$ на всеки агент вижда само собственото си наблюдение, но централизираният оценител $Q_i(s, a_1, \dots, a_N)$ вижда глобалното състояние и действието на *всеки* агент. Асиметрията е същността на подхода. Оценителят за всеки агент, който наблюдава само o_i , се сблъсква с нестационарен, частично наблюдаем свят, така че идентични входи водят до различни възвръщаемости и неговата целева стойност е шумна; централизираният оценител, обусловен върху всичко, вижда целева стойност, която е (почти) детерминистична, така че той е значително по-нисък дисперсионен учител. **MARPO** (Yu и др., 2022) е минималната версия: обикновен PPO с една централизирана функция на стойността $V(\text{global state})$ вместо на агент $V(o_i)$, и често е най-силната базова линия — предупреждение срещу прекалено усложняване.

Има ли централизираният оценител действително по-ниска дисперсия? Измерено върху едностъпкова кооперативна „референтна“ задача (говорителят вижда целта, слушателят не, и двамата трябва да я назоват), обучено до сходимост:

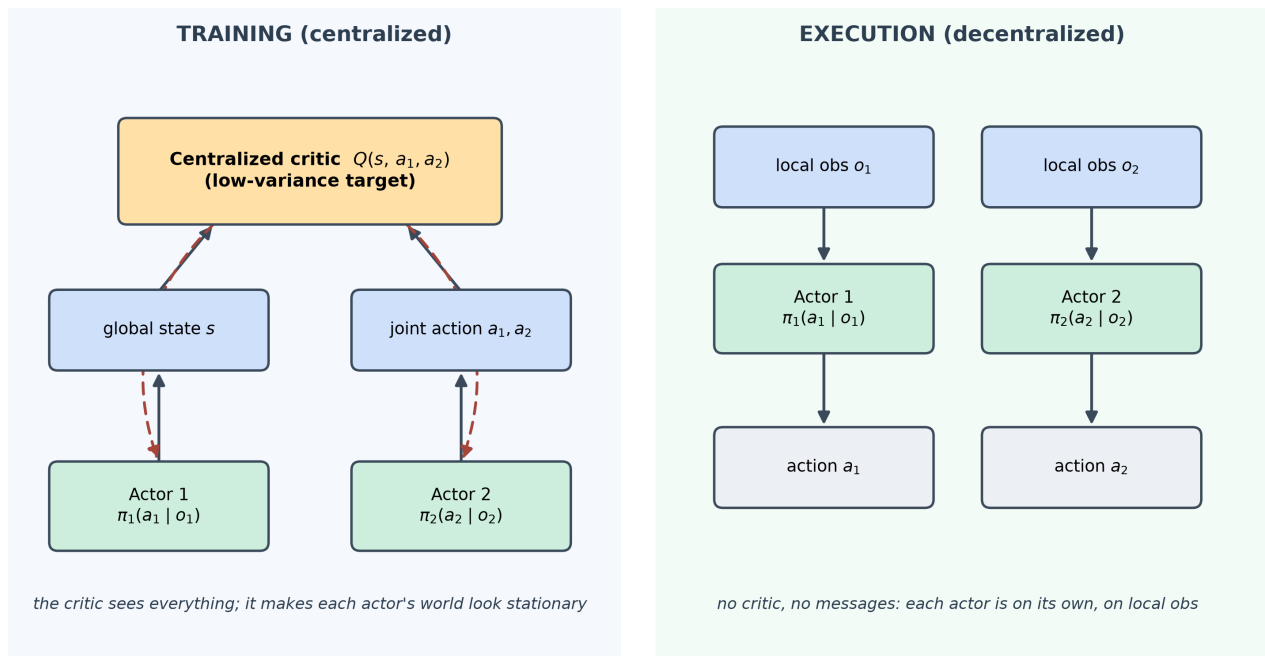
Оценител	Краен остатък (загуба на стойност)
централизиран $Q(s, \text{joint } a)$	3.2×10^{-11}
независим $Q_i(o_i)$	0.077



Фигура 3: Independent learners on the four matrix games: Prisoner's Dilemma collapses to mutual defection, Stag Hunt and Battle of the Sexes settle on a pure equilibrium, and Matching Pennies fails to converge (its trajectory drifts away from the mixed Nash rather than settling on it). The contrast is the visceral case for coordination machinery.



Фигура 4: Zoom on non-stationarity: the distance-to-Nash for Matching Pennies does not shrink (it grows across time-windows, 0.30 to 0.48), while the Prisoner's Dilemma distance-to-(Defect,Defect) collapses to zero. More compute traces a bigger divergence, not convergence — non-stationarity is structural, not a budget problem.



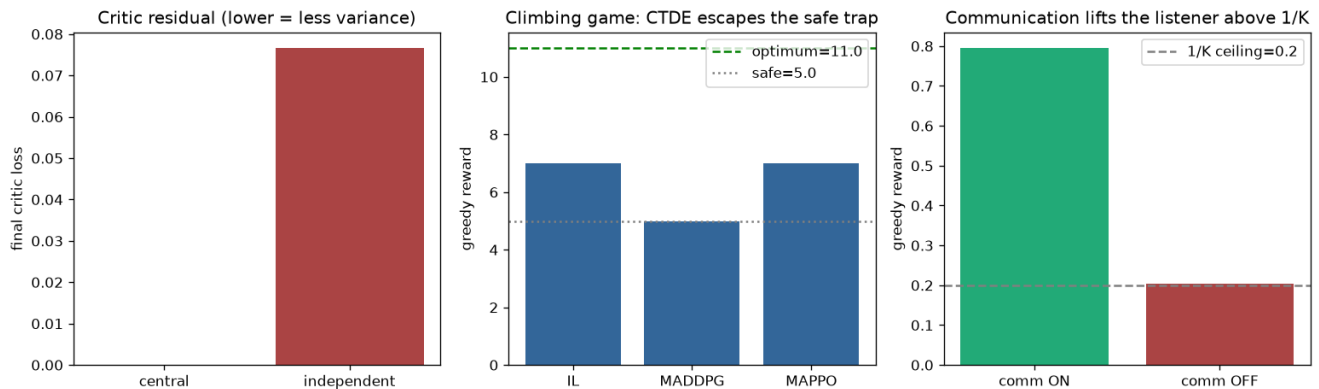
Фигура 5: CTDE architecture. During training a centralized critic (or value function) sees the global state and the joint action and produces low-variance value targets; during execution each actor acts on its own local observation with no access to the critic and no message passing. The training/execution asymmetry is what makes the world look stationary to the learner without cheating at showtime.

Централизираният оценител свежда остатъка си практически до нула — той вижда целта и съвместното действие, така че наградата е детерминистична функция на неговите входове — докато независимият оценител не може да види целта и остава блокиран в прогнозирането на базовата честота. Това потвърждава твърдението за намаляване на дисперсията при CTDE по ясен начин.

Но намаляването на дисперсията не е същото като решаването на координацията и именно тук една прогноза се провали.

Съгласуване (запази прогноза → какво всъщност се случи). Прогнозирах, че в **изкачващата се игра** на Claus–Boutilier — кооперативна матрична игра без състояние, чийто оптимум (11) е фланкиран от -30 санкции за некоординация, с „безопасен“ атрактор при 5 — централизираният оценител ще избяга от капана и ще достигне оптимума, побеждавайки независимите обучаващи агенти. Това не се случи. Измерени алчни награди: **независими обучаващи агенти 7, MADDPG 5, MAPPO 7** (оптимум 11, безопасен 5). Нито един метод не достигна оптимума, а MADDPG всъщност *изостана* както спрямо IL, така и спрямо MAPPO. Честният прочит: централизираният оценител намалява дисперсията на целевата стойност (потвърдено по-горе), но това **не е достатъчно**, за да преодолее относителната свръхобобщеност плюс риска от прекомерно изследване на -30 санкциите — агентите няма да опитат рискованото съвместно действие достатъчно дълго, за да открият 11. Резултатът на MADDPG под нивото на IL конкретно посочва неговото дискретно контрафактично актьорско обновяване с базова линия като частта, която трябва да бъде разгледана следваща. Урокът оцелява в по-сдържан вид: **CTDE купува по-добър оценител, но не и автоматична координация.**

Методологическа бележка, която си струва да се запомни: и двата ефекта на CTDE по-горе бяха **невидими при бързата “smoke config”** — там загубите на оценителя бяха почти еднакви, а ползата от комуникацията (§7) беше нула. Те се проявиха едва след обучение до сходимост при по-голямата “scale” конфигурация. Конфигурацията на дима показва, че кодът функционира; за да се наблюдават явленията, е необходимо обучение до сходимост.

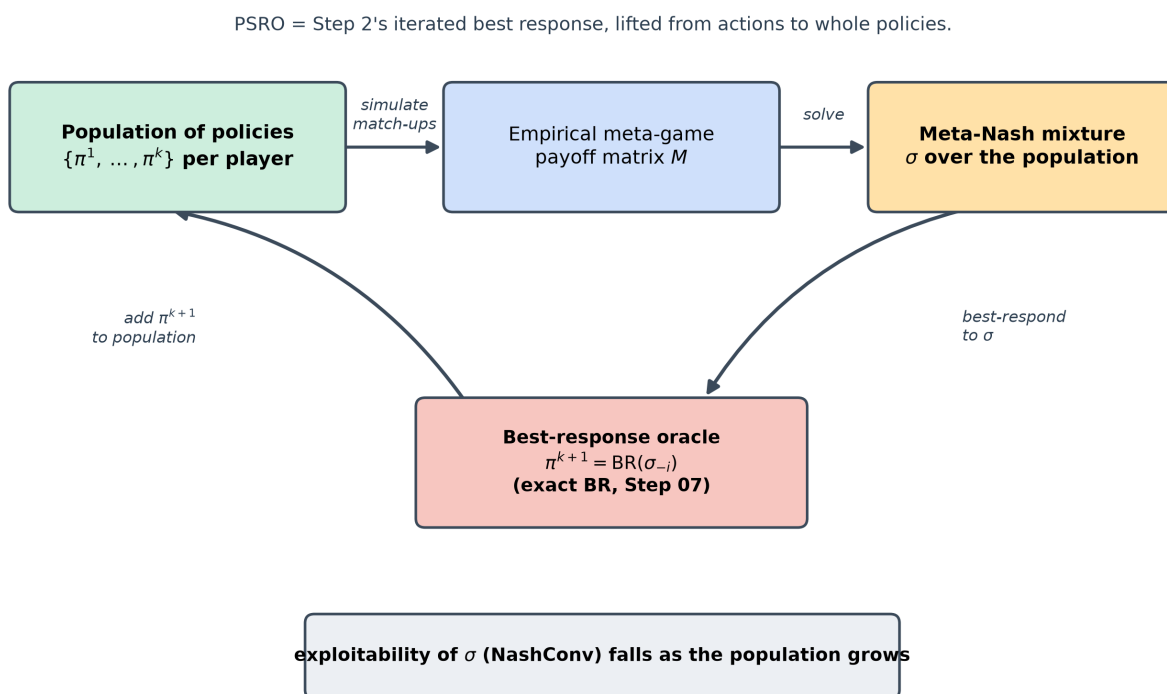


Фигура 6: Cooperative CTDE and communication results (scale config). Left: the centralized critic’s residual is orders of magnitude below the independent critic’s. Middle: on the climbing game, no method reaches the optimum (11); MADDPG (5) trails IL and MAPPO (7) — CTDE reduces critic variance but does not by itself solve hard-exploration coordination. Right: communication lifts the listener far above the $1/K$ guessing ceiling (see §7).

Прочетете повече: Lowe, R. и др. (2017). „Многоагентен актьор-критик за смесени кооперативни и състезателни среди.“ *neurIPS* (MADDPG); и Yu, C. и др. (2022). „Изненадващата ефективност на PPO в кооперативни многоагентни игри.“ *neurIPS* (MAPPO).

9.6 6. PSRO — теория на игрите върху съвкупност от политики

Втората структурна корекция е последователността на стъпката и прекият наследник на **итерирания най-добър отговор** от Стъпка 2. **PSRO** (Policy-Space Response Oracles; Lanctot и др., 2017) разглежда цели стратегии като атомите на една по-висша игра. То поддържа **популация** от стратегии за всеки играч, изгражда емпиричната **мета-игра** матрица на печалбата между популациите, решава **мета-Наш равновесие** върху нея, обучава **най-добър отговор** (оракул) към сместа мета-Наш на опонента и добавя този отговор към популацията — повтори. То обединява самообучението, фиктивната игра и метода на **двойно предсказание** в една рамка и, което е от решаващо значение за този проект, неговата метрика за прогрес е **експлоатируемост** — същата NashConv, използвана в Стъпки 2–8.



Фигура 7: The PSRO double-oracle loop. From the current populations, build the meta-game payoff matrix, solve its meta-Nash mixture, then call a best-response oracle against the opponent’s mixture and add the new policy to the population. Exploitability of the meta-Nash mixture is expected to fall as the population grows. In this project the oracle is Step 07’s exact best response, so PSRO’s convergence is measured with the same exploitability yardstick as the game-theory steps.

Два факта правят имплементацията точна, а не приблизителна. Първо, оракулът използва повторно **точния най-добър отговор** от Стъпка 07 за игрите кун и ледюк. Второ, мета-Наш сместа на опонента върху по-

веденчески стратегии е, съгласно Kuhn's Theorem (тъй като тези игри имат пълна памет), реализационно еквивалентна на **една единствена поведенческа стратегия**; свеждането на сместа по този начин позволява директното прилагане на двигателя за точен най-добър отговор.

Защо популация, а не просто последната стратегия от самообучението? Защото самообучението се сближава в *средното*, а не в последната итерация. Измерено върху **кун** при фиктивна игра със самообучение: експлоатируемостта на **средната** итерация спадна от 0.24 до 0.031 за 200 итерации, докато експлоатируемостта на **последната** итерация продължи да осцилира между 0.33 и 0.83. Доверяването на последната стратегия би означавало да се доверим на стойност, която никога не се установява; мета-сместа на PSRO е популационната версия на осредняването, върху което се основават CFR и фиктивната игра.

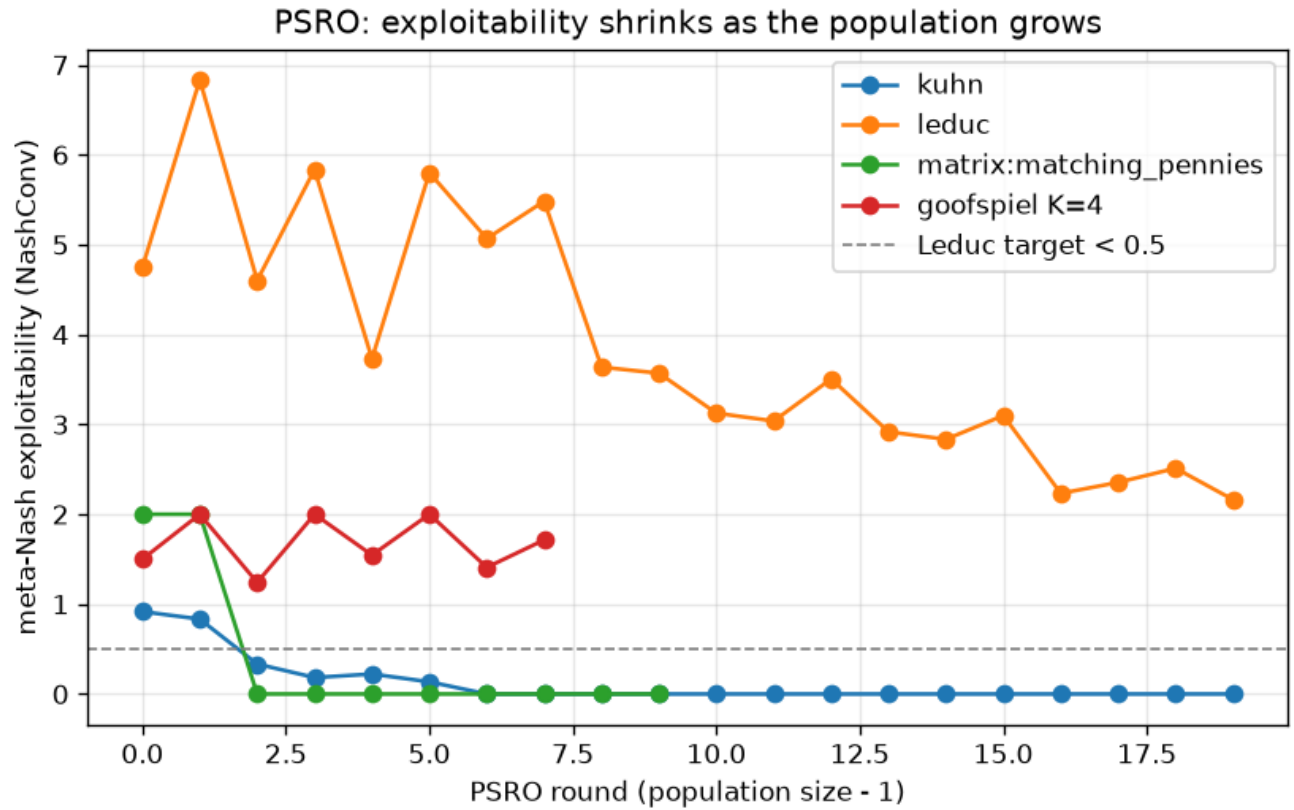
Сходимост на PSRO, измерена (експлоатируемост = NashConv на мета-Наш сместа в цялата игра):

Игра	Траектория на експлоатируемостта	Присъда
кун покер	$0.917 \rightarrow \sim 2 \times 10^{-16}$ до кръг 6	сближава се до Machine Zero
матрица (игра „хвърляне на монета”)	$2.0 \rightarrow 0$ до кръг 2	сближава се
камък-ножица-хартия (изследване)	$2.0 \rightarrow 0.017$; популация $\rightarrow \{R,P,S\}$, смес $\rightarrow$ равномерна	сближава се
Ледюк Холдем	$4.75 \rightarrow 2.16$ за 20 кръга	намалява, но е далеч над целта
Goofspiel ($K = 3$)	$1.33 \rightarrow 0$	сближава се
Goofspiel ($K = 4$)	колебае се $1.4 \leftrightarrow 2.0$	не се установява

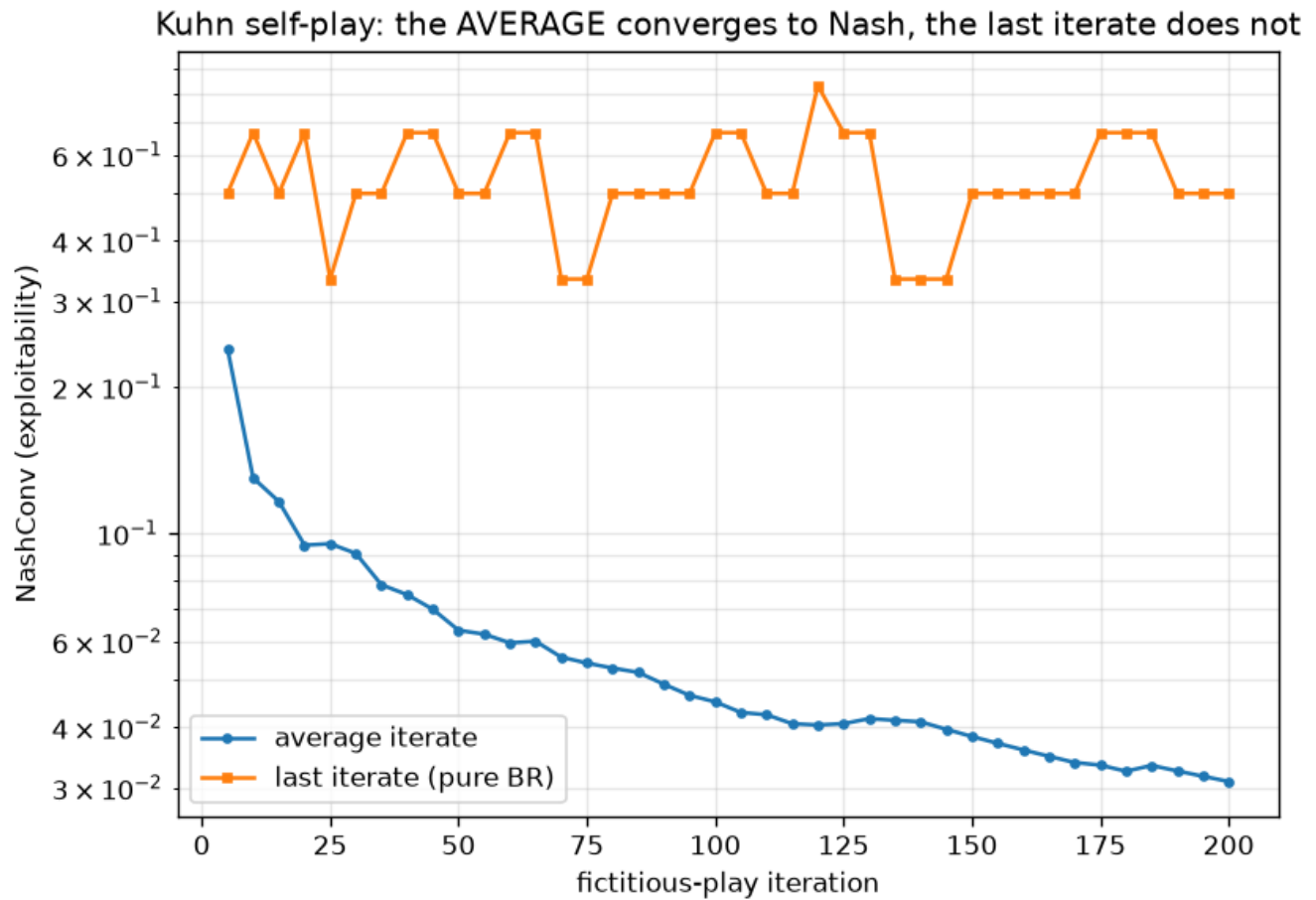
Резултатите за **кун** / матрица / RPS са учебникови: при малките игри популацията бързо обхваща необходимите стратегии и експлоатируемостта рязко спада. Два резултата не се развива според очакванията.

Съгласуване 1 (ледюк). I predicted PSRO would drive Leduc exploitability below 0.5 within 20 iterations. Measured, it fell from 4.75 to 2.16 — a clear, roughly monotone decline, but nowhere near 0.5. This is **genuine slow convergence**, not a bug: Kuhn hit machine zero in 6 rounds, but Leduc's game tree is far larger, and a population of 20 *pure* best responses is simply too small to closely approximate its mixed Nash. The “< 0.5 in 20” target was optimistic. The lesson — exploitability decreases as the population grows — holds; the *rate* is the scaling wall, and it rhymes with Step 8's global-vs-local scaling finding.

Съгласуване 2 (Goofspiel $K = 4$). I predicted non-increasing exploitability. At $K = 3$ it converged to 0 чисто; at $K = 4$ it oscillates between ~ 1.4 and ~ 2.0 and does not settle. This is the one result I cannot yet fully explain, and per the workflow I am **documenting it, not fixing it**. Two concrete suspects for a follow-up session: the Goofspiel PSRO driver never de-duplicates best-response policies (so the meta-game can stall on repeats), and a pure-strategy population is likely too weak to represent the larger game's mixed meta-Nash. Flagged as an open code item, not a validated result.



Фигура 8: PSRO exploitability vs population size across games (scale config). Kuhn, the matrix game, and Rock-Paper-Scissors collapse to (near) zero within a handful of rounds; Leduc declines steadily but stays well above the 0.5 target after 20 rounds (a scaling wall for a pure-strategy population); Goofspiel K=4 oscillates rather than settling (a flagged anomaly). The exact best-response oracle guarantees convergence in principle; the rate is what size controls.



Фигура 9: Self-play on Kuhn: the average-iterate exploitability (NashConv) falls steadily toward zero while the last-iterate exploitability keeps oscillating. This is why self-play and PSRO rely on averaging over a population rather than trusting the most recent policy.

Прочетете повече: Lanctot, M. и др. (2017). „Унифициран игровитеоретичен подход към обучението с подкрепление на множество агенти.“ *neurIPS* (PSRO); и McMahan, H. B., Gordon, G. & Blum, A. (2003). „Планиране в присъствието на функции на разходите, контролирани от противник.“ *ICML* (методът на двойно предсказание, който PSRO обобщава).

9.7 7. Научена комуникация (commNet)

CTDE централизира информацията по време на *обучение*; комуникацията я централизира по време на *изпълнение* чрез канал, който агентите **научават**, а не такъв, който дизайнерът изгражда. В **commNet** (Sukhbaatar и др., 2016) всеки агент кодира своето наблюдение в скрит вектор, излъчва **съобщение**, и след това получава **средната стойност** на съобщенията на останалите агенти като допълнителен вход за своята стратегия. Цялото се обучава от край до край, така че протоколът възниква: в референтната задача говорещият (който единствен вижда целта) трябва да се научи да я кодира, а слушателят – да я декодира.

Тестът е показателен, тъй като без канал слушателят е ограничен единствено до чисто отгатване, $1/K$. Измерено (конфигурация на мащаба, $K = 5$, при което прагът на отгатваемост е 0.2):

Канал	Награда за отборна алчност
комуникация върху	0.795
изключена комуникация	0.204

С канала слушателят се издига далеч над тавана от $1/K$; без него той остава точно на тавана. Комуникацията върши реална работа — и обърнете внимание, че съгласуването от §5 важи и тук: при конфигурацията с дим и двете числа бяха 0.24 (каналът все още не се беше научил да пренася информация), а ползата се появи едва след обучение в мащаб.

Прочетете повече: Sukhbaatar, S., Szlam, A. & Fergus, R. (2016). “Обучение на многоагентна комуникация с обратно разпространение на грешката.” *neurIPS* (commNet).

9.8 8. LOLA — моделиране на опонента като учащ се

Всеки метод досега разглежда стратегията на противника като фиксирана, докато агентът реагира. **LOLA** (обучение с осъзнаване за ученето на противника; Foerster и др., 2018) е изключението и е този, който е най-пряко свързан с дисертацията. Вместо да се оптимизира спрямо *настоящата* стратегия на противника, всеки агент се оптимизира спрямо стратегията, която противникът ще има *след една стъпка на обучение*, като диференцира *през* тази стъпка. Допълнителният член представлява смесена втора производна — как актуализацията на стратегията на противника зависи от *моите* параметри — и именно той превръща егоистичните агенти в кооперативни.

Класическата демонстрация е **Iterated Prisoner's Dilemma** с памет-1, при която стратегията на всеки агент се състои от пет вероятности за сътрудничество, а очакваната дисконтирана възвръщаемост има затворена форма чрез стационарната марковска верига върху двойките резултати. Наивни градиентни обучаеми, всеки максимизиращ собствената си възвръщаемост спрямо текущата стратегия на другия, сходява към взаимно предателство; **IOLA learners**, всеки отчитащ предстоящата актуализация на другия, достига взаимно сътрудничество.

Измерена (дисконтирана възвръщаемост на стъпка; пълно сътрудничество ≈ 3 , взаимно предателство $\square \text{MATHPI} \square$):

Обучаващи се	Възвръщаемост
наивни срещу наивни	1.04
LOLA срещу LOLA	2.82

Посоката е точно резултатът на LOLA — сътрудничеството възниква там, където наивното обучение води до дефектиране. (Моята прогноза за ≈ 3 беше малко завишена; измерената 2.82 е близка до пълно сътрудничество, а изследователското изпълнение с по-голяма **скорост на обучение при предвиждане** достигна ~ 2.9 .) Вградена **проверка за коректност** потвърждава механизма: задаването на **скоростта на обучение при предвиждане** на нула кара **градиента** на LOLA да се редуцира точно до **наивния градиент**, така че сътрудничеството произтича конкретно от втория ред на члена за **предвиждане**.

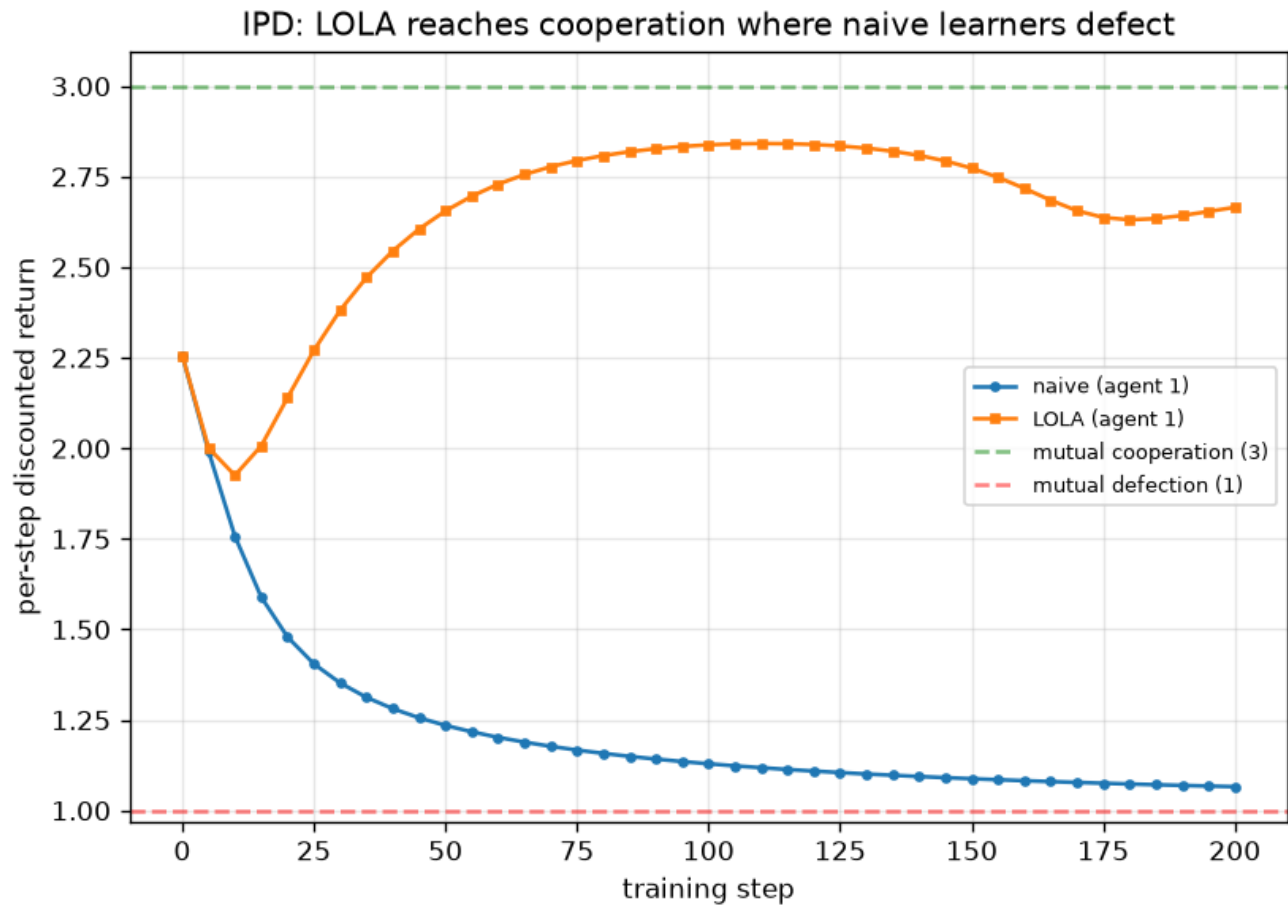
Концептуално това представлява **динамично** моделиране на противника: Стъпка 7 извежда *текущата* стратегия на противника; LOLA предвижда тяхната *обучителна траектория*. Комбинирането на двете — статично четене, което задейства динамично предвиждане — е кандидат за Принос №1, а не нещо вече решено.

Прочетете повече: Foerster, J. и др. (2018). “Обучение с осъзнаване на обучението на противника.” *AAMAS* (LOLA).

9.9 9. Честни бележки, ограничения и къде това раздаване продължава нататък

Какво се задържа. Потвърдените резултати са гръбнакът: самостоятелното обучение се сближава при доминиращи/координационни игри и *се проваля* при игра „хвърляне на монета“; PSRO свежда експлоатируемостта до (почти) нула при Кун, матричната игра и RPS с точен оракул; самостоятелното обучение се сближава в средното, но не и в последната итерация; централизиращият оценител има дисперсия на остатъка с порядъци по-ниска; научената комуникация издига слушателя над прага на отгатваемост; и LOLA превръща дефекторите в IPD сътрудници. Взети заедно, те проследяват предвидената дъга — от „защо наивното обучение се разпада“ до „структурните поправки, които го възстановяват“.

Какво не успя и защо това е от значение. Четири честни уговорки се пренасят напред. (1) Игра „хвърляне на монета“ се отклонява към границата, вместо да обикаля — по-остра версия на урока за несходимост. (2) PSRO при Leduc се сближава *бавно* (експлоатируемост ~ 2.16 след 20 рунда, а не < 0.5) — стена на



Фигура 10: LOLA vs naive learners on the Iterated Prisoner's Dilemma: naive learners' per-step return collapses toward mutual defection (~ 1), while LOLA learners' return climbs toward mutual cooperation (~ 2.8). Anticipating the opponent's next learning step is what reshapes the dynamics from defection to cooperation.

мащабиране на популацията с чисти стратегии. (3) Goofspiel $K = 4$ се колебае, вместо да се сближава — маркирана, необяснена аномалия в кода (документирана, но не поправена). (4) При играта „изкачване“ нито един метод не достигна оптимума и MADDPG се представи по-зле от независимите обучаващи агенти — централизираният оценител намалява дисперсията, но сам по себе си не решава координация при риск от прекомерно изследване. Двата елемента на кода (Goofspiel $K = 4$; контрафактична базова линия на MADDPG) трябва да бъдат разгледани, преди тези части да бъдат използвани отново. И един методологичен момент: ефектите на невронните мрежи бяха невидими при бързата конфигурация на дима и се появиха едва в мащаб, така че именно числата за мащаб са тези, върху които почиват твърденията.

Доверие. Всяка цел на равновесие е *точна* (аналитично равновесие на Наш за матричните игри; точен най-добър отговор и NashConv от Стъпка 07 за кун/ледюк/Goofspiel), така че игровитеоретичните резултати се ограничават от обективната истина, а не от други симулации. Невронните резултати представляват качествени неравенства (централен < независим; комуникация ВКЛЮЧЕНА > комуникация ИЗКЛЮЧЕНА), а не точни стойности, и по конструкция са чувствителни към началното число и версията. PNG изображенията на експериментите, цитирани по-горе, са генерирани от ангажираните JSON artifacts (виж `../figures/README.md`); няколко от тях не бяха запазени при изпълнението и са маркирани като „за затваряне“.

Обратни и бъдещи връзки. Обратно: PSRO представлява итерираният най-добър отговор от Стъпка 2, издигнат до ниво популация, и използва изцяло двигателя за точен най-добър отговор от Стъпка 07; стената на мащабиране при ледюк отеква откритието глобално срещу локално от Стъпка 8. Напред: трите тези вече са конкретизирани — LOLA като динамично моделиране на противника (Принос №1), мета-играта на PSRO като методология за оценяване (Принос №3) и, преди всичко, **липсващата $N > 2$ минимакс котва** (Принос №2), единственото място, където всяка двуйгрова гаранция от Стъпки 2–8 престава да бъде валидна.

Отворени въпроси. Може ли приблизителен (RL) оракул да доближи PSRO при ледюк значително повече до наш равновесие и с каква цена за гаранцията за сходимост? Какво премахва колебанието в Goofspiel $K = 4$ — дедупликация, популация от смесени стратегии или различен мета-решател? Защо централизираният оценител *вреди* при изкачващата се игра и дали контрафактична/COMA базова линия го коригира? И основният въпрос на дисертацията, който стои под всички тях: без минимаксна стойност за $N > 2$, какво означава „безопасно“ дори за популация и може ли мета-играта на PSRO да осигури приложима заместителка?

9.10 Основни изводи за синтеза на дисертацията

- **Нестационарността е основният проблем, и тя е структурна, а не въпрос на изчислителна мощност** — играта „хвърляне на монета“ се разминава, независимо колко дълго тренирате.
- **CTDE централизира оценителя по време на обучението и децентрализира актьора при изпълнението**; измерено е, че осигурява оценител с почти нулева дисперсия (3.2×10^{-11} спрямо 0.077) — но само намаляването на дисперсията не реши изкачващата се игра, така че то е необходимо, но не и

достатъчно.

- **PSRO е мостът между теорията на игрите $\leftrightarrow$ MARL:** измерено е, че довежда кун до експлоатируемост, равна на машинната нула, а RPS — до равномерност; ледюк намалява, но достига стена на машабиране — същата стена, която мотивира скалируемите методи в дисертацията.
- **Самообучението/PSRO успяват в средното/популацията, а не в последната итерация** (средната NashConv на кун $0.24 \rightarrow 0.031$, докато последната итерация се колебае) — причината да съществува популация.
- **LOLA преформулира противника като обучаем** и превръща дефекцията в IPD в кооперация ($1.04 \rightarrow 2.82$) — динамичният комплемент към статичния модел на противника от Стъпка 7 (Принос №1).
- **Минимаксната разлика $N > 2$ е отворената врата към дисертацията** (Принос №2): речникът на стъпки 2–8 преминава в MARL, но предпазният ограничител не.

Стъпка 10 Обобщение — Обучение на базата на популации и Еволюционна теория на игрите

Изследване върху възможностите за прилагане на изкуствен интелект в компютърните игри

Alexander Andreev

July 2026

Съдържание

10 Стъпка 10 — Обучение, базирано на популации и еволюционна теория на игрите	1
10.1 1. Защо популации — и защо структурата определя всичко	2
10.2 2. Семейството от методи	2
10.3 3. Репликаторна динамика — равновесия на популацията	3
10.4 4. Въртящият се пумпал — транзитивна спрямо циклична структура	5
10.5 5. Лигата AlphaStar-style PBT	7
10.6 6. Разнообразие — наистина ли популацията е разнообразна?	9
10.7 7. EGTA — оценка на популацията и неочаквано откритие	12
10.8 8. Честни бележки, ограничения и предаване на щафетата	13
10.9 Основни изводи за синтеза на дисертацията	15

10 Стъпка 10 — Обучение, базирано на популации и еволюционна теория на игрите

Това е цялостна глава за обучение и оценка на **популации** от агенти: еволюционната математика, която определя дали дадена популация изобщо може да се установи в равновесие, транзитивната/циклична структура, която решава дали процесът на самообучение ще се сближи или ще се завърти, и малка **лига** в стил alphaStar, изградена върху решима покер игра. Тя има две цели — да послужи като самостоятелно обобщение за следващите стъпки и като основен източник на приносите на тезата на популационно ниво. Написана е така, че да може да се чете независимо от останалите глави. **Всички експериментални резултати, представени тук, са получени** от възпроизводими изпълнения и, когато е възможно, са ограничени от *точни* референции (аналитично равновесие на Еволюционно стабилни стратегии/Наш за матричните игри; точната експлоатируемост по най-добър отговор от Стъпка 07 за ледюк). Когато дадено изпълнение противоречи на очакванията, произтичащи от теорията, запазвам първоначалното теоретично предвиждане и го съпоставям с получените резултати — именно тези разминавания са най-показателните части от стъпката.

Къде се вписва това в тезата. Стъпки 2–8 решават *една* фиксирана игра; Стъпка 9 добавя *втори* обучаем. Стъпка 10 преминава към *цяла популация*, която се обучава срещу себе си. Тя носи три основни приноса на тезата. **Основни експлоатори** са автоматизирани моделиращи съперника — повдигането на популационно

ниво на статичния прочит от Стъпка 7 (Принос №1). **AlphaStar League** е механизъм за безопасно използване на популационно ниво, но чиято гаранция *липсва* — точното място, където се съдържа Принос №2. А **EGTA / мета-Наш** е методологията за оценяване на популацията (Принос №3).

10.1 1. Защо популации — и защо структурата определя всичко

Самообучението има известен **режим на отказ**: играч, който тренира единствено срещу самия себе си, може да се върти в кръг. Ако играта е **камък-ножица-хартия**, „да станеш по-добър“ няма смисъл — камък побеждава ножица, губи от хартия и подобряването срещу един противник просто те прави по-лош срещу друг. Най-важната идея в тази стъпка е, че игрите имат два вида структура, смесени заедно: **транзитивна** част (истинска **скала на уменията** — съществува *по-добър играч*) и **циклична** част (**колело от броячи** — има само *какво побеждава какво*). Дали една **самообучаваща се популация** ще се сближи до нещо добро или ще се върти вечно, зависи от това *коя част е доминираща*, а не от **скоростта на обучение**.

Една представа, която да запомните: **доджо**. **Основните** ученици са това, което се опитвате да създадете. Поддържате зала, пълна с **спаринг партньори**, чиято единствена задача е да откриват и наказват конкретна слабост в настоящите ученици (експлоататори). И поддържате **музей на минали шампиони** — замразени „снимки“ — така че никой да не печели, като забрави как да победи стар стил. Обучението тогава представлява цикъл: учениците спарингуват, силните се копират и леко мутират (**обучение, базирано на популации**), а периодично копие на всички се замразява в музея. Лигата е това доджо, превърнато в математическа форма.

Прочетете повече: Balduzzi, D. и др. (2019). „Open-ended Learning in Symmetric Zero-sum Games.“ *ICML* — геометрията на пумпала на транзитивната спрямо цикличната структура; и Jaderberg, M. и др. (2017). „Population Based Training of Neural Networks.“ *arXiv:1711.09846*.

10.2 2. Семейството от методи

Всеки от разгледаните методи представлява различен начин за превръщане на **популацията** в тренировъчен сигнал или средство за оценка.

Подход	Какво прави	Кога да се използва	Основна слабост
Самообучение	обучава последната политика срещу (копие на) самата себе си	транзитивни игри; бърза базова линия	цикли / забравя в циклични игри; последната итерация не се сближава

Подход	Какво прави	Кога да се използва	Основна слабост
PSRO	популация + мета-Наш + предсказвач за най-добър отговор; добавя най-добрия отговор, повтаря	състезателни игри; търси се игровитеоретична цел	пълен най-добър отговор на рунд; <i>смесването</i> не е решението (виж §7)
PBT	обучава много агенти; копира по-приспособените (експлоатация) + променя хиперпараметри (изследване)	всяка популация; онлайн търсене на хиперпараметри	може да намали разнообразието или да регресира (виж §5)
AlphaStar League	PBT с три типа агенти (основен / експлоататор на основния / експлоататор на лигата) + замразяване + PFSP	нетранзитивни игри, където наивното самообучение циклира	много агенти, голямо изчислително усилие; евристика, без гаранция за безопасност

Два инструмента за оценка ги пресичат. **Репликаторната динамика** (§3) е непрекъснатата идеализация на подбора — тя показва къде една популация *тече* и къде може да *почине*. **Декомпозицията „пумпал“** (§4) измерва доколко една игра (или популация) е транзитивна или циклична. Заедно те представляват диагностика, която предсказва, *преди* да бъде използвана изчислителната мощност, дали лигата в §5 ще се установи или ще се завърти.

Исторически линията е следната: обучение, базирано на популации като онлайн търсене на хиперпараметри (PBT, 2017) → теория на игрите върху популация от стратегии (PSRO, 2017) → геометрията, която обяснява *защо* е необходима популация („пумпал“, 2019) → и пълната тритипна лига, която победи професионални човешки играчи (AlphaStar, 2019), оценена с емпирична теория на игрите (EGTA, Tuyls и др., 2020).

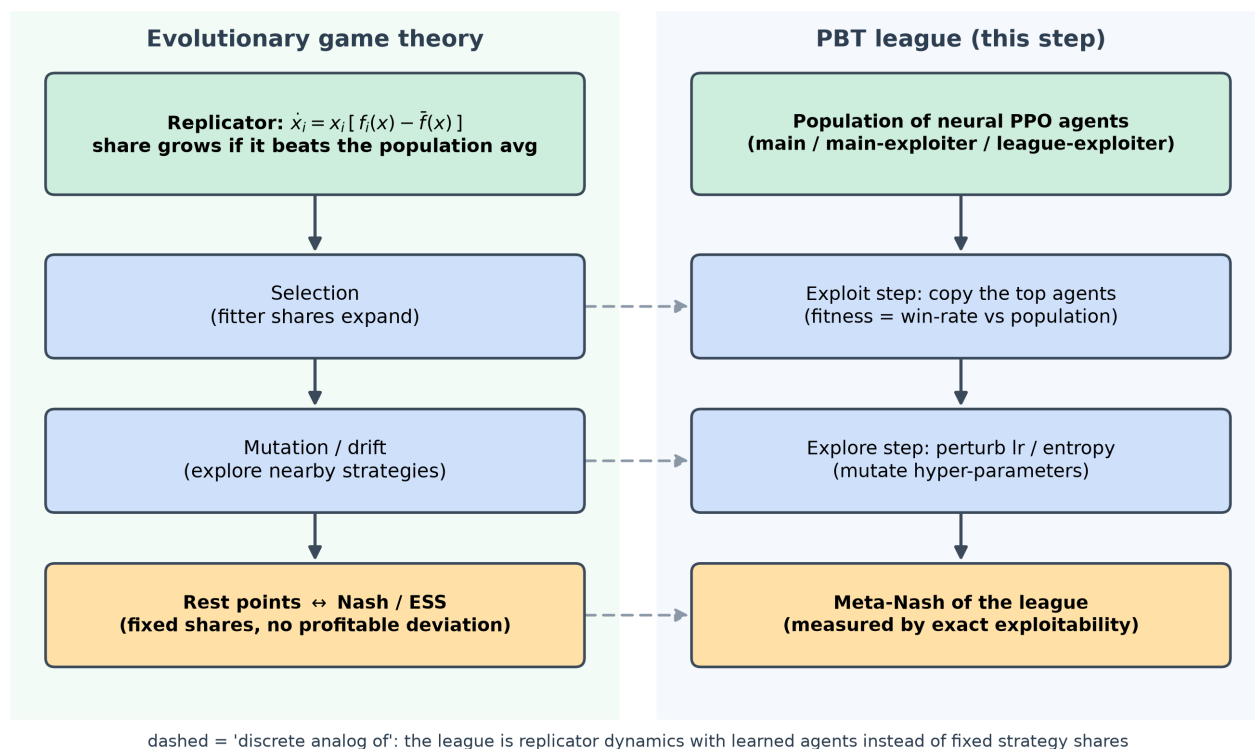
10.3 3. Репликаторна динамика — равновесия на популацията

Репликаторното уравнение е най-простият модел на подбора: делът на дадена стратегия нараства пропорционално на това колко тя превъзхожда средната за популацията.

$$\dot{x}_i = x_i [f_i(x) - \bar{f}(x)], \quad \bar{f}(x) = \sum_j x_j f_j(x).$$

Точките на покой съвпадат точно с равновесията на Наш; *стабилните* точки на покой са **еволюционно**

стабилни стратегии (ESS) — смесени стратегии, които, след като бъдат възприети от цялата популация, не могат да бъдат изместени от малък дял мутанти. Това представлява непрекъснатото идеализиране на това, което една PBT лига прави дискретно: копиране на по-добрите агенти (подбор) и тяхното смущаване (мутация).



Фигура 1: Evolutionary game theory as the continuous idealization of a PBT league: replicator selection $\leftrightarrow$ copying the fitter agents, mutation/drift $\leftrightarrow$ perturbing learning rate and entropy, and replicator rest points $\leftrightarrow$ the meta-Nash / ESS the league is trying to reach. The league is replicator dynamics with learned agents instead of fixed strategy shares.

Изпълнен върху четири канонични симетрични игри (с детерминистична динамика, поради което резултатът е недвусмислен), измерените резултати съвпадат напълно с теорията:

Игра	Аналитична референция	Измерено крайно състояние	Сходим?
Дилемата на затворника	Дефект доминира; ESS (0, 1)	[0.0, 1.0]	да
Ястреб-гълъб	вътрешно ESS $p(\text{Hawk}) = V/C = 0.5$	[0.5, 0.5] (орбитален радиус 0.0)	да
Камък-ножица-хартия	Наш равновесие = равномерно; няма ESS	орбитира около центъра с радиус 0.095	не

Игра	Аналитична референция	Измерено крайно състояние	Сходим?
Лов на елен	две чисти ESS (басейнозависими)	$[0.8, 0.2] \rightarrow [1, 0]$; $[0.2, 0.8] \rightarrow [0, 1]$	да

Дилемата на затворника се свежда до доминиращата стратегия; ястреб-гълъб *достига* вътрешната 0.5 ESS; ловът на елен избира различно чисто ESS в зависимост от басейна, в който попада началната точка. **Камък-ножица-хартия никога не се сближава** — неговата вътрешна неподвижна точка е *център*, така че популацията обикаля около нея завинаги. Този последен ред е цялата мотивация за останалата част от стъпката: цикличната игра няма стабилна популация.

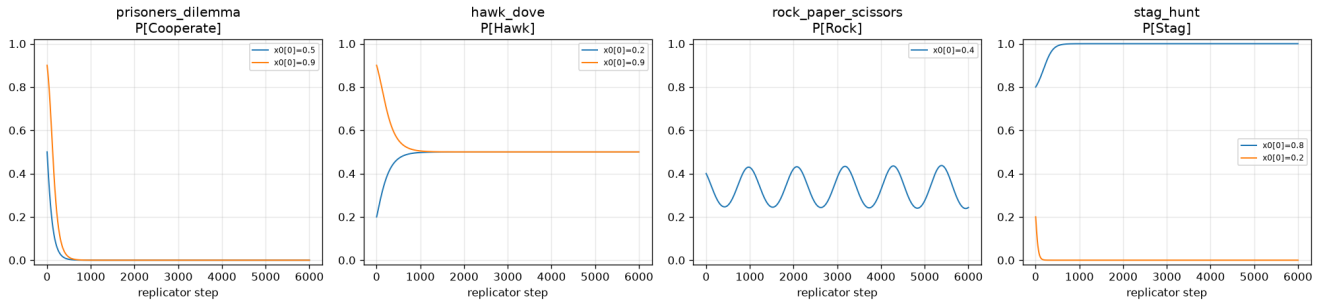
Прочетете още: Hofbauer, J. & Sigmund, K. (1998). *Evolutionary Games and Population Dynamics* (Cambridge) — репликаторна динамика, ESS и центърът на RPS; както и обзорът на Bloembergen–Tuyls (JAIR, 2015), който свързва репликаторната динамика с многоагентното обучение.

10.4 4. Въртящият се пумпал — транзитивна спрямо циклична структура

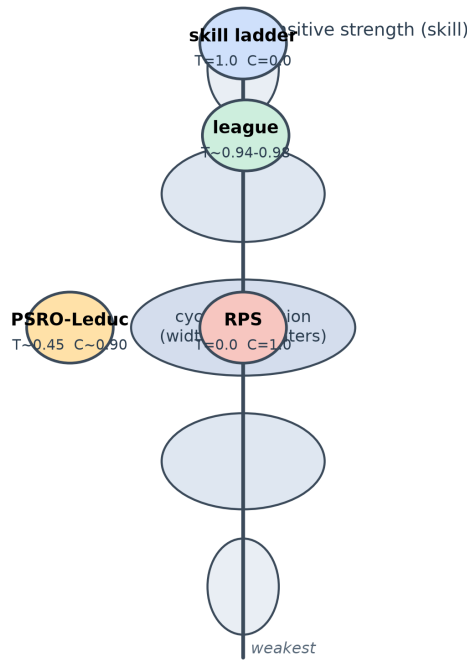
Ако цикличните игри са проблемът, необходимо е да *измерим* степента им на цикличност. **Декомпозицията на пумпала** (Balduzzi и др.) разделя матрицата на печалбата на една игра на **транзитивен** компонент (скала на уменията, изразена чрез оценки за всяка стратегия) и **цикличен** компонент (това, което остава — частта камък-ножица-хартия). Името произлиза от формата: реалните игри са най-широки (най-циклични) сред посредствените стратегии и се стесняват до точка (чисто транзитивни) в крайностите на уменията.

Една тънко̀ст, която си струва да се отбележи: първоначалната стъпка предполагаше **sVD rank-1** декомпозиция, която неправилно съобщава за ≈ 0.707 транзитивно съотношение на **камък-ножица-хартия** — едно **антисиметрично приближение** с ранг 1 просто не може да представи чист цикъл. Внедряването използва вместо това **комбинаторния-Ходж** (разлика в оценките) метод, който правилно определя транзитивното съотношение на RPS като 0.0. Измерено:

Популация	Транзитивен (Ходж)	Цикличен	Структура
камък-ножица-хартия	0.0	1.0	чисто цикличен
чиста скала на уменията	1.0	0.0	чисто транзитивен
PSRO-ледюк най-добър отговор мета-игра	0.41–0.46	0.89–0.91	предимно цикличен (27 трицикли)
Лига моментна снимка мета-игра	0.94–0.98	—	предимно транзитивен



Фигура 2: Replicator phase portraits: Prisoner's Dilemma → all-Defect, Hawk-Dove → the 0.5 interior ESS, Rock-Paper-Scissors → a closed orbit that never converges, and Stag Hunt → two basins (all-Stag or all-Hare). The non-converging RPS orbit is why populations in cyclic games need explicit machinery to avoid spinning.

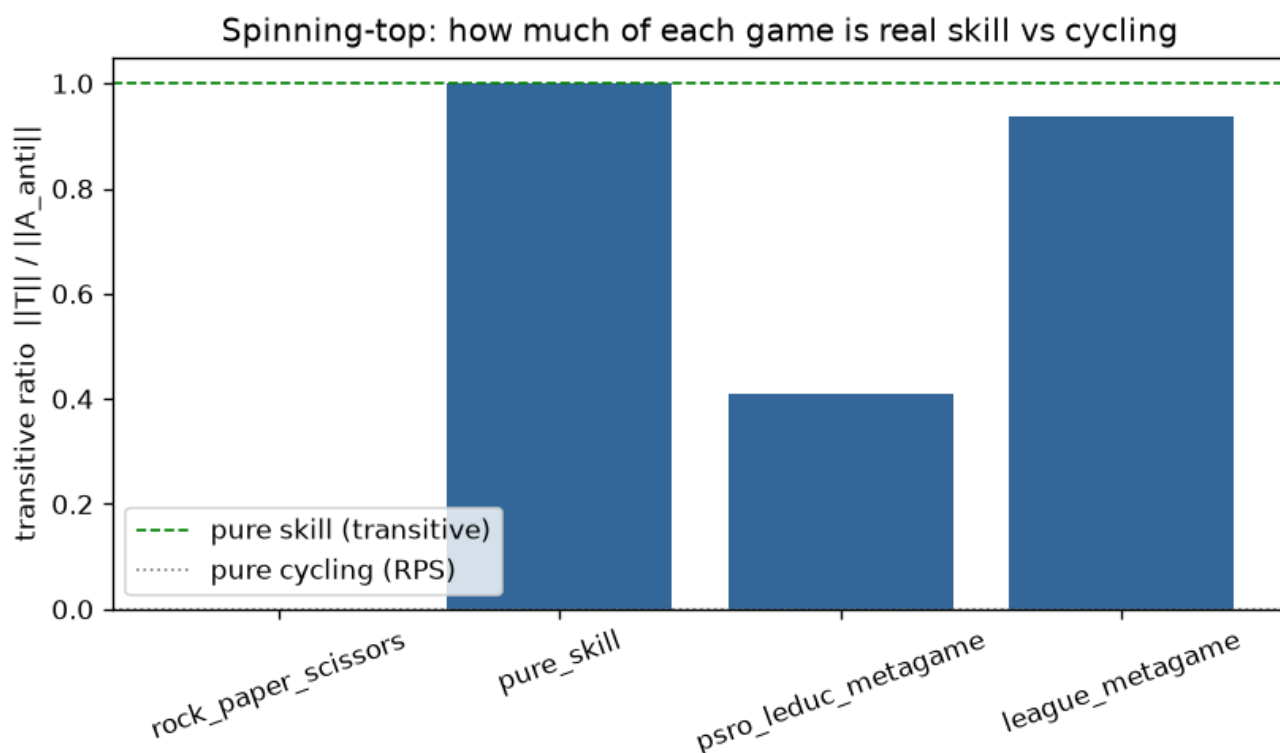


same game, different populations: a best-response population (PSRO) sits in the wide cyclic belly; a training-snapshot population (league) climbs the transitive spine.

Фигура 3: The spinning top: a vertical transitive axis (skill — there is a better player) and a cyclic dimension (width — the number of counters, rock-paper-scissors structure). Rock-Paper-Scissors sits in the widest cyclic belly (transitive ratio 0.0); a pure skill ladder sits on the transitive spine (1.0); the PSRO-Leduc best-response meta-game sits in the wide cyclic belly (~0.45 transitive), while the league's snapshot meta-game climbs the transitive spine (~0.94-0.98).

Двете *реални* популации са основният извод — и те си противоречат в рамките на една и съща игра.

Reconciliation (kept prediction → what actually happened). I framed poker as a skill ladder and expected Leduc’s meta-game to be mostly transitive. Measured, it depends entirely on *which population you decompose*. A population of **best responses** (PSRO) is mostly **cyclic** (≈ 0.45 transitive, 27 three-cycles): the best response beats the current mixture, a newer best response beats *that*, and so on — Balduzzi’s spinning-top in action. A population of **моментни снимки на траекториите на обучение** (the league) is mostly **transitive** (≈ 0.94 - 0.98), because later snapshots are usually stronger than earlier ones, forming a ladder. Neither is a bug; the transitive/cyclic ratio is a property of the *population*, and choosing how you build the population is choosing whether you see a wheel or a ladder.



Фигура 4: Transitive ratio across four populations: RPS (0.0, purely cyclic), a pure skill ladder (1.0), the PSRO-Leduc best-response meta-game (~ 0.41 - 0.46 , mostly cyclic), and the league snapshot meta-game (~ 0.94 - 0.98 , mostly transitive). Same game, opposite structure, depending on how the population is built.

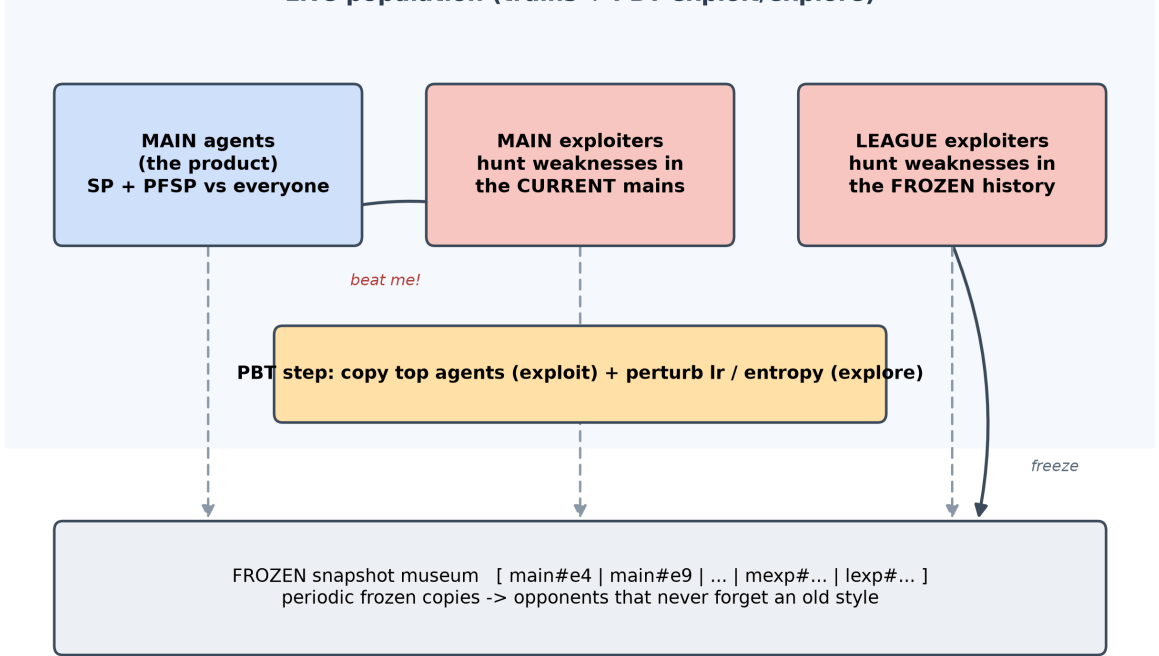
Прочетете повече: Balduzzi, D. и др. (2019). „Open-ended Learning in Symmetric Zero-sum Games“. *ICML*; и Czarnecki, W. M. и др. (2020). „Real World Games Look Like Spinning Tops“. *NeurIPS*.

10.5 5. Лигата AlphaStar-style PBT

С помощта на диагнозата изграждаме доджото. Лигата обучава **невролни** PPO агенти на **Leduc Холдем** с три типа агенти: **основни** агенти (продуктът; те играят срещу всички чрез PFSP самообучение), **основни**

експлоатъори (търсят слабости в *текущите* основни) и **експлоатъори в лигата** (търсят слабости навсякъде в *замразената история*). Обучението, базирано на популации, копира най-добрите агенти (експлоатация) и променя тяхната **скорост на обучение / ентропия** (изследване); периодически се добавя **замразена моментна снимка** на всеки агент в музея; **PFSP** съдвяването избира всеки противник с вероятност, която нараства според това колко е трудно да бъде победен. От решаващо значение е всяка **невронна мрежа** да бъде извлечена до **таблична** стратегия, така че **точният** най-добър отговор на Стъпка 07 да измерва нейната експлоатируемост — същият NashConv, използван още от Стъпка 2.

PFSP matchmaking: sample each opponent with probability rising in how hard it is to beat (focus training where you are losing).



measured: main exploitability falls 4.73 -> ~1.21 then regresses to ~2.05 (scale, 120 epochs); best frozen snapshot 1.31 beats PSRO 2.16 and self-play 3.68.

Фигура 5: The league: main agents (the product), main exploiters (hunt weaknesses in the current mains), and league exploiters (hunt weaknesses in the frozen history), with periodic freezing into a snapshot museum and PFSP matchmaking that focuses training where the agent is losing. PBT copies the top agents and perturbs their hyperparameters.

Измерва ли се подобрение? Оценено върху две конфигурации (smoke: 7 агента, 15 епохи; мащаб: 8 агента, 120 епохи, 48 замразени моментни снимки):

Метрика	Smoke	Машаб
минимална експлоатируемост при основен отговор	4.67 → 3.04 (завършва на мин)	4.73 → мин ≈ 1.21 → 2.05
експлоатируемост на мета-Наш равновесие	4.73 → 3.04	5.01 → мин ≈ 1.32 (плато 1.60) → 2.96
финален ЕЛО (активни агенти)	1176–1211	1198–1210

Метрика	Smoke	Мащаб
---------	-------	-------

Съгласуване (kept prediction → what actually happened). I predicted a *monotone* decrease in експлоатируемост. Smoke’s 15 epochs oblige — a clean drop that ends at its minimum. But scale’s 120 epochs tell the real story: exploitability falls steeply to a minimum near epoch 60 (минимална експлоатируемост при основен отговор ≈ 1.21 , мета-Наш равновесие bottoming ≈ 1.32 then holding a ≈ 1.60 plateau), and then **regresses** back up to ≈ 2.05 / ≈ 2.96 by epoch 119. The best agents are the *frozen snapshots* from mid-run; the *live* основни агенти се влошават късно, преследвайки своите експлоатъори (churn / частично забравяне). This is only visible once training is long enough — a непрекъсната лига is not a monotonically improving one. The remedy (untested here) is съхраняване на най-добрия модел / population регуляризация. Methodologically it echoes Step 9: **scale reveals what smoke hides.**

10.6 6. Разнообразие — наистина ли популацията е разнообразна?

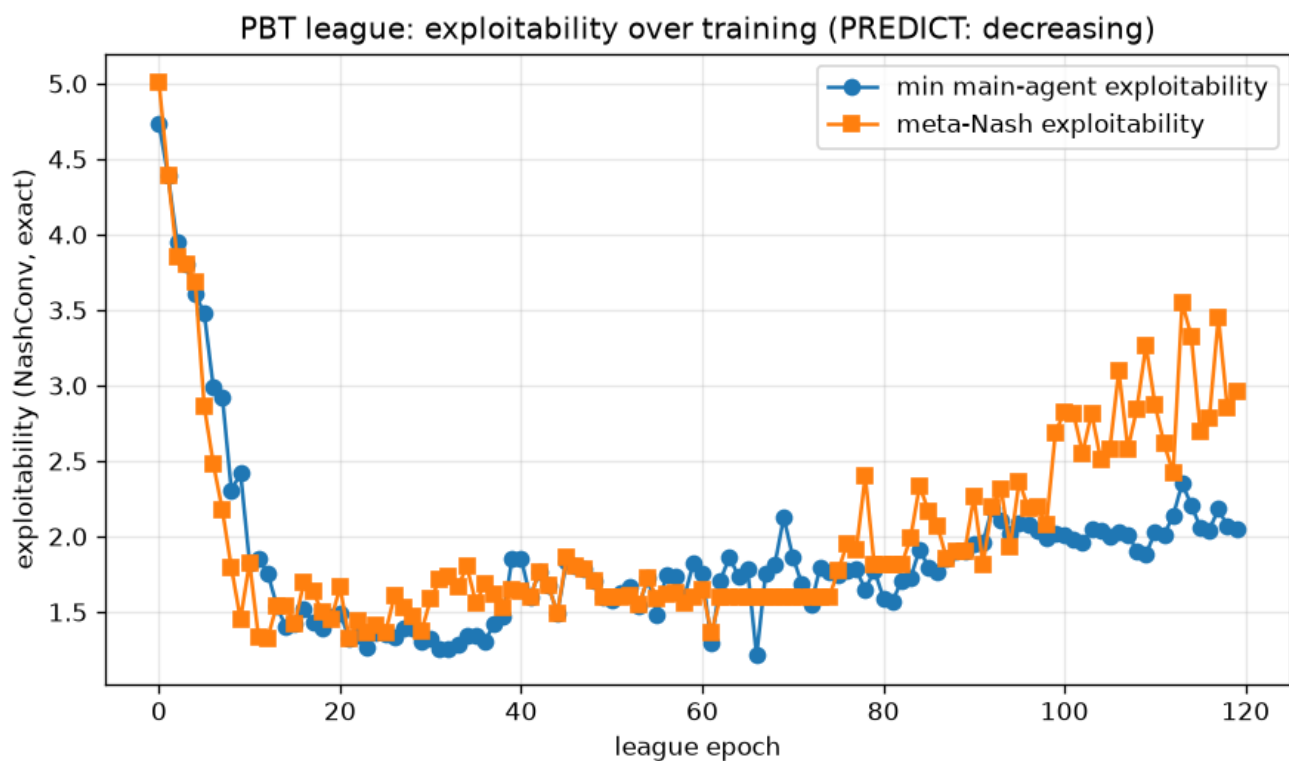
Лигата е толкова добра, колкото и *разнообразието* от стратегии, които съдържа. Измерваме три показателя: **ефективен размер на популацията** (коефициентът на участие на мета-Наш теглата), **поведенческо групиране** (политиките наистина ли са различни?) и **покрытие на експлоатация**. Резултатите показват, че популацията е само слабо разнообразна: коефициентът на участие се увеличава от 1.0 (smoke) до 1.9 (scale), но поведенческото групиране свежда всичко до **един клъстер** и при двата мащаба — дори при scale, където максималното поведенческо разстояние между двойки (0.48) надвишава прага за клъстеризация (0.30), методът на единична връзка обединява веригата. Разнообразието тук е *на ниво тегло* (мета-Наш разпределя подкрепата върху няколко агента), а не *на поведенческо ниво* (агентите играят почти идентично).

Механизмът се проявява най-ясно в опростена среда — мини-PBT върху матрични игри:

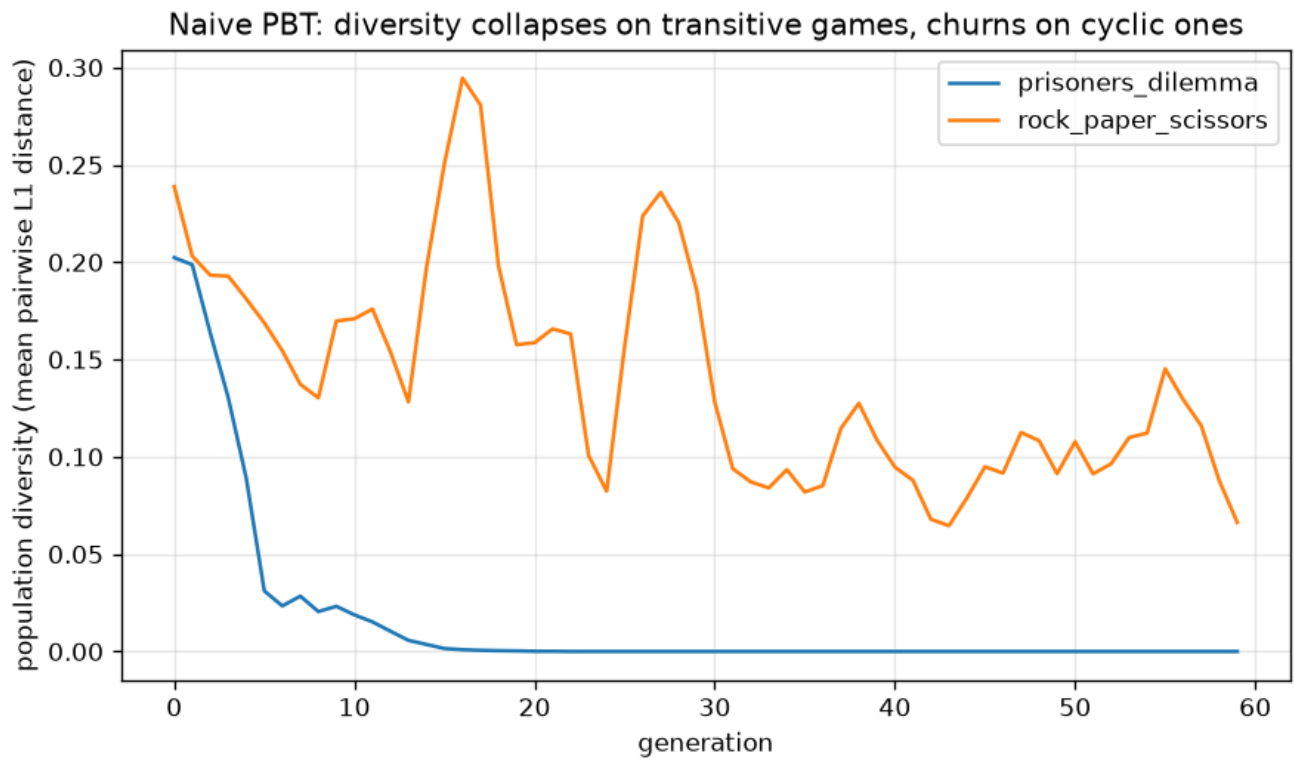
Игра	Разнообразие през поколенията
Дилемата на затворника (транзитивен)	колабира до 0 (всички → дефект)
Камък-ножица-хартия (цикличен)	върти се безкрайно (0.07–0.29), никога не се установява

Транзитивните игри *убиват* разнообразието (съществува един най-добър отговор и всички стратегии се сближават към него), докато цикличните игри го *поддържат* (няма един най-добър отговор, поради което популацията продължава да се променя). Лигата ледюк е близка до транзитивния край, което е и причината за ниското ѝ разнообразие — както и защо ползите от разнообразието на alphaStar, докладвани при приблизително 600 агента, не се проявяват в този мащаб.

Прочетете повече: Vinyals, O. и др. (2019). “Ниво на гросмайстор в StarCraft II с помощта на многоагентно обучение с подкрепление.” *Nature* (alphaStar; лигата и PFSP).



Фигура 6: League exploitability over 120 epochs (scale): both min-main and meta-Nash exploitability fall to a minimum near epoch 60 and then regress upward. The frozen snapshots capture the strong mid-run agents; the live agents degrade late under exploiter pressure.

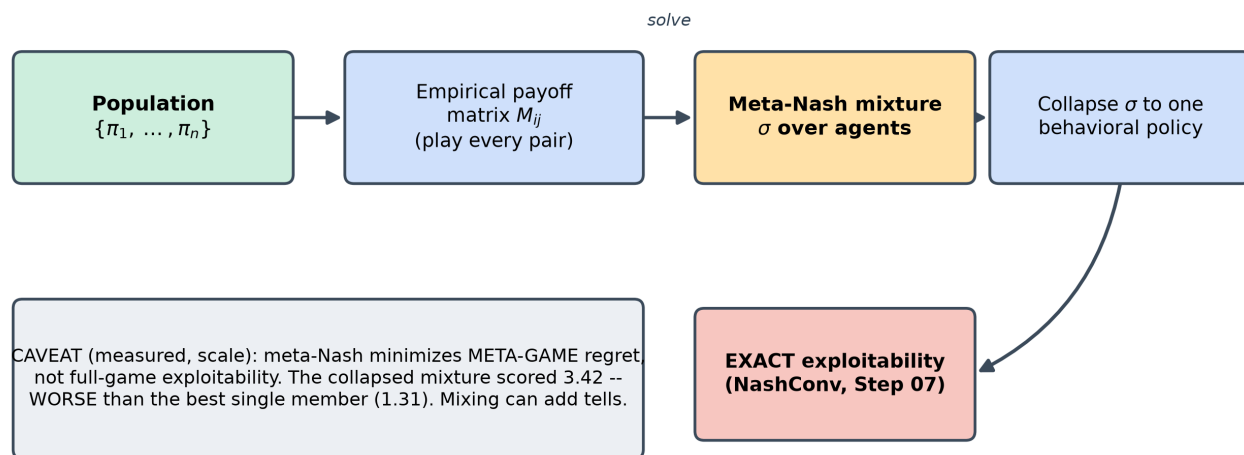


Фигура 7: Mini-PBT diversity: on the transitive Prisoner's Dilemma the population collapses to a single strategy (diversity $\rightarrow 0$), while on cyclic Rock-Paper-Scissors it churns indefinitely (0.07-0.29) as it chases the wheel of counters. Game structure, not population size, decides whether diversity survives.

10.7 7. EGTA — оценка на популацията и неочаквано откритие

Последният инструмент е **емпиричен игровотеоретичен анализ (EGTA)**: разглеждат се цялостните политики като „стратегии“ в по-висша игра, изиграва се всяка двойка с цел попълване на емпирична матрица на печалбата, намисля се нейната **мета-Наш** смес и тя се оценява. Това представлява измерване на експлоатируемостта на ниво популация — наш равновесие в игра, чиито атоми са политики.

EGTA = Nash of a game whose 'strategies' are whole policies -- the population lift of exploitability.



Фигура 8: The EGTA pipeline: play every pair of agents to build an empirical payoff matrix, solve its meta-Nash mixture, collapse the mixture to a single behavioral policy, and measure its EXACT full-game exploitability. The measured caveat: the meta-Nash minimizes meta-game regret, not full-game exploitability, so the mixture can be more exploitable than the best single member.

Очакването (от контролния списък на суровата стъпка) беше, че **мета-Наш** равновесието в **лигата** ще бъде *по-малко* експлоатируемо от всеки отделен член. Измерено:

Конфигурация	експлоатируемост на мета-Наш равновесие	най-добър индивид	мета-Наш ≤ най-добър?
Smoke	2.665	2.665	да (цялото тегло е върху най-добрия агент)
Scale	3.418	1.305	не

Съгласуване (запазена прогноза → какво всъщност се случи). Очаквах **мета-Наш** смес да бъде поне толкова **неексплоатируема**, колкото нейният най-добър елемент. Smoke го потвърди тривиално — **мета-Наш** постави *цялото* тегло върху единствения най-добър агент, така че мета

= най-добър = 2.665. При мащаб **мета-Наш** разпределя тегло (0.645 върху един агент плюс опашка), а колапсираната **поведенческа смес** отбелязва 3.418 — **по-лошо** от най-добрата единична моментна снимка (1.305). Това не е грешка в смесването (същият кодов път даде мета = най-добър в smoke). **Мета-Наш** минимизира **мета-игрово съжаление** — справяне добре *срещу популацията* — което е *различна цел* от минимизирането на **пълна експлоатируемост на играта**; и реализационно претеглена смес от поведенчески политики може да бъде *по-експлоатируема* от своите компоненти, защото блендът въвежда „разкриващи“ информационни множества, които най-добрият отговорчик наказва. Изводът се обръща: това, което една **лига** трябва да *изпрати*, е избран, **най-добър устойчив на отговор елемент** — а не **мета-Наш смес**.

Как се представя **лигата** спрямо останалите методи по показателя за **експлоатируемост в ледюк**?

Метод (мащаб)	Крайна експлоатируемост
cFR-Nash (прагът)	0.0099
Лига — най-добър индивид	1.305
PSRO (точен предсказвач за най-добър отговор)	2.163
Самообучение	3.683
Лига — мета-Наш смес	3.418

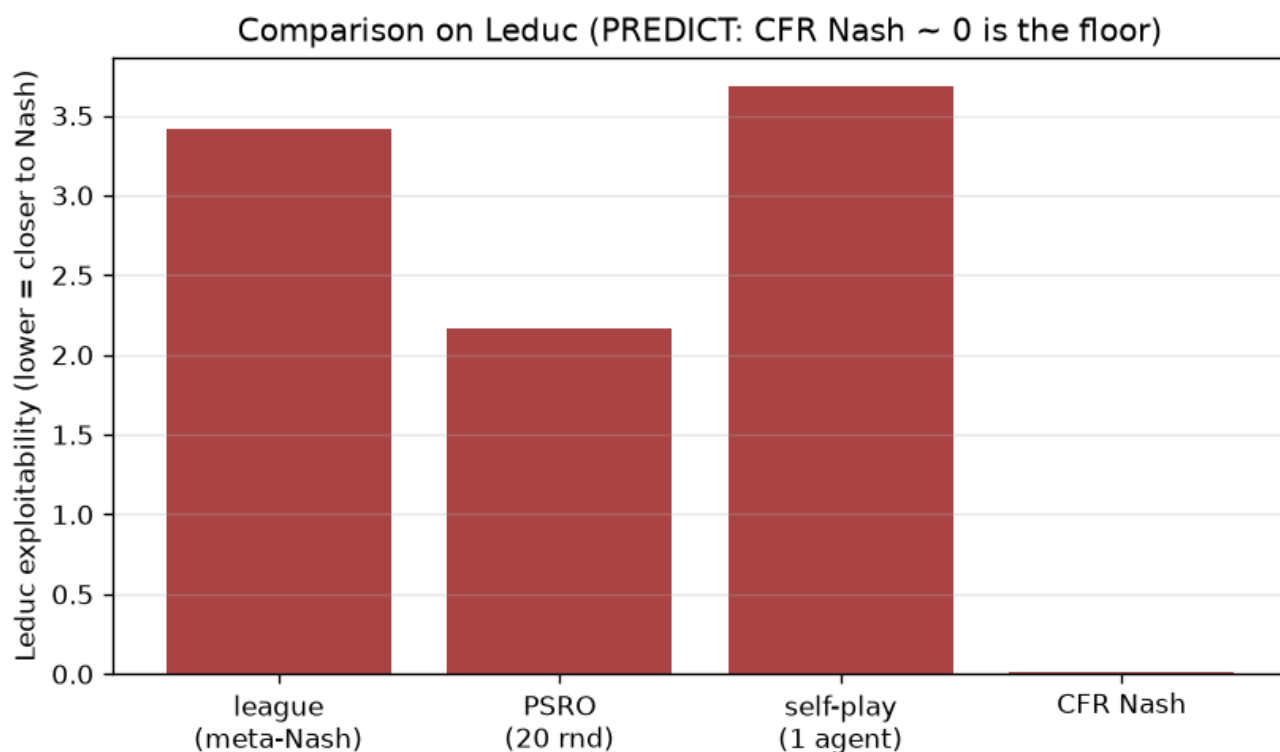
Най-добрият индивид от **лигата** е най-силният научен резултат — надминавайки точния PSRO и самообучението — въпреки че всички научени методи остават далеч над долната граница на Наш. Сместа от **лигата** е най-слабата, дори по-лоша от самообучението (съгласуване в §7). Изпратете индивида, а не сместа.

Прочетете повече: Lanctot, M. и др. (2017). „Унифициран игровотеоретичен подход към обучението с подкрепление на множество агенти.“ *NeurIPS* (PSRO); и Tuyls, K. и др. (2020). „Граници и динамика за емпиричен игровотеоретичен анализ.“ *AAMAS/JAAMAS* (EGTA).

10.8 8. Честни бележки, ограничения и предаване на щафетата

Какво се потвърди. Потвърденият гръбнак: репликаторната динамика възпроизвежда всяко аналитично равновесие на Еволюционно стабилни стратегии/Наш, включително неконвергиращата орбита на RPS; декомпозицията тип пумпал на Ходж чисто отделя умението от цикли и правилно обозначава чистите случаи; лигата води до силно *ранно* подобрене и създава най-добър индивид (1.305), който превъзхожда PSRO (2.163) и самообучение (3.683); а EGTA дава работеща експлоатируемост на ниво популация. Заедно те проследяват дъгата от „в кои игри една популация дори може да се установи?“ до „какво се случва, когато една популация сама се обучава?“.

Какво не се получи и защо това има значение. Три честни уговорки вървят напред. (1) Експлоатируемостта на лигата **претърпява късна регресия** в мащаб ($4.73 \rightarrow \approx 1.21 \rightarrow \approx 2.05$) — непрекъснатата лига не е монотонно подобряваща се; най-добрите агенти са замразени моментни снимки. (2) **Мета-Наш сместа е**



Фигура 9: Final Leduc exploitability by method (scale): CFR-Nash floor ~ 0.01 ; the league's best individual (1.31) beats PSRO (2.16) and self-play (3.68); the league's meta-Nash mixture (3.42) is the weakest learned result.

по-експлоатируема от своя най-добър член в мащаб (3.42 срещу 1.31) — смесването на поведенчески политики не гарантира устойчивост, затова изпратете член, а не сместа. (3) Мета-играта на ледюк е **циклична като популация на най-добри отговори**, но **транзитивна като моментна популация** — структурата е свойство на популацията, която се изгражда. И един методологичен момент: късната регресия и провалът при смесването са невидими в бързата конфигурация на дима — мащабът разкрива това, което димът скрива.

Доверие. Всяка еволюционна цел е *аналитична* (ESS/Наш равновесие на матричните игри), а всяка експлоатируемост на лигата е *точният* NashConv от стъпка 07 върху таблично извлечена политика — така че игровитеоретичните резултати са ограничени от обективната истина. Невронните резултати са едно PBT изпълнение на конфигурация: *посоките* (ранно подобрение, най-добър индивид < PSRO < самообучение, къс-на регресия, мета-Наш > най-добър член) представляват надеждните твърдения, а не величините с третия десетичен знак. Експерименталните PNG изображения по-горе са генерирани от ангажираните JSON artifacts (виж `../figures/README.md`); концептуалните диаграми са създадени от скриптовете `make_*_figure.py`.

Обратни и бъдещи връзки. Обратно: лигата е асинхронизиран вариант на PSRO от стъпка 9, използващ невронни оракули, и тя изцяло възприема точния най-добър отговор от стъпка 07; „мета-Наш на популация“ представлява Наш от стъпка 2, повдигнат на едно ниво. Бъдеще: основните експлоататори са автоматизирани моделиращи съперника (Принос №1); EGTA/мета-Наш е методологията за оценяване (Принос №3); а липсващата гаранция на лигата — тя може да претърпи регресия и нейната смес може да бъде експлоатируема — е популационната форма на **липсващия $N > 2$ предпазен ограничител** (Принос №2). Транзитивна-

та/циклична диагностика предсказва, че коалиционните игри от стъпка 11 ще бъдат силно циклични, така че naïve PBT там ще се държи циклично.

Отворени въпроси. Отстраняват ли съхраняването на най-добрия модел и регуляризацията на популацията късната регресия (поправка в обучението), или тя е присъща на тритипния дизайн (поправка в дизайна)? Трябва ли оценката на популацията да представя избран член, вместо мета-Наш сместа, или пък **мета-решавател с регуляризация за разнообразие**? И в основата на всичко: без минимаксна стойност за популация, какво всъщност означава „безопасно“ и може ли експлоатационният механизъм на лигата да получи гаранция, вместо да остане евристика?

10.9 Основни изводи за синтеза на дисертацията

- **Структурата определя сходимостта.** Транзитивното/цикличното (пумпал) съотношение е *диагностика при предварително обучение*: транзитивните игри се установяват (и потискат разнообразието), цикличните игри се въртят (и налагат разнообразие). Измерено: RPS 0.0 е транзитивна, скала на уменията 1.0, PSRO-Leduc ≈ 0.45 (циклична), моментни снимки на лигата $\approx 0.94\text{--}0.98$ (транзитивни).
- **Лигата произвежда силни индивиди, но не носи гаранция.** Най-добрият модел 1.305 побеждава PSRO 2.163 и самообучението 3.683 — въпреки това експлоатируемостта *регресира* късно ($\rightarrow 2.05$), а мета-Наш *сместа* (3.42) е по-лоша от най-добрия си член (1.31).
- **Мета-Наш оптимизира мета-игровото съжаление, а не пълната експлоатируемост на играта** — двете не съвпадат, така че оценката на популацията трябва директно да оценява колапсиралата смес и да изпраща избран член.
- **Точната оценка е котва.** Извличането на невронни политики в табличен вид и оценяването им с точния най-добър отговор от Стъпка 07 прави всяко твърдение за популацията проверено на място — същият NashConv от Стъпка 2.
- **Разликата в безопасността за населението е отворената врата (Принос №2):** експлоатационният механизъм на AlphaStar е аналогът на популацията на безопасната експлоатация от Стъпка 8, но той е евристичен — нито гарантира монотонно подобрене, нито неексплоатируема смес.

Стъпка 11 Обобщение — Динамично формиране на коалиции в състезателни FFA игри (So Long Sucker)

Изследване върху възможностите за прилагане на изкуствен интелект в компютърните игри

Alexander Andreev

July 2026

Съдържание

11 Стъпка 11 — Динамично формиране на коалиции в състезателни игри всеки срещу всеки	1
11.1 Защо появата на трети играч е повратна точка	2
11.2 Детекторът на коалиции — извличане на съюзи от ходовете	2
11.3 Стойност на Шейпли в състезателна игра	3
11.4 Coalition-Aware MAPPO — и кога всъщност се проявяват коалиции	7
11.5 EGTA и въртящият се пумпал — SLS колело ли е, или стълба?	8
11.6 Честни бележки, ограничения и къде това раздаване се поема оттук нататък	11
11.7 Основни изводи за синтеза на дисертацията	12

11 Стъпка 11 — Динамично формиране на коалиции в състезателни игри всеки срещу всеки

Това е цялостна глава за нещо, което възниква едва когато една игра има **трети играч**: временни **съюзи**, които се формират, експлоатират и предават. Тя изгражда първото обучение с подкрепление, отчитащо **коалиции** в **So Long Sucker (SLS)** — играта от 1950 г., която Наш, Шапли, Шубик и Хауснер проектираха именно за да изучават това — и я използва, за да осмисли прехода от $N = 2$ към $N \geq 3$, където **нашево равновесие и експлоатируемост спират да бъдат управляеми и губят смисъла си**, така че „сработи ли?“ вече не може да бъде едно-единствено число. Тя служи за две цели — самостоятелно въведение за по-късни стъпки и основен източник за многоагентните приноси на дисертацията. Написана е така, че да може да се чете самостоятелно. **Всички експериментални числа, докладвани тук, са измерени** във възпроизводими изпълнения и, когато е възможно, са ограничени от *точни* референции (учебникови стойности на Шапли/ядро за кооперативни игри; точен двуиграчев минимакс решавач за крайната игра на SLS). Когато някое изпълнение противоречи на очакванията ми — включително и реален бъг в двигателя — запазвам първоначалното очакване и го съгласувам с това, което се е случило; тези пропуски са най-поучителните части от стъпката.

Къде се вписва това в дисертацията. Стъпки 2–10 разчитаха на игра с двама играчи с *точен* предсказвач за най-добър отговор. Стъпка 11 го премахва и навлиза във фронта. Тук се съдържат три основни приноса на дисертацията. **Детекторът на коалиции** издига моделирането на противника от „какъв тип играч е това?“ до „с кого е в съюз?“ (Принос №1). **Безопасната базова линия губи своята нашава опорна точка** — при

празно ядро няма стабилно разпределение, така че „безопасно“ трябва да стане поведенческа/популационна (piKL), пропусъкът, който тази стъпка очертава, но не запълва (Принос №2). И EGTA Meta-Game + стойност на Шейпли заменя експлоатируемостта, която няма смисъл срещу коалиция (Принос №3).

11.1 Защо появата на трети играч е повратна точка

С двама играчи една игра с нулева сума има *стойност*: съществува оптимална по минимакс стратегия и „колко далеч сте от оптималната?“ (експлоатируемост) е едно-единствено, смислено число, което поддържа всяка стъпка от Стъпка 2 насам. Добавянето на трети играч променя основата. Сега двама играчи могат да **се обединят** срещу третия и интересният въпрос вече не е „какво е равновесието?“, а „с кого се съюзява кой, за колко дълго и кой предава пръв?“. Равновесие на Наш в игра всеки срещу всеки с четирима играчи е едновременно **неразрешим** от гледна точка на изчислителна мощност и **стратегически безсъдържателен** — то не дава никаква информация за коалициите, които всъщност определят изхода от играта.

Картина, която да задържите в ума си: **вечеря** с четирима гости и една къща. Всеки се съюзява по двойки, за да свърши работата, но най-умният ход е тихо да предадете партньора си един ход *преди* той да ви предаде. Со Лонг Съкър прави това буквално: държите цветни **жетони**, поставяте ги в купчините на други играчи (ръкостискане) или завладявате техните купчини (ножът), и бивате елиминирани, когато останете без **жетони**. Целият съюз е **кодиран в самите ходове** — няма отделна **фаза** за преговори, която да се моделира, което е именно причината детекторът да може да прочете коалицията директно от потока на **ходовете**.

Последствието за методологията е най-важната идея в тази стъпка: **на $N \geq 3$ се преминава от точна оценка към емпирична оценка**. Вместо експлоатируемост се използват процент победи, резултат на коалиция от детектора и циклично съотношение на една емпирична мета-игра — всичко това е закотвено в единствената под-игра, която *е* точно решима, а именно крайната игра с двама играчи.

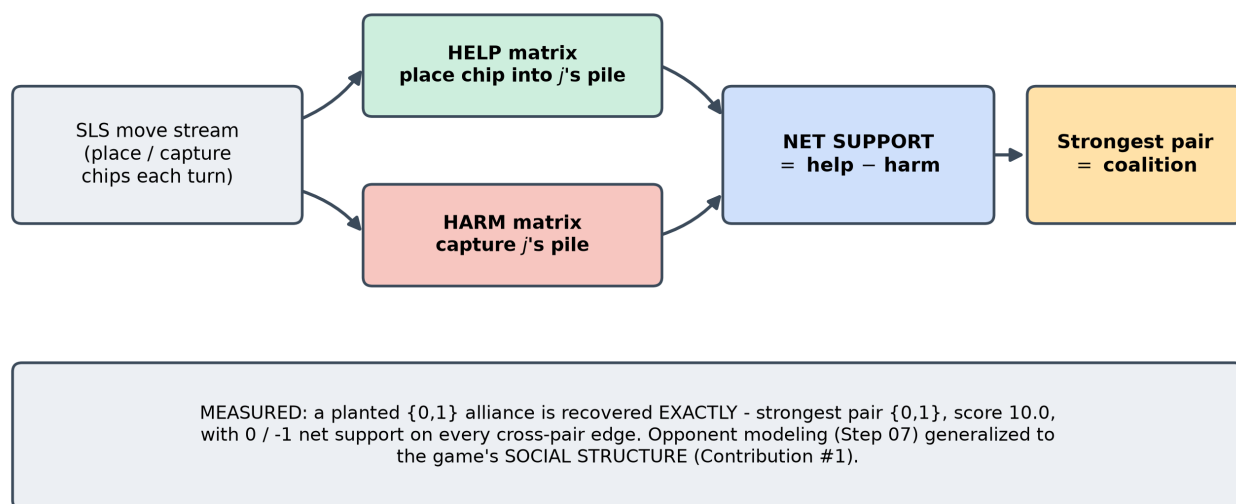
Read more: Nash, Shapley, Shubik & Hausner (1950s), *So Long Sucker* (the game itself); and De Carufel, J. & Jerade, M. — the 2-player SLS анализ на крайна игра, който дава единствената точна опора.

11.2 Детекторът на коалиции — извличане на съюзи от ходовете

Първият инструмент обобщава моделирането на противника. Вместо да извежда скрит *тип* или *ръка* (Стъпка 7), детекторът извежда скрита *социална структура*. Той наблюдава хода и натрупва две матрици: **помощ** (игращ *i* поставя чип в купчината на играч *j*) и **вреда** (игращ *i* залови купчината на играч *j*). Разликата между тях е **нетна подкрепа**, а двойката с най-висока взаимна нетна подкрепа представлява най-силната коалиция.

Работи ли? Скриптираме двама играчи да си помагат систематично и да вредят на останалите, без да предоставяме никаква информация на детектора, и го караме да посочи коалицията. В резултат той възстановява засадената $\{0, 1\}$ коалиция **точно**, с отчетливо по-висок резултат (10.0 за съюзническата двойка, 0 или -1 навсякъде другаде). Коалицията е напълно разпознаваема единствено от начина на поставяне на чиповете.

The alliance is encoded in the moves themselves - no separate negotiation phase to model.



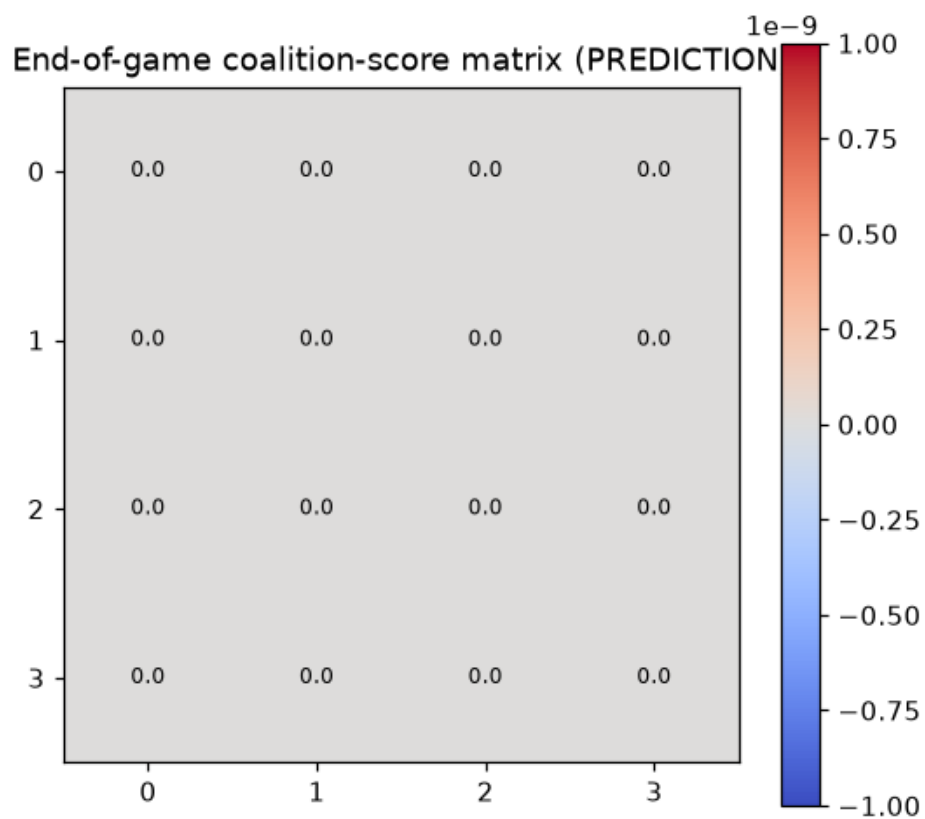
Фигура 1: The coalition detector: from the SLS move stream, placing a chip into another player's pile counts as HELP and capturing a pile counts as HARM. Accumulated into help/harm matrices and differenced into net support, a reciprocal alliance shows up as a strong mutual edge. Opponent modeling (Step 7) lifted from “what hand?” to “who is allied with whom?” (Contribution #1).

Прочетете повече: двигателят за моделиране на противника от Стъпка 07 (това хранилище) — принципът „наблюдавай действия -> актуализирай вярвания“, който детекторът преоткрива като матрици на помощ/вреда.

11.3 Стойност на Шейпли в състезателна игра

Ако искаме един **агент** да се *научи* да формира коалиции, е необходим сигнал за **награда**, който да показва „ти допринесе за една коалиция“. Класическият инструмент е **стойността на Шейпли**: единственият справедлив начин за разпределяне на стойността на една коалиция между нейните членове чрез осредняване на пределния принос на всеки член във всички възможни редове на присъединяване. Две учебникови игри определят какво означават „справедливо“ и „стабилно“, като и двете се възпроизвеждат точно:

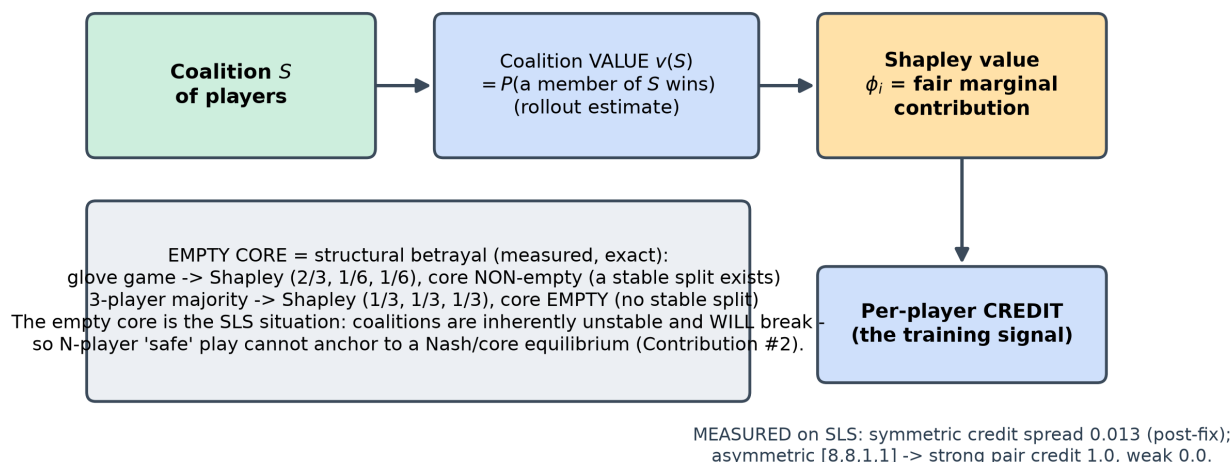
Кооперативна игра-играчка	Стойност на Шейпли	Ядро (стабилност)
Glove Game	$(2/3, 1/6, 1/6)$	непразно (съществува стабилно разпределение)
тричленно мнозинство	$(1/3, 1/3, 1/3)$	празно (няма стабилно разпределение)



Фигура 2: Coalition graph inferred purely from chip placement: the planted $\{0,1\}$ alliance appears as a strong reciprocal help edge; cross-pair edges are neutral or hostile. The detector cleanly recovers a coalition it was never told about.

Празното ядро на играта на мнозинството е концептуалното сърце на стъпката: това е игра, при която *всяко* разпределение може да бъде оспорено от някаква коалиция, така че сътрудничеството е *по своята същност нестабилно*. Това е ситуацията в SLS — и причината n-игращен „безопасен“ стил на игра да не може да бъде основан на стабилно равновесие (Принос №2).

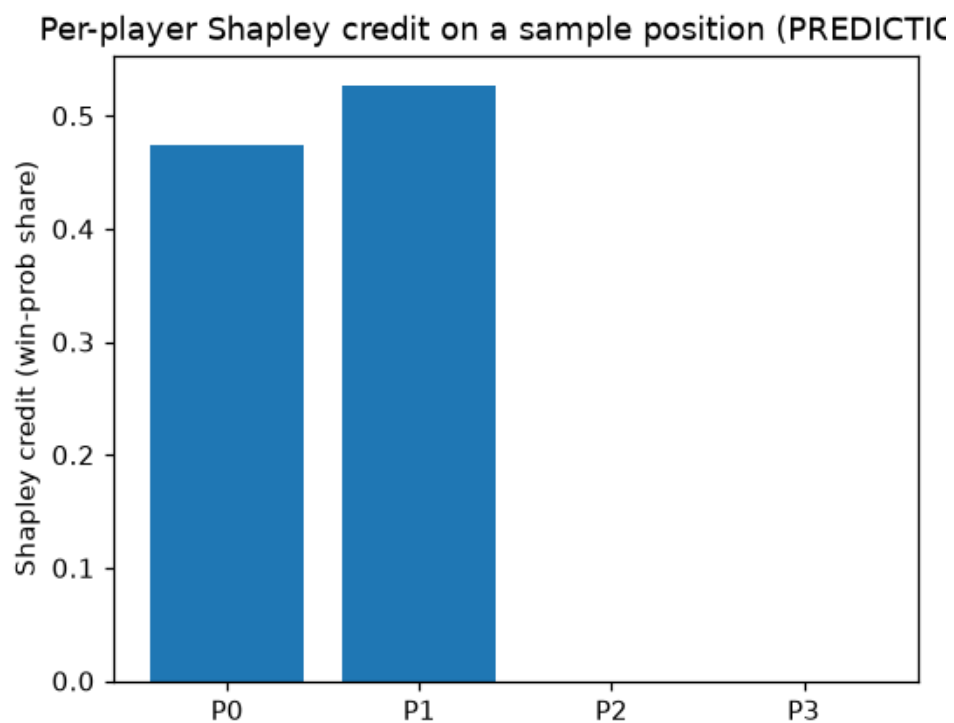
Shapley = the UNIQUE fair split; adapted to a competitive game via win-probability as the value.



Фигура 3: Shapley credit adapted to a purely competitive game: the coalition’s “value” is redefined as the PROBABILITY a coalition member wins (estimated by rollouts), and each player’s Shapley value of that win-probability function is their credit. The empty core of the majority game (no stable allocation) is the structural signature of SLS coalitions - they will break.

SLS е състезателна, а не кооперативна игра — няма общ пот за разделяне — така че ние **предефинираме стойността на коалиция** като *вероятността член на коалицията да спечели*, оценена чрез Монте Карло симулации. Кредитът на всеки играч след това се изчислява като стойността на Шейпли на тази функция за вероятност от печалба. Измерено върху SLS позиции: една наистина симетрична позиция води до почти равностойно разпределение на кредита (разпределение 0.013), а една асиметрична [8, 8, 1, 1] позиция при-съжда *целия* кредит на силната двойка (стойност на коалиция 1.0). Симетричният резултат е докладван **след** отстраняване на грешка — виж съгласуването по-долу.

Съгласуване (kept prediction -> what actually happened). I predicted a symmetric position would give a symmetric credit spread (< 0.15). The first run gave 0.54 — a red FAIL — with Player 0 winning $\sim 2x$ its fair share across three independent scripts. Suspecting the engine before the prediction, I found the mechanism: **$\sim 99.5\%$ of random SLS games end in a deadlock** (all live hands empty), so the winner is decided by a most-chips **tie-break** — whose lowest-index rule quietly handed seat 0 its edge. An **unbiased random tie-break** fixed it: symmetric spread $0.54 \rightarrow 0.013$, all-random winners now uniform. It also revealed that an impressive ~ 0.87 hero win-rate had been the *same* artifact (the



Фигура 4: Разпределение на заслуги по Шейпли върху SLS позиции: почти равномерно между местата в симетричния случай (разпределение 0.013, след отстраняване на грешка), и изцяло концентрирано върху силната двойка в асиметричния случай $[8,8,1,1]$. Заслугите проследяват реалния принос, след като развързката на двигателя бъде безпристрастна.

hero always sat in seat 0); the fair number is ~ 0.41 . The lesson: in a game that almost always ends in a near-tie, the tie-break rule is the most load-bearing line in the engine, and a symmetric *position* is not a symmetric *outcome* until it is unbiased.

Read more: Shapley, L. S. (1953). “A value for n-person games.” *Contributions to the Theory of Games*; Chalkiadakis, Elkind & Wooldridge (2011), *Computational Aspects of Cooperative Game Theory* (core, shapley, nucleolus); и Wang и др. относно разпределянето на заслугата, базирано на Shapley в MARL.

11.4 Coalition-Aware MAPPO — и кога всъщност се проявяват коалиции

Сега провеждаме обучението. Всяко място в SLS е маскиран агент на Episodic PPO, който се обучава чрез самообучение. Наградата е **смес**:

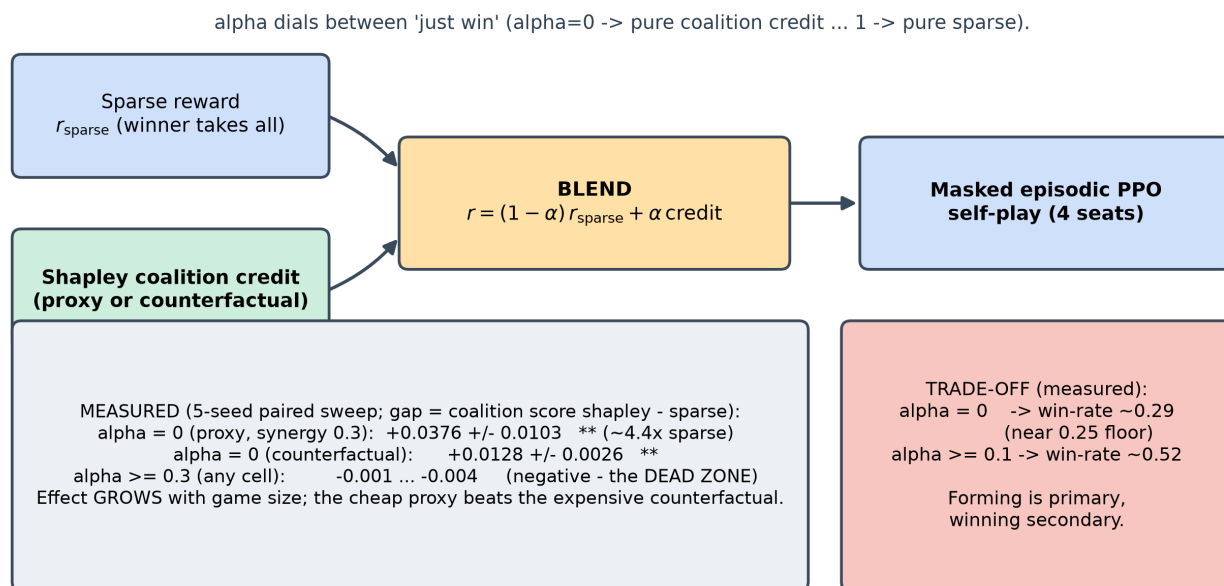
$$r = (1 - \alpha) r_{\text{sparse}} + \alpha (\text{Shapley coalition credit}),$$

където r_{sparse} е наградата „всичко или нищо“, а кредитният член представлява стойността на Шейпли за вероятността от победа от раздела *коалиционен кредит* (евтин заместител с критична стойност, или скъпо контрафактично разгръщане). Теглото на смесване α определя баланса между „просто победи“ и „формирай коалиции“.

Формират ли коалиция-осъзнатите агенти повече коалиции отколкото оскъдните агенти? Честният отговор изискваше **петкратно кръстосано сравнение по двойки** върху $\alpha \times$ мода на кредита $\times$ синергия, тъй като една единствена конфигурация ме подведе сериозно (по-долу). Измерено (сдвоена разлика = коалиционен резултат на Shapley агентите минус този на оскъдните агенти; ** = значимо при $> 2 \times \text{SE}$):

Режим	Сдвоена разлика (Shapley - оскъдна базова линия)
мащаб, заместител, $\alpha = 0$, синергия 0.3	$+0.0376 \pm 0.0103$ (~4.4 пъти над оскъдната базова линия)
мащаб, контрафактичен, $\alpha = 0$	$+0.0128 \pm 0.0026$
всяко ниво, $\alpha \geq 0.3$	$-0.001 \dots - 0.004$ (отрицателна)
smoke, заместител, $\alpha = 0$, синергия 0.1	$+0.0024 \pm 0.0008$ (незначителна)

Съгласуване (запазена прогноза -> какво всъщност се случи). My single-config runs used the default $\alpha = 0.3$ and showed the coalition signal collapse at scale — which I first read as “the proxy credit is too weak once training is longer.” The sweep overturned that completely: **α is the dominant knob, and 0.3 is a dead zone.** Coalitions emerge significantly *only at low α* (at $\alpha \approx 0$ the Shapley agent beats sparse by $+0.038$, ~4.4x), while *every $\alpha \geq 0.3$ cell is negative* — the sparse всичко или нищо term drowns the coalition signal. Two further surprises: the effect **grows with game size** (opposite to my “smoke-positive / scale-null” read, which was an artifact of holding $\alpha = 0.3$ at both tiers), and the **cheap proxy beats the expensive контрафактичен**. So the primary thesis signal (коалиционно



Фигура 5: Coalition-aware MAPPO reward blend: the sparse winner-takes-all signal is mixed with the Shapley coalition credit by weight alpha. Measured, alpha is the dominant knob: coalitions emerge significantly only at low alpha (heavy credit weight), and every alpha ≥ 0.3 suppresses the signal. The cheap critic-value proxy beats the expensive counterfactual credit.

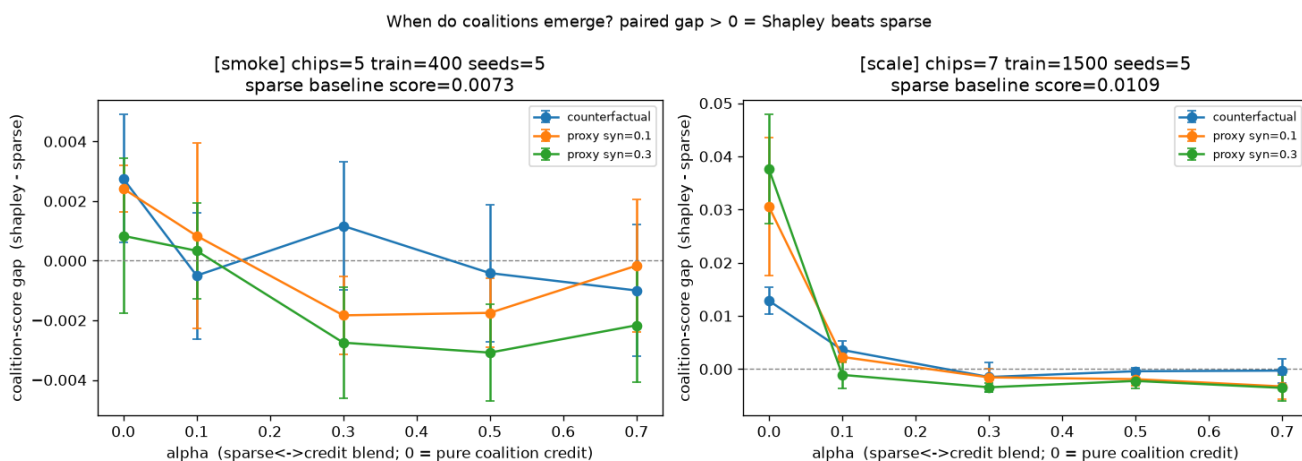
формирание) is real and robust — I had simply measured it in the one regime where the sparse term hides it. The fix is “weight the coalition credit heavily,” not “compute a truer credit.”

Няма безплатен обяд: при чисто коалиционен кредит ($\alpha = 0$) процентът победи спада до ~ 0.29 (близо до 0.25 случайния минимум), докато $\alpha \geq 0.1$ го запазва на ~ 0.52 . Основната цел е формирането на коалиции, а победата е второстепенна — това е рамката на суровата стъпка, вече количествено изразена.

Read more: the Step 09 MARL stack (this repo) for MAPPO with a centralized critic; and Bakhtin и др. (2022) on **piKL** — regularizing toward a behavioral prior instead of Nash, the n-игращен безопасен начин на игра recipe this step points at.

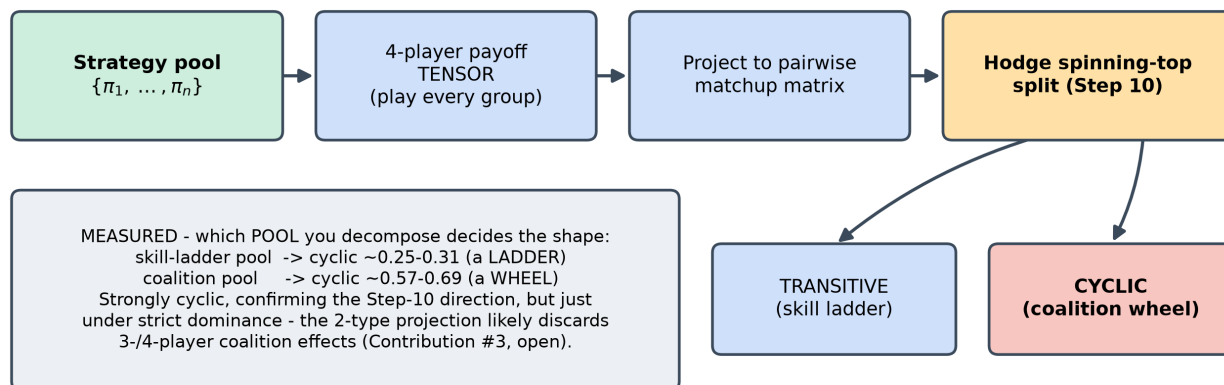
11.5 EGTA и въртящият се пумпал — SLS колело ли е, или стълба?

Крайният инструмент оценява *популацията*. Емпиричният игровотеоретичен анализ (EGTA) разглежда целите стратегии като атомите на мета-игра, изиграва всяка двойка, за да попълни емпирична матрица на печалбата, и анализира нейната структура. За да се използват повторно решаващът на мета-Наш от Стъпка 9 и **пумпалът** (Hodge) разлагане от Стъпка 10 — и двата двуигрови инструмента — 4-игровият тензор на печалбата се **проектира** към матрица на двустранни сблъсъци, след което се разделя на **транзитивен** (стълба на умения) компонент и **цикличен** (камък-ножица-хартия) компонент.



Фигура 6: Paired coalition-score gap across the alpha x credit x synergy sweep (5 seeds, error bars): large and significant only in the low-alpha regime (peaking at +0.038 with the proxy at alpha=0), negative for every alpha ≥ 0.3 . The earlier null came from measuring in the alpha=0.3 dead zone.

EGTA = Nash of a game whose 'strategies' are whole policies (exploitability has no meaning vs a coalition).

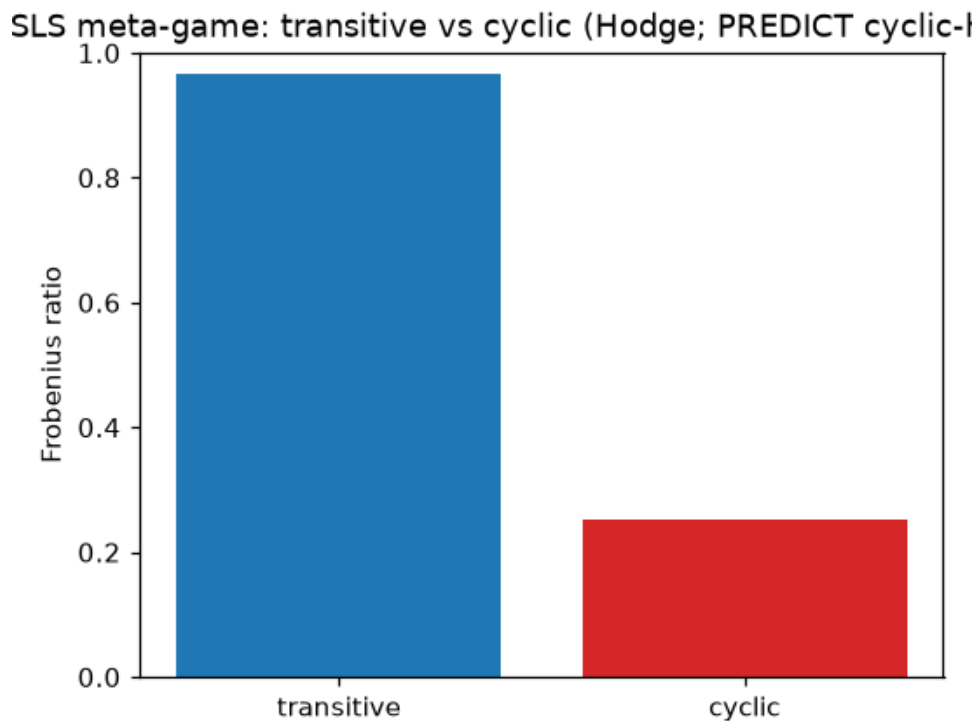


Фигура 7: The EGTA + spinning-top pipeline for SLS: play every pair of strategies to fill a payoff tensor, project the 4-player tensor to a pairwise matchup matrix, and Hodge-decompose it into transitive (skill ladder) and cyclic (coalition counters) parts. Measured caveat: the 2-type projection likely discards 3-/4-player coalition effects, so the cyclic ratio is a lower bound.

Стъпка 10 предвижда, че коалиционните игри във FFA ще бъдат силно циклични. Измерено, това зависи изцяло от **популацията, която се разлага** — същият урок, който Стъпка 10 научава:

Популация	Циклично съотношение	Структура
Пул от нива на умения	0.25-0.31	транзитивно-доминантен (стълба)
Коалиционен пул (стратегии „съюзник/предател“)	$\sim 0.57-0.69$	силно цикличен (колело)

Reconciliation (kept prediction -> what actually happened). I expected a large cyclic component and, at first, saw a near-perfect skill ladder (cyclic ~ 0.07). That was partly the seat-0 bug (the *Shapley credit* section) and partly **pool composition**: the default baseline pool *is* a skill ladder. A coalition pool (fixed-ally + betrayer strategies) pushes the cyclic ratio to $\sim 0.57-0.69$ — a large нетранзитивна компонента, която силно потвърждава *посоката* на прогнозата, но остава **честно под строго доминиране** (cyclic² малко под 0.5). Основният заподозрян за остатъка е, че **2-type projection discards 3-/4-player coalition effects** — a tensor-native decomposition is the open question. Neither reading is a bug; which population you build decides whether SLS looks like a wheel or a ladder.



Фигура 8: Spinning-top transitive/cyclic ratios: the skill-ladder pool is transitive-dominant (cyclic $\sim 0.25-0.31$), while the coalition pool is strongly cyclic ($\sim 0.57-0.69$). Coalition play injects large non-transitivity, confirming the Step-10 direction while staying just under strict cyclic dominance.

Прочетете повече: Balduzzi, D. и др. (2019). „Open-ended Learning in Symmetric Zero-sum Games“. *ICML* (въртележката); и Lanctot, M. и др. (2017), линията PSRO / EGTA (стек от стъпка 9–10).

11.6 Честни бележки, ограничения и къде това раздаване се поема оттук нататък

Какво се задържа. Точните опори са стабилни: sLS двигателят съвпада с 2-игровата минимакс крайна игра без нито едно несъответствие; детекторът възстановява засадена коалиция **точно** ($\{0, 1\}$, резултат 10.0); кодът на Shapley възпроизвежда кооперативни игри-игралки — *включително тяхното ядро* — до четири десетични знака. И основното твърдение за обучението оцелява при петкратно кръстосано сравнение по двойки: обучението с отчитане на коалициите произвежда значително повече поведение на коалиция (+0.038, $\sim 4.4x$) в ниския α режим.

Какво не се получи и защо това е важно. Две честни корекции се предават напред. (1) „Симетричната“ игра не беше симетрична — **грешка при разпределение на равни резултати** с най-много жетони даде на място 0 приблизително два пъти по-голям дял от справедливия и завиши процента победи на героя от истински ~ 0.41 до фалшив ~ 0.87 ; поправено е, но това доказва, че *правилата* на двигателя, а не неговият решаващ, са мястото, където се крие рискът. (2) „Коалициите не възникват в голям мащаб“ беше **неправилно зададено тегло на смесване** ($\alpha = 0.3$ е мъртва зона), а не провал на метода. Методологично ехо от Стъпки 9-10: **една единствена конфигурация скрива онова, което обхождане с фиксирани начални стойности разкрива.**

Доверие. Всяка точна цел (минимакс на крайната игра, кооперативни игри-игралки) е детерминистичен и възпроизводим; твърденията за обучението се основават на петкратно кръстосано сравнение по двойки с грешки, така че *посоките* са надеждните твърдения, а не величините до третия знак след десетичната запетая. Постоянното предупреждение е **точност на двигателя**: двигателят е сертифициран спрямо собствения си правилник, а не спрямо транскрипция на De Carufel & Jerade — и грешката при разпределение на равни резултати показва, че съгласуването не е козметично. (Подаденият `scale_results.JSON` е представка-артефакт, цитиран единствено като доказателство за грешката; авторитетните данни за мащаб идват от обхождането.)

Обратни и напредни връзки. Обратно: детекторът е моделът на противника от Стъпка 7, разширен до социална структура; мета-Наш и пумпалът са Стъпки 9–10, приложени повторно върху проектираната мета-игра; празното ядро прави невъзможно определянето на „безопасно = ограничено отклонение от Нашово равновесие“ от Стъпка 8, което налага използването на поведенческия априорен разпредел на `piKL`. Напред: празнината в безопасността, свързана с поведенческия априорен разпредел, насочва към **Стъпка 12** (език / преговори, `CICERO / Welfare-Diplomacy`), а `eGTA-tensor evaluation` е многоагентното обобщаване на експлоатируемост, което **Стъпка 14** наследява (Принос #3).

Отворени въпроси. Съвпада ли sLS двигателят с точните правила на De Carufel & Jerade (грешката при разпределение на равни резултати показва, че това е от значение)? Дали EGTA Projection, съобразена с 3-/4-игралки, изтласква цикличното съотношение отвъд строгото доминиране? Достатъчно добър ли е евтиният заместващ кредитен сигнал като цяло или само при SLS? И под всичко това: без стойност на Наш и с празно ядро, **какво изобщо означава „безопасен“** в n -игрална коалиционна среда — празнината от Принос #2, която тази стъпка формулира, но не затваря.

11.7 Основни изводи за синтеза на дисертацията

- **При $N \geq 3$ точната оценка отпада.** Стойността на Наш е неразрешима и стратегически празна при 4-игрочна FFA; оценката става емпирична (процент на победи + коалиционен резултат + EGTA Cyclic Ratio), закотвена в единствената точно решима под-игра (2-игрочният минимакс игра в крайна фаза, 0 несъответствия).
- **Коалициите могат да бъдат научени — но само ако се претегли кредитът.** Сигналят за коалиция на Шапли превъзхожда оскъдната награда с $+0.038$ ($\sim 4.4x$, 5 семена) при ниска α и е *потиснат* при $\alpha \geq 0.3$; ефектът нараства с размера на играта, а евтиният заместващ сигнал превъзхожда скъпия контрафактичен.
- **Коалиционното формиране намалява шансовете за победа.** $\alpha = 0$ понижава процента на победи до ~ 0.25 минимум; $\alpha \geq 0.1$ restores ~ 0.52 — формирането е първично, победата вторична, количествено определено.
- **Празното ядро $\Rightarrow$ означава структурна нестабилност.** Празното ядро на играта на мнозинството е ситуацията в SLS: не съществува стабилно разпределение, така че коалициите *ще* се разпаднат — поради което „безопасната“ игра при n -играчи изисква поведенчески априорен разпредел (piKL), а не закотвяне в Наш/ядро (Принос #2).
- **Коя популация се разлага определя стълбата срещу колелото** (урокът от Стъпка 10, потвърден): пул от нива на умения изглежда транзитивен (~ 0.3 цикличен), коалиционният пул изглежда силно цикличен (~ 0.57 - 0.69).
- **Вярност към правилата на двигателя е по-важна от кода на решаващия алгоритъм.** И двете останали слабости (пристрастието към мястото, поддоминантният цикличен коефициент) се намират в опростяванията на двигателя, а не в математиката на Шапли/EGTA — която премина точно през своите учебникови проверки.

Стъпка 12 Обобщение — Последователни модели и LLM агенти в стратегически среди

Изследване върху възможностите за прилагане на изкуствен интелект в компютърните игри

Alexander Andreev

July 2026

Съдържание

12 Стъпка 12 — Последователни модели и агенти с голям езиков модел в стратегически ситуации	1
12.1 Защо да се разглежда обучението с подкрепление като прогнозиране на последователност . .	1
12.2 Капанът между късмета и умението	2
12.3 абстрактен дървовиден модел на решенията — условност върху гарантираното	2
12.4 Какво всъщност представлява условното връщане в покера	3
12.5 Езикови модели като стратегически агенти	4
12.6 Експлоатация, адаптация и ограниченията на опростената игра	5
12.7 Основни изводи за синтеза на дисертацията	7

12 Стъпка 12 — Последователни модели и агенти с голям езиков модел в стратегически ситуации

12.1 Защо да се разглежда обучението с подкрепление като прогнозиране на последователност

Класическото обучение с подкрепление оптимизира чрез оценка на функция на стойността, връщане на наградите и подобряване на стратегията. Трансформърът за вземане на решения (Chen и др., 2021) предлага различен подход — разглежда целия проблем като **супервизирано моделиране на последователности**. На gPT-style модел се подава траектория от тройки

$$(\hat{R}_1, s_1, a_1, \hat{R}_2, s_2, a_2, \dots)$$

където $\hat{R}_t = \sum_{t' \geq t} r_{t'}$ е *очакваната възвръщаемост до края*, моделът се обучава да предсказва a_t въз основа на всичко, случващо се до s_t . Не се използва оператор на Белман и не се извършва стъпка за подобряване на стратегията. При извод просто **условие**: задайте желана висока очаквана възвръщаемост до края и моделът генерира действията, които исторически са я предшестващи.

Привлекателността на тази теза е, че тя работи изцяло **извън линия**, върху фиксиран набор от данни — в режима, в който се намират логовете на Playtech на Step 13, където самообучението не е възможно.

Read more: Chen и др., *Трансформър за вземане на решения: Обучение с подкрепление чрез моделиране на последователности*, neurIPS 2021 (arXiv:2106.01345).

12.2 Капанът между късмета и умението

Преформулирането прикрива едно предположение: че очакваната възвръщаемост до края е нещо, което агентът *е заслужил*. В стохастична среда това не е вярно. Paster и др. (2022) показват, че стратегиите, обусловени от възвръщаемостта, систематично преследват късмета, а най-простият пример го прави очевидно.

Разгледайте едностъпков еднорък бандит. Действие А изплаща 0.5 детерминистично, а действие В изплаща 1.0 с вероятност 0.4 и 0 в останалите случаи, така че очакваната му стойност е $\mathbb{E}[B] = 0.4 < 0.5$. Действие А е оптимално по очаквана стойност. Единственият начин обаче една траектория да постигне възвръщаемост от 1.0 е чрез избор на действие В и получаване на късмет. Ако обусловим върху „възвръщаемост = 1.0“, ще получим единствено траектории с действие В:

$$P(a = B \mid \hat{R} = 1.0) = 1.00 \quad (\text{measured})$$

Обучаващият се, обусловен от възвръщаемостта, уверено избира действието с по-ниска **очаквана стойност**. Това не е грешка в модела; това е смисълът на **условност върху изход**, когато самият изход съдържа елемент на случайност.

Прочетете повече: Paster, McIlraith & Ba, *You Can't Count on Luck: Why Decision Transformers and RvS Fail in Stochastic Environments*, neurIPS 2022 (arXiv:2205.15967).

12.3 абстрактен дървовиден модел на решенията — условност върху гарантираното

Решението е да се обуславя не върху възвръщаемостта, която *се е случила*, а върху възвръщаемостта, която главният герой може да **гарантира срещу противник в най-лошия случай** — минимакс очакваната възвръщаемост до края. ARDT (Tang и др., 2024) я оценява чрез **регресия с експектили**, която минимизира асиметричната квадратична загуба.

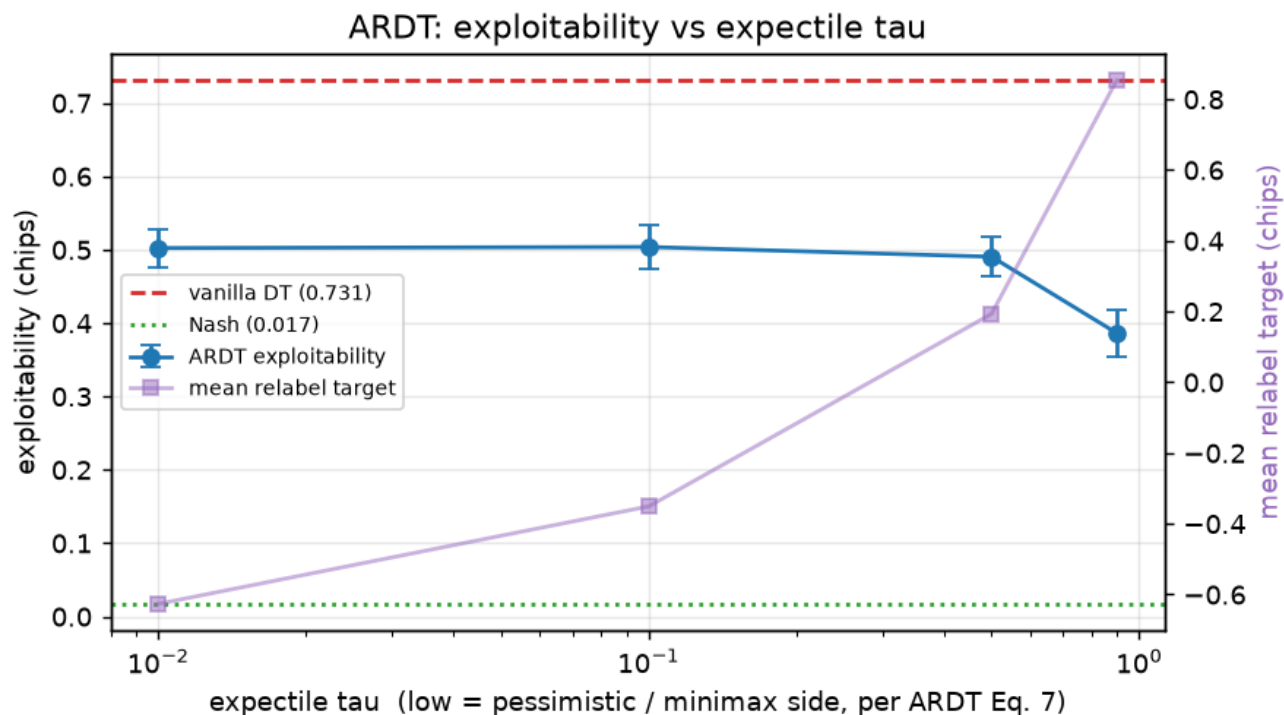
$$L_{\text{ER}}^{\alpha}(u) = \mathbb{E}_u[|\alpha - \mathbf{1}(u > 0)| \cdot u^2]$$

чиито граници дават необходимите оператори: $\alpha \rightarrow 0$ възстановява **минимума**, а $\alpha \rightarrow 1$ — **максимума**. Траекториите се преетикетират с оценената минимакс доходност и върху тях се обучава стандартен дървовиден модел на решенията.

Два детайла от публикувания метод имат значение и двата бяха потвърдени чрез прочитане на източника, а не само резюмето. Първо, песимистичната страна е **ниска** α (твърдението в суровия текст, че $\tau = 0.9$ е песимистичен, е обърнато); самата статия изпълнява $\alpha = 0.01$. Второ — по-важното — целта за преозначаване е

$$\tilde{R}_t = \tilde{Q}_\nu(s_t, a_t)$$

стойност на **състояние-действие**, получена от два *свързани* оценителя, които се напасват последователно със загуби α и $1 - \alpha$. Стойност на състояние $V(s)$ не може да различи „това състояние е лошо“ от „това действие в това състояние е лошо“, което е точно дискриминацията, върху която методът се основава.



Фигура 1: ARDT exploitability against the expectile tau, with the mean relabel target on the second axis.

Измерена върху **кун** с умишлено опростен заместител, отчитащ само състоянието, целта за преозначаване се изменя монотонно с τ , както изисква теорията — но експлоатируемостта е *най-ниска* от оптимистичната страна, което е белегът на липсващия аргумент на **действието**, а не доказателство срещу **абстрактния дървовиден модел на решенията**.

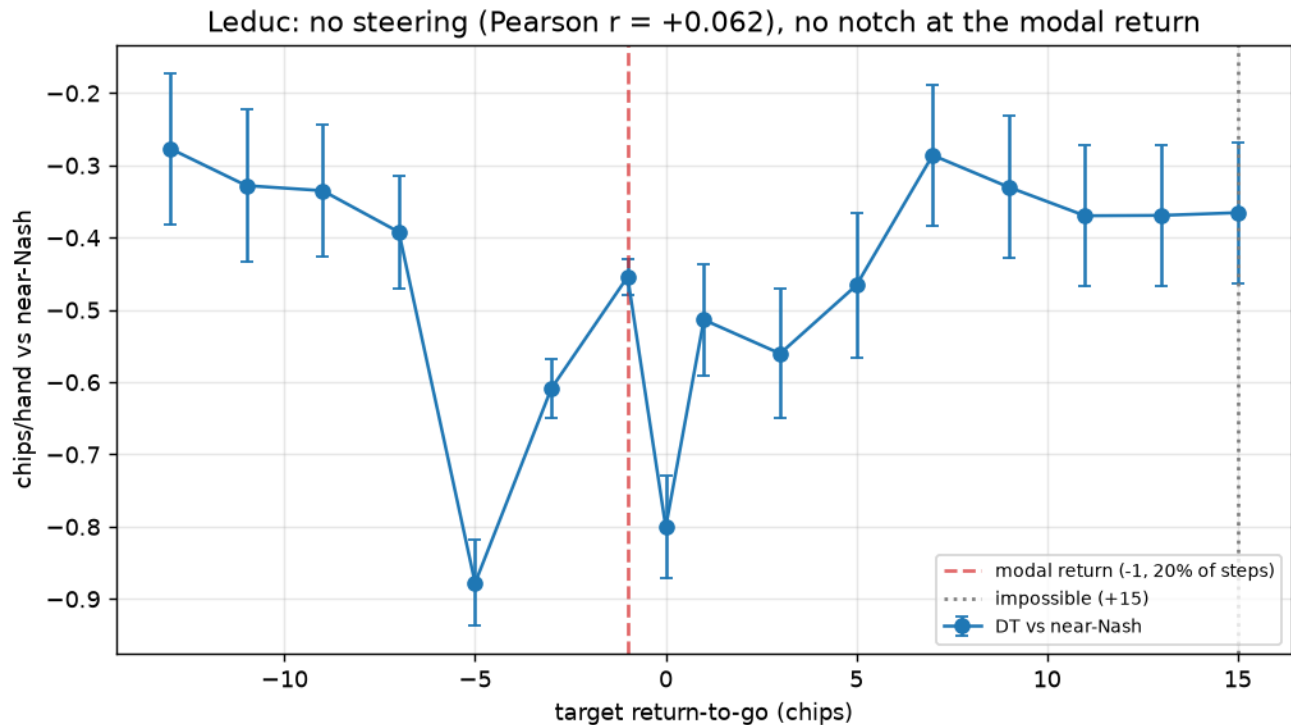
Read more: Tang, Zhang, Gu и др., *Adversarially Robust Decision Transformer*, NeurIPS 2024 (arXiv:2407.18414) — Section 3 и Algorithm 1.

12.4 Какво всъщност представлява условното връщане в покера

Равновесието и експлоатируемостта на **кун покер** могат да бъдат изчислени точно, поради което въпросът може да бъде решен, а не само обсъждан. Резултатът е, че условното връщане **променя** стратегията съществено, но не я **насочва**: експлоатируемостта остава непроменена в диапазона на реалните **добити цели**, с рязък спад при една конкретна стойност — модалната печалба, която в кун е печалбата от **пас**.

Естественото обяснение е, че в игра с четири печалби *големината* на връщането кодира кой **залог** е бил

изигран, а неговият *знак* — кой държи по-добрата карта. Следователно условното връщане определя формата на ръката, а не качеството на играта.



Фигура 2: Decision Transformer performance against the conditioned target return on Leduc Hold'em, with standard errors.

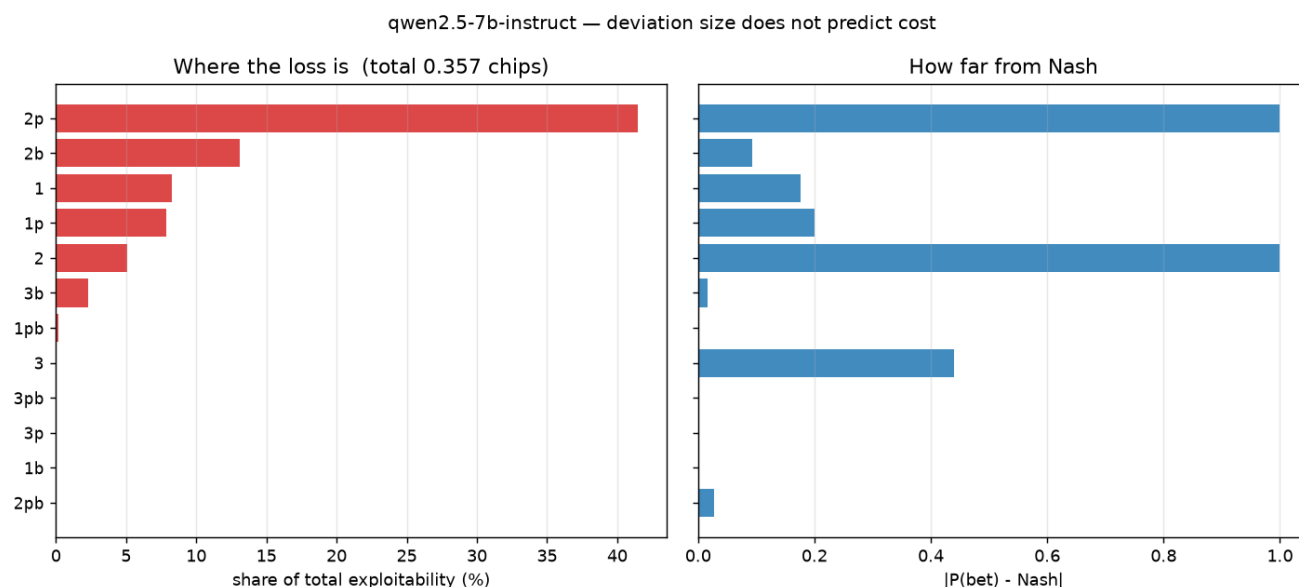
Ледюк Холдем тества това обяснение с петнадесет стойности на печалбата вместо четири, два кръга на залагане и обща карта. **Сривът изчезва точно както предвижда обяснението — и насочването все още не се наблюдава** (Пирсън $r = +0.062$). Следователно обяснението с алфабета на изплащанията обяснява срива, но не и провала. Провалът е по-фундаментален: в игра с нулева сума и непълна информация реализираната възвръщаемост се определя основно от тайната карта и действията на опонента, които главният герой не контролира. Никакво количество гранулираност на печалбата не прави един неконтролируем сигнал управляем.

Това е аргументът, който се пренася в Стъпка 13. При фиксирани логове трансформърът с обуславяне по връщането е неподходящият инструмент — и стойността на преетикетирането при абстрактния дървовиден модел на решенията се състои именно в това, че то заменя неконтролируема **цел на обуславяне** с контролируема.

12.5 Езикови модели като стратегически агенти

Втората парадигма изцяло пропуска обучението: опишете правилата на английски и оставете езиков модел да играе. Измерени по същия точен критерий, ЕМ са **честно експлоатируеми** — но интересният резултат е *къде* те се провалят.

Те разбират правилно **ранга** на ръцете, но грешат в **честотите** на смесените стратегии. Всеки тестван бекенд залага със стойност краля с вероятност 1,00, докато равновесието изисква смесване при 0,68, и никой не блъфира с валето изобщо при обикновена подкана. Модел от 7В съвпада с модел от 20В, защото играта възнаграждава правилното смесване, а не знанието или мащаба.



Фигура 3: Each information set’s share of total exploitability, beside its deviation from equilibrium.

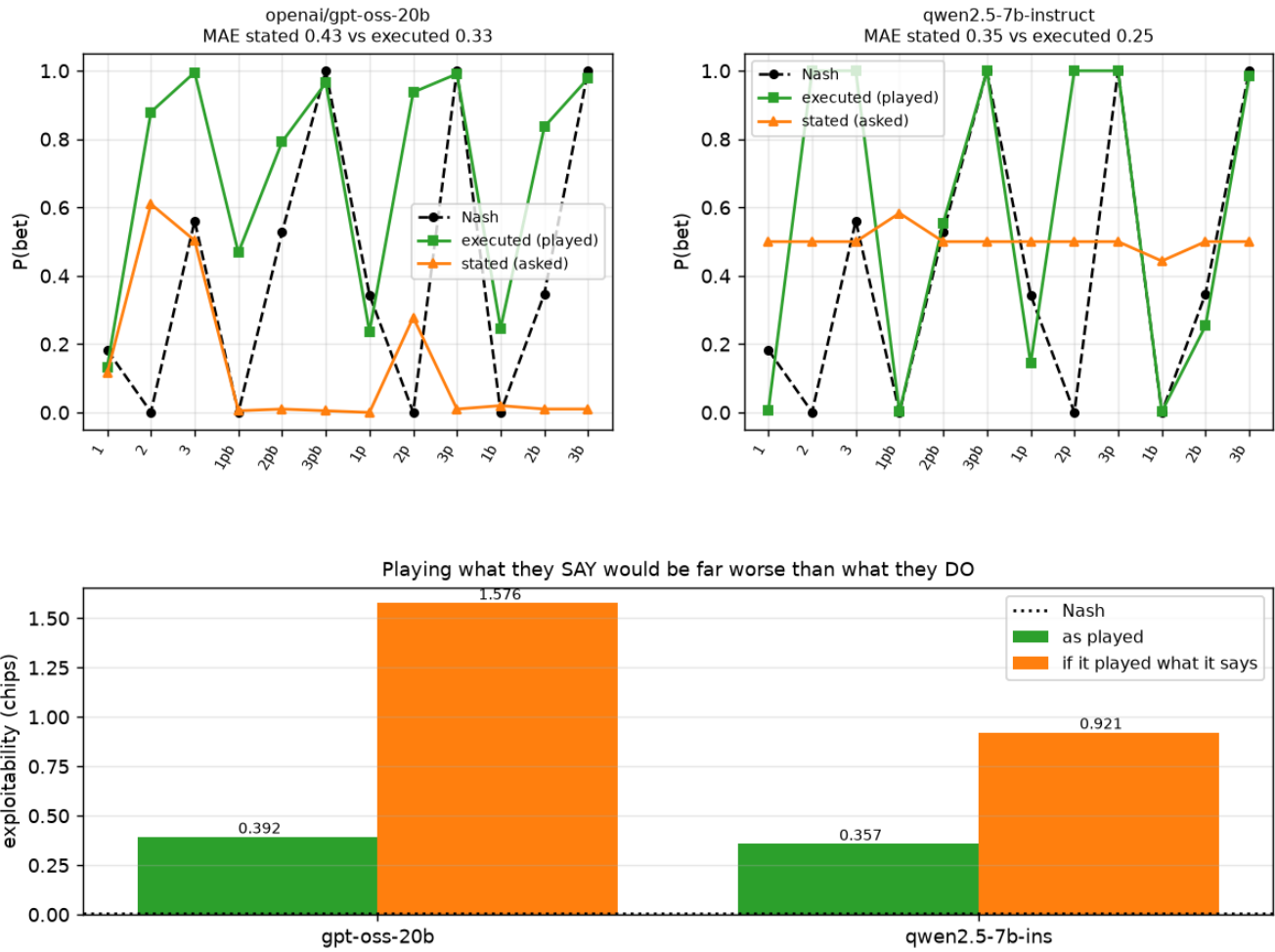
Разлагането на загубата по решение обръща очевидното тълкуване. „Провалът“ с краля — най-голямото видимо отклонение от равновесие — представлява **0,1%** от общата загуба, тъй като свръхзалагането на ръка, която никога не е по-слаба, е почти безплатно. Едно единствено решение с дама струва **41,4%**. Величината на отклонението и цената са почти некорелирани, което е конкретният аргумент за докладване на диагностика по решения, вместо да се представя само едно число за експлоатируемост в рамката за оценяване от Стъпка 14.

Питането на модел какво възнамерява е *по-лошо* от измерването на това, което прави. Оценени като стратегии, заявените от тези модели честоти са 2,6 и 4,0 пъти по-експлоатируеми в сравнение със стратегиите, които те реално играят. Един модел отговаря „50%“ почти навсякъде, а другият — „почти 0%“ почти навсякъде, включително и когато държи най-силната ръка. Поведенческото проучване тук не е просто по-удобно от самоанализа — то е по-точно.

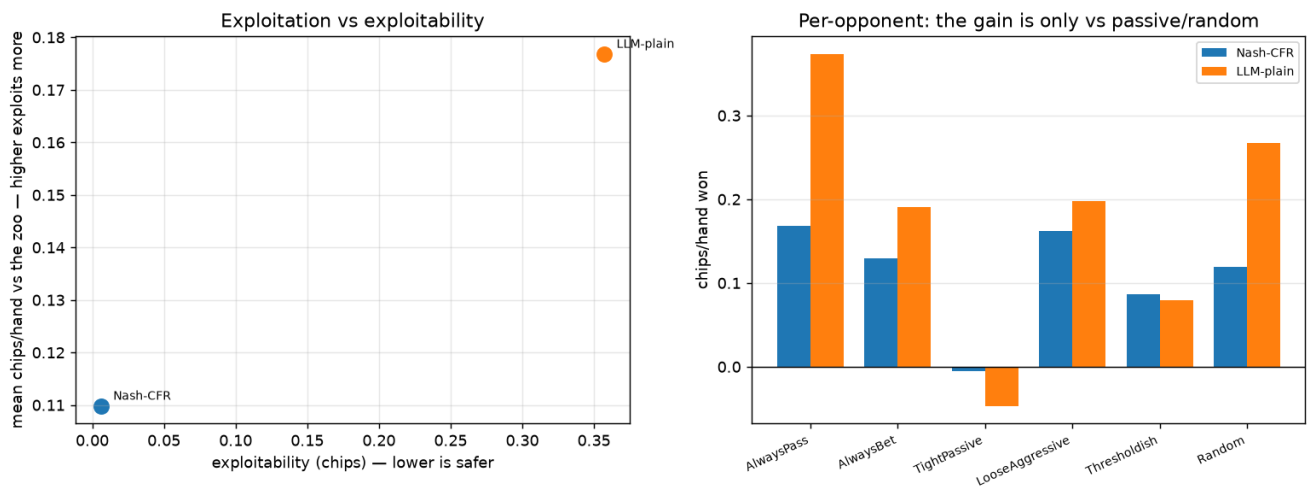
Прочетете повече: Guertler и др., *TextArena: A Framework for Text-Based Game Environments*, 2025 (arXiv:2504.11442).

12.6 Експлоатация, адаптация и ограниченията на опростената игра

Мерките за експлоатируемост показват доколко съвършен противник ви наказва; те не казват нищо за това колко добре вие сами наказвате слабостите. Измерено спрямо набор от умишлено експлоатируеми компетентни архетипи, езиковият модел **експлоатира с 61% по-силно от равновесната игра, като същевре-**



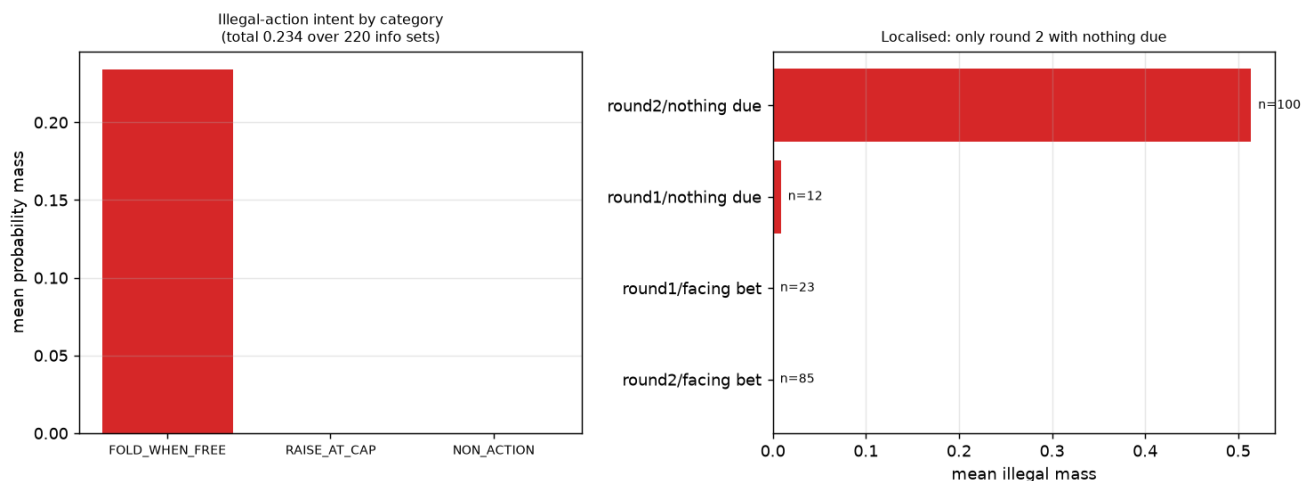
Фигура 4: Stated versus executed betting frequencies for two models, with the exploitability of each strategy.



Фигура 5: Exploitation against exploitability, with the per-opponent breakdown.

менно е 59 пъти по-експлоатируем — но цялата печалба идва от пасивни и случайни противници. Срещу двата най-компетентни архетипа той се представя *по-зле* от равновесието. Това е компромисът безопасност–експлоатация, който представлява вторият принос на дисертацията, наблюдаван, а не предположен.

Като се има предвид наблюдаемата история на една сесия в контекст, моделът **не** се адаптира: той улавя 83% от наличната експлоатация срещу тривиално пасивен противник още от първата половина на сесията и не показва никакво подобрене (средно обучение -0.22). Това, което изглежда като моделиране на противника, всъщност е фиксирано предварително убеждение, благоприятстващо разпуснат-агресивни стратегии.



Фигура 6: Illegal-action intent by category and by situation on Leduc Hold'em.

Накрая опростената игра ласкае тези модели. Една улица и една обща карта по-късно големият езиков модел е статистически неразличим от трансформъра за вземане на решения, изобщо не може да победи слаби противници и има склонност към недопустимо действие с приблизително една четвърт от своята вероятностна маса. Този провал се дължи на *едно* погрешно схващане, а не на разпръснатата обърканост — моделът иска да **пасува, когато проверката е безплатна**, почти изключително със слаби несдвоени раздавания срещу висока маса, докато никога не прави опит за повишаване над лимита за залагане. Всеки оптимизъм, произтичащ от опростената игра, трябва да се възприема като свойство на самата опростена игра.

12.7 Основни изводи за синтеза на дисертацията

- **Обуславянето на величина, която агентът не контролира, не може да го насочва.** Демонстрирано в две игри; това е причината преетикетирането на ARDT — а не самият трансформър за вземане на решения — да се прехвърля към фиксирани покер логове.
- **Публикуваният ARDT преетикетира със стойност на състояние-действие.** Възпроизвеждането на ползата от метода изисква $\tilde{Q}(s, a)$, а не $V(s)$ — идентифицираното и подкрепено с доказателства решение за Стъпка 13.
- **Скалярна оценка класира методите, но не ги диагностицира.** Едно решение носи 41,4% от загубата, докато най-забележимото отклонение носи 0,1%.
- **Измервайте поведението, а не самоотчета.** Вербализираната стратегия е съществено по-лоша от

изиграната стратегия за всеки тестван модел.

- **Адаптация към съперника в контекста не се появява**, така че е необходим експлицитен модел на противника, а не да се приема за даденост — пряко свързано с първия принос на дисертацията.
- **Протоколът за измерване е част от резултата**. При алчен избор на последователност езиковият модел играе чиста стратегия и всяка измерена честота дегенерира до 0 или 1; четири отделни открития в тази стъпка се оказаха измервателни артефакти, всеки от които беше хванат чрез евтина проверка спрямо точно изчислимо количество.